

The review form should look as follows:

Add review

Linkage

Creator:

bob

▼

✎ + 👁

Book:

Hands-On Machine Learning for Algorithmic Trading (9781789346411)

▼

✎ + 👁

The Book that this review is for.

Review content

Content:

An excellent introduction to the topic.

⏏

The Review text.

Rating:

5

⬆ ⬇ ⬆

The rating the reviewer has given.

SAVE

Save and add another

Save and continue editing

Figure 4.61 – The review form with fieldsets

If we want to omit the title on a fieldset, we can do so by assigning the None value to it:

```
fieldsets = (
    (None, {"fields": ("creator", "book")}),
    ("Review content", {"fields": ("content", "rating")}),
)
```

Now, the review form should appear as shown in the following screenshot:

검토 양식은 다음과 같아야 합니다:

Add review

Linkage

Creator:

bob

▼

✎ + 👁

Book:

Hands-On Machine Learning for Algorithmic Trading (9781789346411)

▼

✎ + 👁

The Book that this review is for.

Review content

Content:

An excellent introduction to the topic.

⌵

The Review text.

Rating:

5

⬆ ⬇ ⬆

The rating the reviewer has given.

SAVE

Save and add another

Save and continue editing

그림 4.61 - 필드셋이 있는 검토 양식

필드셋에서 제목을 생략하고 싶다면, None 값을 할당하여 그렇게 할 수 있습니다:

```
fieldsets = (
    (None, {"fields": ("creator", "book")}),
    ("검토 내용", {"fields": ("content", "rating")}),
)
```

이제 검토 양식은 다음 스크린샷과 같이 나타나야 합니다:

Add review

Creator:   

Book:   
The Book that this review is for.

Review content

Content:
The Review text.

Rating: 
The rating the reviewer has given.

Figure 4.62 – The review form with the first fieldset untitled

Now that we have learned about change lists and form customizations, we can put our knowledge into practice with a comprehensive activity.

Activity 4.03 – Customizing the model admins

In our data model, the Contributor class is used to store data for book contributors; they can be authors, contributors, or editors. This activity focuses on modifying the Contributor class and adding a ContributorAdmin class to improve the user-friendliness of the admin app. At present, the Contributor change list defaults to a single column, FirstNames, based on the `__str__` method created in *Chapter 2*. We will investigate some alternative ways of representing this. These steps will help you complete the activity:

1. Edit `reviews/models.py` to add additional functionality to the Contributor model.
2. Add an `initialled_name` method to Contributor that takes no arguments (like the `Book.isbn13` method).
3. The `initialled_name` method will return a string containing Contributor.`last_names` followed by a comma and the initials of the given names. For example, for a Contributor object with `first_names` of Jerome David and `last_names` of Salinger, `initialled_name` will return Salinger, JD. If the

Add review

Creator:   

Book:   
The Book that this review is for.

Review content

Content: 
The Review text.

Rating: 
The rating the reviewer has given.

그림 4.62 - 첫 번째 필드셋이 제목 없는 리뷰 양식

변경 목록과 양식 사용자 정의에 대해 배웠으므로, 포괄적인 활동을 통해 우리의 지식을 실천에 옮길 수 있습니다.

활동 4.03 – 모델 관리자 사용자 정의

우리의 데이터 모델에서 Contributor 클래스는 책 기여자를 위한 데이터를 저장하는 데 사용됩니다. 그들은 저자, 기여자 또는 편집자가 될 수 있습니다. 이 활동은 Contributor 클래스를 수정하고 ContributorAdmin 클래스를 추가하여 관리 앱의 사용자 친화성을 향상시키는 데 중점을 둡니다. 현재 Contributor 변경 목록은 Chapter 2에서 생성된 `__str__` 메서드를 기반으로 하는 단일 열, `FirstNames`로 기본 설정되어 있습니다. 우리는 이를 나타내는 몇 가지 대체 방법을 조사할 것입니다. 이 단계는 활동을 완료하는 데 도움이 될 것입니다:

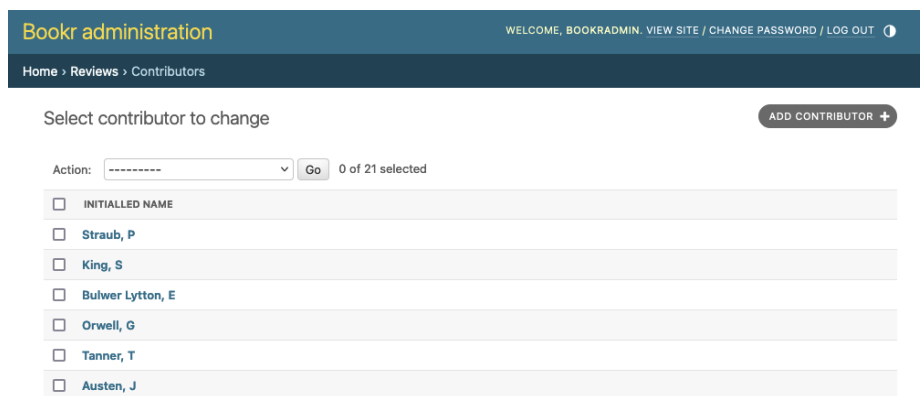
1. `reviews/models.py`를 편집하여 Contributor 모델에 추가 기능을 추가합니다
2. Contributor에 인수를 받지 않는 `initialled_name` 메서드를 추가합니다 (예: `Book.isbn13` 메서드와 같이).
3. `initialled_name` 메서드는 `Contributor.last_names` 뒤에 심표와 주어진 이름의 이니셜이 포함된 문자열을 반환합니다. 예를 들어, `first_names`가 `Jerome David`이고 `last_names`가 `Salinger`인 Contributor 객체의 경우, `initialled_name`은 `Salinger, JD`를 반환합니다. 만약

`Contributor.last_names` is blank, then `Contributor.first_names` will be returned. Here is an example:

```
>>> Contributor(first_names="Plato",
                 last_names="").initialled_name()

'Plato'
```

4. Replace the `__str__` method for `Contributor` with one that calls `initialled_name()`. At this point, the **Contributors** display list will look like this:



The screenshot shows the 'Bookr administration' interface. At the top, there's a header with 'Bookr administration' on the left and 'WELCOME, BOOKRADMIN. VIEW SITE / CHANGE PASSWORD / LOG OUT' on the right. Below the header is a breadcrumb trail: 'Home > Reviews > Contributors'. The main content area is titled 'Select contributor to change' and includes an 'ADD CONTRIBUTOR +' button. Below this is an 'Action:' dropdown menu with a 'Go' button and a status '0 of 21 selected'. A table lists contributors with checkboxes and their names:

☐	INITIALLED NAME
☐	Straub, P
☐	King, S
☐	Bulwer Lytton, E
☐	Orwell, G
☐	Tanner, T
☐	Austen, J

Figure 4.63 – The Contributors display list

5. Edit the `reviews/admin.py` file. We will modify `ContributorAdmin`, which was subclassed from `admin.ModelAdmin` in the *The list display fields* section.
6. Modify it so that on the Contributors change list, records are displayed with two sortable columns: Last Names and First Names. We can remove the existing `list_display` attribute from `ContributorAdmin` along with the `initialled_name` function.
7. Add a search bar that searches on Last Names and First Names. Modify it so that the Last Names search matches the start of Last Names.
8. Add a filter on Last Names.

By completing the activity, you should see something like this:

Contributor.last_names가 비어 있으면 Contributor.first_names가 반환됩니다. 예시는 다음과 같습니다:

```
>>> Contributor(first_names="Plato",
                  last_names="").initialled_name()

'Plato'
```

4. Contributor의 __str__ 메서드를 initialled_name()을 호출하는 것으로 교체하십시오. 이 시점에서 Contributors 표시 목록은 다음과 같이 보일 것입니다:

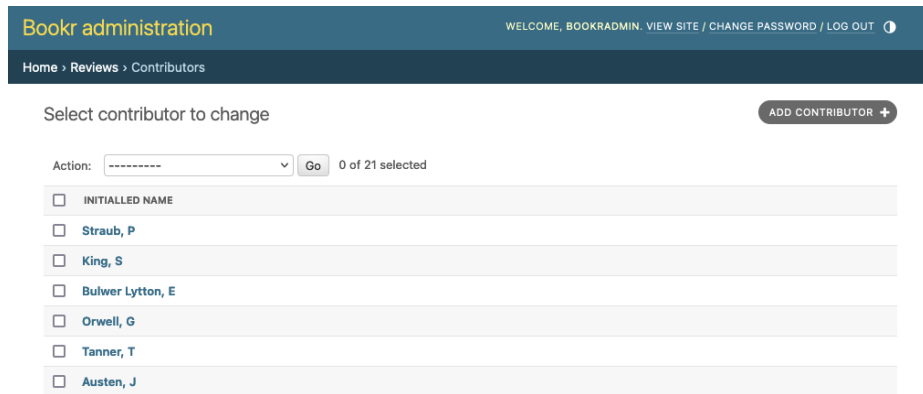


그림 4.63 - 기여자 표시 목록

5. reviews/admin.py 파일을 수정합니다. 우리는 기여자 관리자인 ContributorAdmin을 수정할 것입니다. 이는 The list display fields 섹션에서 admin.ModelAdmin에서 서브클래스화되었습니다.
6. 기여자 변경 목록에서 Last Names와 First Names라는 두 개의 정렬 가능한 열로 레코드가 표시되도록 수정합니다. ContributorAdmin에서 기존 list_display 속성과 initialled_name 함수를 제거할 수 있습니다.
7. Last Names와 First Names에서 검색하는 searchidx_9dc391a5 바를 추가합니다. Last Names 검색이 Last Names의 시작과 일치하도록 수정합니다.
8. Last Names에 대한 필터를 추가합니다.

활동을 완료하면 다음과 같은 것을 볼 수 있어야 합니다:

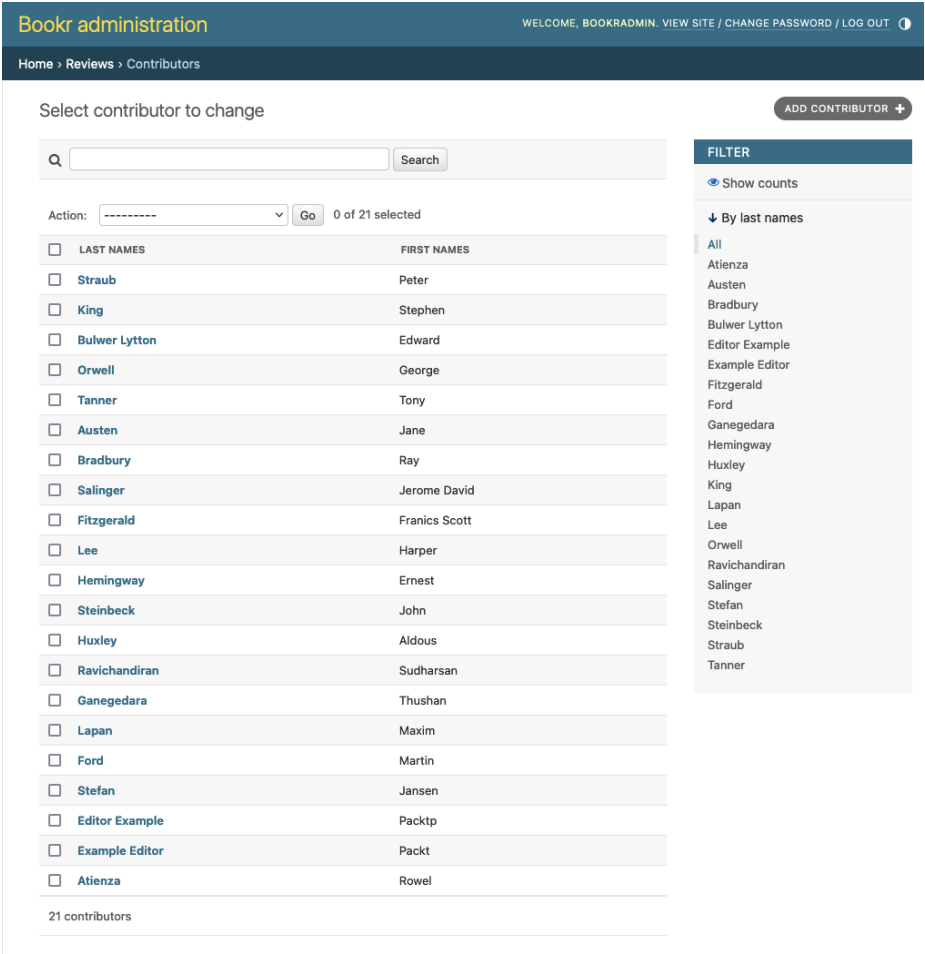


Figure 4.64 – Expected output

Changes such as these can be made to improve the functionality of the admin user interface. By implementing the First Names and Last Names columns as separate columns in the Contributors change list, we are giving the user an option to sort on either of the fields. By considering what columns are most useful in search retrieval and filter selections, we can improve the efficient retrieval of records.

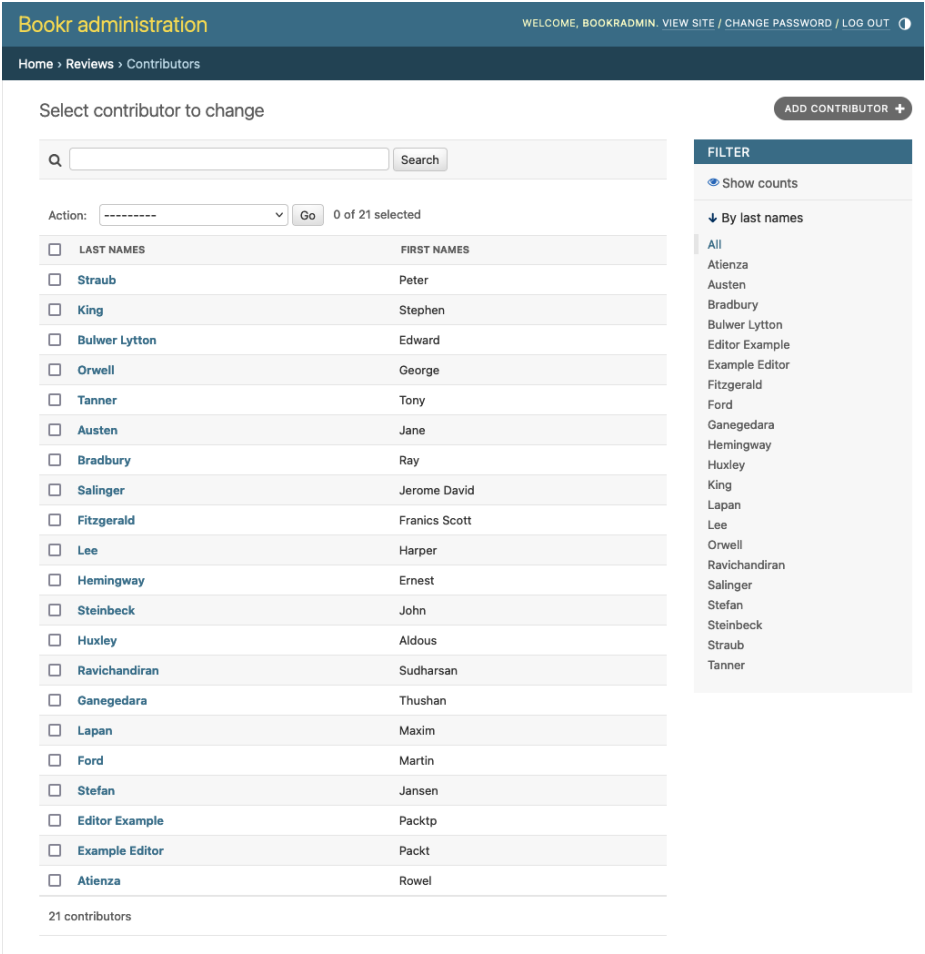


그림 4.64 - 예상 출력

이러한 변경 사항은 관리 사용자 인터페이스의 기능을 개선하기 위해 이루어질 수 있습니다. 기여자 변경 목록에서 이름과 성을 별도의 열로 구현함으로써 사용자가 두 필드 중 하나로 정렬할 수 있는 옵션을 제공합니다. 검색 검색 및 필터 선택에서 가장 유용한 열이 무엇인지 고려함으로써 우리는 기록의 효율적인 검색을 개선할 수 있습니다.

Summary

In this chapter, we saw how to create superusers through the Django command line and how to use them to access the admin app. Then, after a brief tour of the admin app's basic functionality, we examined how to register our models with it to produce a CRUD interface for our data.

Then, we learned how to refine this interface by modifying site-wide features. We altered how the admin app presents model data to the user by registering custom model admin classes with the admin site. This allowed us to make fine-grained changes to the representation of our models' interfaces. These modifications included customizing change list pages by adding additional columns, filters, date hierarchies, and search bars. We also modified the layout of the model admin pages by grouping and excluding fields.

This was only a very shallow dive into the functionality of the admin app. We will revisit the rich functionality of `AdminSite` and `ModelAdmin` in *Chapter 10*. But first, we need to learn some more intermediate features of Django. In the next chapter, we will learn how to organize and serve static content, such as CSS, JavaScript, and images, from a Django app.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

요약

이 장에서는 Django 명령줄을 통해 슈퍼유저를 생성하는 방법과 이를 사용하여 관리 앱에 접근하는 방법을 살펴보았습니다. 그런 다음 관리 앱의 기본 기능을 간단히 둘러본 후, 데이터에 대한 CRUD 인터페이스를 생성하기 위해 모델을 등록하는 방법을 검토했습니다.

그런 다음 사이트 전반의 기능을 수정하여 이 인터페이스를 개선하는 방법을 배웠습니다. 우리는 관리 사이트에 사용자 정의 모델 관리 클래스를 등록하여 관리 앱이 사용자에게 모델 데이터를 표시하는 방식을 변경했습니다. 이를 통해 모델 인터페이스의 표현을 세밀하게 조정할 수 있었습니다. 이러한 수정 사항에는 추가 열, 필터, 날짜 계층 및 검색 표시줄을 추가하여 변경 목록 페이지를 사용자 정의하는 것이 포함되었습니다. 또한 필드를 그룹화하고 제외하여 모델 관리 페이지의 레이아웃을 수정했습니다.

이것은 관리 앱의 기능에 대한 매우 얇은 탐구에 불과했습니다. 우리는 10장에서 AdminSite와 ModelAdmin의 풍부한 기능을 다시 살펴볼 것입니다. 하지만 먼저 Django의 중급 기능을 좀 더 배워야 합니다. 다음 장에서는 Django 앱에서 CSS, JavaScript 및 이미지와 같은 정적 콘텐츠를 구성하고 제공하는 방법을 배울 것입니다.

이 책의 PDF 사본, 코드 번들 및 기타 자료를 받으세요

QR 코드를 스캔하거나 packtpub.com/unlock으로 이동하세요. 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장 사본을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

5

Serving Static Files

A web application with just plain **HyperText Markup Language (HTML)** is quite limiting. However, we can enhance the look of web pages with **Cascading Style Sheets (CSS)** and images, and add interaction with JavaScript. We call all these kinds of files "static files," which are developed and then deployed as part of the application. We can compare this to dynamic responses, which are generated in real time when a request is made. All the views you have written generate a dynamic response by rendering a template.

Note

We will not consider templates to be static files as they are not sent verbatim to a client; instead, they are rendered first and sent as part of a dynamic response.

During development, the static files are created on the developer's machine, and then must be moved to the production web server. If you must move an application to production in a short timeframe (say, a few hours), then it can be time-consuming to collect all the static assets, move them to the correct directory, and upload them to the server. When developing web applications using other frameworks or languages, you might need to manually put all of your static files into a specific directory that your web server hosts. Making changes to the URL from which static files are served might mean updating values throughout your code.

Django can manage static assets for us to make this process easier. It provides tools for serving them with its development server during development. When your application goes to production, it can also collect all your assets and copy them to a folder for a dedicated web server to host. This allows you to keep your static files segregated in a meaningful way during development, and automatically bundle them for deployment.

5

정적 파일 제공

단순한 하이퍼텍스트 마크업 언어(HTML)로만 구성된 웹 애플리케이션 idx_b514d5a4는 상당히 제한적입니다. 그러나 우리는 웹 페이지의 모양을 향상시키기 위해 캐스케이딩 스타일 시트(CSS)와 이미지로 개선하고, 자바스크립트를 사용하여 상호작용을 추가할 수 있습니다. 우리는 이러한 모든 종류의 파일을 "정적 파일"이라고 부르며, 이는 개발된 후 애플리케이션의 일부로 배포됩니다. 이는 요청이 있을 때 실시간으로 생성되는 동적 응답과 비교할 수 있습니다. 여러분이 작성한 모든 뷰는 템플릿을 렌더링하여 동적 응답을 생성합니다.

참고

우리는 템플릿이 클라이언트에 문자 그대로 전송되지 않기 때문에 정적 파일로 간주하지 않을 것입니다. 대신, 템플릿은 먼저 렌더링되고 동적 응답의 일부로 전송됩니다.

개발 중에 정적 파일은 개발자의 머신에서 생성되며, 이후 프로덕션 웹 서버로 이동해야 합니다. 만약 애플리케이션을 짧은 시간 내에 프로덕션으로 이동해야 한다면(예: 몇 시간), 모든 정적 자산을 수집하고 올바른 디렉토리로 이동한 후 서버에 업로드하는 데 시간이 많이 걸릴 수 있습니다. 다른 프레임워크나 언어를 사용하여 웹 애플리케이션을 개발할 때, 웹 서버가 호스팅하는 특정 디렉토리에 모든 정적 파일을 수동으로 넣어야 할 수도 있습니다. 정적 파일이 제공되는 URL을 변경하는 것은 코드 전반에 걸쳐 값을 업데이트해야 할 수 있음을 의미합니다.

장고는 이 과정을 쉽게 만들기 위해 정적 자산을 관리할 수 있습니다. 개발 중에 개발 서버를 통해 정적 자산을 제공하는 도구를 제공합니다. 애플리케이션이 프로덕션으로 이동할 때, 모든 자산을 수집하고 전용 웹 서버가 호스팅할 폴더로 복사할 수 있습니다. 이를 통해 개발 중에 정적 파일을 의미 있는 방식으로 분리하고, 배포를 위해 자동으로 번들링할 수 있습니다.

This functionality is provided by Django's built-in `staticfiles` app. It adds several useful features for working with and serving static files:

- The `static` template tag automatically builds the static URL for an asset and includes it in your HTML.
- A view (called `static`) serves static files in development.
- Static file finders can be used to customize where assets are found on your filesystem.
- The `collectstatic` management command finds all static files and moves them into a single directory for deployment.
- The `findstatic` management command shows which static file on the disk has been loaded for a particular request. This also helps us debug if a particular file is not being loaded.

In the exercises and activities of this chapter, we will be adding static files (images and CSS) to the Bookr application. Each file will be stored inside the Bookr project directory during development. We will need to generate a URL for each so that the templates can reference them and the browser can download them. Once the URL has been generated, Django will need to serve these files. When we deploy the Bookr application to production, all the static files need to be found and moved to a directory where they can be served by the production web server. If there are static files that are not loading as expected, we need a method of determining what the cause is.

In this chapter, you will start by learning the difference between static and dynamic responses. You will then see how the Django `staticfiles` app helps manage static files. As you continue to work on the Bookr app, you will enhance it with images and CSS. You'll see the different ways you can lay out your static files for your project and examine how Django consolidates them for production deployment. Django includes tools for referencing static files in templates; you'll see how these tools help reduce the amount of work you must do when deploying the application to production. After this, you'll explore the `findstatic` command, which can be used to debug issues with your static files. Later, you'll get an overview of how to write code for storing static files on a remote service. Finally, you'll look at caching web assets and how Django can help with cache invalidation.

We will cover the following main topics in this chapter:

- Static file serving
- Introduction to static files finders
- Generating static URLs with the static template tag
- FileSystem Finder

이 기능은 Django의 내장 staticfiles 앱에서 제공됩니다. 정적 파일을 작업하고 제공하는 데 유용한 여러 기능을 추가합니다:

- 정적 템플릿 태그는 자산의 정적 URL을 자동으로 생성하고 이를 HTML에 포함시킵니다.
- 정적 파일을 제공하는 뷰(정적이라고 함)가 개발 중에 정적 파일을 제공합니다.
- 정적 파일 탐색기는 자산이 파일 시스템에서 어디에 있는지를 사용자 정의하는 데 사용할 수 있습니다. collectstatic 관리 명령은 모든 정적 파일을 찾아 배포를 위해 단일 디렉토리로 이동합니다.
- findstatic 관리 명령은 특정 요청에 대해 디스크에서 어떤 정적 파일이 로드되었는지를 보여줍니다. 이는 특정 파일이 로드되지 않는 경우 디버깅하는 데 도움이 됩니다.

이 장의 연습 및 활동에서는 Bookr 애플리케이션에 정적 파일(이미지 및 CSS)을 추가할 것입니다. 각 파일은 개발 중에 Bookr 프로젝트 디렉토리 내에 저장됩니다. 템플릿이 이를 참조하고 브라우저가 다운로드할 수 있도록 각 파일에 대한 URL을 생성해야 합니다. URL이 생성되면 Django는 이러한 파일을 제공해야 합니다. Bookr 애플리케이션을 프로덕션에 배포할 때 모든 정적 파일을 찾아 프로덕션 웹 서버가 제공할 수 있는 디렉토리로 이동해야 합니다. 예상대로 로드되지 않는 정적 파일이 있는 경우 원인을 파악할 방법이 필요합니다.

이 장에서는 정적 응답과 동적 응답의 차이를 배우는 것으로 시작합니다. 그런 다음 Django 정적 파일 앱이 정적 파일을 관리하는 데 어떻게 도움이 되는지 살펴봅니다. Bookr 앱 작업을 계속하면서 이미지와 CSS로 향상시킬 것입니다. 프로젝트에 대한 정적 파일을 배치하는 다양한 방법을 보고 Django가 이를 프로덕션 배포를 위해 어떻게 통합하는지 살펴봅니다. Django는 템플릿에서 정적 파일을 참조하는 도구를 포함하고 있으며, 이러한 도구가 애플리케이션을 프로덕션에 배포할 때 수행해야 하는 작업의 양을 줄이는 데 어떻게 도움이 되는지 확인할 수 있습니다. 그 후, 정적 파일의 문제를 디버깅하는 데 사용할 수 있는 findstatic 명령을 탐색합니다. 나중에 원격 서비스에 정적 파일을 저장하기 위한 코드 작성 방법에 대한 개요를 제공합니다. 마지막으로 웹 자산 캐싱과 Django가 캐시 무효화에 어떻게 도움이 되는지 살펴봅니다.

이 장에서는 다음과 같은 주요 주제를 다룰 것입니다

- 다: 정적 파일 제공
- 정적 파일 탐색기 소개
- 정적 템플릿 태그 파일 시스템 파인더로 정적
- URL 생성하기

- Static file finders – use during collectstatic
- STATICFILES_DIRS prefixed mode
- The findstatic command
- Serving the latest files (for cache validation)
- Custom storage engines

Technical requirements

Find the solution in the Chapter05 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Django and static file serving

In the introduction, we mentioned that Django includes a view function called `static` that serves static files. The first important point to make is that *Django does not intend to serve static files in production*. It is not Django's role, and in production, this is better suited to the web server. If Django is reading from the filesystem and sending out a file, then it has no advantage over a normal web server, which will probably be more performant at this task. Further, if you serve static files with Django, you will keep the Python process busy for the duration of this request, and it will be unable to serve the dynamic requests for which it is more suited.

For these reasons, the Django `static` view is designed only for use during development and will not work if your `DEBUG` setting is `False`. Since, during development, we usually only have one person accessing the site at a time (the developer), then Django is fine to serve static files. We will discuss how the `staticfiles` app supports production deployment more shortly. Plus, the entire production deployment process will be covered in *Chapter 18, Deploying Django*.

A URL mapping to the `static` view is automatically set up when running the Django development server, provided that your `settings.py` file consists of the following elements:

- Has `DEBUG` set to `True`
- Contains `"django.contrib.staticfiles"` in its `INSTALLED_APPS`

Both of these settings are configured on a newly created app.

The URL mapping that is created is roughly equivalent to having the following map in your `urlpatterns`:

```
path(settings.STATIC_URL, django.conf.urls.static)
```

- 정적 파일 찾기 –
- collectstaticSTATICFILES_DIRS 접두
- 사 모드에서 사용findstatic 명령
- 최신 파일 제공 (캐시 검증용)커스텀 스
- 토리지 엔진

기술 요구 사항

이 책의 GitHub 저장소의 Chapter05 폴더에서 솔루션을 찾으세요. 저장소 링크에 접근하려면 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르세요.

장고와 정적 파일 제공

소개에서 우리는 Django에 정적 파일을 제공하는 static이라는 뷰 함수가 포함되어 있다고 언급했습니다. 첫 번째 중요한 점은 Django가 프로덕션에서 정적 파일을 제공할 의도가 없다는 것입니다. 이는 Django의 역할이 아니며, 프로덕션에서는 웹 서버에 더 적합합니다. Django가 파일 시스템에서 읽고 파일을 전송하는 경우, 이는 아마도 이 작업에서 더 성능이 좋은 일반 웹 서버에 비해 아무런 이점이 없습니다. 또한 Django로 정적 파일을 제공하면 이 요청의 지속 시간 동안 Python 프로세스가 바쁘게 유지되며, 이는 더 적합한 동적 요청을 제공할 수 없게 됩니다.

이러한 이유로 Django 정적 뷰는 개발 중에만 사용하도록 설계되었으며 DEBUG 설정이 False인 경우 작동하지 않습니다. 개발 중에는 일반적으로 한 번에 한 사람(개발자)만 사이트에 접근하므로 Django가 정적 파일을 제공하는 것은 괜찮습니다. 정적 파일 앱이 프로덕션 배포를 지원하는 방법에 대해서는 곧 논의할 것입니다. 또한 전체 프로덕션 배포 프로세스는 18장, Django 배포에서 다룰 것입니다.

Django 개발 서버를 실행할 때 정적 뷰에 대한 URL 매핑이 자동으로 설정됩니다. 단, settings.py 파일이 다음 요소로 구성되어 있어야 합니다:

- DEBUG가 True로 설정되어 있음
- INSTALLED_APPS에 "django.contrib.staticfiles"가 포함되어

어 있음. 이 두 설정은 새로 생성된 앱에서 구성됩니다.

생성된 URL 매핑은 대략적으로 다음과 같은 urlpatterns에 맵이 있는 것과 같습니다:

```
path(settings.STATIC_URL, django.conf.urls.static)
```

Any URL starting with `settings.STATIC_URL` (which is `/static/` by default) gets mapped to the static view.

Note

You could use the static view without having staticfiles in `INSTALLED_APPS`, but you must set up an equivalent URL mapping manually.

Next, we'll talk about how Django locates static files using Static Files Finder.

Introduction to static files finders

Static files finders can be thought of as a plugin component of the Django architecture. They are classes that implement methods for converting URL paths into file paths, iterating through the project directory to find static files. There are three situations when Django needs to locate static files on disk, using static file finders to do so:

- The first is when the Django static view receives a request to load a particular static file; here, it needs to convert the path in the URL into a location on disk. For example, let's say the URL's path is `/static/logo.png`, and it is converted into the `bookr/static/logo.png` path on the disk. As we noted in the previous section, this only happens during development. On a production server, Django should not receive this request as it will be handled directly by the web server.
- The second is when using the `collectstatic` management command. This gathers all the static files in the project directory and copies them to a single directory to be served by the production web server. `bookr/static/logo.png` will get copied to the web server root – for example, `/var/www/bookr/static/logo.png`. The static file finder contains code for locating all the static files inside your project directory.
- The third use of static file finders is while executing the `findstatic` management command. This is similar to the first usage, where it accepts a static file name (such as `logo.png`), but it outputs the full path (`bookr/static/logo.png`) to the terminal instead of loading the file's content.

Django comes with some built-in finders, but you can also write your own if you want to store static files in a custom directory layout. The list of finders Django uses is defined by the `STATICFILES_FINDERS` setting in `settings.py`. In this chapter, we will cover the behavior of the default static file finders, `AppDirectoriesFinder` and `FileSystemFinder`, in the *AppDirectoriesFinder* subsection and the *FileSystemFinder* section, respectively.

settings.STATIC_URL로 시작하는 모든 URL idx_5ca35b19(/static/가 기본값)은 정적 뷰에 매핑됩니다.

참고

INSTALLED_APPS에 정적 파일이 없더라도 정적 뷰를 사용할 수 있지만, 동등한 URL 매핑을 수동으로 설정해야 합니다.

다음으로, Django가 정적 파일을 Static Files Finder를 사용하여 찾는 방법에 대해 이야기하겠습니다.

정적 파일 파인더 소개

정적 파일 파인더는 Django 아키텍처의 플러그인 구성 요소로 생각할 수 있습니다. 이들은 URL 경로를 파일 경로로 변환하고, 정적 파일을 찾기 위해 프로젝트 디렉토리를 반복하는 메서드를 구현하는 클래스입니다. Django가 디스크에서 정적 파일을 찾아야 할 세 가지 idx_fa10432상황이 있습니다. 정적 파일 파인더를 사용하여 이를 수행합니다:

- 첫 번째는 Django 정적 뷰가 특정 정적 파일을 로드하라는 요청을 받을 때입니다. 이 경우 URL의 경로를 디스크의 위치로 변환해야 합니다. 예를 들어, URL의 경로가 /static/logo.png라고 가정하면, 이는 디스크의 bookr/static/logo.png 경로로 변환됩니다. 이전 섹션에서 언급했듯이, 이는 개발 중에만 발생합니다. 프로덕션 서버에서는 Django가 이 요청을 받지 않아야 하며, 이는 웹 서버에 의해 직접 처리됩니다.
- 두 번째는 collectstatic 관리 명령을 사용할 때입니다. 이는 프로젝트 디렉토리의 모든 정적 파일을 수집하고, 이를 단일 디렉토리로 복사하여 프로덕션 웹 서버에서 제공할 수 있도록 합니다. bookr/static/logo.png는 웹 서버 루트로 복사됩니다 - 예를 들어, /var/www/bookr/static/logo.png. 정적 파일 파인더는 프로젝트 디렉토리 내의 모든 정적 파일을 찾기 위한 코드를 포함하고 있습니다.
- 정적 파일 찾기의 세 번째 사용은 findstatic 관리 명령을 실행할 때입니다. 이는 정적 파일 이름(예: logo.png)을 받아들이는 첫 번째 사용과 유사하지만, 파일의 내용을 로드하는 대신 전체 경로(bookr/static/logo.png)를 터미널에 출력합니다.

Django는 몇 가지 내장된 찾기를 제공하지만, 정적 파일을 사용자 정의 디렉토리 레이아웃에 저장하려는 경우 직접 작성할 수도 있습니다. Django가 사용하는 찾기 목록은 settings.py의 STATICFILES_FINDERS 설정에 정의되어 있습니다. 이 장에서는 기본 정적 파일 찾기인 AppDirectoriesFinder와 FileSystemFinder의 동작을 각각 AppDirectoriesFinder 하위 섹션과 FileSystemFinder 섹션에서 다룰 것입니다.

Note

If you look in `settings.py`, you won't see the `STATICFILES_FINDERS` setting defined by default. This is because Django will use its built-in default for the setting, which is defined as `["django.contrib.staticfiles.finders.FileSystemFinder", "django.contrib.staticfiles.finders.AppDirectoriesFinder"]`. If you add the `STATICFILES_FINDERS` setting to your `settings.py` file to include a custom finder, be sure to include these defaults if you're using them.

In this section, we will first discuss static file finders and their use in the first case – responding to a request. Then, we will introduce some more concepts and return to the behavior of `collectstatic` and how it uses static file finders.

Static file finders – used during a request

When Django receives a request for a static file (remember, Django will only serve static files during development), each static file finder that has been defined will be queried until a file on the disk has been found. Or, if none of the finders can locate a file, the static view will return an HTTP 404 Not Found response.

For example, the URL of the request will be something such as `/static/main.css` or `/static/reviews/logo.png`. Each finder will be queried in turn with the path from the URL and will return a path such as `bookr/static/main.css` for the first file and `bookr/reviews/static/reviews/logo.png` for the second.

Each finder will use its logic to convert from a URL path into a filesystem path – we will discuss this logic in the *AppDirectoriesFinder* section (which comes next) and the *FileSystemFinder* section (which will come later).

AppDirectoriesFinder

The *AppDirectoriesFinder* class is used to find static files inside each app directory, in a directory called `static`. The application must be listed in the `INSTALLED_APPS` setting in your `settings.py` file (we did this in *Chapter 1, An Introduction to Django*). As we also mentioned in *Chapter 1, An Introduction to Django*, it is good for apps to be self-contained. By letting each application have a static directory, we can continue the self-contained design by also storing app-specific static files inside the app directory.

Before we use *AppDirectoriesFinder*, we will explain a problem that can occur if multiple static files have the same name, and also how to solve this problem.

노트

settings.py를 보면 기본적으로 정의된 STATICFILES_FINDERS 설정이 보이지 않을 것입니다. 이는 Django가 설정에 대해 기본적으로 정의된 ["django.contrib.staticfiles.finders.FileSystemFinder", "django.contrib.staticfiles.finders.AppDirectoriesFinder"]를 사용할 것이기 때문입니다. 사용자 정의 파인더를 포함하기 위해 settings.py 파일에 STATICFILES_FINDERS 설정을 추가하는 경우, 이를 사용하는 경우 이러한 기본값을 포함해야 합니다.

이 섹션에서는 먼저 정적 파일 파인더와 요청에 응답하는 첫 번째 경우에서의 사용에 대해 논의할 것입니다. 그런 다음 몇 가지 개념을 소개하고 collectstatic의 동작과 정적 파일 파인더를 사용하는 방법으로 돌아갈 것입니다.

정적 파일 파인더 – 요청 중에 사용됨

Django가 정적 파일에 대한 요청을 받을 때(idx_3407b73), 정의된 각 정적 파일 파인더가 쿼리되어 디스크에서 파일이 발견될 때까지 진행됩니다. 또는, 어떤 파인더도 파일을 찾을 수 없는 경우, 정적 뷰는 HTTP 404 Not Found 응답을 반환합니다.

예를 들어, 요청의 URL은 /static/main.css 또는 /static/reviews/logo.png와 같은 형태가 될 것입니다. 각 파인더는 URL의 경로로 차례로 쿼리되며, 첫 번째 파일에 대해 bookr/static/main.css와 두 번째 파일에 대해 bookr/reviews/static/reviews/logo.png와 같은 경로를 반환합니다.

각 파인더는 URL 경로를 파일 시스템 경로로 변환하기 위해 자신의 논리를 사용할 것입니다. 이 논리는 다음 섹션인 AppDirectoriesFinder와 이후에 나올 FileSystemFinder 섹션에서 논의할 것입니다.

AppDirectoriesFinder

AppDirectoriesFinder 클래스는 각 앱 디렉토리 내의 정적 파일을 찾는 데 사용됩니다. static이라는 이름의 aidx_44a09122 디렉토리 안에 있습니다. idx_86736b67 애플리케이션은 yoursettings.py 파일의 INSTALLED_APPS 설정에 나열되어야 합니다(우리는 1장, Django 소개에서 이것을 했습니다). 1장, Django 소개에서도 언급했듯이, 앱이 자급자족하는 것이 좋습니다. 각 애플리케이션이 정적 디렉토리를 갖도록 함으로써, 앱 디렉토리 내에 앱 전용 정적 파일을 저장하여 자급자족 설계를 계속할 수 있습니다.

AppDirectoriesFinder를 사용하기 전에, 여러 정적 파일이 동일한 이름을 가질 경우 발생할 수 있는 문제와 이 문제를 해결하는 방법을 설명하겠습니다.

Static file namespacing

In the prior section, *Static file finders – used during a request*, we discussed serving a file named `logo.png`. This would provide a logo for the reviews application. The filename (`logo.png`) could be quite common – you could imagine that if we added a store app (for purchasing books), it would also have a logo. Not to mention that third-party Django apps might also want to use a common name, such as `logo.png`. The problem we are about to describe could apply to any static file that has a common name, such as `styles.css` or `main.js`.

Let's consider the reviews and stores examples. We can add a static directory in each of these apps. Then, each static directory would have a `logo.png` file (although it would be a different logo). The directory structure is shown in *Figure 5.1*:

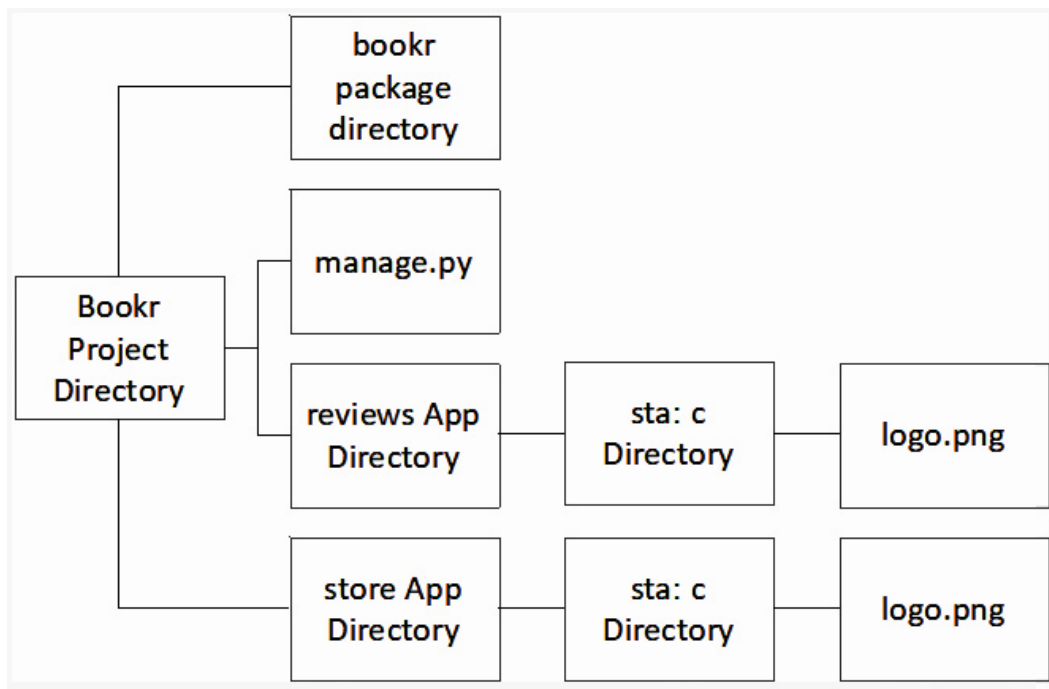


Figure 5.1 – Directory layout with static directories inside app directories

The URL path that we use to download the static file is relative to the static directory. Therefore, it is unclear which `logo.png` is being referenced if we make an HTTP request for `/static/logo.png`. Django will check the static directory for each application in turn (in the order they are specified in the `INSTALLED_APPS` setting). The first `logo.png` that it locates, it will serve. There is no way, in this directory layout, to specify which `logo.png` you want to load.

정적 파일 네임스페이싱

이전 섹션에서는 요청 중에 사용되는 정적 파일 탐색기에서 logo.png라는 파일을 제공하는 것에 대해 논의했습니다. 이는 리뷰 애플리케이션에 대한 로고를 제공할 것입니다. 파일 이름(logo.png)은 꽤 일반적일 수 있습니다. 예를 들어, 우리가 상점 앱(책 구매용)을 추가한다면, 그것도 로고가 있을 것입니다. 타사 Django 앱도 logo.png와 같은 일반 이름을 사용하고 싶어할 수 있습니다. 우리가 설명하려는 문제는 styles.css나 main.js와 같이 일반 이름을 가진 모든 정적 파일에 적용될 수 있습니다.

리뷰와 상점 예제를 고려해 봅시다. 각 앱에 정적 디렉토리를 추가할 수 있습니다. 그런 다음 각 정적 디렉토리에는 logo.png 파일이 있을 것입니다(비록 다른 로고일지라도). 디렉토리 구조는 그림 5.1에 나와 있습니다:

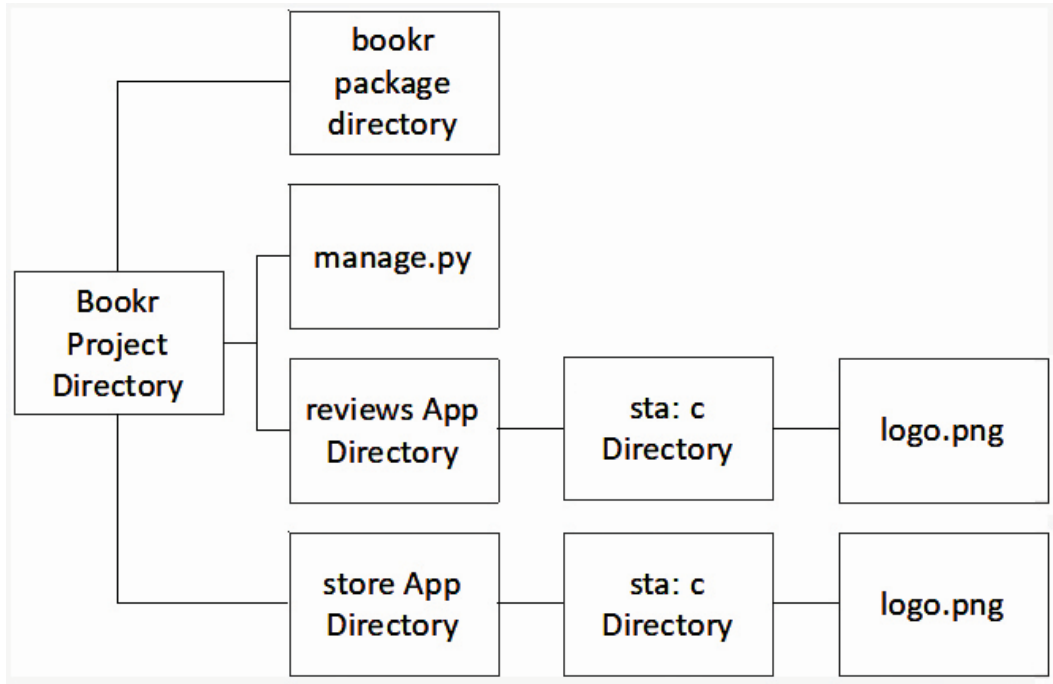


그림 5.1 - 앱 디렉토리 내의 정적 디렉토리가 있는 디렉토리 레이아웃

정적 파일을 다운로드하는 데 사용하는 URL 경로는 정적 디렉토리를 기준으로 합니다. 따라서 /static/logo.png에 대한 HTTP 요청을 할 경우 어떤 logo.png가 참조되고 있는지 불분명합니다. Django는 INSTALLED_APPS 설정에 지정된 순서대로 각 애플리케이션의 정적 디렉토리를 차례로 확인합니다. 찾은 첫 번째 logo.png를 제공할 것입니다. 이 디렉토리 레이아웃에서는 어떤 logo.png를 로드할지 지정할 방법이 없습니다.

We can solve this problem by **namespacing** our static files. This is the process of using another directory inside the static directory, named the same as the app. The reviews app has a reviews directory inside its static directory, and the store app has a store directory inside its static directory. The respective `logo.png` files are then moved inside these sub-directories.

The new directory layout is shown in *Figure 5.2*:

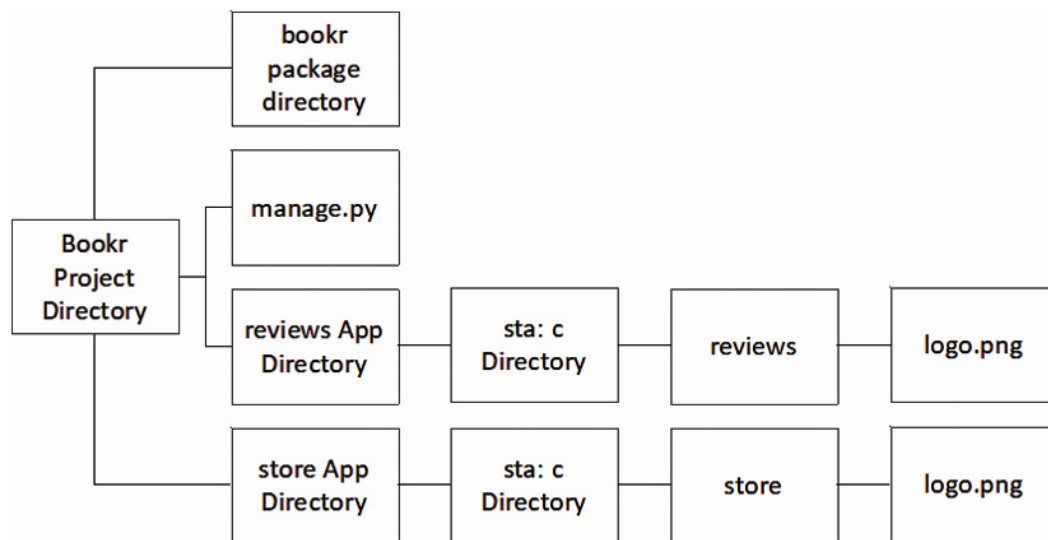


Figure 5.2 – Directory layout with namespaced directories

To load a specific file, we must include the namespaced directory too. For the reviews logo, the URL path is `/static/reviews/logo.png`, which maps to `bookr/reviews/static/reviews/logo.png` on the disk. Similarly, for the store logo, the path is `/static/store/logo.png`, which maps to `bookr/store/static/store/logo.png`. You might have noticed that the example paths for the `logo.png` file were already namespaced in the *Static file finders – used during a request* section.

Now that we have introduced `AppDirectoriesFinder` and static file namespacing, we can use them to serve our first static file. In the first exercise of this chapter, we will create a new Django project for a basic business site. We will then serve a logo file from an app called `landing` that we will create in this project. The `AppDirectoriesFinder` class is used to find static files inside each app directory, in a directory called `static`. The application must be listed in the `INSTALLED_APPS` setting in your `settings.py` file.

정적 파일의 네임스페이스를 사용하여 idx_6d6ab659 이 문제를 해결할 수 있습니다. 이는 정적 디렉토리 내에 앱과 동일한 이름을 가진 다른 디렉토리를 사용하는 과정입니다. 리뷰 앱은 정적 디렉토리 내에 reviews 디렉토리를 가지고 있으며, 스토어 앱은 정적 디렉토리 내에 store 디렉토리를 가지고 있습니다. 각각의 logo.png 파일은 이러한 하위 디렉토리로 이동됩니다. 새로운 디렉토리 레이아웃은 그림 5.2에 나와 있습니다:

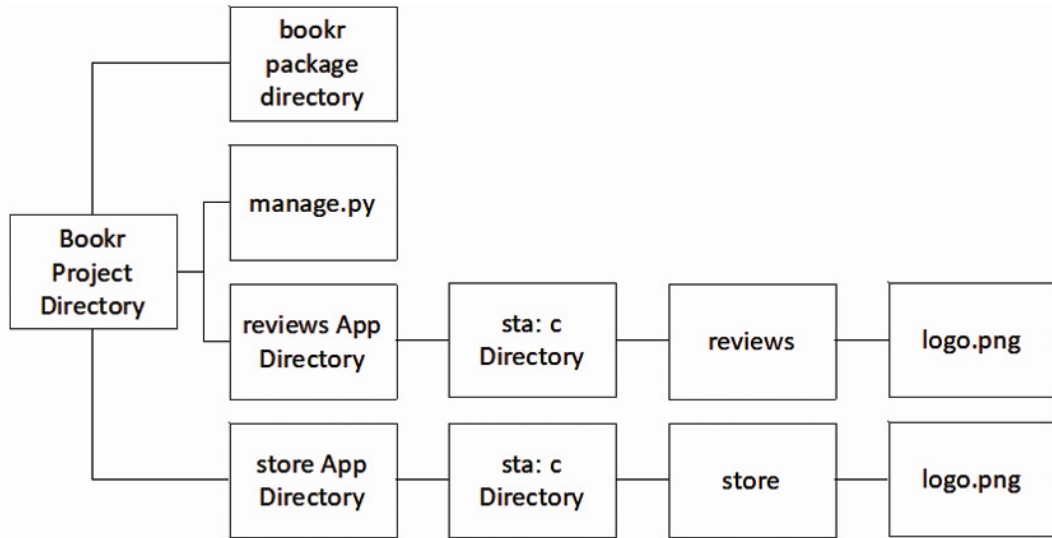


그림 5.2 - 네임스페이스가 있는 디렉토리 레이아웃

특정 파일을 로드하려면 네임스페이스가 있는 디렉토리도 포함해야 합니다. 리뷰 로고의 경우 URL 경로는 /static/reviews/logo.png이며, 이는 디스크의 bookr/reviews/static/reviews/logo.png에 매핑됩니다. 마찬가지로, 스토어 로고의 경우 경로는 /static/store/logo.png이며, 이는 bookr/store/static/store/logo.png에 매핑됩니다. logo.png 파일의 예제 경로가 이미 정적 파일 탐색기에서 네임스페이스가 지정되었음을 알 수 있습니다 - 요청 섹션에서 사용됩니다.

이제 AppDirectoriesFinder와 정적 파일 네임스페이싱을 도입했으므로, 이를 사용하여 첫 번째 정적 파일을 제공할 수 있습니다. 이 장의 첫 번째 연습에서는 기본 비즈니스 사이트를 위한 새로운 Django 프로젝트를 생성할 것입니다. 그런 다음 이 프로젝트에서 생성할 landing이라는 앱에서 로고 파일을 제공할 것입니다. AppDirectoriesFinder 클래스는 각 앱 디렉토리 내의 정적 파일을 찾기 위해 사용되며, static이라는 디렉토리 내에 있습니다. 애플리케이션은 settings.py 파일의 INSTALLED_APPS 설정에 나열되어야 합니다.

As we mentioned in *Chapter 1, An Introduction to Django*, it is good for apps to be self-contained. By letting each application have a `static` directory, we can continue the self-contained design by also storing app-specific static files inside the app directory.

The easiest way to serve a static file is from an app directory. This is because we don't need to make any settings changes. Instead, we just need to create the files in the correct directory, and they will be served using the default Django configuration.

Note

The business site project

For the exercises in this chapter, we'll create a new Django project and use it to demonstrate the static file concepts. The project will be a basic business site with a simple landing page that has a logo. The project will have one app, called *landing*.

You can refer to *Exercise 1.01 – Creating a project and app, and starting the dev server* from *Chapter 1, An Introduction to Django*, to refresh your memory on creating a Django project.

For detailed instructions on how to set up and configure PyCharm to work with your Django project, refer to *Exercise 1.02 – project setup in PyCharm*, in *Chapter 1, An Introduction to Django*.

Exercise 5.01 – serving a file from an app directory

In this exercise, you will add a logo file for the *landing* app. This will be done by putting a `logo.png` file in a `static` directory inside the *landing* app directory. Once you've done this, you can test that the static file is being served correctly and confirm the URL that will serve it:

1. Start by creating the new Django project. You can reuse the Bookr virtual environment that already has Django installed.
2. Open a new terminal and activate the virtual environment (refer to the *Preface* for instructions on how to create and activate a virtual environment).
3. Run the `django-admin` command in the terminal (or Command Prompt) to start a Django project named `business_site`.

```
django-admin startproject business_site
```

There won't be any output. This command will scaffold the Django project in a new directory named `business_site`.

1장에서 언급했듯이, Django 소개, 앱이 독립적으로 구성되는 것이 좋습니다. 각 애플리케이션에 정적 디렉토리를 두어 앱 전용 정적 파일을 앱 디렉토리 내에 저장함으로써 독립적인 디자인을 계속할 수 있습니다.

정적 파일을 제공하는 가장 쉬운 방법은 앱 디렉토리에서 제공하는 것입니다. 이는 설정 변경이 필요 없기 때문입니다. 대신, 올바른 디렉토리에 파일을 생성하기만 하면 기본 Django 구성으로 제공됩니다.

참고

비즈니스 사이트 프로젝트

이번 장의 연습을 위해 새로운 Django 프로젝트를 만들고 이를 사용하여 정적 파일 개념을 설명할 것입니다. 이 프로젝트는 로고가 있는 간단한 랜딩 페이지를 가진 기본 비즈니스 사이트가 될 것입니다. 이 프로젝트에는 landing이라는 하나의 앱이 있을 것입니다.

Django 프로젝트를 만드는 방법에 대한 기억을 되살리기 위해 1장, Django 소개의 연습 1.01 - 프로젝트 및 앱 만들기, 개발 서버 시작을 참조할 수 있습니다.

Django 프로젝트와 함께 작업하기 위해 PyCharm을 설정하고 구성하는 방법에 대한 자세한 지침은 1장, Django 소개의 연습 1.02 - PyCharm에서 프로젝트 설정을 참조하십시오.

연습 5.01 - 앱 디렉토리에서 파일 제공하기

이 연습에서는 랜딩 앱에 로고 파일을 추가합니다. 이는 랜딩 앱 디렉토리 내의 정적 디렉토리에 alogo.png 파일을 넣음으로써 이루어집니다. 이 작업을 완료하면 정적 파일이 올바르게 제공되고 있는지 테스트하고 이를 제공할 idx_2972f73aURL을 확인할 수 있습니다:

1. 새로운 Django 프로젝트를 생성하는 것으로 시작합니다. 이미 Django가 설치된 Bookr 가상 환경을 재사용할 수 있습니다.
2. 새로운 터미널을 열고 가상 환경을 활성화합니다(가상 환경을 생성하고 활성화하는 방법에 대한 지침은 서문을 참조하십시오).
3. 터미널(또는 명령 프롬프트)에서 django-admin 명령을 실행하여 business_site라는 이름의 Django 프로젝트를 시작합니다.

```
django-admin startproject business_site
```

출력은 없습니다. 이 명령은 business_site라는 새 디렉토리에 Django 프로젝트를 스캐폴딩합니다.

4. Create a new Django app in this project by using the `startapp` management command. The app should be called *landing*. To do this, `cd` into the `business_site` directory, then run the following:

```
python manage.py startapp landing
```

Note that there won't be any output again. The command will create the `landing` app directory inside the `business_site` directory.

5. Launch PyCharm and open the `business_site` directory:
 - If you already have a project open, you can do this by clicking **File | Open**.
 - Otherwise, just click **Open** in the **Welcome to PyCharm** window. Navigate to the `business_site` directory, select it, then click **Open**.
 - If prompted, choose **Trust Project**.

The `business_site` project window should look similar to what's shown in *Figure 5.3*. Click the dropdown that currently says **Current File** and choose **Edit Configurations...**

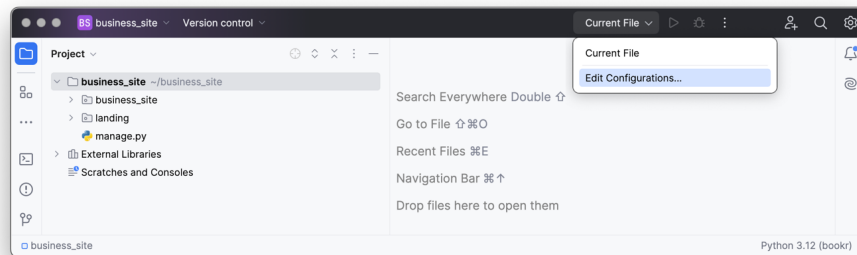


Figure 5.3 – The `business_site` project

6. Then follow the **Add new run Configuration...** link.

4. startapp 관리 명령어를 사용하여 이 프로젝트에 새로운 Django 앱을 만듭니다. 앱의 이름은 landing이어야 합니다. 이를 위해 business_site 디렉토리로 이동한 다음 다음을 실행합니다:

```
python manage.py startapp landing
```

출력이 다시 없을 것입니다. 이 명령은 business_site 디렉토리 내에 landing 앱 디렉토리를 생성합니다.

5. PyCharm idx_cf11b263을 실행하고 business_site 디렉토리를 엽니다:

- 이미 프로젝트가 열려 있다면, 파일 | 열기를 클릭하여 이 작업을 수행할 수 있습니다.
- 그렇지 않으면, PyCharm 환영 창에서 열기를 클릭하세요. business_site 디렉토리로 이동하여 선택한 다음 열기를 클릭합니다.
- 메시지가 표시되면 프로젝트를 신뢰하도록 선택하세요.

business_site 프로젝트 창은 그림 5.3에 표시된 것과 유사해야 합니다. 현재 'Current File'이라고 표시된 드롭다운을 클릭하고 'EditConfigurations...'를 선택하세요.

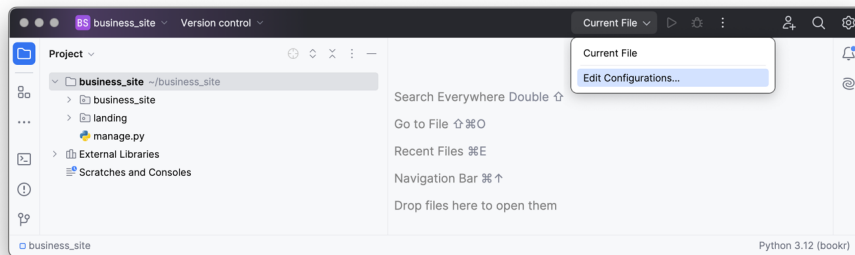


그림 5.3 - business_site 프로젝트

6. 그런 다음 'Add new run Configuration...' 링크를 따르세요.

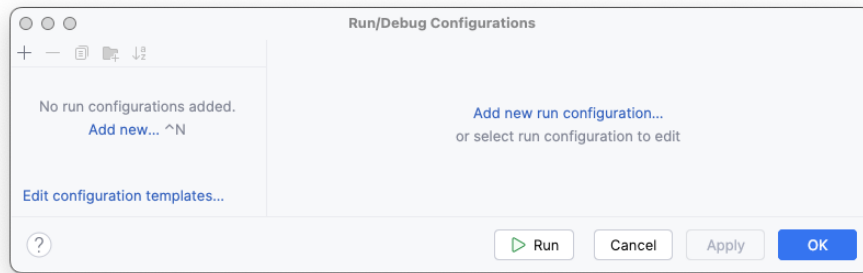


Figure 5.4 – Run/Debug Configurations

7. Create a new run configuration to execute `manage.py runserver` for the project. You can reuse the Bookr virtual environment again. The **Run/Debug Configurations** window should look similar to what's shown in Figure 5.5 when you're done:

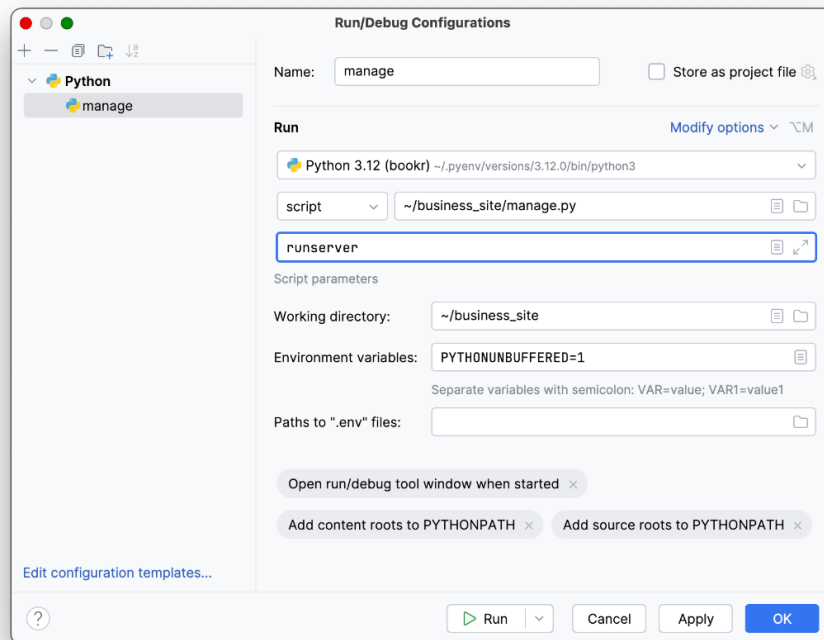


Figure 5.5 – Run/Debug Configurations for runserver

You can test that the configuration has been set up correctly by clicking the **Run** button and then visiting `http://127.0.0.1:8000/` in your browser. You should see the Django

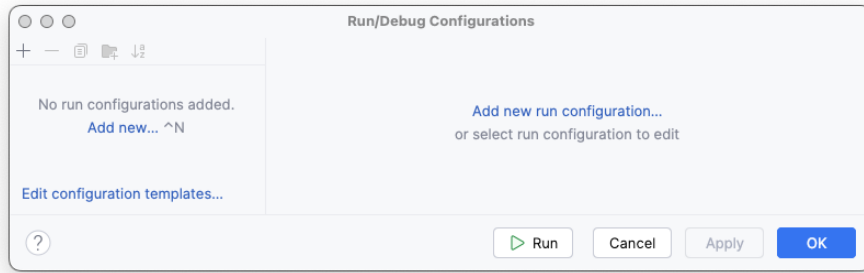


그림 5.4 - 실행/디버그 구성

7. 프로젝트를 위해 `manage.py runserver`를 실행할 새로운 실행 `idx_8ae4800d` 구성을 만드세요. Bookr 가상 환경을 다시 사용할 수 있습니다. 완료되면 실행/디버그 구성 창은 그림 5.5에 표시된 것과 유사해야 합니다:

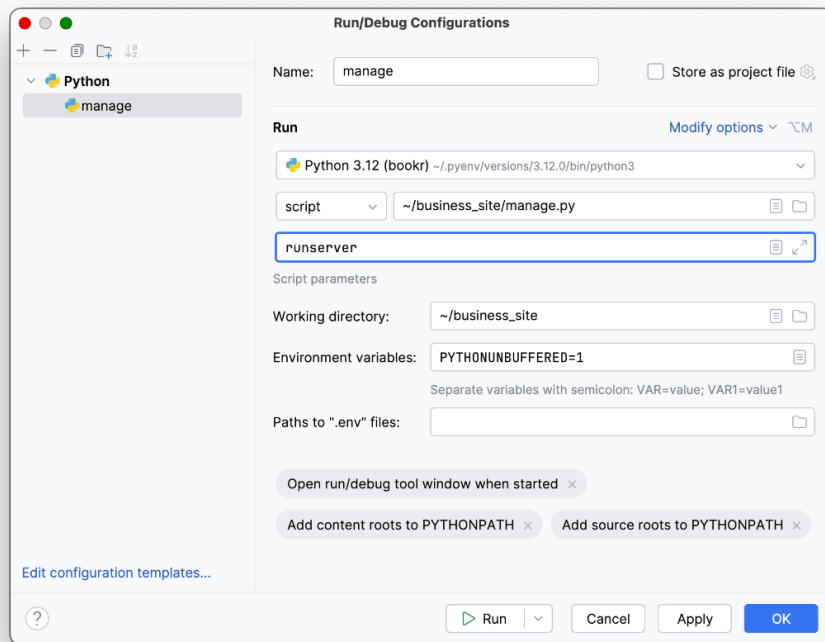


그림 5.5 - runserver에 대한 실행/디버그 구성

`idx_f6b52595` 구성이 올바르게 설정되었는지 확인하려면 실행 버튼을 클릭한 다음 브라우저에서 `http://127.0.0.1:8000/`를 방문하세요. Django가 표시되어야 합니다.

welcome screen. If the debug server fails to start or you see the Bookr main page, then you probably still have the Bookr project running. Try stopping the Bookr runserver process (press *Ctrl* + *C* in the terminal that's running it) and then starting the new one you just set up.

Now that we have completed the project configuration in PyCharm, we will begin the coding steps for the exercise.

1. Open `settings.py` in the `business_site` directory and add 'landing' to the `INSTALLED_APPS` setting. We learned how to do this in *Step 1 of Exercise 1.05 – creating a templates directory and base template*, in *Chapter 1, An Introduction to Django*.
2. In PyCharm, right-click on the `landing` directory in the project panel and select **New | Directory**.
3. Enter `static` and press the *Enter* key:

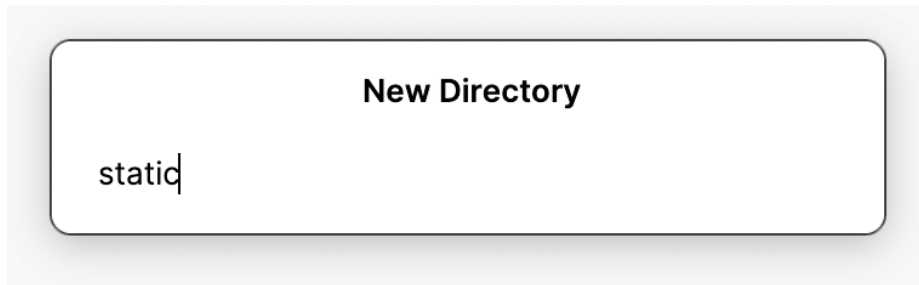


Figure 5.6 – Naming the directory `static`

4. Right-click on the `static` directory you just created and select **New | Directory** again.
5. Enter `landing` and press *Enter*. This will implement namespacing of the static files directory, as we discussed earlier:

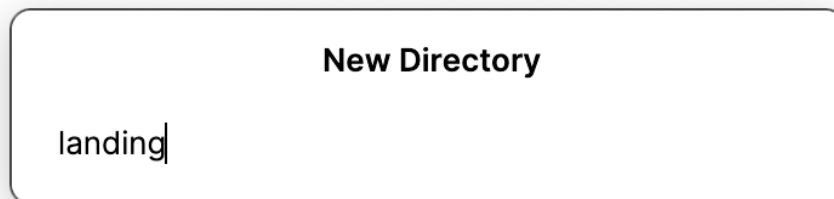


Figure 5.7 – Naming the new directory `reviews` to implement namespacing

환영 화면. 디버그 서버가 시작되지 않거나 Bookr 메인 페이지가 보이면, 아마도 여전히 Bookr 프로젝트가 실행 중일 것입니다. Bookr runserver 프로세스를 중지해 보세요(실행 중인 터미널에서 Ctrl + C를 누르세요) 그리고 방금 설정한 새 서버를 시작하세요.

이제 PyCharm에서 idx_04560440 프로젝트 구성을 완료했으므로, 연습을 위한 코딩 단계를 시작하겠습니다.

1. business_site 디렉토리에서 settings.py를 열고 INSTALLED_APPS 설정에 'landing'을 추가하세요. 우리는 1장, Django 소개에서 템플릿 디렉토리 및 기본 템플릿을 만드는 연습 1.05의 1단계에서 이를 수행하는 방법을 배웠습니다.
2. PyCharm에서 프로젝트 패널의 landing 디렉토리를 마우스 오른쪽 버튼으로 클릭하고 새로 만들기 | 디렉토리를 선택하세요.
3. static을 입력하고 Enter 키를 누르세요:

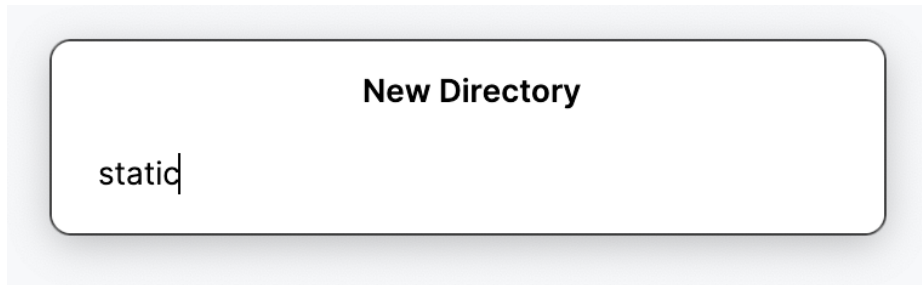


그림 5.6 - 디렉토리 이름 지정 정적

4. 방금 생성한 정적 디렉토리를 마우스 오른쪽 버튼으로 클릭하고 다시 새로 만들기 | 디렉토리를 선택합니다. landing을 입력하고 Enter를 누릅니다. 이렇게
5. 하면 앞서 논의한 대로 정적 파일 디렉토리의 네임스페이스가 구현됩니다:

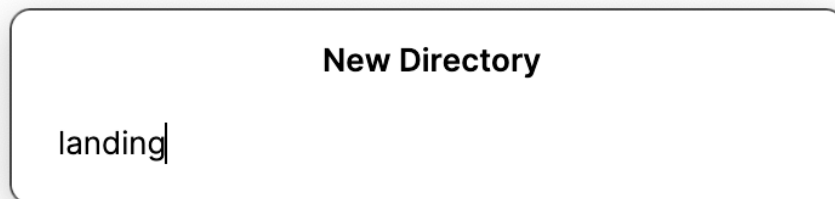


그림 5.7 - 네임스페이스를 구현하기 위해 새 디렉토리 reviews의 이름 지정

6. Download the `logo.png` file from the `Chapter05` folder in this book's GitHub repo and move it into the `landing/static/landing` directory.
7. Start the Django dev server. If it is not already running, then navigate to `http://127.0.0.1:8000/static/landing/logo.png`. You should see the image being served in your browser, illustrating that you have set up static file serving correctly:

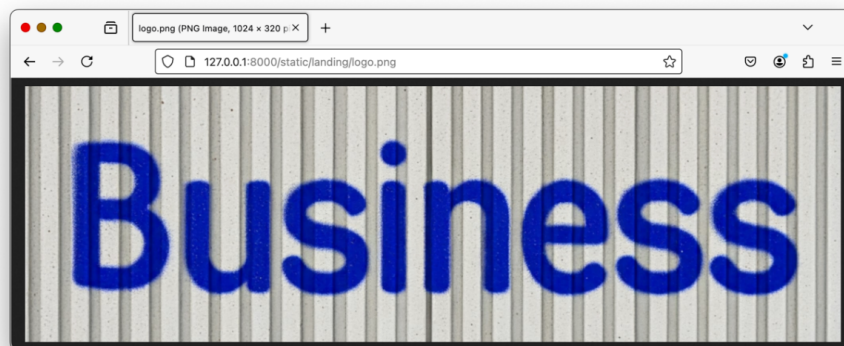


Figure 5.8 – Image served by Django

Now that you've seen that loading static images works correctly, we'll learn more about how to prevent hardcoding these URLs. In the next section, you'll see how Django can automatically generate correct static URLs for use in templates.

Generating static URLs with the static template tag

In *Exercise 5.01 – serving a file from an app directory*, you set up an image file to be served by Django. We saw that the URL of the image was `http://127.0.0.1:8000/static/landing/logo.png`, which you could use inside an HTML template. For example, to display the image with an `img` tag, you could use this code in your template:

```

```

Or, since Django is also serving the media and has the same host as the dynamic template response, you can simplify this by just including the path, as follows:

```

```

Both addresses (URLs and paths) have been hardcoded into the template – that is, we include the full path to the static file and make assumptions about where the file is being hosted. This

6. 이 책의 GitHub 저장소의 Chapter05 폴더에서 logo.png 파일을 다운로드하고 landing/static/landing 디렉토리로 이동하세요.
7. idx_a331da30Django 개발 서버를 시작하세요. 이미 실행 중이지 않다면 <http://127.0.0.1:8000/static/landing/logo.png>로 이동하세요. 브라우저에서 이미지가 제공되는 것을 볼 수 있어야 하며, 이는 정적 파일 제공이 올바르게 설정되었음을 보여줍니다:



그림 5.8 - Django에 의해 제공된 이미지

이제 idx_d073778정적 이미지를 로드하는 것이 올바르게 작동하는 것을 보았으므로, 이러한 URL을 하드코딩하지 않는 방법에 대해 더 배울 것입니다. 다음 섹션에서는 Django가 템플릿에서 사용할 수 있는 올바른 정적 URL을 자동으로 생성하는 방법을 보여줍니다.

정적 템플릿 태그로 정적 URL 생성하기

연습 5.01 – 앱 디렉토리에서 파일 제공하기에서, Django에 의해 제공될 이미지 파일을 설정했습니다. 이미지의 URL은 <http://127.0.0.1:8000/static/landing/logo.png>였으며, 이를 idx_4194308HTML 템플릿 내에서 사용할 수 있었습니다. 예를 들어, img 태그로 이미지를 표시하려면 템플릿에서 다음 코드를 사용할 수 있습니다:

```

```

또는 Django가 media를 제공하고 동적 템플릿 응답과 동일한 호스트를 가지고 있으므로, 다음과 같이 경로만 포함하여 간단하게 만들 수 있습니다:

```

```

두 주소(URL 및 경로)는 템플릿에 하드코딩되어 있습니다. 즉, 정적 파일의 전체 경로를 포함하고 파일이 호스팅되는 위치에 대한 가정을 합니다. 이것

works fine with the Django development server or if you host your static files and Django website on the same domain. For better performance as your site becomes more popular, you might consider serving static files from their own domain or **Content Delivery Network (CDN)**.

Note

A CDN is a service that can host parts or all of your website for you. It provides several web servers and can seamlessly speed up the loading of your website. For example, it might serve files to a user from the server that is geographically closest to them. There are several CDN providers, and depending on how they are set up, they might require you to specify a certain domain from which to serve your static files.

Take, for instance, a common separation approach: using a different domain for static file serving. You host your main website at `https://www.example.com` but want to serve static files from `https://static.example.com`. During development, we could use just the path to the logo file, as in the example we just saw. But when we deploy to the production server, our URLs would need to change so that they include the domain, like so:

```

```

Since all the links are hardcoded, this would need to be done for every URL throughout our templates, every time we deploy to production. Once they have been changed, though, the URL will no longer work in the Django dev server. Luckily, Django provides a solution to this problem.

The static template tag

The `staticfiles` app provides a template tag, `static`, for dynamically generating the URL to a static file inside a template. Since the URLs are all being dynamically generated, we can change the URL for all of them by changing just one setting (`STATIC_URL` in `settings.py` – more on this soon). Furthermore, later, we will introduce a method for invalidating browser caches for static files that relies on the use of the `static` template tag.

The `static` tag is very simple – it takes a single argument, which is the project-relative path to a static asset. It will then output this path, prepended by the `STATIC_URL` setting. But first, it must be loaded into the template with the `{% load static %}` template tag.

Django has a set of default template tags and filters (or tag sets) that it automatically makes available to every template. Django (and third-party libraries) also provide tag sets that are not automatically loaded. In these cases, we need to load these extra template tags and filters into

Django 개발 서버와 잘 작동하거나 정적 파일과 Django 웹사이트를 동일한 도메인에서 호스팅하는 경우 잘 작동합니다. 사이트가 더 인기를 끌면서 더 나은 성능을 위해 정적 파일을 자체 도메인이나 콘텐츠 전송 네트워크(CDN)에서 제공하는 것을 고려할 수 있습니다.

참고

CDN은 귀하의 웹사이트의 일부 또는 전체를 호스팅할 수 있는 서비스입니다. 여러 웹 서버를 제공하며 귀하의 웹사이트 로딩 속도를 원활하게 높일 수 있습니다. 예를 들어, 사용자의 지리적으로 가장 가까운 서버에서 파일을 제공할 수 있습니다. 여러 CDN 제공업체가 있으며, 설정 방법에 따라 정적 파일을 제공할 특정 도메인을 지정해야 할 수도 있습니다.

예를 들어, 정적 파일 제공을 위한 일반적인 분리 접근 방식을 살펴보겠습니다. 귀하는 `https://www.example.com`에서 주요 웹사이트를 호스팅하지만 `https://static.example.com`에서 정적 파일을 제공하고 싶습니다. 개발 중에는 방금 본 예와 같이 로고 파일의 경로만 사용할 수 있습니다. 그러나 프로덕션 서버에 배포할 때는 URL이 도메인을 포함하도록 변경해야 합니다. 다음과 같이요:

```

```

모든 `idx_55eca55d` 링크가 하드코딩되어 있으므로 프로덕션에 배포할 때마다 모든 템플릿의 모든 URL에 대해 이 작업을 수행해야 합니다. 그러나 변경된 후에는 Django 개발 서버에서 URL이 더 이상 작동하지 않습니다. 다행히도 Django는 이 문제에 대한 해결책을 제공합니다.

정적 템플릿 태그

`staticfiles` 앱 `idx_7b962744`은 템플릿 내에서 정적 파일에 대한 URL을 동적으로 생성하기 위한 템플릿 태그인 `static`을 제공합니다. URL이 모두 동적으로 생성되므로 하나의 설정(`STATIC_URL` in `settings.py` – 이에 대한 자세한 내용은 곧 다룰 예정)를 변경하여 모든 URL을 변경할 수 있습니다. 또한 나중에 정적 파일에 대한 브라우저 캐시를 무효화하는 방법을 소개할 예정이며, 이는 정적 템플릿 태그의 사용에 의존합니다.

정적 태그는 매우 간단합니다. 단일 인수를 사용하며, 이는 정적 자산에 대한 프로젝트 상대 경로입니다. 그런 다음 이 경로는 `STATIC_URL` 설정으로 접두사가 붙여져 출력됩니다. 그러나 먼저 `{% load static %}` 템플릿 태그를 사용하여 템플릿에 로드해야 합니다.

장고는 모든 템플릿에 자동으로 제공되는 기본 템플릿 태그 및 필터(또는 태그 세트) 집합을 가지고 있습니다. 장고(및 서드파티 라이브러리) 또한 자동으로 로드되지 않는 태그 세트를 제공합니다. 이러한 경우, 우리는 이러한 추가 템플릿 태그와 필터를 로드해야 합니다.

a template before we can use them. This can be done with the `load template` tag, which should come near the start of a template (although it must be after the `extends template` tag if one is used). The `load template` tag takes one or more packages/libraries to load. Here's an example:

```
{% load package_one package_two package_three %}
```

This would load the template tag and filters set provided by the (made up) `package_one`, `package_two`, and `package_three` packages.

The `load template` tag must be used in the actual template that requires the loaded package. For example, if your template extends another template, and that base template has loaded a certain package, your dependent template does not automatically have access to that package. Your template must still load the package to access the new tag set. The `static template` tag is not part of the default set, which is why we need to load it.

Then, it can be used to interpolate anywhere inside the template file. For example, by default, Django uses `/static/` as `STATIC_URL`. If we wanted to generate the static URL for our `logo.png` file, we would use the tag in a template like this:

```
{% static 'landing/logo.png' %}
```

The output inside the template would be this:

```
/static/landing/logo.png
```

An example of the static template tag

This will become clearer with an example, so let's look at how the `static` tag could be used to generate a URL for several different assets.

We can include the logo as an image on the page with an `img` tag, as follows:

```

```

This is rendered in the template like so:

```

```

Alternatively, we could use the `static` tag to generate the URL for a linked CSS file, as follows:

```
<link href="{% static 'path/to/file.css' %}" rel="stylesheet">
```

템플릿을 사용하기 전에 템플릿을 로드해야 합니다. 이는 템플릿의 시작 부분 근처에 있어야 하는 `load` 템플릿 태그를 사용하여 수행할 수 있습니다(단, 사용되는 경우 `extends` 템플릿 태그 이후에 와야 합니다). `load` 템플릿 태그는 하나 이상의 패키지/라이브러리를 로드합니다. 다음은 예입니다:

```
{% load package_one package_two package_three %}
```

이것은 (가상의) `package_one`, `package_two` 및 `package_three` 패키지에서 제공하는 템플릿 태그 및 필터 세트를 로드합니다.

`load` 템플릿 태그는 로드된 패키지가 필요한 실제 템플릿에서 사용해야 합니다. 예를 들어, 템플릿이 다른 템플릿을 확장하고 그 기본 템플릿이 특정 패키지를 로드한 경우, 종속 템플릿은 자동으로 해당 패키지에 접근할 수 없습니다. 템플릿은 여전히 패키지를 로드하여 새로운 태그 세트에 접근해야 합니다. `static` 템플릿 태그는 기본 세트의 일부가 아니므로 로드해야 합니다.

그런 다음 템플릿 파일 내의 어디에서나 `idx_675fa933`를 보간하는 데 사용할 수 있습니다. 예를 들어, 기본적으로 Django는 `/static/`을 `STATIC_URL`로 사용합니다. `logo.png` 파일의 정적 URL을 생성하려면 템플릿에서 다음과 같이 태그를 사용합니다:

```
{% static 'landing/logo.png' %}
```

템플릿 내의 출력 `idx_7ea18951`은 다음과 같습니다:

```
/static/landing/logo.png
```

정적 템플릿 태그의 예

예를 통해 더 명확해질 것이므로, 정적 태그를 사용하여 여러 다른 자산의 URL을 생성하는 방법을 살펴보겠습니다.

페이지에 `img` 태그로 로고를 이미지로 포함할 수 있습니다. 다음과 같이:

```

```

템플릿에서 다음과 같이 렌더링됩니다:

```

```

또는 정적 태그를 사용하여 링크된 CSS 파일의 URL을 생성할 수 있습니다. 다음과 같이:

```
<link href="{% static 'path/to/file.css' %}" rel="stylesheet">
```

This will be rendered like so:

```
<link href="/static/path/to/file.css" rel="stylesheet">
```

It can be used in a script tag to include a JavaScript file, using the following line of code:

```
<script src="{% static 'path/to/file.js' %}"></script>
```

This is rendered as follows:

```
<script src="/static/path/to/file.js"></script>
```

You can even use it to generate a link to a static file for download, as we've done here:

```
<a href="{% static 'path/to/document.pdf' %}">Download PDF</a>
```

Note

Note that this won't generate the actual PDF content – it will just create a link to an already existing file. We will cover the generation of PDF and image files in *Chapter 13*.

This is rendered as follows:

```
<a href="/static/path/to/document.pdf">Download PDF</a>
```

While referring to these examples, we can now demonstrate the advantage of using the static tag instead of hardcoding. When we are ready to deploy to production, we can just change the `STATIC_URL` value in `settings.py`. None of the values in the templates need to be changed.

For example, we can change `STATIC_URL` to `https://static.example.com/`, and then when the page gets rendered next, the examples we've seen will automatically update. The following line shows this for the image:

```

```

The following is for the CSS link:

```
<link href="https://static.example.com/path/to/files.css" rel="stylesheet">
```

이것은 다음과 같이 렌더링됩니다:

```
<link href="/static/path/to/file.css" rel="stylesheet">
```

다음 코드 줄을 사용하여 JavaScript 파일을 포함하는 스크립트 태그에서 사용할 수 있습니다:

```
<script src="{% static 'path/to/file.js' %}"></script>
```

이것은 다음과 같이 렌더링됩니다:

```
<script src="/static/path/to/file.js"></script>
```

여기서 한 것처럼 다운로드를 위한 정적 파일 링크를 생성하는 데에도 사용할 수 있습니다:

```
<a href="{% static 'path/to/document.pdf' %}">PDF 다운로드</a>
```

메모

실제 PDF 콘텐츠가 생성되지 않음을 유의하십시오. – 이미 존재하는 파일에 대한 링크만 생성됩니다. PDF 및 이미지 파일 생성은 13장에서 다룰 것입니다.

다음과 같이 렌더링됩니다:

```
<a href="/static/path/to/document.pdf">PDF 다운로드</a>
```

이 예제를 참조하면서, 이제 하드코딩 대신 static 태그를 사용하는 장점을 보여줄 수 있습니다. 프로덕션에 배포할 준비가 되면 settings.py에서 STATIC_URL 값을 변경하기만 하면 됩니다. 템플릿의 값은 변경할 필요가 없습니다.

예를 들어, STATIC_URL을 https://static.example.com/으로 변경하면 페이지가 다음에 렌더링될 때 우리가 본 예제가 자동으로 업데이트됩니다. 다음 줄은 이미지를 보여줍니다:

```

```

다음은 CSS 링크에 대한 것입니다:

```
<link href="https://static.example.com/path/to/files.css" rel="stylesheet">
```

For the script, it's as follows:

```
<script src="https://static.example.com/path/to/file.js"></script>
```

And finally, the following is for the link:

```
<a href="https://static.example.com/path/to/document.pdf">Download PDF</a>
```

Note that in all these examples, a literal string is being passed as an argument (it is quoted). You can also use a variable as an argument – for example, if you were rendering a template with a context, such as in this example code:

```
def view_function(request):
    context = {"image_file": "logfile.png"}
    return render(request, "example.html", context)
```

We are rendering the `example.html` template with a variable called `image_file`. This variable has a value of `logo.png`.

You would pass this variable to the `static` tag without quotes:

```

```

It would render like this (assuming we changed `STATIC_URL` back to `/static/`):

```

```

The template tag can also be used with the `as [variable]` suffix to assign the result to a variable for use later in the template. This can be useful if the static file lookup takes a long time and you want to refer to the same static file multiple times (such as by including an image in multiple places).

The first time you refer to the static URL, give it a variable name to assign to. In this case, we are creating the `logo_path` variable:

```

```

This renders the same as the examples we've seen before:

```

```

스크립트는 다음과 같습니다:

```
<script src="https://static.example.com/path/to/file.js"></script>
```

마지막으로, 링크는 다음과 같습니다:

```
<a href="https://static.example.com/path/to/document.pdf">PDF 다운로드</a>
```

이 모든 idx_e799c203예제에서 리터럴 문자열이 인수로 전달되고 있습니다(따옴표로 묶여 있습니다). 변수도 인수로 사용할 수 있습니다 – 예를 들어, 이 예제 코드와 같이 컨텍스트로 템플릿을 렌더링할 때:

```
def view_function(request):
    context = {"image_file": "logfile.png"}
    return render(request, "example.html", context)
```

우리는 image_file이라는 변수를 사용하여 example.html 템플릿을 렌더링하고 있습니다. 이 변수의 값은 logo.png입니다.

이 변수를 따옴표 없이 static 태그에 전달합니다:

```

```

이것은 다음과 같이 렌더링됩니다 (STATIC_URL을 /static/으로 변경했다고 가정할 때):

```

```

템플릿 태그는 as [변수] 접미사와 함께 사용하여 결과를 나중에 템플릿에서 사용할 변수에 할당할 수 있습니다. 정적 파일 조회에 시간이 오래 걸리고 동일한 정적 파일을 여러 번 참조하고 싶을 때 유용할 수 있습니다 (예: 여러 곳에 이미지를 포함할 때).

정적 URL을 처음 참조할 때, 할당할 변수 이름을 지정합니다. 이 경우, 우리는 logo_path 변수를 생성하고 있습니다:

```

```

이것은 우리가 이전에 본 예제와 동일하게 렌더링됩니다:

```

```

However, we can then use the assigned variable (`logo_path`) again later in the template:

```

```

This renders the same again:

```

```

This variable is now just a normal context variable in the template scope and can be used anywhere in the template. Be careful, though, as you might override a variable that has already been defined – although this is a general warning when using any of the template tags that assign variables (for example, `{% with %}`).

In the next exercise, we will put the static template into practice by adding a template to the `business_site` project, then including the example image.

Exercise 5.02 – using the static template tag

In *Exercise 5.01 – serving a file from an app directory*, you tested serving `logo.png` from the `static` directory. In this exercise, you will continue with the `business_site` project and create an `index.html` file as the template for our landing page. Then, you'll include the logo in this page using the `{% static %}` template tag:

1. In PyCharm (make sure you're in the `business_site` project), right-click on the `landing` directory and create a new folder called `templates`.
2. Right-click on the new `templates` directory and select **New | HTML File**. Select **HTML 5 file** and name it `index.html`:

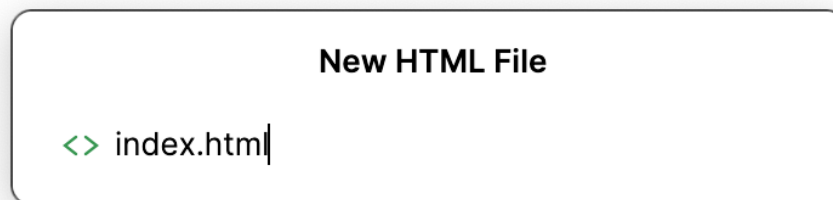


Figure 5.9 – New `index.html`

그러나 나중에 템플릿에서 할당된 변수(`logo_path`)를 다시 사용할 수 있습니다:

```

```

이것은 다시 동일하게 렌더링됩니다:

```

```

이 변수는 이제 템플릿 범위 내의 일반적인 컨텍스트 변수이며 템플릿 어디에서나 사용할 수 있습니다. 그러나 `idx_5035e16d`로 이미 정의된 변수를 덮어쓸 수 있으므로 주의해야 합니다. 이는 변수를 할당하는 템플릿 태그(예: `{% with %}`)를 사용할 때의 일반적인 경고입니다.

다음 연습에서는 비즈니스 사이트 프로젝트에 템플릿을 추가하여 정적 템플릿을 실습할 것입니다. 그런 다음 예제 이미지를 포함합니다.

연습 5.02 – 정적 템플릿 태그 사용

연습 5.01 – 앱 디렉토리에서 파일 제공에서, 정적 디렉토리에서 `logo.png`를 제공하는 것을 테스트했습니다. 이번 연습에서는 비즈니스 `idx_5a5061cb` 사이트 프로젝트를 계속 진행하고 랜딩 페이지의 템플릿으로 `index.html` 파일을 생성합니다. 그런 다음 `{% static %}` 템플릿 태그를 사용하여 이 페이지에 로고를 포함합니다:

1. PyCharm에서 (`business_site` 프로젝트에 있는지 확인하세요), `landing` 디렉토리를 오른쪽 클릭하고 `templates`라는 새 폴더를 만드세요.
2. 새 `templates` 디렉토리를 오른쪽 클릭하고 `New | HTML File`을 선택하세요. `HTML5` 파일을 선택하고 이름을 `index.html`로 지정하세요:

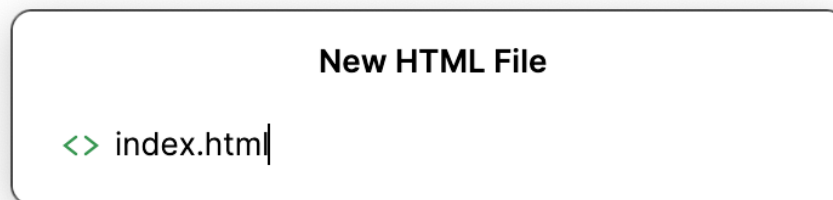


그림 5.9 – 새 `index.html`

3. Open the `index.html` file and, first, load the static tag library to make the static tag available in the template. Do this with the `load template` tag. On the second line of the file (just after `<!DOCTYPE html>`), add this line to load the static library:

```
{% load static %}
```

4. You can also make the template a bit nicer with some extra content. Enter the Business Site text inside the `<title>` tags:

```
<title>Business Site</title>
```

Then, inside the body, add an `<h1>` element with the `Welcome to my Business Site` text:

```
<h1>Welcome to my Business Site</h1>
```

5. Underneath the heading text, use the `{% static %}` template tag to set the source of an `<img>`. You will use it to refer to the logo from *Exercise 5.01 – serving a file from an app directory*:

```

```

6. Finally, to flesh out the site a bit, add a `<p>` element under `<img>`. Give it some text about the business:

```
<p>Welcome to the site for my Business. For all your Business needs!</p>
```

Although the extra text and title are not too important, they give us an idea of how to use the `{% static %}` template tag around the rest of the content. Save the file. It should look like in the `Chapter05` folder in this book's GitHub repo.

7. Next, set up a URL to use to render the template. You will also use the built-in `TemplateView` to render the template without having to create a view. Open `urls.py` in the `business_site` package directory. At the start of the file, import `TemplateView` as follows:

```
from django.views.generic import TemplateView
```

3. index.html 파일을 열고, 먼저 정적 태그 라이브러리를 로드하여 템플릿에서 정적 태그를 사용할 수 있도록 합니다. load 템플릿 태그를 사용하여 이 작업을 수행합니다. 파일의 두 번째 줄(<!DOCTYPE html> 바로 뒤)에 이 줄을 추가하여 정적 라이브러리를 로드합니다:

```
{% load static %}
```

4. 템플릿을 약간 더 멋지게 만들기 위해 추가 콘텐츠를 넣을 수도 있습니다. <title> 태그 안에 BusinessSite 텍스트를 입력하세요:

```
<title>Business Site</title>
```

그런 다음, 본문 안에 Welcome to my Business Site 텍스트가 있는 <h1> 요소를 추가하세요:

```
<h1>내 비즈니스 사이트에 오신 것을 환영합니다</h1>
```

5. idx_570cdfe9 제목 텍스트 아래에 {% static %} 템플릿 태그를 사용하여 소스를 설정하세요. . 이것은 연습문제 5.01 – 앱에서 파일 제공하기의 로그를 참조하는 데 사용할 것입니다. 디렉토리:

```

```

6. 마지막으로, 사이트를 좀 더 풍성하게 만들기 위해 아래에 <p> 요소를 추가하세요. 텍스트를 넣어주세요.

사업에 대하여:

```
<p>저의 비즈니스 사이트에 오신 것을 환영합니다. 모든 비즈니스 필요를 위해!</p>
```

추가 텍스트와 제목은 그리 중요하지 않지만, 나머지 콘텐츠 주위에 {% static %} 템플릿 태그를 사용하는 방법에 대한 아이디어를 제공합니다. 파일을 저장하세요. 이 책의 GitHub 리포지토리의 Chapter05 폴더에 있는 것처럼 보여야 합니다.

7. 다음으로, 템플릿을 렌더링하는 데 사용할 URL을 설정합니다. 뷰를 생성하지 않고도 템플릿을 렌더링하기 위해 내장 TemplateView를 사용할 것입니다. business_site 패키지 디렉토리에서 urls.py를 엽니다. 파일의 시작 부분에서 다음과 같이 TemplateView를 가져옵니다:

```
from django.views.generic import TemplateView
```

You can also remove this Django `import admin` line since we're not using it in this project:

```
from django.contrib import admin
```

8. Add a URL map from the root URL to a `TemplateView`. The `as_view` method of `TemplateView` takes `template_name` as an argument, which is used in the same way as a path that you might pass to the render function. Your `urlpatterns` should look like this:

```
urlpatterns = [  
    path("",  
        TemplateView.as_view(template_name="index.html")),  
]
```

Save the `urls.py` file. Once complete, it should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

9. Start the Django dev server, if it's not already running. Navigate to `http://127.0.0.1:8000/` in your browser. You should see your new landing page, as shown in *Figure 5.10*:

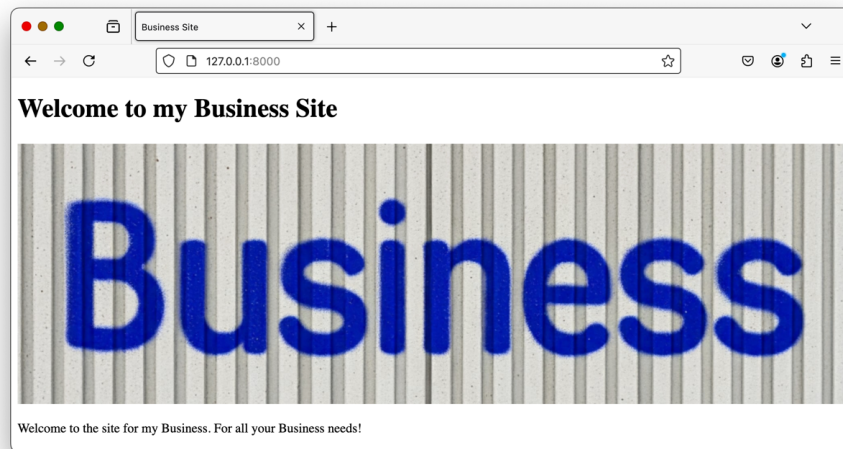


Figure 5.10 – My business site with a logo

이 프로젝트에서 사용하지 않으므로 이 Django import admin 줄도 제거할 수 있습니다:

```
from django.contrib import admin
```

8. 루트 URL에서 TemplateView로 URL idx_008cecf6map을 추가합니다. as_view 메서드의 TemplateView는 template_name을 인수로 받으며, 이는 render 함수에 전달할 수 있는 경로(apath)와 같은 방식으로 사용됩니다. urlpatterns는 다음과 같이 작성되어야 합니다:

```
urlpatterns = [
    ...
    경로("")
    TemplateView.as_view(template_name="index.html"),
    ...
]
```

urls.py 파일을 저장하세요. 완료되면 이 책의 GitHub 저장소의 Chapter05 폴더에 있는 것처럼 보여야 합니다.

9. Djangoidx_1ac20295 개발 서버를 시작하세요. 이미 실행 중이지 않다면, 브라우저에서 <http://127.0.0.1:8000/>로 이동하세요. 새로운 랜딩 페이지가 표시되어야 하며, Figure 5.10과 같이 보여야 합니다:

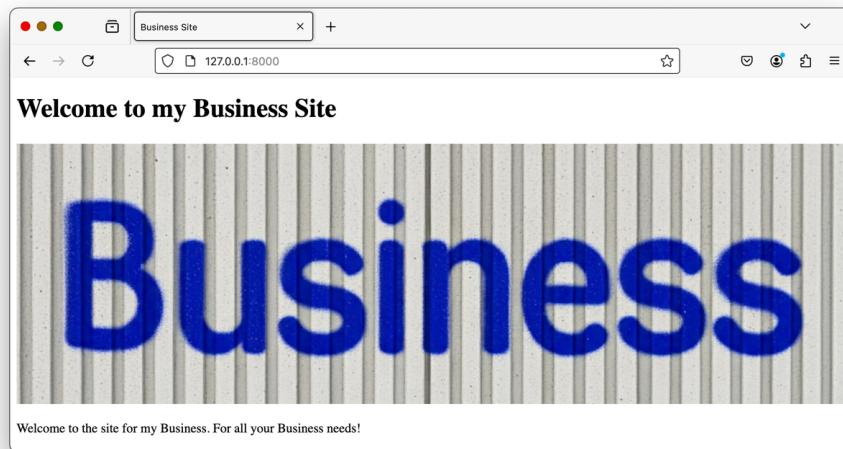


Figure 5.10 - 로고가 있는 내 비즈니스 사이트

In this exercise, we added a base template for landing and loaded the static library into the template. Once the static library was loaded, we were able to use the `static` template tag to load an image. Then, we were able to see our business logo rendered in the browser.

So far, the static file loading has used `AppDirectoriesFinder` because it required no extra configuration to use it. In the next section, we will look at `FileSystemFinder`, which is more flexible but requires a small amount of configuration to use it.

FileSystemFinder

So far, we've learned about `AppDirectoriesFinder`, which loads static files inside Django app directories. However, we expect well-designed apps to be self-contained, and therefore, they should only contain static files that they rely on. If we have other static files that are used throughout the website or across different apps, we should store them outside the app directory.

Note

As a general rule, your CSS is probably consistent throughout your site and could be kept in a global directory. Some images and JavaScript code could be specific to apps, so these would be stored in the static directory for that application. This is just general advice, though: you can store static files anywhere that makes the most sense for your project.

In our business site application, we will be storing a CSS file in a site static directory, as it will be used not only in the landing app but also throughout the site as we add more apps.

Django provides support for serving static files from arbitrary directories using its `FileSystemFinder` static file finder. The directories can be anywhere on the disk. Usually, you will have a static directory inside your project directory, but if your company has a global static directory that is used in many different projects (including non-Django web applications), then you could use this as well.

`FileSystemFinder` uses the `STATICFILES_DIRS` setting in the `settings.py` file to determine which directories to search for static files in. This is not present when the project is created and must be set by the developer. We will add it in the next exercise. There are two options for building this list:

- Setting a list of directories
- Setting a list of tuples in the form of `(prefix, directory)`

이 연습에서는 랜딩을 위한 기본 템플릿을 추가하고 정적 라이브러리를 템플릿에 로드했습니다. 정적 라이브러리가 로드된 후, 우리는 정적 템플릿 태그 `idx_8037b73dtoload`를 사용하여 이미지를 불러올 수 있었습니다. 그 후, 우리는 브라우저에서 비즈니스 로고가 렌더링된 것을 볼 수 있었습니다.

지금까지 정적 파일 로딩은 추가 구성 없이 사용할 수 있었기 때문에 `AppDirectoriesFinder`를 사용했습니다. 다음 섹션에서는 더 유연하지만 사용하기 위해 약간의 구성이 필요한 `FileSystemFinder`를 살펴보겠습니다.

FileSystemFinder

지금까지 우리는 Django `idx_0f01bfbdappdirectories` 내에서 정적 파일을 로드하는 `AppDirectoriesFinder`에 대해 배웠습니다. 그러나 우리는 잘 설계된 앱이 독립적이기를 기대하며, 따라서 그들이 의존하는 정적 파일만 포함해야 합니다. 웹사이트 전반이나 다른 앱에서 사용되는 다른 정적 파일이 있다면, 우리는 그것들을 앱 디렉토리 외부에 저장해야 합니다.

참고

일반적으로 귀하의 CSS는 사이트 전반에 걸쳐 일관성이 있을 것이며, 전역 디렉토리에 보관될 수 있습니다. 일부 이미지와 JavaScript 코드는 앱에 특정할 수 있으므로, 이러한 것들은 해당 애플리케이션의 정적 디렉토리에 저장됩니다. 그러나 이것은 일반적인 조언일 뿐입니다: 귀하의 프로젝트에 가장 적합한 곳에 정적 파일을 저장할 수 있습니다.

우리의 비즈니스 사이트 애플리케이션에서는 CSS 파일을 사이트 정적 디렉토리에 저장할 것입니다. 이는 랜딩 앱뿐만 아니라 더 많은 앱을 추가하면서 사이트 전반에 걸쳐 사용될 것입니다.

Django는 `FileSystemFinder` 정적 파일 탐색기를 사용하여 임의의 디렉토리에서 정적 파일을 제공하는 지원을 제공합니다. 디렉토리는 디스크의 어디에나 있을 수 있습니다. 일반적으로 프로젝트 디렉토리 내에 정적 디렉토리가 있지만, 회사에 여러 다른 프로젝트(비-Django 웹 애플리케이션 포함)에서 사용되는 전역 정적 디렉토리가 있다면, 이것도 사용할 수 있습니다.

`FileSystemFinder`는 `settings.py` 파일의 `STATICFILES_DIRS` 설정을 사용하여 정적 파일을 검색할 디렉토리를 결정합니다. 프로젝트가 생성될 때는 이 설정이 없으며 개발자가 설정해야 합니다. 다음 연습에서 추가할 것입니다. 이 목록을 작성하는 두 가지 옵션이 있습니다:

- 디렉토리 목록 설정
- (prefix, directory) 형식의 튜플 목록 설정

The second use case will be easier to understand once we have covered some more of the fundamentals, so we will return to it after explaining and demonstrating the first case. This will be covered after *Exercise 5.04 – collecting static files for production*, in the `STATICFILES_DIRS` *prefixed mode* section. For now, we will just explain the first use case, which is just a list of one or more directories.

In `business_site`, we will add a `static` directory inside the project directory (that is, in the same directory that contains the landing app and `manage.py` file). We can use the `BASE_DIR` setting when building the list to assign to `STATICFILES_DIRS`:

```
STATICFILES_DIRS = [BASE_DIR / "static"]
```

We also mentioned earlier in this section that you might want to set multiple directory paths in this list; for example, if you had some company-wide static data shared by multiple web projects. Simply add extra directories to the `STATICFILES_DIRS` list:

```
STATICFILES_DIRS = [BASE_DIR / "static",  
                    "/Users/username/projects/company-static/"  
]
```

Each of these directories would be checked, in the order specified, to find a matching file. If a file existed in both directories, the first one found would be served. For example, if the `static/main.css` (inside the `business_site` project directory) and `/Users/username/projects/company-static/bar/main.css` files both existed, a request for `/static/main.css` would serve the `business_site` project's `main.css` as it is first in the list. Consider then, when deciding the order in which you add directories to `STATICFILES_DIRS`, you may choose to prioritize your project static files over the global ones or vice versa.

In our business site (and later with Bookr), we will only use one static directory in this list, so we won't have to worry about this problem.

In the next exercise, we will add a static directory with a CSS file inside. Then, we will configure the `STATICFILES_DIRS` setting so that it can be served from the `static` directory.

Exercise 5.03 – serving from a project static directory

We showed an example of serving an application-specific image file in *Exercise 5.01 – serving a file from an app directory*. Now, we want to serve a CSS file that is to be used throughout our project to set styles, so we will serve this from a static directory right inside the project folder.

두 번째 사용 사례는 몇 가지 기본 사항을 다룬 후에 이해하기 쉬워질 것이므로, 첫 번째 사례를 설명하고 시연한 후에 다시 돌아올 것입니다. 이는 Exercise 5.04 – 프로덕션을 위한 정적 파일 수집 후에 STATICFILES_DIRS 접두사 모드 섹션에서 다룰 것입니다. 지금은 단순히 하나 이상의 디렉토리 목록인 첫 번째 사용 사례를 설명하겠습니다.

business_site에서 우리는 프로젝트 디렉토리(즉, 랜딩 앱과 manage.py 파일이 포함된 동일한 디렉토리) 내에 정적 디렉토리를 idx_b93354efadd 추가할 것입니다. STATICFILES_DIRS에 할당할 목록을 만들 때 BASE_DIR 설정을 사용할 수 있습니다:

```
STATICFILES_DIRS = [BASE_DIR / "static"]
```

이 섹션에서 언급했듯이, 이 목록에 여러 디렉토리 경로를 설정하고 싶을 수 있습니다. 예를 들어, 여러 웹 프로젝트에서 공유되는 회사 전체의 정적 데이터가 있는 경우입니다. STATICFILES_DIRS 목록에 추가 디렉토리를 간단히 추가하십시오:

```
STATICFILES_DIRS = [BASE_DIR / "static",
                    "/Users/username/projects/company-static/"
                    ]
```

이 idx_c408d8d2 디렉토리 각각은 일치하는 파일을 찾기 위해 지정된 순서대로 확인됩니다. 두 디렉토리에 파일이 모두 존재하는 경우, 먼저 발견된 파일이 제공됩니다. 예를 들어, business_site 프로젝트 디렉토리 내의 static/main.css와 /Users/username/projects/company-static/bar/main.css 파일이 모두 존재하는 경우, /static/main.css에 대한 요청은 목록에서 첫 번째인 business_site 프로젝트의 main.css를 제공할 것입니다. 따라서 STATICFILES_DIRS에 디렉토리를 추가할 순서를 결정할 때, 프로젝트 정적 파일을 전역 파일보다 우선시하거나 그 반대로 선택할 수 있습니다.

우리의 비즈니스 사이트(그리고 나중에 Bookr와 함께)에서는 이 목록에서 하나의 정적 디렉토리만 사용할 것이므로 이 문제에 대해 걱정할 필요가 없습니다.

다음 연습에서는 내부에 CSS 파일이 있는 정적 디렉토리를 추가할 것입니다. 그런 다음 STATICFILES_DIRS 설정을 구성하여 정적 디렉토리에서 제공될 수 있도록 할 것입니다.

연습 5.03 – 프로젝트 정적 디렉토리에서 제공하기

우리는 연습 5.01 – 앱 디렉토리에서 파일 제공하기에서 애플리케이션 특정 이미지 idx_b1148892 파일을 제공하는 예를 보여주었습니다. 이제 우리는 프로젝트 전반에 걸쳐 스타일을 설정하는 데 사용될 CSS 파일을 제공하고자 하므로, 이를 프로젝트 폴더 바로 안에 있는 정적 디렉토리에서 제공할 것입니다.

In this exercise, you'll set up your project to serve static files from a specific directory, and then use the `{% static %}` template tag again to include it in the template. This will be done using the `business_site` example project:

1. Open the `business_site` project in PyCharm if it's not already open. Then, right-click on the `business_site` project directory (the top-level `business_site` directory, not the `business_site` package directory) and select **New | Directory**.
2. In the **New Directory** dialog, enter `static` and click **OK**.
3. Right-click on the `static` directory you just created and select **New | File**.
4. In the **Name New File** dialog, enter `main.css` and click **OK**.
5. The blank `main.css` file should open automatically. Enter a couple of simple CSS rules to center the text and set a font and background color, like so:

```
body {  
    font-family: Arial, sans-serif;  
    text-align: center;  
    background-color: #f0f0f0;  
}
```

Now save and close `main.css`.

You can also take a look at the complete file for reference in the `Chapter05` folder in this book's GitHub repo.

6. Next, open `business_site/settings.py`. Here, set a list of directories to the `STATICFILES_DIRS` settings. In this case, the list will have just one item. Define a new variable called `STATICFILES_DIRS` at the bottom of the `settings.py` file using the following code:

```
STATICFILES_DIRS = [BASE_DIR / "static"]
```

In the `settings.py` file, `BASE_DIR` is a variable that contains the path to the project directory. You can build the full path to the static directory you created in *Step 2* by joining `static` to `BASE_DIR`. Then, you can put this inside a list. The complete `settings.py` file should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

7. Start the Django dev server if it is not running. You can verify that the settings are correct by checking if you can load the `main.css` file. Note that this is not namespaced,

이 연습에서는 특정 디렉토리에서 정적 파일을 제공하도록 프로젝트를 설정한 다음, {% static %} 템플릿 태그를 다시 사용하여 템플릿에 포함시킬 것입니다. 이는 business_site 예제 프로젝트를 사용하여 수행됩니다:

1. PyCharm에서 business_site 프로젝트를 열고, 아직 열려 있지 않다면 비즈니스 사이트 프로젝트 디렉토리(최상위 business_site 디렉토리, business_site 패키지 디렉토리가 아님)를 마우스 오른쪽 버튼으로 클릭하고 새로 만들기 | 디렉토리를 선택합니다.
2. 새 디렉토리 대화 상자에서 static을 입력하고 확인을 클릭합니다.
3. 방금 생성한 static 디렉토리를 마우스 오른쪽 버튼으로 클릭하고 새로 만들기 | 파일을 선택합니다.
4. 이름 새 파일 대화 상자에 main.css를 입력하고 확인을 클릭합니다.
5. 빈 main.css 파일이 자동으로 열려야 합니다. 텍스트를 중앙에 정렬하고 글꼴 및 배경 색상을 설정하는 몇 가지 간단한 CSS 규칙을 입력합니다, 다음과 같이:

```
body {
    font-family: Arial, sans-serif;
    text-align: center;
    background-color: #f0f0f0;
}
```

이제 main.css를 저장하고 닫습니다.

이 책의 GitHub 저장소의 Chapter05 폴더에서 참조용으로 전체 파일을 살펴볼 수도 있습니다.

6. 다음으로, business_site/settings.py를 엽니다. 여기에서 STATICFILES_DIRS 설정에 디렉토리 목록을 설정합니다. 이 경우 목록에는 하나의 항목만 있습니다. 다음 코드를 사용하여 settings.py 파일의 맨 아래에 STATICFILES_DIRS라는 새 변수를 정의합니다:

```
STATICFILES_DIRS = [BASE_DIR / "static"]
```

settings.py 파일에서 BASE_DIR은 프로젝트 디렉토리의 경로를 포함하는 변수입니다. 2단계에서 생성한 정적 디렉토리의 전체 경로를 BASE_DIR에 static을 결합하여 만들 수 있습니다. 그런 다음 이것을 목록 안에 넣을 수 있습니다. 완전한 settings.py 파일은 이 책의 GitHub 리포지토리의 Chapter05 폴더에 있는 파일과 같아야 합니다.

7. idx_a0ecd072Django 개발 서버가 실행되고 있지 않다면 시작합니다. main.css 파일을 로드할 수 있는지 확인하여 설정이 올바른지 확인할 수 있습니다. 이것은 네임스페이스가 없다는 점에 유의하세요,

so the URL is `http://127.0.0.1:8000/static/main.css`. Open this URL in your browser and check that the content matches what you just entered and saved:



Figure 5.11 – CSS served by Django

If the file does not load, check your `STATICFILES_DIRS` settings. You may need to restart the Django dev server if it was running while you made changes to `settings.py`.

8. Now, you need to include `main.css` in your index template. Open `index.html` in the `templates` folder. Before the closing `</head>` tag, add this `<link>` tag to load the CSS:

```
<link rel="stylesheet" href="{% static 'main.css' %}">
```

This links in the `main.css` file, using the `{% static %}` template tag. As mentioned earlier, since `main.css` is not namespaced, you can just include its name. Save the file; it should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

9. Load `http://127.0.0.1:8000/` in your browser; you should see the background color, fonts, and alignment all change:

따라서 URL은 `http://127.0.0.1:8000/static/main.css`입니다. 이 URL을 브라우저에서 열고 입력하고 저장한 내용이 일치하는지 확인하세요:

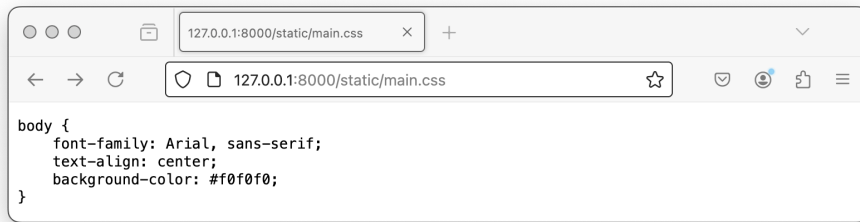


그림 5.11 - Django에서 제공하는 CSS

파일이 로드되지 않으면 `STATICFILES_DIRS` 설정을 확인하세요. `settings.py`를 변경하는 동안 Django 개발 서버가 실행 중이었다면 다시 시작해야 할 수도 있습니다.

8. 이제 `index` 템플릿에 `main.css`를 포함해야 합니다. `templates` 폴더에서 `index.html`을 엽니다. 닫는 `</head>` 태그 전에 CSS를 로드하기 위해 이 `<link>` 태그를 추가하세요:

```
<link rel="stylesheet" href="{% static 'main.css' %}">
```

이것은 `{% static %}` 템플릿 태그를 사용하여 `main.css` 파일을 연결합니다. 앞서 언급했듯이 `main.css`는 네임스페이스가 없으므로 이름만 포함하면 됩니다. 파일을 저장하세요; 이 책의 GitHub 리포지토리의 `Chapter05` 폴더에 있는 파일과 비슷하게 보여야 합니다.

9. 브라우저에서 `http://127.0.0.1:8000/`를 로드하세요; 배경 `idx_f04cb441` 색상, 글꼴 및 정렬이 모두 변경된 것을 볼 수 있어야 합니다:

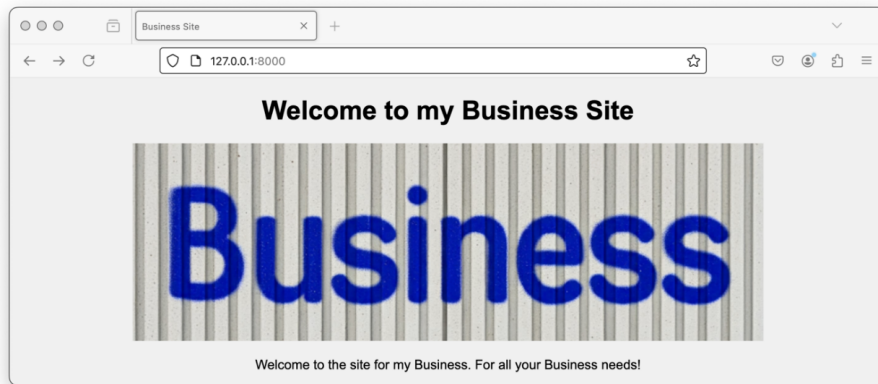


Figure 5.12 – CSS applied with custom fonts visible

Your business landing page should look like what's shown in *Figure 5.12*. Since you included the CSS in the `index.html` template, it will be available in all the templates that extend this template (although none do at the moment, it's good planning for the future).

In this exercise, we put some CSS rules into a file and served them using Django's `FileSystemFinder`. This was accomplished by creating a `static` directory inside the `business_site` project directory and specifying it in the Django settings (the `settings.py` file) using the `STATICFILES_DIRS` setting. We linked the `main.css` file to the `index.html` template using the `static` template tag. We loaded the main page in our browser and saw the font and color changes applied.

We've now covered how static file finders are used during a request (to load a specific static file when given a URL). We'll now look at their other use case: finding and copying static files for production deployment when running the `collectstatic` management command.

Static file finders – use during `collectstatic`

Once we have finished working on our static files, they need to be moved into a specific directory that can be served by our production web server. We can then deploy our website by copying our Django code and static files to our production web server. In the case of `business_site`, we will want to move `logo.png` and `main.css` (along with other static files that Django itself includes) into a single directory that can be copied to the production web server. This is the role of the `collectstatic` management command.

Without having to run `collectstatic`, a web server would not be able to map a URL back to a path. For example, it would not know that `main.css` must be loaded from the project static



그림 5.12 - 사용자 정의 글꼴이 적용된 CSS

귀하의 비즈니스 랜딩 페이지는 그림 5.12에 표시된 것처럼 보여야 합니다. `index.html` 템플릿에 CSS를 포함했기 때문에 이 템플릿을 확장하는 모든 템플릿에서 사용할 수 있습니다(현재는 아무것도 확장하지 않지만, 미래를 위한 좋은 계획입니다).

이번 연습에서는 CSS 규칙을 파일에 넣고 Django의 `FileSystemFinder`를 사용하여 제공했습니다. 이는 `business_site` 프로젝트 디렉토리 내에 정적 디렉토리를 생성하고 `STATICFILES_DIRS` 설정을 사용하여 Django 설정(`설정.py` 파일)에서 지정함으로써 이루어졌습니다. 우리는 `static` 템플릿 태그를 사용하여 `main.css` 파일을 `index.html` 템플릿에 연결했습니다. 브라우저에서 `idx_760209b1main` 페이지를 로드하고 글꼴 및 색상 변경 사항이 적용된 것을 보았습니다.

이제 요청 중에 정적 파일 찾기가 어떻게 사용되는지 다루었습니다(주어진 URL에서 특정 정적 파일을 로드하기 위해). 이제 그들의 다른 사용 사례를 살펴보겠습니다: `collectstatic` 관리 명령을 실행할 때 프로덕션 배포를 위해 정적 파일을 찾고 복사하는 것입니다.

정적 파일 찾기 – `collectstatic` 중 사용

정적 파일 작업을 마친 후에는 `idx_b1cf318c`가 프로덕션 웹 서버에서 제공될 수 있는 특정 디렉토리로 이동해야 합니다. 그런 다음 Django 코드와 정적 파일을 프로덕션 웹 서버에 복사하여 웹사이트를 배포할 수 있습니다. `business_site`의 경우 `logo.png`와 `main.css`(및 Django 자체가 포함하는 다른 정적 파일)를 프로덕션 웹 서버에 복사할 수 있는 단일 디렉토리로 이동해야 합니다. 이것이 `collectstatic` 관리 명령의 역할입니다.

`collectstatic`을 실행하지 않고는 웹 서버가 URL을 경로로 다시 매핑할 수 없습니다. 예를 들어, `main.css`가 프로젝트 정적에서 로드되어야 한다는 것을 알지 못할 것입니다.

directory while `logo.png` is to be loaded from the landing app directory – it has no concept of the Django directory layout. Upon running the `collectstatic` management command, Django uses each finder to list static files on the disk. Every static file that is found is then copied into the `STATIC_ROOT` directory (also defined in `settings.py`). This is a little bit like the reverse of handling a request. Instead of getting a URL path and mapping it to a filesystem path, the filesystem path is copied to a location that can be predicted by the frontend web server. This allows the frontend web server to handle a request for a static file independently of Django.

Note

A frontend web server is a web server that's designed to route requests to applications (such as Django) or read static files from disk. They can handle requests faster, but aren't able to generate dynamic content in the same way as something such as Django. Frontend web servers are software such as Apache HTTPD, nginx, and lighttpd.

For some specific examples of how `collectstatic` works, we'll use the two files from *Exercise 5.01 – serving a file from an app directory* and *Exercise 5.03 – serving from a project status directory*, respectively: `landing/static/landing/logo.png` and `static/main.css`.

Let's assume that `STATIC_ROOT` has been set to a directory being served by a normal web server – this would be something such as `/var/www/business_site/static`. The destination for these files would be `/var/www/business_site/static/reviews/logo.png` and `/var/www/business_site/static/main.css`, respectively.

Now, when a request for a static file comes in, the web server will be able to serve it easily because the paths are mapped consistently:

- `/static/main.css` is served from the `/var/www/business_site/static/main.css` file
- `/static/reviews/logo.png` is served from the `/var/www/business_site/static/reviews/logo.png` file

This means the web server root is `/var/www/business_site/` and static paths are loaded directly from disk in the usual manner that a web server would load files.

With that, we have demonstrated how Django locates static files during development and can serve them itself. In production, we need the frontend web server to be able to serve static files without involving Django, for both safety and speed.

You might be tempted to serve files directly from the Django project directory by setting your web server root to this directory – **don't do this**. There is a **security risk** in sharing your entire

디렉토리에서 logo.png는 랜딩 앱 디렉토리에서 로드되어야 하며, Django 디렉토리 레이아웃에 대한 개념이 없습니다. collectstatic 관리 명령을 실행하면 Django는 각 파일을 사용하여 디스크의 정적 파일을 나열합니다. 발견된 모든 정적 파일은 STATIC_ROOT 디렉토리(또한 settings.py에 정의됨)로 복사됩니다. 이는 요청을 처리하는 것의 반대와 비슷합니다. URL 경로를 가져와서 파일 시스템 경로에 매핑하는 대신, 파일 시스템 경로가 프론트엔드 웹 서버가 예측할 수 있는 위치로 복사됩니다. 이를 통해 프론트엔드 웹 서버는 Django와 독립적으로 정적 파일에 대한 요청을 처리할 수 있습니다.

참고

프론트엔드 웹 서버는 요청을 애플리케이션(예: Django)으로 라우팅하거나 디스크에서 정적 파일을 읽도록 설계된 웹 서버입니다. 이들은 요청을 더 빠르게 처리할 수 있지만, Django와 같은 방식으로 동적 콘텐츠를 생성할 수는 없습니다. 프론트엔드 웹 서버는 Apache HTTPD, nginx 및 lighttpd와 같은 소프트웨어입니다.

collectstatic이 작동하는 방법에 대한 몇 가지 구체적인 예로, 우리는 Exercise 5.01의 두 파일 - 앱 디렉토리에서 파일 제공 및 Exercise 5.03 - 프로젝트 상태 디렉토리에서 제공을 사용할 것입니다: landing/static/landing/logo.png 및 static/main.css.

STATIC_ROOT가 일반 웹 서버에 의해 제공되는 디렉토리로 설정되었다고 가정해 보겠습니다. 이는 /var/www/business_site/static과 같은 것이 될 것입니다. 이러한 파일의 목적지는 각각 /var/www/business_site/static/reviews/logo.png 및 /var/www/business_site/static/main.css가 될 것입니다.

이제 정적 파일에 대한 요청이 들어오면, 웹 서버는 경로가 일관되게 매핑되기 때문에 쉽게 제공할 수 있습니다:

- /static/main.css는 /var/www/business_site/static/main.css 파일에서 제공되며 /static/reviews/logo.png는
- /var/www/business_site/static/reviews/logo.png 파일에서 제공됩니다.

이것은 웹 서버 루트가 /var/www/business_site/이고 정적 경로가 웹 서버가 파일을 로드하는 일반적인 방식으로 디스크에서 직접 로드된다는 것을 의미합니다.

이로써 우리는 Django가 개발 중에 정적 파일을 찾는 방법과 이를 스스로 제공할 수 있는 방법을 보여주었습니다. 프로덕션에서는 프론트엔드 웹 서버가 Django를 포함하지 않고 정적 파일을 제공할 수 있어야 합니다. 이는 안전성과 속도 모두를 위해 필요합니다.

웹 서버 루트를 이 디렉토리로 설정하여 Django 프로젝트 디렉토리에서 파일을 직접 제공하고 싶을 수 있지만 - 그렇게 하지 마십시오. 전체를 공유하는 데 보안 위험이 있습니다.

Django project directory, as it would make it possible to download `settings.py` or other sensitive files. Running `collectstatic` will copy the files to a directory that can be moved outside the Django project directory to the web server root for security.

Here, we have talked about using `collectstatic` to copy static files directly to the web server root. You could also have Django copy them to an intermediary directory and have your deployment process move to a CDN or another server afterward. However, we won't go into detail on specific deployment processes; how you choose to copy static files to the web server will depend on you or your company's existing setup (for example, a continuous delivery pipeline).

Note

The `collectstatic` command does not take into consideration the use of `static` template tags. It will collect all the static files inside static directories, even those that your project does not include inside a template.

In the next exercise, we will see the `collectstatic` command in action. We'll use it to copy all the `business_site` static files that we have so far into a temporary directory.

Exercise 5.04 – collecting static files for production

While we won't be covering deployment to a web server in this chapter, we can still use the `collectstatic` management command and see its result. In this exercise, we will create a temporary holding location for the static files to be copied into. This directory will be called `static_production_test` and will be located inside the `business_site` project directory.

As part of the deployment process, you could copy this directory to your production web server. However, since we won't be setting up a web server until *Chapter 18, Deploying Django*, we will just examine its contents to understand how files are copied and organized.

Here's how to get started:

1. In PyCharm, create a temporary directory to put the collected files in. Right-click on the `business_site` project directory (this is the top-level folder, not the `business_site` module) and select **New | Directory**.
2. In the **New Directory** dialog, enter `static_production_test` and hit *Enter*.
3. Open `settings.py` and, at the bottom of the file, define a new setting for `STATIC_ROOT`. Set it to the path of the directory you just created:

```
STATIC_ROOT = BASE_DIR / "static_production_test"
```

Django 프로젝트 디렉토리로, settings.py 또는 기타 민감한 파일을 다운로드할 수 있게 합니다. collectstatic을 실행하면 파일이 Django 프로젝트 디렉토리 외부의 웹 서버 루트로 이동할 수 있는 디렉토리에 복사됩니다.

여기에서는 collectstatic을 사용하여 정적 파일을 웹 서버 루트로 직접 복사하는 방법에 대해 이야기했습니다. Django가 중간 디렉토리에 복사하도록 하고 배포 프로세스가 이후에 CDN 또는 다른 서버로 이동하도록 할 수도 있습니다. 그러나 특정 배포 프로세스에 대한 자세한 내용은 다루지 않을 것입니다; 정적 파일을 웹 서버로 복사하는 방법은 귀하 또는 귀하의 회사의 기존 설정(예: 지속적 배포 파이프라인)에 따라 달라질 것입니다.

참고

collectstatic 명령은 정적 템플릿 태그의 사용을 고려하지 않습니다. 프로젝트가 템플릿에 포함하지 않은 것들조차도 정적 디렉토리 내의 모든 정적 파일을 수집합니다.

다음 연습에서는 collectstatic 명령이 작동하는 모습을 볼 것입니다. 지금까지 가지고 있는 business_site 정적 파일을 임시 디렉토리에 복사하는 데 사용할 것입니다.

연습 5.04 – 프로덕션을 위한 정적 파일 수집

이번 장에서는 웹 서버에 배포하는 내용을 다루지 않지만, collectstatic 관리 명령을 사용하고 그 결과를 볼 수 있습니다. 이 연습에서는 정적 파일이 복사될 임시 보관 위치를 생성할 것입니다. 이 디렉토리는 static_production_test라고 불리며 business_site 프로젝트 디렉토리 내에 위치할 것입니다.

배포 프로세스의 일환으로 이 디렉토리를 프로덕션 웹 서버로 복사할 수 있습니다. 그러나 18장, Django 배포까지 웹 서버를 설정하지 않을 것이므로, 파일이 어떻게 복사되고 정리되는지 이해하기 위해 그 내용을 살펴보겠습니다.

시작하는 방법은 다음과 같습니다:

1. PyCharm에서 수집된 파일을 넣을 임시 디렉토리를 만듭니다. business_site 프로젝트 디렉토리(비즈니스_사이트 모듈이 아닌 최상위 폴더)를 마우스 오른쪽 버튼으로 클릭하고 새로 만들기 | 디렉토리를 선택합니다.
2. 새 디렉토리 대화 상자에서 static_production_test를 입력하고 Enter 키를 누릅니다.
3. settings.py를 열고 파일의 맨 아래에 STATIC_ROOT에 대한 새 설정을 정의합니다. 방금 만든 디렉토리의 경로로 설정합니다:

```
STATIC_ROOT = BASE_DIR / "static_production_test"
```

This will join `static_dir` to `BASE_DIR` (the `business_site` project path) to generate the full path. Save the `settings.py` file. It should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

4. In a terminal, run the `collectstatic` manage command:

```
python manage.py collectstatic
```

You should see an output like the following:

```
129 static files copied to '/Users/chrisquest/business_site/
static_production_test'.
```

This might seem like a lot if you were expecting it to copy just two files, but remember that it will copy all the files for all installed apps. In this case, as you have the Django admin app installed, most of the 129 files are to support that.

5. Let's look through the `static_production_test` directory to check what has been created. An expanded view of this directory (from the PyCharm project page) is shown in *Figure 5.13*, for reference. Yours should be similar:

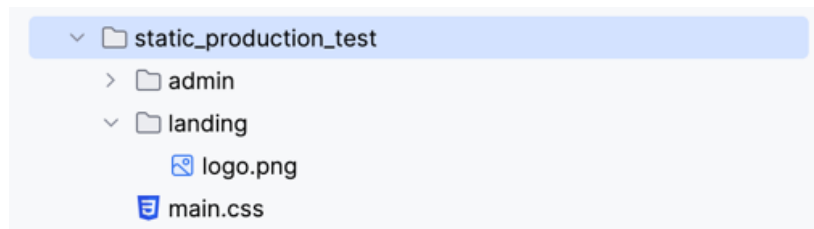


Figure 5.13 – Destination directory of the `collectstatic` command

You should notice three items inside it:

- The `admin` directory: This contains files from the Django admin app. If you look inside this, you'll see it has been organized into subfolders for `css`, `fonts`, `img`, and `js`.

이것은 `static_dir`을 `BASE_DIR`(비즈니스 사이트 프로젝트 경로)에 결합하여 전체 경로를 생성합니다. `settings.py` 파일을 저장하세요. 이 파일은 이 책의 GitHub 리포지토리의 Chapter05 폴더에 있는 파일과 비슷해야 합니다.

4. `idx_e5864fb8` 터미널에서 `collectstatic` 관리 명령을 실행하세요:

```
python manage.py collectstatic
```

다음과 같은 출력이 표시되어야 합니다:

```
129개의 정적 파일이
'/Users/chrisquest/business_site/static_production_test'에 복사되었습니다.
```

이것은 단 두 개의 파일만 복사할 것으로 예상했다면 많은 것처럼 보일 수 있지만, 설치된 모든 앱의 모든 파일을 복사한다는 것을 기억하세요. 이 경우, Django admin 앱이 설치되어 있으므로 129개의 파일 중 대부분은 이를 지원하기 위한 것입니다.

5. 정적_생산_테스트 디렉토리를 살펴보고 생성된 내용을 확인해 봅시다. 이 디렉토리의 확장된 뷰(파이썬 프로젝트 페이지에서)는 참고를 위해 그림 5.13에 표시되어 있습니다. 여러분의 것도 비슷해야 합니다:

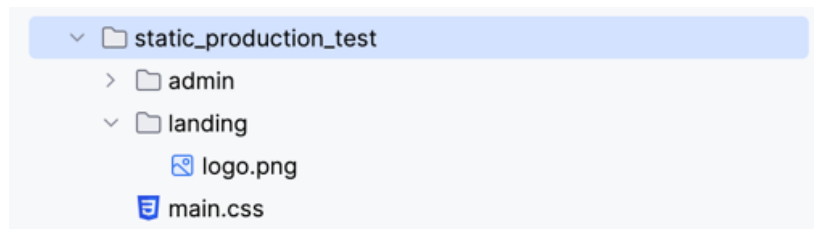


그림 5.13 - `collectstatic` 명령의 대상 디렉토리

`idx_5322f98e` 안에 세 가지 항목이 있어야 합니다:

- `admin` 디렉토리: 여기에는 Django 관리 앱의 파일이 포함되어 있습니다. 이 안을 들여다보면 `css`, `fonts`, `img`, `js`를 위한 하위 폴더로 정리되어 있는 것을 볼 수 있습니다.

- The `landing` directory: This is the static directory from your landing app. Inside it is the `logo.png` file. This directory has been created to match the namespacing of the directory that we created.
- The `main.css` file: This is from your project static directory. Since you didn't place it inside a namespacing directory, this has been placed directly inside `STATIC_ROOT`.

If you want, you can open any of these files and verify that their content matches the files you have just been working on – they should do as they are simply copies of the original files.

In this exercise, we collected all the static files from `business_site` (including the admin static files that Django includes). They were copied into the directory defined by the `STATIC_ROOT` setting (`static_production_test` inside the `business_site` project directory). We saw that `main.css` was directly inside this folder, but that other static files were namespaced inside their app directories (admin and reviews). This folder could have been copied to a production web server to deploy our project.

In the next section, we'll look at how a prefix can be added to each static file directory source to customize how static files are organized after being collected.

STATICFILES_DIRS prefixed mode

As mentioned earlier, the `STATICFILES_DIRS` setting also accepts items as tuples in the form of `(prefix, directory)`. These modes of operation are not mutually exclusive; `STATICFILES_DIRS` may contain both non-prefixed (string) or prefixed (tuple) items. Essentially, this allows you to map a certain URL prefix to a directory.

In Bookr, we do not have enough static assets to warrant setting this up, but it can be useful if you want to organize your static assets differently. For example, you can keep all your images in a certain directory, and all your CSS in another directory. You might need to do this if you use a third-party CSS generation tool such as Node.js with LESS.

Note

LESS is a CSS pre-processor that uses Node.js. It allows you to write CSS using variables and other programming-like concepts that don't exist natively. Node.js will then compile this to CSS. A more in-depth explanation is outside the scope of this book, though the site lesscss.org offers a comprehensive introduction and further documentation. If you use it (or a similar tool), then you might want to serve directly from the directory to which it saves its compiled output.

- 랜딩 디렉토리: 이것은 당신의 랜딩 앱의 정적 디렉토리입니다. 그 안에는 logo.png 파일이 있습니다. 이 디렉토리는 우리가 만든 디렉토리의 네임스페이스에 맞게 생성되었습니다.
- main.css 파일: 이것은 당신의 프로젝트 정적 디렉토리에서 온 것입니다. 네임스페이스 디렉토리 안에 두지 않았기 때문에, 이것은 직접 STATIC_ROOT 안에 배치되었습니다.

원한다면, 이 파일들 중 어떤 것이든 열어보고 그 내용이 당신이 방금 작업한 파일과 일치하는지 확인할 수 있습니다 - 그들은 단순히 원본 파일의 복사본이기 때문에 일치해야 합니다.

이번 연습에서는 business_site에서 모든 정적 파일을 수집했습니다 (Django가 포함하는 관리자 정적 파일 포함). 이들은 STATIC_ROOT 설정에 의해 정의된 디렉토리(비즈니스 사이트 프로젝트 디렉토리 안의 static_production_test)로 복사되었습니다. 우리는 main.css가 이 폴더 안에 직접 있었지만, 다른 정적 파일들은 그들의 앱 디렉토리(관리자 및 리뷰) 안에 네임스페이스가 지정되어 있음을 보았습니다. 이 폴더는 우리의 프로젝트를 배포하기 위해 프로덕션 웹 서버로 복사될 수 있었습니다.

다음 섹션에서는 수집된 후 정적 파일이 어떻게 구성되는지를 사용자 정의하기 위해 각 정적 파일 디렉토리 소스에 접두사를 추가하는 방법을 살펴보겠습니다.

STATICFILES_DIRS 접두사 모드

앞서 언급했듯이, STATICFILES_DIRS 설정은 (prefix, directory) 형태의 튜플로 항목을 수락합니다. 이러한 작동 모드는 상호 배타적이지 않습니다; STATICFILES_DIRS는 비 접두사(문자열) 또는 접두사(튜플) 항목을 모두 포함할 수 있습니다. 본질적으로, 이는 특정 URL 접두사를 디렉토리에 매핑할 수 있게 해줍니다.

Bookr에서는 idx_05ed0485 정적 자산이 충분하지 않아 이를 설정할 필요는 없지만, 정적 자산을 다르게 구성하고 싶다면 유용할 수 있습니다. 예를 들어, 모든 이미지를 특정 디렉토리에 보관하고, 모든 CSS를 다른 디렉토리에 보관할 수 있습니다. Node.js와 LESS와 같은 서드파티 CSS 생성 도구를 사용하는 경우 이렇게 해야 할 수도 있습니다.

메모

LESS는 Node.js를 사용하는 CSS 전처리기입니다. 이를 통해 기본적으로 존재하지 않는 변수 및 기타 프로그래밍과 유사한 개념을 사용하여 CSS를 작성할 수 있습니다. 그런 다음 Node.js가 이를 CSS로 컴파일합니다. 더 깊이 있는 설명은 이 책의 범위를 벗어납니다.

lesscss.org 사이트는 포괄적인 소개와 추가 문서를 제공합니다.

이 도구(또는 유사한 도구)를 사용하는 경우, 컴파일된 출력이 저장되는 디렉토리에서 직접 제공할 수 있습니다.

The easiest way to explain how prefixed mode works is with a short example. This will expand on the `STATICFILES_DIRS` setting we created in *Exercise 5.03 – serving from a project static directory*; however, this is not an exercise. In this example, two prefixed directories are being added to this setting – one for serving images and one for serving CSS:

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
    ("images", BASE_DIR / "static_images"),  
    ("css", BASE_DIR / "static_css"),  
]
```

As well as the `static` directory that was already being served with no prefix, we have added serving of the `static_images` directory inside the `business_site` project directory. This has a prefix of `images`. We have also added serving for the `static_css` directory inside the `Bookr` project directory, with a prefix of `css`.

Then, we can serve three files – `main.js`, `main.css`, and `main.jpg` – from the `static`, `static_css`, and `static_images` directories, respectively. The directory layout will be as shown in *Figure 5.14*:

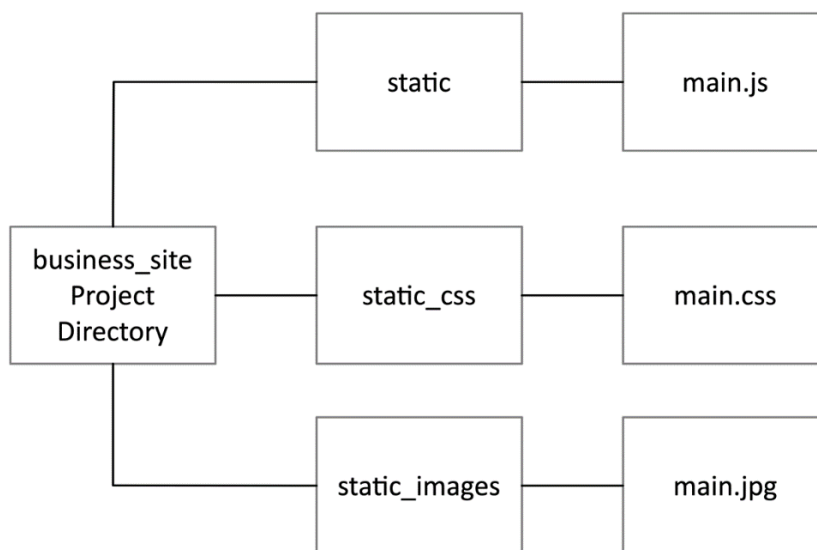


Figure 5.14 – Directory layout for use with prefixed URLs

In terms of accessing these via a URL, the mapping is as shown in *Figure 5.15*:

접두사 모드가 작동하는 방식을 설명하는 가장 쉬운 방법은 짧은 예제를 사용하는 것입니다. 이는 연습 5.03에서 생성한 STATICFILES_DIRS 설정을 확장합니다 - 프로젝트 정적 디렉토리에서 제공하는 것이지만, 이는 연습이 아닙니다. 이 예제에서는 두 개의 접두사 디렉토리가 이 설정에 추가되고 있습니다 - 하나는 이미지를 제공하기 위한 것이고, 다른 하나는 CSS를 제공하기 위한 것입니다:

```
STATICFILES_DIRS = [
    BASE_DIR / "static",
    ("images", BASE_DIR / "static_images"),
    ("css", BASE_DIR / "static_css"),
]
```

이미지 접두사가 있는 business_site 프로젝트 디렉토리 내의 static_images 디렉토리 제공을 추가했습니다. 또한 css 접두사가 있는 Bookr 프로젝트 디렉토리 내의 static_css 디렉토리 제공도 추가했습니다.

그런 다음, 우리는 각각 static, static_css 및 static_images 디렉토리에서 main.js, main.css 및 main.jpg의 세 파일을 제공할 수 있습니다. 디렉토리 레이아웃은 그림 5.14에 표시된 대로 될 것입니다:

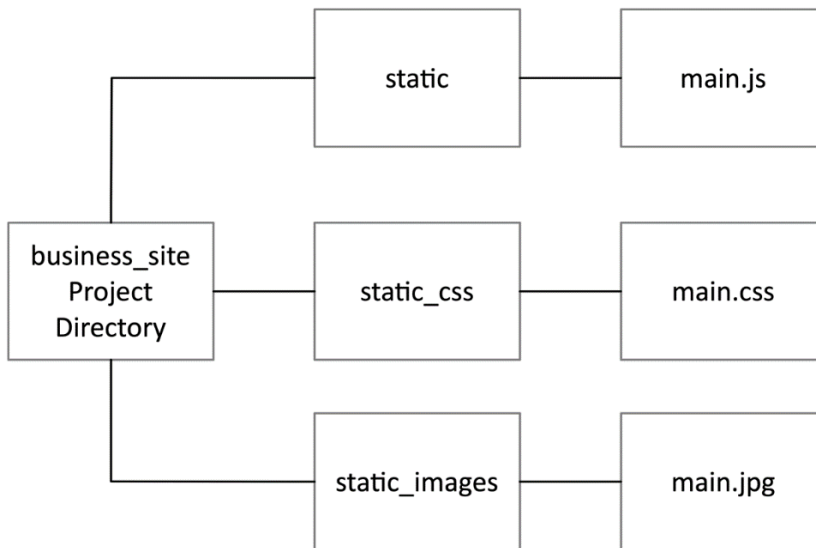


그림 5.14 - 접두사가 있는 URL에 사용하기 위한 디렉토리 레이아웃

이 URL을 통해 접근하는 측면에서, 매핑은 그림 5.15와 같이 표시됩니다:

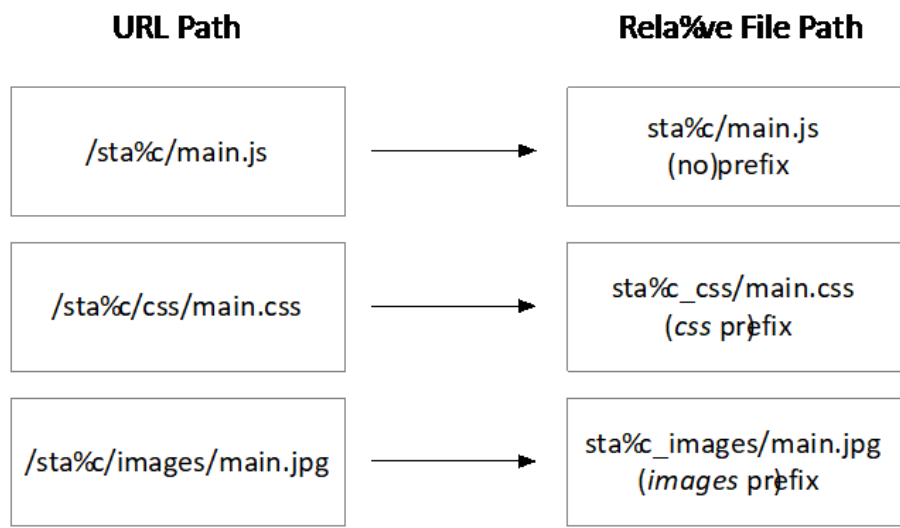


Figure 5.15 – Mapping a URL to a file based on the prefix

Django routes any static URL with a prefix to the directory that matches that prefix. When used with the `static` template tag, use the prefix and filename, not the directory name. Here's an example:

```
{% static 'images/main.jpg' %}
```

When the static files are gathered using the `collectstatic` command, they are moved into a directory with the prefix name, inside `STATIC_ROOT`. The source paths and the target paths inside the `STATIC_ROOT` directory are shown in *Figure 5.16*:

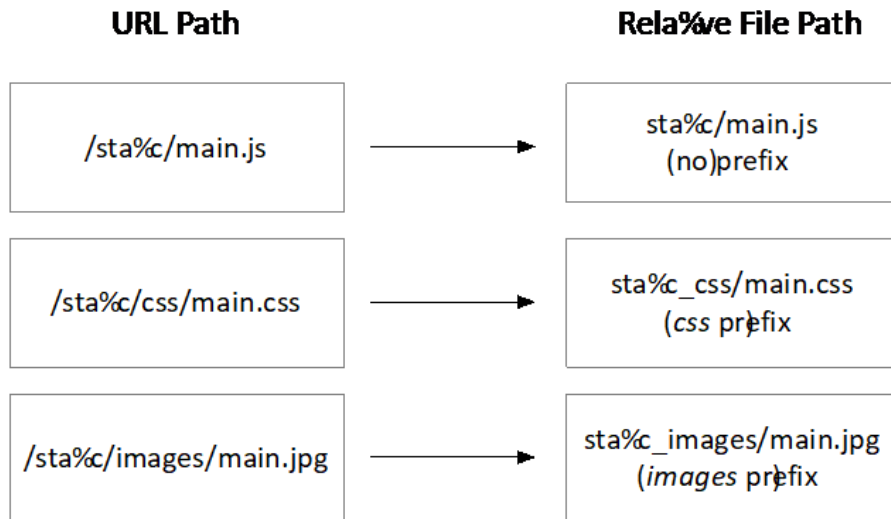


그림 5.15 - 접두사에 따라 URL을 파일에 매핑하기

Django는 접두사가 있는 정적 URL을 해당 접두사와 일치하는 디렉토리로 라우팅합니다.

정적 템플릿 태그와 함께 사용할 때는 디렉토리 이름이 아니라 접두사와 파일 이름을 사용하세요. 예를 들면 다음과 같습니다:

```
{% static 'images/main.jpg' %}
```

정적 파일이 collectstatic 명령을 사용하여 수집되면, STATIC_ROOT 내부의 접두사 이름이 있는 디렉토리로 이동됩니다. STATIC_ROOT 디렉토리 내의 소스 경로와 대상 경로는 그림 5.16에 나와 있습니다:

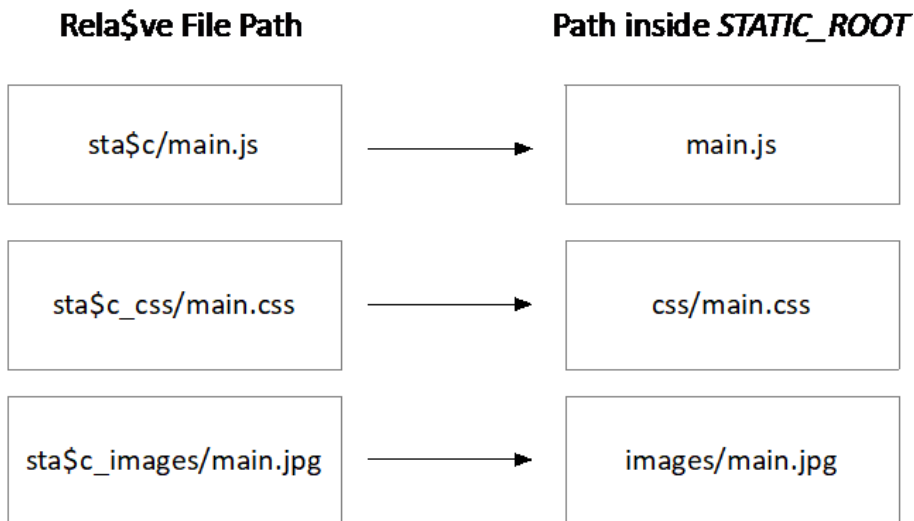


Figure 5.16 – Mapping from the path in the project directory to the path in *STATIC_ROOT*

Django creates the prefix directories inside *STATIC_ROOT*. Because of this, the paths can be kept consistent, even when using a web server and not routing the URL lookup through Django.

Next, we'll look at a management command that's useful for troubleshooting when static files aren't being loaded as you expect: `findstatic`.

The `findstatic` command

The *staticfiles* application also provides one more management command: `findstatic`. This command allows you to enter the relative path to a static file (the same as what would be used inside a static template tag) and Django will tell you where that file was located. It can also be used in verbose mode to output the directories it is searching through.

Note

You may not be familiar with the concept of verbosity or verbose mode. Having a higher verbosity (or simply turning on verbose mode) will cause a command to generate more output. Many command-line applications can be executed with more or less verbosity. This can be helpful when you're trying to debug programs you are using. To see an example of verbose mode in action, you can try running the Python shell in verbose mode. Enter `python -v` (instead of just `python`) and press *Enter*. Python will start in verbose mode, which prints out the path of every file it imports.

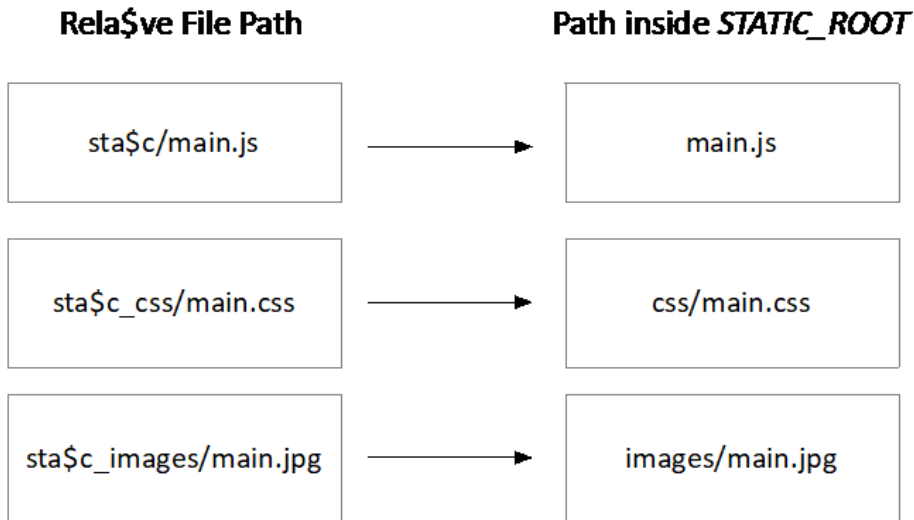


그림 5.16 - 프로젝트 디렉토리의 경로에서 *STATIC_ROOT*의 경로로 매핑

Django는 *STATIC_ROOT* 내부에 접두사 디렉토리를 생성합니다. 이로 인해 경로는 일관되게 유지될 수 있으며, 웹 서버를 사용하고 Django를 통해 URL 조회를 라우팅하지 않을 때도 마찬가지입니다.

다음으로, 정적 파일이 예상대로 로드되지 않을 때 문제 해결에 유용한 관리 명령어를 살펴보겠습니다: `findstatic`.

findstatic 명령어

`staticfiles` 애플리케이션은 또 하나의 관리 명령어를 제공합니다: `findstatic`. 이 명령어는 정적 파일에 대한 상대 경로를 입력할 수 있게 해주며(정적 템플릿 태그 내에서 사용되는 것과 동일) Django는 해당 파일이 어디에 위치했는지 알려줍니다. 또한 검색 중인 디렉토리를 출력하는 자세한 모드로 사용할 수도 있습니다.

참고

당신은 자세함 또는 자세한 모드의 개념에 익숙하지 않을 수 있습니다. 더 높은 자세함(또는 단순히 자세한 모드를 켜는 것)은 명령어가 더 많은 출력을 생성하게 만듭니다. 많은 명령줄 애플리케이션은 더 많거나 적은 자세함으로 실행될 수 있습니다. 이는 사용 중인 프로그램을 디버깅할 때 유용할 수 있습니다. 자세한 모드의 예를 보려면, Python 셸을 자세한 모드로 실행해 볼 수 있습니다. `python -v`를 입력하고(그냥 `python` 대신) Enter를 누르세요. Python은 자세한 모드로 시작되며, 가져오는 모든 파일의 경로를 출력합니다.

This command is mostly useful for debugging/troubleshooting purposes. If the wrong file is loading, or a particular file can't be found, you can use this command to try to find out why. The command will display which file on disk is being loaded for a specific path, or let you know that the file can't be found and what directories were searched.

This can help solve issues where multiple files have the same name, and the precedence is not what you expect. See the *FileSystemFinder* section for a note about precedence in the `STATICFILES_DIRS` setting. You may also see that Django is not searching in a directory you expect for the file, in which case the static directory might need to be added to the `STATICFILES_DIRS` setting.

In the next exercise, you will execute the `findstatic` management command so that you are familiar with what the outputs are for good (file found correctly) and bad (file missing) scenarios.

Exercise 5.05 – finding files using `findstatic`

You will now run the `findstatic` command with a variety of options and understand what its output means. First, we will use it to find a file that exists and see that it displays the path to the file. Then, we will try to find a file that doesn't exist and check the error that is output. After that, we will repeat this process with multiple levels of verbosity and different ways of interacting with the command.

While this exercise will not make changes to or progress the Bookr project, it is good to be familiar with the command in case you need to use it when working on your own Django applications.

Let's get started:

1. Open a Terminal and navigate to the `business_site` project directory.
2. Execute the `findstatic` command with no options. It will output some help explaining how it is used:

```
python manage.py findstatic
```

The following help output will be displayed:

```
usage: manage.py findstatic [-h] [--first] [--version]
                             [-v {0,1,2,3}]
                             [--settings SETTINGS]
                             [--pythonpath PYTHONPATH]
                             [--traceback] [--no-color]
                             [--force-color]
```

이 명령은 주로 디버깅/문제 해결 목적으로 유용합니다. 잘못된 파일이 로드되거나 특정 파일을 찾을 수 없는 경우, 이 명령을 사용하여 그 이유를 알아낼 수 있습니다. 이 명령은 특정 경로에 대해 어떤 파일이 디스크에서 로드되고 있는지를 표시하거나, 파일을 찾을 수 없고 어떤 디렉토리를 검색했는지 알려줍니다.

이것은 여러 파일이 동일한 이름을 가질 때 문제를 해결하는 데 도움이 될 수 있으며, 우선 순위가 예상과 다를 수 있습니다. STATICFILES_DIRS 설정에서 우선 순위에 대한 주석을 보려면 FileSystemFinder 섹션을 참조하십시오. 또한 Django가 파일을 위해 예상하는 디렉토리에서 검색하지 않는 것을 볼 수 있으며, 이 경우 정적 디렉토리를 STATICFILES_DIRS 설정에 추가해야 할 수 있습니다.

다음 연습에서는 findstatic 관리 명령을 실행하여 올바른(파일이 올바르게 발견됨) 및 잘못된(파일이 누락됨) 시나리오에 대한 출력이 무엇인지 익숙해질 것입니다.

연습 5.05 – findstatic을 사용하여 파일 찾기

이제 다양한 옵션으로 findstatic 명령을 실행하고 그 출력이 의미하는 바를 이해할 것입니다. 먼저, 존재하는 파일을 찾기 위해 사용하고 idx_9cd5459f 파일의 경로를 표시하는 것을 볼 것입니다. 그런 다음 존재하지 않는 파일을 찾으려고 시도하고 출력되는 오류를 확인할 것입니다. 그 후, 여러 수준의 자세한 정보와 명령과 상호작용하는 다양한 방법으로 이 과정을 반복할 것입니다.

이 연습은 Bookr 프로젝트에 변경을 가하거나 진행하지 않겠지만, 자신의 Django 애플리케이션 작업 시 필요할 경우 이 명령에 익숙해지는 것이 좋습니다.

시작해 봅시다:

1. 터미널을 열고 business_site 프로젝트 디렉토리로 이동합니다.
2. 옵션 없이 findstatic 명령을 실행합니다. 사용 방법에 대한 도움말이 출력됩니다:

```
python manage.py findstatic
```

다음과 같은 도움말 출력이 표시됩니다:

```
usage: manage.py findstatic [-h] [--first] [--version]
                             [-v {0,1,2,3}]
                             [--settings SETTINGS]
                             [--pythonpath PYTHONPATH]
                             [--traceback] [--no-color]
                             [--force-color]
```

```
[--skip-checks]
staticfile [staticfile ...]
manage.py findstatic: error: Enter at least one label.
```

3. You can find one or more files at a time. Let's start with one that we know exists: `main.css`. We'll use the `findstatic` management command to locate it on disk by entering the following command:

```
python3 manage.py findstatic main.css
```

The preceding command outputs the path at which `main.css` was found:

```
Found 'main.css' here:
/Users/chrisguest/business_site/static/main.css
```

Your full path will be different (unless you are also called Chris), but you can see that when Django locates `main.css` in a request, it will load the `main.css` file from the project static directory.

This can be useful if a third-party application you have installed has not namespaced its static files correctly and is conflicting with one of your files.

4. Let's try finding a file that does not exist, `logo.png`:

```
python manage.py findstatic logo.png
```

Django will display an error stating that the file could not be found:

```
No matching file found for 'logo.png'.
```

Django is unable to locate this file because we have namespaced it – we must include the full relative path, the same way we used it in the static template tag.

5. Try finding the `logo.png` again, but this time using the full path:

```
python manage.py findstatic landing/logo.png
```

Django can find the file now:

```
Found 'landing/logo.png' here:
/Users/chrisguest/business_site/landing/static/landing/logo.png
```

```
[--skip-checks]
staticfile [staticfile ...]
manage.py findstatic: 오류: 최소한 하나의 레이블을 입력하세요.
```

3. 한 번에 하나 이상의 파일을 찾을 수 있습니다. 존재하는 파일부터 시작해 보겠습니다: main.css. 다음 명령어를 입력하여 findstatic 관리 명령을 사용하여 디스크에서 찾겠습니다:

```
python3 manage.py findstatic main.css
```

이전 명령은 main.css가 발견된 경로를 출력합니다:

```
'main.css'를 여기에서 찾았습니다
다:/Users/chrisquest/business_site/static/main.
css
```

전체 경로는 다를 것입니다(당신도 Chris가 아니라면), 하지만 Django가 요청에서 main.css를 찾을 때 프로젝트 정적 디렉토리에서 main.css 파일을 로드한다는 것을 알 수 있습니다.

타사 애플리케이션이 네임스페이스를 올바르게 설정하지 않은 경우 유용할 수 있습니다. 정적 파일이 올바르게 설정되지 않아 귀하의 파일 중 하나

와 충돌하고 있습니다. 찾으려고 해봅시다, logo.png:

```
python manage.py findstatic logo.png
```

Django는 파일을 찾을 수 없다는 오류를 표시합니다:

```
'logo.png'에 대한 일치하는 파일을 찾을 수 없습니다.
```

Django는 이 파일을 찾을 수 없습니다. 왜냐하면 우리는 네임스페이스를 설정했기 때문입니다. 정적 템플릿 태그에서 사용한 것과 같은 방식으로 전체 상대 경로를 포함해야 합니다.

5. 다시 logo.png를 찾아보세요. 하지만 이번에는 전체 경로를 사용하세요:

```
python manage.py findstatic landing/logo.png
```

이제 Django가 파일을 찾을 수 있습니다:

```
여기에서 'landing/logo.png'를 찾았습니다
다:/Users/chrisquest/business_site/landing/static/landing/logo.p
ng
```

6. You can find multiple files at once by adding each file as an argument:

```
python manage.py findstatic landing/logo.png  
missing-file.js main.css
```

The location status for each file is shown as follows:

```
No matching file found for 'missing-file.js'.  
Found 'landing/logo.png' here:  
  /Users/chrisguest/business_site/landing/static/landing/logo.png  
Found 'main.css' here:  
  /Users/chrisguest/business_site/static/main.css
```

7. The command can be executed with a verbosity of 0, 1, or 2. By default, it executes at verbosity 1. To set the verbosity, use the `--verbosity` or `-v` flag. Decreasing the verbosity to 0 only outputs the paths it locates without any extra information. No errors are displayed for missing paths:

```
python manage.py findstatic -v0 landing/logo.png missing-file.js main.css
```

The output shows only found paths – notice that no error is shown for the missing file, `missing-file.js`:

```
/Users/chrisguest/business_site/landing/static/landing/logo.png  
/Users/chrisguest/business_site/static/main.css
```

This level of verbosity can be useful if you are piping the output to another file or command.

8. To get more information about which directories Django is searching in for the file you have requested, increase the verbosity to 2:

```
python manage.py findstatic -v2 landing/logo.png  
missing-file.js main.css
```

The output contains much more information, including the directories that have been searched for the requested file. You can see that as the admin application is installed, Django is also searching in the Django admin application directory for static files:

6. 각 파일을 인수로 추가하여 한 번에 여러 파일을 찾을 수 있습니다:

```
python manage.py findstatic landing/logo.png
missing-file.js main.css
```

각 파일의 위치 상태는 다음과 같이 표시됩니다:

```
'missing-file.js'에 대한 일치하는 파일을 찾을 수 없습니다.
여기에서 'landing/logo.png'를 찾았습니다
  다:/Users/chrisquest/business_site/landing/static/landing/logo.p
ng
'main.css'를 여기에서 찾았습니다
  다:/Users/chrisquest/business_site/static/main.
css
```

7. 명령은 0, 1 또는 2의 상세 수준으로 실행할 수 있습니다. 기본적으로는 다음과 같이 실행됩니다.
 verbosity 1. verbosity를 설정하려면 --verbosity 또는 -v 플래그를 사용하세요.
 verbosity를 0으로 낮추면 추가 정보 없이 찾은 경로만 출력됩니다. 경로가 없을
 경우 오류는 표시되지 않습니다:

```
python manage.py findstatic -v0 landing/logo.png missing-file.js main.css
```

출력은 발견된 경로만 보여줍니다 – 누락된 파일 missing-file.js에 대한 오류가
 표시되지 않는 것을 주목하세요:

```
/Users/chrisquest/business_site/landing/static/landing/logo.png
/Users/chrisquest/business_site/static/main.css
```

이 정도의 자세한 설명은 출력을 다른 파일이나 명령어로 파이핑할 때 유용
 할 수 있습니다.

8. 요청한 파일을 찾기 위해 Django가 검색하는 디렉터리에 대한 자세한 정보를 얻
 으려면, 상세 수준을 2로 높이세요:

```
python manage.py findstatic -v2 landing/logo.png
missing-file.js main.css
```

출력에는 요청된 파일을 찾기 위해 검색된 디렉토리를 포함하여 idx_3b6384a1에 대한 훨씬
 더 많은 정보가 포함되어 있습니다. 관리 애플리케이션이 설치되면 Django가 정적 파
 일을 위해 Django 관리 애플리케이션 디렉토리에서도 검색하고 있음을 알 수 있습니다:

```
> python manage.py findstatic -v2 landing/logo.png missing-file.js main.css
No matching file found for 'missing-file.js'.

Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
Found 'landing/logo.png' here:
/Users/chrisquest/business_site/landing/static/landing/logo.png
Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
Found 'main.css' here:
/Users/chrisquest/business_site/static/main.css
Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
```

Figure 5.17 – `findstatic` executed with verbosity 2, showing exactly which directories were searched

The `findstatic` command is not something that you will use day-to-day when working with Django, but it is useful to know about when you're trying to troubleshoot problems with static files. Here, we saw the command's output, the full path to a file that existed, as well as the error messages when files did not exist. We also ran the command and supplied multiple files at once, and saw that information about all the files was output. Finally, we ran the command with different levels of verbosity. The `-v0` flag suppressed errors about missing files. `-v1` was the default and displayed found paths and errors. Increasing the verbosity using the `-v2` flag also printed out the directories that were being searched through for a particular static file.

In the next section, we'll take a look at caching static files and how Django helps us ensure our users are served the latest versions.

```
> python manage.py findstatic -v2 landing/logo.png missing-file.js main.css
No matching file found for 'missing-file.js'.

Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
Found 'landing/logo.png' here:
/Users/chrisquest/business_site/landing/static/landing/logo.png
Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
Found 'main.css' here:
/Users/chrisquest/business_site/static/main.css
Looking in the following locations:
/Users/chrisquest/business_site/static
/Users/chrisquest/business_site/static_images
/Users/chrisquest/business_site/static_css
/Users/chrisquest/.pyenv/versions/djangoenv/lib/python3.12/site-packages/djang
o/contrib/admin/static
/Users/chrisquest/business_site/landing/static
```

그림 5.17 - *verbosity 2*로 실행된 *findstatic*, 검색된 디렉토리를 정확히 보여줌

findstatic 명령은 Django로 작업할 때 매일 사용하는 것은 아니지만, 정적 파일 문제를 해결하려고 할 때 알아두면 유용합니다. 여기서 우리는 명령의 출력, 존재하는 파일의 전체 경로와 존재하지 않는 파일에 대한 오류 메시지를 보았습니다. 또한 명령을 실행하고 여러 파일을 한 번에 제공했으며, 모든 파일에 대한 정보가 출력되는 것을 보았습니다. 마지막으로, 우리는 다양한 수준의 *verbosity*로 명령을 실행했습니다. *-v0* 플래그는 누락된 파일에 대한 오류를 억제했습니다. *-v1*은 기본값이며 발견된 경로와 오류를 표시했습니다. *-v2* 플래그를 사용하여 *verbosity*를 증가시키면 특정 정적 파일을 검색하는 데 사용된 디렉토리도 출력되었습니다.

다음 섹션에서는 정적 파일 캐싱과 Django가 사용자에게 최신 버전을 제공하는 데 어떻게 도움을 주는지 살펴보겠습니다.

Serving the latest files (for cache invalidation)

If you are not familiar with caching, the basic idea is that some operations can take a long time to perform. We can speed up a system by storing the results of the operation in a place that is faster to access so that the next time we need them, they can be retrieved quickly. The "operation" that takes a long time can be anything – a function that takes a long time to run, an image that takes a long time to render, or a large asset that takes a long time to download over the internet. We are interested in this last scenario.

You might have noticed that the first time you ever visit a particular website, it can be slow to load, but then the next time, it loads much faster. This is because your browser has cached some (or all) of the static files the site needs to load.

To use our business site as an example, we have a page that includes the `logo.png` file. The first time we visit the business site, we must download the dynamic HTML, which is small and quick to transfer. Our browser parses the HTML and sees that the `logo.png` file should be included. It can then download this file too, which is much larger and can take longer to download. Note that this scenario assumes that our business site is now hosted on a remote server and not on our local machine – which is very fast for us to access.

If the web server has been set up correctly, your browser will store `logo.png` on your computer. The next time we visit the *landing* page (or indeed any page that includes the `logo.png` file), our browser will recognize the URL and can load the file from disk instead of having to download it again, thus speeding up the browsing experience.

Note

We said that the browser will cache "if the web server is set up correctly." What does this mean? The frontend web server should be configured to send special HTTP headers as part of a static file response. It can send a `Cache-Control` header, which can have values such as `no-cache` (the file should never be cached – that is, the latest version should be requested every time), or `max-age=seconds` (the file should only be downloaded again if it was last retrieved more than *seconds* ago). The response could also contain the `Expires` header, with the value being a date. The file is considered to be "stale" once this date has been reached, and at that point, the new version should be requested.

One of the hardest problems in computer science is cache invalidation. For instance, if we change `logo.png`, how does our browser know it should download the new version?

캐시 무효화를 위한 최신 파일 제공

캐싱에 익숙하지 않다면, 기본 아이디어는 일부 작업이 수행하는 데 오랜 시간이 걸릴 수 있다는 것입니다. 우리는 작업의 결과를 더 빠르게 접근할 수 있는 장소에 저장함으로써 시스템을 가속화할 수 있습니다. 그래서 다음 번에 우리가 그것들을 필요로 할 때, 빠르게 검색할 수 있습니다. 오랜 시간이 걸리는 "작업"은 무엇이든 될 수 있습니다. 실행하는 데 오랜 시간이 걸리는 함수, 렌더링하는 데 오랜 시간이 걸리는 이미지, 또는 인터넷을 통해 다운로드하는 데 오랜 시간이 걸리는 큰 자산 등이 있습니다. 우리는 이 마지막 시나리오에 관심이 있습니다.

특정 웹사이트를 처음 방문할 때 로드 속도가 느릴 수 있지만, 다음 번에는 훨씬 더 빠르게 로드된다는 것을 눈치챌 것입니다. 이는 브라우저가 사이트가 로드하는 데 필요한 일부(또는 모든) 정적 파일을 캐시했기 때문입니다.

우리의 비즈니스 사이트를 예로 들면, logo.png 파일이 포함된 페이지가 있습니다. 비즈니스 사이트를 처음 방문할 때, 우리는 작고 빠르게 전송할 수 있는 동적 HTML을 다운로드해야 합니다. 브라우저는 HTML을 파싱하고 logo.png 파일이 포함되어야 한다고 인식합니다. 그런 다음 이 파일도 다운로드할 수 있으며, 이는 훨씬 더 크고 다운로드하는 데 더 오랜 시간이 걸릴 수 있습니다. 이 시나리오는 우리의 비즈니스 사이트가 이제 원격 서버에 호스팅되고 있으며, 로컬 머신이 아닌 경우를 가정합니다. 로컬 머신은 우리가 접근하기에 매우 빠릅니다.

웹 서버가 올바르게 설정되었다면, 브라우저는 logo.png를 컴퓨터에 저장할 것입니다. 다음 번에 랜딩 페이지(또는 logo.png 파일이 포함된 모든 페이지)를 방문할 때, 브라우저는 URL을 인식하고 파일을 다시 다운로드할 필요 없이 디스크에서 로드할 수 있으므로 브라우징 경험이 빨라집니다.

참고

우리는 브라우저가 "웹 서버가 올바르게 설정되었다면 캐시할 것"이라고 말했습니다. 이것은 무슨 의미인가요? 프론트엔드 웹 서버는 정적 파일 응답의 일환으로 특별한 HTTP 헤더를 보내도록 구성되어야 합니다. Cache-Control 헤더를 보낼 수 있으며, 이 헤더는 no-cache(파일은 절대 캐시되지 않아야 함 - 즉, 최신 버전이 매번 요청되어야 함) 또는 max-age=seconds(파일은 마지막으로 검색된 지 초과된 경우에만 다시 다운로드되어야 함)와 같은 값을 가질 수 있습니다. 응답은 또한 Expires 헤더를 포함할 수 있으며, 그 값은 날짜가 됩니다. 이 날짜에 도달하면 파일은 "오래된" 것으로 간주되며, 그 시점에서 새로운 버전을 요청해야 합니다.

컴퓨터 과학에서 가장 어려운 문제 중 하나는 캐시 무효화입니다. 예를 들어, logo.png를 변경하면 브라우저는 새로운 버전을 다운로드해야 한다는 것을 어떻게 알까요?

The only surefire way of knowing it had changed would be to download the file again and compare it with the version we had already saved every time. Of course, this defeats the purpose of caching since we would still be downloading every time the file changed (or not).

We can cache for an arbitrary or server-specified amount of time, but if the static file changed before that time was up, we wouldn't know. We would use the old version until we considered it expired, at which time we'd download the new version. If we had a 1-week expiry and the static file changed the next day, we'd still be using the old one for 6 days. Of course, the browser can be made to reload the page without using the cache (how this is done depends on the browser; for example, *Cmd/Ctrl + Shift + R*) if you want to forcefully download all static assets again.

There is no need to try to cache our dynamic responses (rendered templates). Since they are designed to be dynamic, we would want to make sure that the user gets the latest version on every page load, so they should not be cached. They are also quite a small size (compared to assets such as images), so there is not much speed advantage when caching them.

Django provides a built-in solution. During the `collectstatic` phase, when the files are copied, Django can append a hash of their content to the filename. For example, the source file, `logo.png`, will be copied to `static_production_test/landing/logo.f30ba08c60ba.png`. This is only done when using the `ManifestFilesStorage` storage engine.

Since the filename only changes when the content changes, the browser will always download the new content.

Using `ManifestFilesStorage` is just one way of invalidating caches. There may be other options that are more suitable for your particular application.

Note

A hash is a one-way function that generates a string of a fixed length, regardless of the length of the input. There are several different hash functions available, and Django uses **MD5** for content hashing. While no longer cryptographically secure, it is adequate for this purpose. To illustrate the fixed length property, the MD5 hash of the a string is `0cc175b9c0f1b6a831c399e269772661`. The MD5 hash of the a much longer string is `69fc4316c18cdd594a58ec2d59462b97`. They are both 32 characters long.

You can choose the storage engine by changing the `STATICFILES_STORAGE` value in `settings.py`. This is a string with a dotted path to the module and class to use. The class that implements the hash-addition functionality is `django.contrib.staticfiles.storage.ManifestStaticFilesStorage`.

변경되었는지 확실히 아는 유일한 방법은 파일을 다시 다운로드하고 우리가 이미 저장한 버전과 비교하는 것이었습니다. 물론, 이는 캐싱의 목적을 무색하게 하며, 파일이 변경되었든 아니든 매번 다운로드해야 합니다.

우리는 임의의 시간 또는 서버에서 지정한 시간 동안 캐시할 수 있지만, 그 시간이 지나기 전에 정적 파일이 변경되면 알 수 없습니다. 우리는 그것이 만료되었다고 생각할 때까지 이전 버전을 사용할 것이며, 그때 새로운 버전을 다운로드할 것입니다. 만약 1주일의 만료 기간이 있고 정적 파일이 다음 날 변경되면, 우리는 여전히 6일 동안 이전 파일을 사용할 것입니다. 물론, 브라우저는 캐시를 사용하지 않고 페이지를 새로 고칠 수 있도록 설정할 수 있습니다(이 방법은 브라우저에 따라 다릅니다; 예를 들어, Cmd/Ctrl + Shift + R) 모든 정적 자산을 다시 강제로 다운로드하려면 이 방법을 사용하세요.

동적 응답(렌더링된 템플릿)을 캐시하려고 할 필요는 없습니다. 이들은 동적으로 설계되었기 때문에 사용자가 매 페이지 로드 시 최신 버전을 받을 수 있도록 해야 하며, 따라서 캐시되어서는 안 됩니다. 또한 이들은 이미지와 같은 자산에 비해 크기가 작기 때문에 캐시할 때 속도 이점이 크지 않습니다.

Django는 내장 솔루션을 제공합니다. collectstatic 단계에서 파일이 복사될 때, Django는 파일 이름에 내용의 해시를 추가할 수 있습니다. 예를 들어, 원본 파일 logo.png는 static_production_test/landing/logo.f30ba08c60ba.png로 복사됩니다. 이는 ManifestFilesStorage 저장 엔진을 사용할 때만 수행됩니다.

파일 이름은 내용이 변경될 때만 변경되므로, 브라우저는 항상 새로운 내용을 다운로드 합니다.

ManifestFilesStorage를 사용하는 것은 캐시를 무효화하는 한 가지 방법일 뿐입니다. 특정 애플리케이션에 더 적합한 다른 옵션이 있을 수 있습니다.

참고

해시는 입력의 길이에 관계없이 고정 길이의 문자열을 생성하는 단방향 함수입니다. 여러 가지 해시 함수가 있으며, Django는 콘텐츠 해싱에 MD5를 사용합니다. 더 이상 암호학적으로 안전하지는 않지만, 이 목적에는 적합합니다. 고정 길이 속성을 설명하기 위해, 문자열의 MD5 해시는 0cc175b9c0f1b6a831c399e269772661입니다. 훨씬 긴 문자열의 MD5 해시는 69fc4316c18cdd594a58ec2d59462b97입니다. 두 해시는 모두 32자입니다.

settings.py에서 STATICFILES_STORAGE 값을 변경하여 저장소 엔진을 선택할 수 있습니다. 이것은 사용할 모듈과 클래스에 대한 점으로 구분된 경로를 가진 문자열입니다. 해시 추가 기능을 구현하는 클래스는 django.contrib.staticfiles.storage.ManifestStaticFilesStorage입니다.

Using this storage engine doesn't require you to make any changes to your HTML templates, provided you are including static assets with the `static` template tag. Django generates a manifest file (`staticfiles.json`, in JSON format) that contains a mapping between the original filename and the hashed filename. It will automatically insert the hashed filename when using the `static` template tag. If you are including your static files without using the `static` tag and instead just manually inserting the static URL, then your browser will attempt to load the non-hashed path, and the URL will not automatically be updated when the cache should be invalidated.

For example, you can include `logo.png` with the `static` tag:

```

```

When the page is rendered, the latest hash will be retrieved from `staticfiles.json` and the output will be like this:

```

```

Whereas, if we had not used the `static` tag and instead hardcoded the path, it would always appear as written:

```

```

Since this does not contain a hash, our browser will not see the path changing and thus never attempt to download the new file.

Django retains the previous version of files with the old hash when running `collectstatic`, so older versions of your application can still refer to it if they need to. The latest version of the file is also copied with no hash so that non-Django applications can refer to it without needing to look up the hash.

In the next exercise, we will change our project settings to use the `ManifestFilesStorage` engine, and then run the `collectstatic` management command. This will copy all the static assets as in *Exercise 5.04 – collecting static files for production*; however, they will now have their hash included in the filename.

이 idx_9c751a89 저장 엔진을 사용하면 HTML 템플릿에 idx_f6a75adb 변경을 할 필요가 없습니다. 정적 자산을 정적 템플릿 태그와 함께 포함하고 있다면 말이죠. Django는 원래 파일 이름과 해시된 파일 이름 간의 매핑을 포함하는 매니페스트 파일 (staticfiles.json, JSON 형식)을 생성합니다. 정적 템플릿 태그를 사용할 때 해시된 파일 이름을 자동으로 삽입합니다. 정적 태그를 사용하지 않고 정적 URL을 수동으로 삽입하는 경우, 브라우저는 해시되지 않은 경로를 로드하려고 시도하며, 캐시가 무효화되어야 할 때 URL이 자동으로 업데이트되지 않습니다.

예를 들어, 정적 태그를 사용하여 logo.png를 포함할 수 있습니다:

```

```

페이지가 렌더링될 때, 최신 해시는 staticfiles.json에서 검색되며 출력은 다음과 같습니다:

```

```

반면, 정적 태그를 사용하지 않고 경로를 하드코딩했다면 항상 다음과 같이 나타납니다:

```

```

해시가 포함되어 있지 않기 때문에, 브라우저는 경로가 변경되는 것을 보지 못하고 따라서 새로운 파일을 다운로드하려고 시도하지 않습니다.

장고는 collectstatic을 실행할 때 이전 해시가 있는 파일의 이전 버전을 유지하므로, 필요할 경우 애플리케이션의 이전 버전이 여전히 이를 참조할 수 있습니다. 파일의 최신 버전은 해시 없이 복사되어 비장고 애플리케이션 idx_3c3efeb0이 해시를 조회할 필요 없이 이를 참조할 수 있습니다.

다음 연습인 idx_cc52af5a에서 우리는 프로젝트 설정을 ManifestFilesStorage 엔진을 사용하도록 변경한 다음 collectstatic 관리 명령을 실행할 것입니다. 이는 Exercise 5.04와 같이 모든 정적 자산을 복사할 것입니다 - 프로덕션을 위한 정적 파일 수집; 그러나 이제 파일 이름에 해시가 포함될 것입니다.

Exercise 5.06 – exploring the ManifestFilesStorage storage engine

In this exercise, you will temporarily update `settings.py` to use `ManifestFilesStorage`, then run `collectstatic` to see how the files are generated with a hash:

1. In PyCharm (still in the `business_site` project), open `settings.py` and add a `STATICFILES_STORAGE` setting at the bottom of the file:

```
STATICFILES_STORAGE =  
"django.contrib.staticfiles.storage.ManifestStaticFilesStorage"
```

The completed file should look like the one placed in the `Chapter05` folder in this book's GitHub repo.

2. Open a terminal, navigate to the `business_site` project directory, and run the `collectstatic` command like before:

```
python manage.py collectstatic
```

If your `static_production_test` directory is not empty (which will probably be the case as files were moved there in *Exercise 5.04 – collecting static files for production*), then you will be prompted to allow the overwrite of the existing files:

```
> python manage.py collectstatic  
  
You have requested to collect static files at the destination  
location as specified in your settings:  
  
    /Users/chrisquest/business_site/static_production_test  
  
This will overwrite existing files!  
Are you sure you want to do this?  
  
Type 'yes' to continue, or 'no' to cancel:
```

Figure 5.18 – Prompt to allow overwrite during static collection

Just type `yes` and then press `Enter` to allow the overwrite.

The output from this command will tell you the number of files that were copied, as well as the number that were processed and had the hash added to the filename:

```
0 static files copied to '/Users/chrisquest/business_site/  
static_production_test', 129 unmodified, 101 post-processed.
```

연습 5.06 – ManifestFilesStorage 스토리지 엔진 탐색

이 idx_c18b9b1cexercise에서는 settings.py를 일시적으로 업데이트하여 ManifestFilesStorage를 사용한 다음 collectstatic을 실행하여 파일이 해시와 함께 생성되는 방식을 확인합니다:

1. PyCharm(여전히 business_site 프로젝트에서)에서 settings.py를 열고 파일 하단에 STATICFILES_STORAGE 설정을 추가합니다:

```
STATICFILES_STORAGE =
    "django.contrib.staticfiles.storage.ManifestStaticFilesStorage"
```

완성된 파일은 이 책의 GitHub 리포지토리의 Chapter05 폴더에 있는 파일과 유사해야 합니다.

2. 터미널을 열고 business_site 프로젝트 디렉토리로 이동한 다음 이전과 같이 collectstatic 명령을 실행합니다:

```
python manage.py collectstatic
```

정적_production_test 디렉토리가 비어 있지 않다면(아마도 5.04 연습에서 정적 파일을 수집하기 위해 파일이 이동되었기 때문에 그럴 것입니다), 기존 파일의 덮어쓰기를 허용하라는 메시지가 표시됩니다:

```
> python manage.py collectstatic

You have requested to collect static files at the destination
location as specified in your settings:

    /Users/chrisquest/business_site/static_production_test

This will overwrite existing files!
Are you sure you want to do this?

Type 'yes' to continue, or 'no' to cancel:
```

그림 5.18 – 정적 수집 중 덮어쓰기를 허용하라는 메시지 그냥 yes를 입력한 다음 Enter를 눌러 덮어쓰기를 허용합니다.

이 명령의 출력 idx_c33dc51a는 복사된 파일 수와 처리된 파일 수, 그리고 파일 이름에 해시가 추가된 파일 수를 알려줍니다:

```
0개의 정적 파일이 '/Users/chrisquest/business_site/static_production_test'에
복사되었고, 129개는 수정되지 않았으며, 101개는 후처리되었습니다.
```

Since you haven't changed any files since we last ran `collectstatic`, no files have been copied. Instead, Django post-processes the files (28 of them) – that is, it generates their hashes and appends them to the filename.

3. The static files were copied into the `static_production_test` directory as they were before; however, there are now two copies of each file: one named with the hash and one without. For example, `static/main.css` has been copied to `static_production_test/main.856c74fb7029.css` (this filename might be different if your CSS file contents differ; for example, if it has extra spaces or newlines):



Figure 5.19 – Expanded `static_production_test` directory with hashed filenames

Figure 5.19 shows the expanded `static_production_test` directory layout. You can see two copies of each file's static file, as well as the `staticfiles.json` manifest file. To take `logo.png` as an example, you can see that `landing/static/landing/logo.png` has been copied to `static_production_test/landing/logo.0745d1077c9e.png`.

Let's make a change to the `main.css` file and see how the hash changes.

4. Add some blank lines to the end of the file and save it (make sure that you edit the source file and not the one that was already copied to the `static_production_test` directory). This won't change the effect of the CSS, but the change in the file will affect its hash. Rerun the `collectstatic` command in a terminal:

```
python3 manage.py collectstatic
```

마지막으로 collectstatic을 실행한 이후로 파일을 변경하지 않았기 때문에, 복사된 파일이 없습니다. 대신, Django는 파일(28개)을 후처리합니다. 즉, 해시를 생성하고 파일 이름에 추가합니다.

3. 정적 파일은 이전과 같이 static_production_test 디렉토리에 복사되었습니다. 그러나 이제 각 파일의 두 개의 복사본이 있습니다: 하나는 해시가 포함된 이름이고, 다른 하나는 그렇지 않습니다. 예를 들어, static/main.css는 static_production_test/main.856c74fb7029.css로 복사되었습니다(이 파일 이름은 CSS 파일 내용이 다르면 다를 수 있습니다; 예를 들어, 여분의 공백이나 줄 바꿈이 있는 경우):

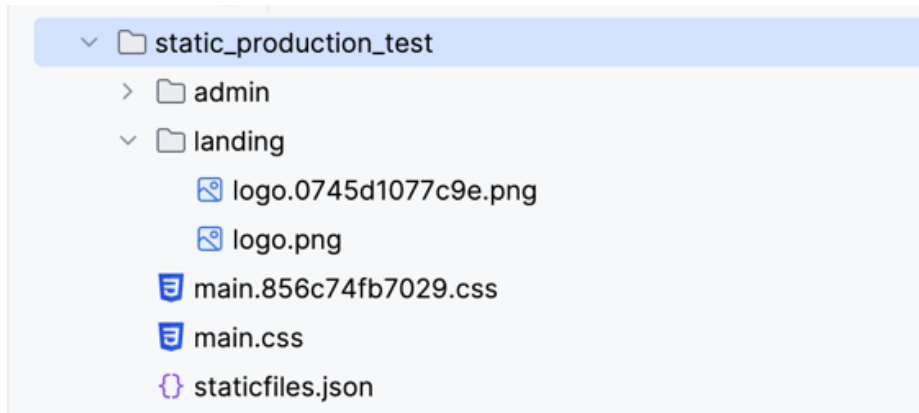


그림 5.19 - 해시된 파일 이름이 있는 확장된 static_production_test 디렉토리

그림 5.19는 확장된 static_production_test 디렉토리 레이아웃을 보여줍니다. 각 파일의 정적 파일 두 개와 staticfiles.json 매니페스트 파일을 볼 수 있습니다. logo.png를 예로 들면, landing/static/landing/logo.png가 static_production_test/landing/logo.0745d1077c9e.png로 복사된 것을 볼 수 있습니다.

main.css 파일을 변경하고 해시가 어떻게 변하는지 봅시다.

4. 파일 끝에 빈 줄을 추가하고 저장합니다(정적_production_test 디렉토리에 이미 복사된 파일이 아닌 소스 파일을 편집하는지 확인하세요). 이것은 CSS의 효과를 변경하지 않지만, 파일의 변경은 해시에 영향을 미칩니다. 터미널에서 collectstatic 명령을 다시 실행하세요:

```
python3 manage.py collectstatic
```

Once again, you may have to enter yes to confirm the overwrite, which will give you the following output:

```
You have requested to collect static files at the destination location as
specified in your settings:
    /Users/chrisgust/business_site/static_production_test

This will overwrite existing files!
Are you sure you want to do this?

Type 'yes' to continue, or 'no' to cancel: yes

1 static file copied to '/Users/chrisgust/business_site/
static_production_test', 128 unmodified, 101 post-processed.
```

Since only one file was changed, only one static file was copied (`main.css`).

5. Look inside the `static_production_test` directory again. You should see that the old file with the old hash was retained and that a new file with a new hash has been added:

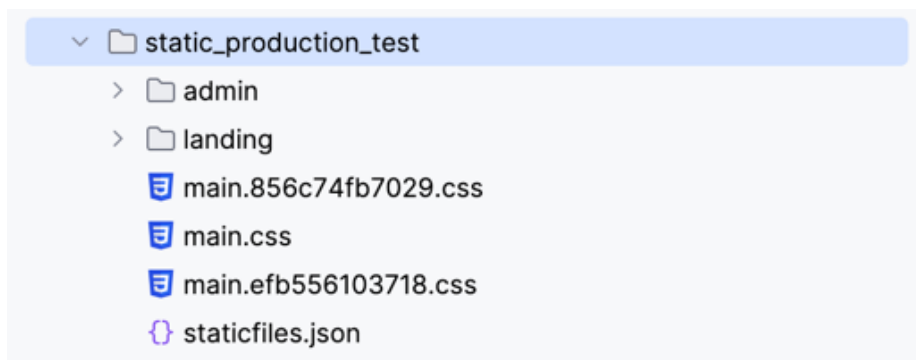


Figure 5.20 – Another `main.css` file with the latest hash was added

In this case, we have `main.856c74fb7029.css` (existing), `main.efb556103718.css` (new), and `main.css`. Your hashes may differ.

The `main.css` file (no hash) always contains the newest content – that is, the contents of the `main.efb556103718.css` and `main.css` files are identical. During the execution of `collectstatic`, Django will copy the file with a hash, as well as without a hash.

6. Now, examine the `staticfiles.json` file that Django generates. This is the mapping that allows Django to look up the hashed path from the normal path. Open

다시 한 번, 덮어쓰기를 확인하기 위해 yes를 입력해야 할 수도 있으며, 이는 다음 출력을 제공합니다:

설정에 지정된 대로 목적지 위치에서 정적 파일을 수집하도록 요청했습니다:

```
/Users/chrisquest/business_site/static_production_test
```

이것은 기존 파일을 덮어씁니다! 정말로
이렇게 하시겠습니까?

계속하려면 'yes'를 입력하고, 취소하려면 'no'를 입력하세요: yes

1개의 정적 파일이 '/Users/chrisquest/business_site/static_production_test'에
복사되었고, 128개는 수정되지 않았으며, 101개는 후처리되었습니다.

하나의 파일만 변경되었으므로 하나의 정적 파일만 복사되었습니다 (main.css).

5. static_production_test 디렉토리를 다시 확인하세요. 이전 해시를 가진 oldfile이 유지되고 새로운 해시를 가진 새 파일이 추가된 것을 볼 수 있어야 합니다:

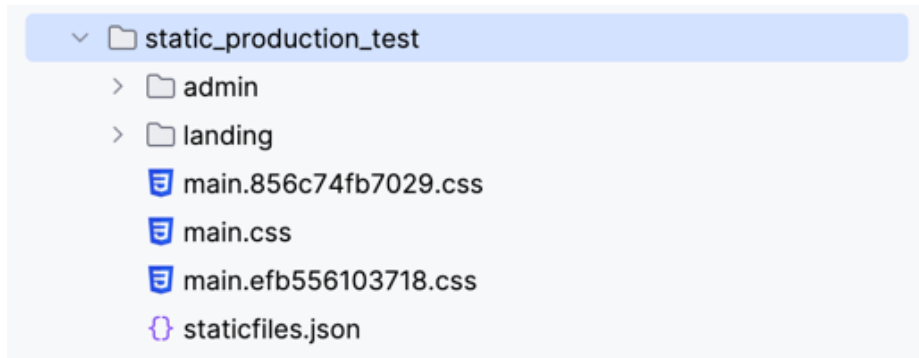


그림 5.20 - 최신 해시를 가진 또 다른 main.css 파일이 추가되었습니다

이 경우, 우리는 main.856c74fb7029.css (기존), main.efb556103718.css(새로운), 그리고 main.css를 가지고 있습니다. 당신의 해시는 다를 수 있습니다.

main.css 파일(해시 없음)은 항상 최신 내용을 포함합니다 – 즉, main.efb556103718.css와 main.css 파일의 내용이 동일합니다. collectstatic 실행 중에 Django는 해시가 있는 파일과 해시가 없는 파일을 모두 복사합니다.

6. 이제 Django가 생성하는 staticfiles.json 파일을 살펴보세요. 이것은 Django가 일반 경로에서 해시된 경로를 조회할 수 있게 해주는 매핑 idx_28109346입니다. 열기

`static_production_test/staticfiles.json`. All the content may appear in one line. If it does, enable text soft wrapping from the **View** menu | **Active Editor** | **Soft Wrap**. Scroll to the end of the file; you should see an entry for the `main.css` file. Here's an example:

```
"main.css": " main.efb556103718.css"
```

This is how Django can populate the correct URL in a template when using the static template tag: by looking up the hashed path in this mapping file.

7. We've finished using `business_site`, which we were just using for testing. You can delete the project or keep it around for reference during the activities.

Note

Unfortunately, we can't examine how the hashed URL is interpolated in the template because, when running in debug mode, Django does not look up the hashed version of the file. As we know, the Django dev server only runs in debug mode, so if we were to turn debug mode off to try to view the hashed interpolation, the Django dev server would not start. You will need to examine this interpolation yourself when going to production and when using a frontend web server.

In this exercise, we configured Django to use `ManifestFilesStorage` for its static file storage by adding the `STATICFILES_STORAGE` setting to `settings.py`. Then, we executed the `collectstatic` command to see how the hashes are generated and added to the filename of the copied files. We saw the `staticfiles.json` manifest file, which stored a lookup from the original path to the hashed path. Finally, we cleaned up the settings and directories that we added in this exercise and *Exercise 5.04 – collecting static files for production*. These were the `STATIC_ROOT` setting, the `STATICFILES_STORAGE` setting, and the `static_production_test` directory.

So far, we've only seen how to locate static files stored on our disk. Django can be customized to locate static files stored in other locations (such as in the cloud). In the next section, we'll briefly look at how to use custom storage engines.

static_production_test/staticfiles.json. 모든 내용이 한 줄에 나타날 수 있습니다. 그렇다면 보기 메뉴 | 활성 편집기 | 줄 바꿈을 활성화하세요. 파일의 끝으로 스크롤하면 main.css 파일에 대한 항목이 표시됩니다. 예시는 다음과 같습니다:

```
"main.css": " main.efb556103718.css"
```

Django가 statictemplate 태그를 사용할 때 템플릿에 올바른 URL을 채우는 방법입니다. 이 매핑 파일에서 해시된 경로를 조회하여.

7. 우리는 테스트를 위해 사용했던 business_site 사용을 마쳤습니다. 프로젝트를 삭제하거나 활동 중 참조용으로 유지할 수 있습니다.

참고

안타깝게도 해시된 URL이 템플릿에서 어떻게 보관되는지 확인할 수 없습니다. 디버그 모드에서 실행할 때 Django는 파일의 해시 버전을 조회하지 않기 때문입니다. 우리가 아는 바와 같이 Django 개발 서버는 디버그 모드에서만 실행되므로 해시 보관을 보기 위해 디버그 모드를 끄면 Django 개발 서버가 시작되지 않습니다. 프로덕션으로 가고 프론트엔드 웹 서버를 사용할 때 이 보관을 직접 확인해야 합니다.

이 연습에서는 STATICFILES_STORAGE 설정을 settings.py에 추가하여 Django가 정적 파일 저장소로 ManifestFilesStorage를 사용하도록 구성했습니다. 그런 다음 collectstatic 명령을 실행하여 해시가 생성되는 방법을 확인하고 복사된 파일의 파일 이름에 idx_e1838253을 추가했습니다. 우리는 원래 경로에서 해시된 경로로의 조회를 저장한 staticfiles.json 매니페스트 파일을 보았습니다. 마지막으로, 이 연습과 연습 5.04 - 프로덕션을 위한 정적 파일 수집에서 추가한 설정과 디렉토리를 정리했습니다. 여기에는 STATIC_ROOT 설정, STATICFILES_STORAGE 설정 및 static_production_test 디렉토리가 포함되었습니다.

지금까지 우리는 디스크에 저장된 정적 파일을 찾는 방법만 보았습니다. Django는 클라우드로 같은 다른 위치에 저장된 정적 파일을 찾도록 사용자 정의할 수 있습니다. 다음 섹션에서는 사용자 정의 저장 엔진을 사용하는 방법을 간략하게 살펴보겠습니다.

Custom storage engines

In the previous section, we set the storage engine to `ManifestFilesStorage`. This class is provided by Django, but it is also possible to write a custom storage engine. For example, you could write a storage engine that uploads your static files to a CDN, Amazon S3, or Google Cloud bucket when you run `collectstatic`.

Writing a custom storage engine is beyond the scope of this book. Third-party libraries already exist that support uploading to a variety of cloud services. One such library is `Django Storages`, which can be found at <https://django-storages.readthedocs.io/>.

The following code is a short skeleton indicating which methods you should implement to create a custom file storage engine:

```
from django.conf import settings
from django.contrib.staticfiles import storage

class CustomFilesStorage(storage.StaticFilesStorage):
    def __init__(self):
        """The class must be able to be instantiated
        without any arguments.
        Create custom settings in settings.py and read them
        instead."""
        self.setting = settings.CUSTOM_STORAGE_SETTING
```

The class you define (in this example, it's called `CustomFilesStorage`) must be able to be instantiated without any arguments. The `__init__` function must be able to load any settings from global identifiers (in this case, from our Django settings):

```
def delete(self, name):
    """Implement delete of the file from the remote
    service."""
```

The `delete` method should be able to delete the file specified by the `name` argument from the remote service:

```
def exists(self, name):
    """Return True if a file with name exists in the
    remote service."""
```

사용자 정의 저장 엔진

이전 섹션에서는 저장 엔진을 ManifestFilesStorage로 설정했습니다. 이 클래스는 Django에서 제공하지만, 사용자 정의 저장 엔진을 작성하는 것도 가능합니다. 예를 들어, collectstatic을 실행할 때 정적 파일을 CDN, Amazon S3 또는 Google Cloud 버킷에 업로드하는 저장 엔진을 작성할 수 있습니다.

사용자 정의 저장 엔진을 작성하는 것은 이 책의 범위를 넘어섭니다. 다양한 클라우드 서비스에 업로드를 지원하는 서드파티 라이브러리가 이미 존재합니다. 그 중 하나는 <https://django-storages.readthedocs.io/>에서 찾을 수 있는 Django Storages입니다.

다음 코드는 사용자 정의 파일 저장 엔진을 만들기 위해 구현해야 할 메서드를 나타내는 간단한 골격입니다:

```
from django.conf import settings
from django.contrib.staticfiles import storage

class CustomFilesStorage(storage.StaticFilesStorage):
    def __init__(self):
        """클래스는 인수 없이 인스턴스화될 수 있어야 합니다.

        settings.py에서 사용자 정의 설정을 만들고 읽습니다
        대신."""
        self.setting = settings.CUSTOM_STORAGE_SETTING
```

정의한 클래스(이 예에서는 CustomFilesStorage라고 함)는 인수 없이 인스턴스화될 수 있어야 합니다. __init__ 함수는 전역 식별자에서 모든 설정을 로드할 수 있어야 합니다(이 경우, 우리의 Django 설정에서):

```
def delete(self, name):
    """원격 서비스에서 파일 삭제를 구현합니다."""
```

delete 메서드는 원격 서비스에서 name 인수로 지정된 파일을 삭제할 수 있어야 합니다:

```
def exists(self, name):
    """원격 서비스에 name이라는 파일이 존재하면 True를 반환합니다."""
```

The `exists` method should query the remote service to check if the file specified by name exists. It should return `True` if the file exists, or `False` if it doesn't:

```
def listdir(self, path):
    """List a directory in the remote service. Return
    should be a 2-tuple of lists, the first a list
    of directories, the second a list of files."""
```

The `listdir` method should query the remote service to list the directory at `path`. It should then return a 2-tuple of lists. The first element of this tuple is a list of directories inside `path`, while the second element is a list of files. Here's an example:

```
return (["directory1", "directory2"], ["main.css", "document.txt", "image.jpg"])
```

If `path` contains no directories or no files, then an empty list should be returned for that element. You would return two empty lists if the directory was empty:

```
def size(self, name):
    """Return the size in bytes of the file with
    name."""
```

The `size` method should query the remote service and get the size of the file specified by name:

```
def url(self, name):
    """Return the URL where the file of with name can
    be access on the remote service. For example,
    this might be URL of the file after it has been
    uploaded to a specific remote host with a
    specific domain."""
```

The `url` method should determine the URL to access the file specified by name. This can be built by appending name to a specific static hosting URL:

```
def _open(self, name, mode="rb"):
    """Return a File-like object pointing to file with
    name. For example, this could be a URL handle
    for a remote file."""
```

exists 메서드는 이름으로 지정된 파일이 존재하는지 확인하기 위해 원격 서비스를 쿼리해야 합니다. 파일이 존재하면 True를 반환하고, 존재하지 않으면 False를 반환해야 합니다:

```
def listdir(self, path):
    """원격 서비스에서 디렉토리를 나열합니다. 반환값은 2
    개의 튜플로, 첫 번째는 디렉토리 목록, 두 번째는
    파일 목록이어야 합니다."""
```

listdir 메서드는 원격 서비스에 쿼리하여 경로에 있는 디렉토리를 나열해야 합니다. 그런 다음 2개의 튜플로 된 목록을 반환해야 합니다. 이 튜플의 첫 번째 요소는 경로 내의 디렉토리 목록이고, 두 번째 요소는 파일 목록입니다. 예를 들면 다음과 같습니다:

```
return (["directory1", "directory2"], ["main.css", "document.txt", "image.jpg"])
```

경로에 디렉토리나 파일이 없으면 해당 요소에 대해 빈 목록이 반환되어야 합니다. 디렉토리가 비어 있으면 두 개의 빈 목록을 반환합니다:

```
def size(self, name):
    """이름으로 지정된 파일의 크기를 바이트 단위로
    반환합니다."""
```

size 메서드는 원격 서비스를 쿼리하여 이름으로 지정된 파일의 크기를 가져와야 합니다:

```
def url(self, name):
    """이름으로 지정된 파일에 접근할 수 있는 URL을 반환합
    니다. 예를 들어, 이는 특정 도메인을 가진 특정 원격
    호스트에 업로드된 후의 파일 URL일 수 있습니다."""
```

url 메서드는 이름으로 지정된 파일에 접근할 URL을 결정해야 합니다. 이는 특정 정적 호스팅 URL에 이름을 추가하여 구성할 수 있습니다:

```
def _open(self, name, mode="rb"):
    """이름으로 지정된 파일을 가리키는 파일과 유사한 객체
    를 반환합니다. 예를 들어, 이는 원격 파일에 대한
    URL 핸들일 수 있습니다."""
```

The `_open` method will provide a remote file handle, specified by name. How you implement this will depend on the type of remote service. You might have to download the file and then use a memory buffer (such as an `io.BytesIO` object) to simulate opening the file:

```
def _save(self, name, content):  
    """Write the content for a file with name. In this  
    method you might upload the content to a remote  
    service."""
```

The `_save` method should save content to the remote file at `name`. The method of implementing this will depend on your remote service. It might transfer the file over SFTP or upload it to a CDN.

While this example does not implement any transferring to or from a remote service, you can refer to it to get an idea of how to implement a custom storage engine.

After implementing your custom storage engine, you can make it active by setting its dotted module path in the `STATICFILES_STORAGE` setting in `settings.py`. We won't proceed further with custom storage, but will take some time to practice what we have learned about static files with activities that implement features of the Bookr project.

```
STATICFILES_STORAGE = "myapp.storages.CustomFilesStorage"
```

You'll now work on the first activity in this chapter. In this activity, you'll add a static logo image to the Bookr application. The logo will be displayed in the **Reviews** section of the website.

`_open` 메서드는 이름으로 지정된 원격 파일 핸들을 제공합니다. 이를 구현하는 방법은 원격 서비스의 유형에 따라 다릅니다. 파일을 다운로드한 다음 메모리 버퍼(예: `io.BytesIO` 객체)를 사용하여 파일을 여는 것을 시뮬레이션해야 할 수도 있습니다:

```
def _save(self, name, content):
    """이름이 있는 파일의 내용을 작성합니다. 이 메서드에서
        는 원격 서비스에 내용을 업로드할 수 있습니다."""
```

`_save` 메서드는 이름에 있는 원격 파일에 내용을 저장해야 합니다. 이를 구현하는 방법은 원격 서비스에 따라 다릅니다. SFTP를 통해 파일을 전송하거나 CDN에 업로드할 수 있습니다.

이 예제 `idx_3831a1e2`는 원격 서비스로의 전송을 구현하지 않지만, 사용자 정의 저장 엔진을 구현하는 방법에 대한 아이디어를 얻기 위해 참조할 수 있습니다.

사용자 정의 저장 엔진을 구현한 후, `settings.py`의 `STATICFILES_STORAGE` 설정에서 점으로 구분된 모듈 경로를 설정하여 활성화할 수 있습니다. 우리는 사용자 정의 저장에 대해 더 진행하지 않겠지만, Bookr 프로젝트의 기능을 구현하는 활동을 통해 정적 파일에 대해 배운 내용을 연습할 시간을 가질 것입니다.

```
STATICFILES_STORAGE = "myapp.storages.CustomFilesStorage"
```

이제 이 장의 첫 번째 활동을 진행할 것입니다. 이 활동에서는 Bookr 애플리케이션에 정적 로고 이미지를 추가합니다. 로고는 웹사이트의 리뷰 섹션에 표시됩니다.

Activity 5.01 – adding a Reviews logo

The Bookr app should have a logo that is specific to pages in the Reviews app. This will involve adding a base template just for the Reviews app and updating our current Reviews templates to inherit from it. Then, you'll include the Bookr Reviews logo on this base template. These steps will help you complete this activity:

1. Add a CSS rule to position the logo. Put this rule into the existing `base.html` file, after the `.navbar-brand` rule:

```
.navbar-brand > img {  
    height: 60px;  
}
```

2. Add a brand block template tag that inheriting templates can override. Put this inside the `<a>` element with the `navbar-brand` class. The default contents of block should be left as *Book Review*.
3. Add a static directory inside the Reviews app, containing a namespaced directory. Download the `Reviews logo.png` file from the `Chapter05` folder in this book's GitHub repo and put it inside this directory.
4. Create a `templates` directory for the Bookr project (inside the Bookr project directory). Then, move the Reviews app's current `base.html` into this directory so that it becomes a base template for the whole project.
5. Add the new templates directory's path to the `TEMPLATES["DIRS"]` setting in `settings.py`.
6. Create another `base.html` template specifically for the Reviews app. Put it inside the Reviews app's templates directory. The new template should extend the existing (now global) `base.html` file.
7. The new `base.html` file should override the content of the brand block. This block should contain just an `<img>` whose `src` attribute is set using the `{% static %}` template tag. The image source should be the logo we added in *Step 2*.

Refer to the following screenshots to see what your pages should look like after these changes. Note that, although you are making changes to the base template, it will not change the layout of the main page:

활동 5.01 – 리뷰 로고 추가하기

Bookr 앱은 리뷰 앱의 페이지에 특정한 로고를 가져야 합니다. 이를 위해 리뷰 앱 전용 기본 템플릿을 추가하고 현재의 리뷰 템플릿이 이를 상속하도록 업데이트해야 합니다. 그런 다음, 이 기본 템플릿에 Bookr 리뷰 로고를 포함시킬 것입니다. 이 단계들은 이 활동을 완료하는 데 도움이 될 것입니다:

1. 로고의 위치를 지정하는 CSS 규칙을 추가합니다. 이 규칙을 기존 base.html 파일의 .navbar-brand 규칙 뒤에 넣습니다:

```
.navbar-brand > img {
  height: 60px;
}
```

2. 상속 템플릿이 재정의할 수 있는 브랜드 블록 템플릿 태그를 추가합니다. 이를 navbar-brand 클래스가 있는 <a> 요소 안에 넣습니다. 블록의 기본 내용은 Book Review로 남겨두어야 합니다.
3. 리뷰 앱 안에 네임스페이스가 있는 디렉토리를 포함하는 정적 디렉토리를 추가합니다. 이 책의 GitHub 저장소의 Chapter05 폴더에서 Reviews logo.png 파일을 다운로드하고 이 디렉토리 안에 넣습니다.
4. Bookr 프로젝트 디렉토리 내에 templates 디렉토리를 만드세요. 그런 다음, Reviews 앱의 현재 base.html 파일을 이 디렉토리로 이동하여 전체 프로젝트의 기본 템플릿이 되도록 하세요.
5. 새 템플릿 디렉토리 경로를 settings.py의 TEMPLATES["DIRS"] 설정에 추가하세요.
6. Reviews 앱 전용 base.html 템플릿을 하나 더 만드세요. 그것을 Reviews 앱의 templates 디렉토리 안에 넣으세요. 새 템플릿은 기존의 (현재 global인) base.html 파일을 확장해야 합니다.
7. 새로운 base.html 파일은 brand 블록의 내용을 재정의해야 합니다. 이 블록은 src 속성이 {% static %}을 사용하여 설정된 태그만 포함해야 합니다.

템플릿 태그. 이미지 소스는 2단계에서 추가한 로고여야 합니다. 변경 후 페이지가 어떻게 보여야 하는지 확인하려면 다음 idx_6d0b04d0 스크린샷을 참조하세요. 기본 템플릿을 변경하더라도 메인 페이지의 레이아웃은 변경되지 않는다는 점에 유의하세요.

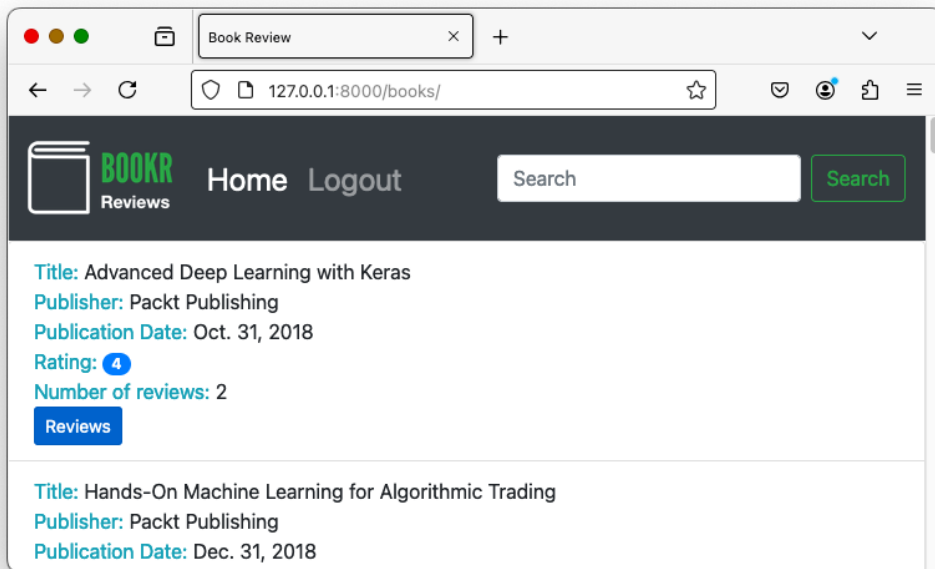


Figure 5.21 – The Book List page after adding the Reviews logo

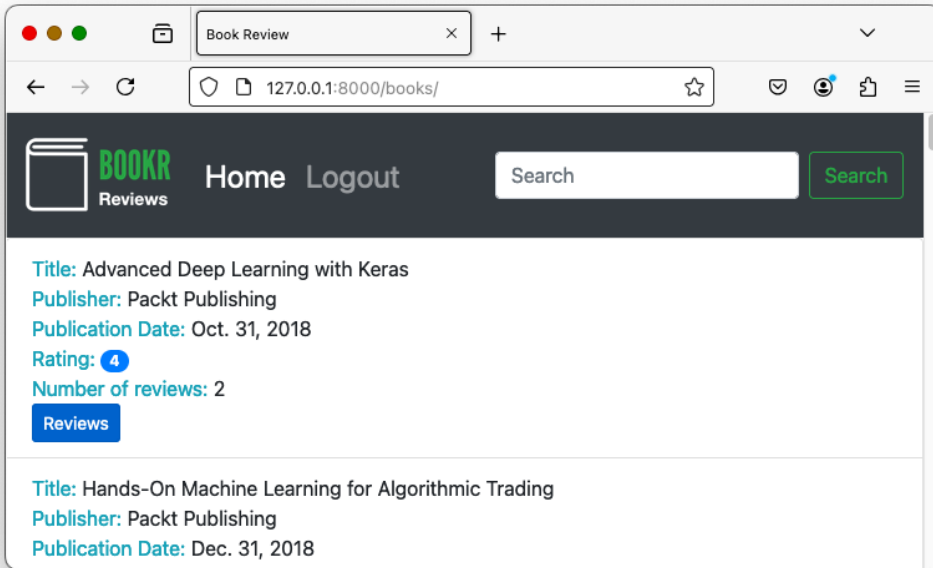


그림 5.21 - 리뷰 로고를 추가한 후의 도서 목록 페이지

On the **Book Details** page, you will also see the Reviews logo:

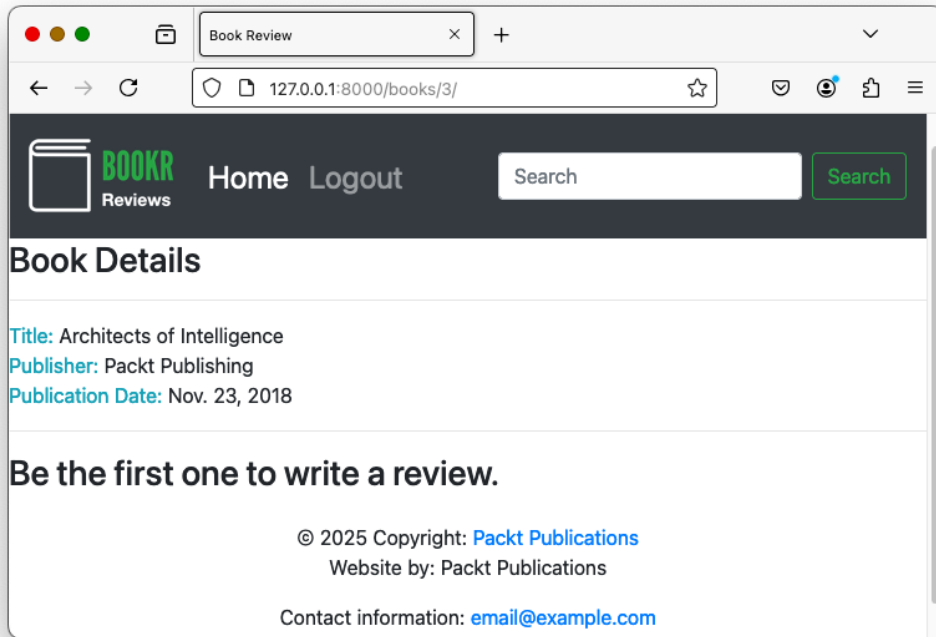


Figure 5.22 – The Book Details page after adding the Reviews logo

Activity 5.02 – CSS enhancements

Currently, the CSS is kept inline in the `base.html` template. As a best practice, it should be moved into its own file so that it can be cached separately and decrease the size of the HTML downloads. As part of this, you'll also add some CSS enhancements, such as fonts and colors, and link in Google Fonts CSS to support these changes. These steps will help you complete this activity:

1. Create a directory named `static` in the Bookr project directory. Then, create a new file inside it named `main.css`.

도서 세부정보 페이지에서 idx_5e682c78리뷰 로고도 볼 수 있습니다:

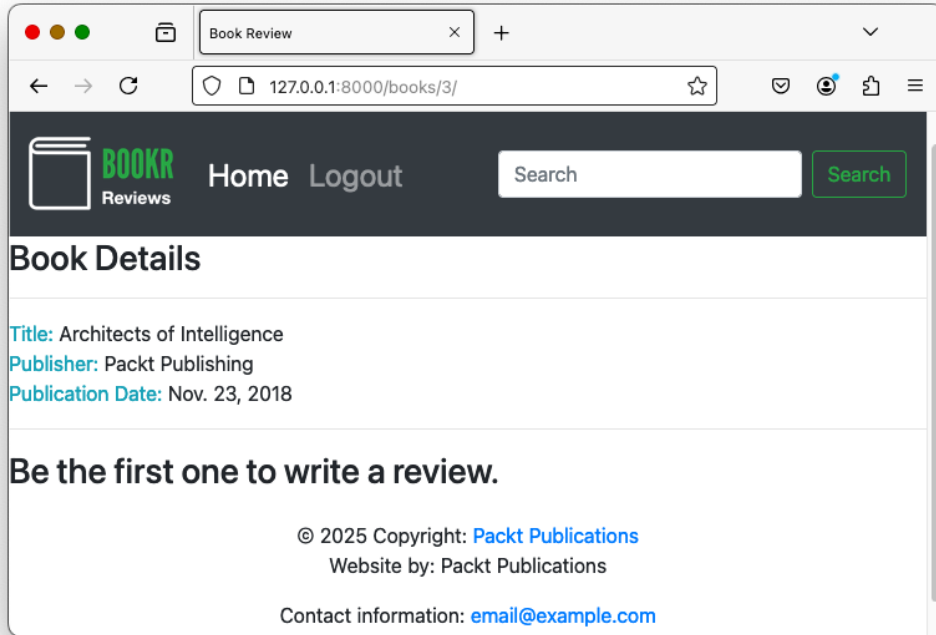


그림 5.22 - 리뷰 로고를 추가한 후의 도서 세부정보 페이지

활동 5.02 – CSS 개선

현재 CSS는 base.html 템플릿에 인라인으로 유지되고 있습니다. 모범 사례로, 별도의 파일로 이동하여 캐시할 수 있도록 하고 HTML 다운로드 크기를 줄여야 합니다. 이와 함께 글꼴 및 색상과 같은 CSS 개선 사항을 추가하고 이러한 변경 사항을 지원하기 위해 Google Fonts CSS를 링크합니다. 이러한 단계는 이 활동을 완료하는 데 도움이 될 것입니다:

1. Bookr 프로젝트 디렉토리에 static이라는 이름의 디렉토리를 만듭니다. 그런 다음 그 안에 main.css라는 새 파일을 만듭니다.

2. Copy the contents of the `<style>` element from the main `base.html` template into the new `main.css` file, then remove the `<style>` element from the template. Add these extra rules to the end of the CSS file:

```
body {  
    font-family: 'Source Sans Pro', sans-serif;  
    background-color: #e6efe8  
    color: #393939;  
}  
  
h1, h2, h3, h4, h5, h6 {  
    font-family: 'Libre Baskerville', serif;  
}
```

3. Link to the new `main.css` file with a `<link rel="stylesheet" href="...">` tag. Use the `{% static %}` template tag to generate the URL for the `href` attribute, and don't forget to load the static library.
4. Link in the Google Fonts CSS by adding this code to the base template:

```
<link rel="stylesheet" href="https://fonts.googleapis.com/css?  
family=Libre+Baskerville|Source+Sans+Pro&display=swap">
```

(You will need to have an active internet connection so that your browser can include this remote CSS file.)

5. Update your Django settings to add `STATICFILES_DIRS`, which is set to the static directory you created in *Step 1*. When you're finished, your Bookr application should look like *Figure 5.23*:

2. main base.html 템플릿에서 <style> 요소의 내용을 새로운 main.css 파일로 복사한 다음, 템플릿에서 <style> 요소를 제거합니다. CSS 파일 끝에 이 추가 규칙을 추가하세요:

```
body {
    font-family: 'Source Sans Pro', sans-serif;
    background-color: #e6efe8
    color: #393939;
}

h1, h2, h3, h4, h5, h6 {
    font-family: 'Libre Baskerville', serif;
}
```

3. 새로운 main.css 파일에 대한 링크를 <link rel="stylesheet" href="..."> 태그로 추가하세요. {% static %} 템플릿 태그를 사용하여 href 속성의 URL을 생성하고, static 라이브러리를 로드하는 것을 잊지 마세요.
4. 기본 템플릿에 이 코드를 추가하여 Google Fonts CSS에 링크하세요:

```
<link rel="stylesheet" href="https://fonts.googleapis.com/css?
family=Libre+Baskerville|Source+Sans+Pro&display=swap">
```

(브라우저가 이 원격 CSS 파일을 포함할 수 있도록 활성 인터넷 연결이 필요합니다.)

5. Django 설정을 업데이트하여 Step 1에서 생성한 staticdirectory로 설정된 STATICFILES_DIRS를 추가하세요. 완료되면, 당신의 Bookr 애플리케이션은 Figure 5.23과 같아야 합니다:

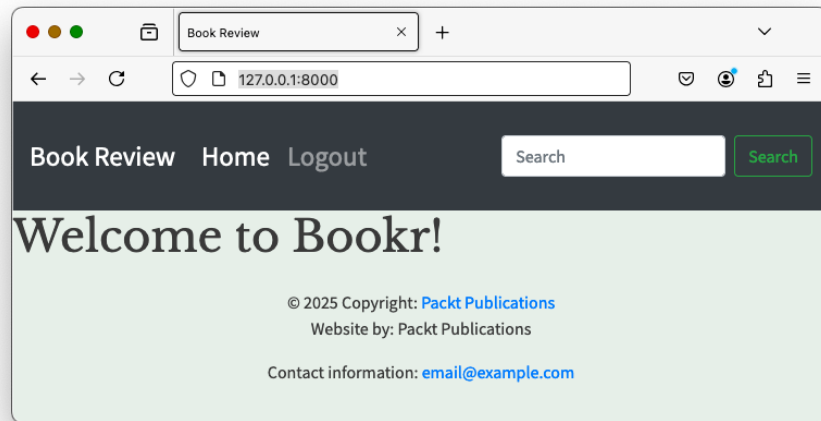


Figure 5.23 – Main page with a new font and background color

Notice the new font and background color. These should be displayed on all the Bookr pages.

Activity 5.03 – adding a global logo

You have already added a logo that is served on pages for the Reviews app. We have another logo to be used globally by default, but other apps will be able to override it:

1. Download the Bookr logo (logo.png) from the Chapter05 folder in this book's GitHub repo.
2. Save it in the main static directory for the project.
3. Edit the main base.html file. We already have a block for the logo (*brand*), so an `<img>` can be placed inside here. Use the static template tag to refer to the logo you just downloaded.
4. Check that your pages work. On the main URL, you should see the Bookr logo, but on the **Book List** and **Book Details** pages, you should see the Bookr Reviews logo.

When you're finished, you should see the Bookr logo on the main page:

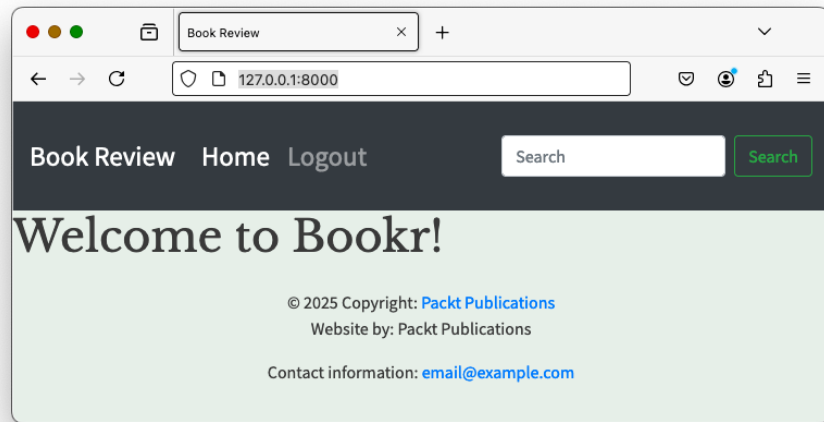


그림 5.23 - 새로운 글꼴과 배경색이 있는 메인 페이지

새로운 글꼴과 배경색을 주목하세요. 이 `idx_0cc795f1`은 모든 Bookr 페이지에 표시되어야 합니다.

활동 5.03 – 글로벌 로고 추가하기

당신은 `idx_6a80d9f3`이미 리뷰 앱의 페이지에서 제공되는 로고를 추가했습니다. 우리는 기본적으로 전역적으로 사용될 또 다른 로고가 있지만, 다른 앱들이 이를 재정의할 수 있습니다:

1. 이 책의 GitHub 저장소의 Chapter05 폴더에서 Bookr 로고(`logo.png`)를 다운로드하세요.
2. 프로젝트의 주요 정적 디렉토리에 저장하세요.
3. 주요 `base.html` 파일을 수정하세요. 로고(브랜드)를 위한 블록이 이미 있으므로, 여기 안에 `<img>`를 배치할 수 있습니다. 방금 다운로드한 로고를 참조하기 위해 정적 템플릿 태그를 사용하세요.
4. `idx_f96b3dcdyour` 페이지가 작동하는지 확인하세요. 주요 URL에서 Bookr 로고를 볼 수 있어야 하지만, Book List 및 Book Details 페이지에서는 Bookr Reviews 로고를 볼 수 있어야 합니다. 작업이 완료되면, 주요 페이지에서 Bookr 로고를 볼 수 있어야 합니다:

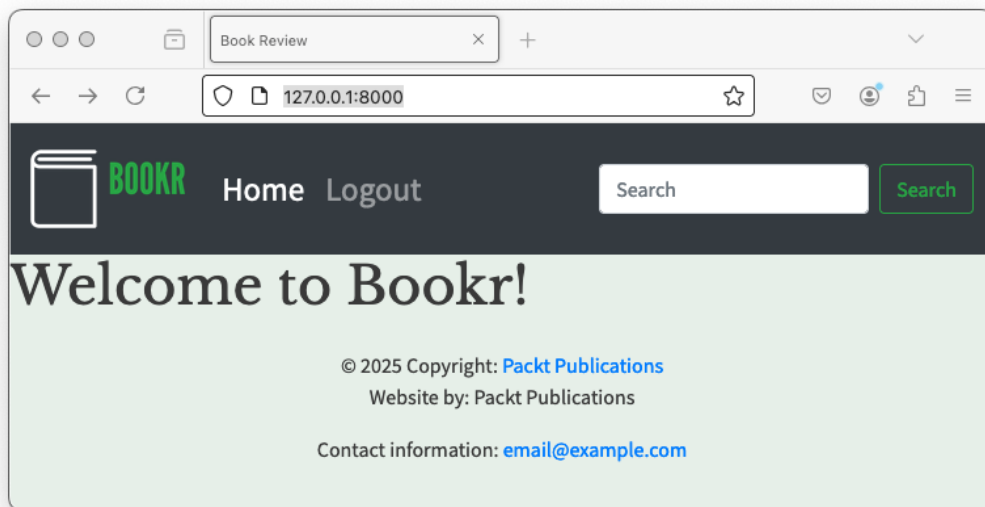


Figure 5.24 – Bookr logo on the main page

When you visit a page that had the Bookr Reviews logo before, such as the **Book List** page, it should still show the Bookr Reviews logo:

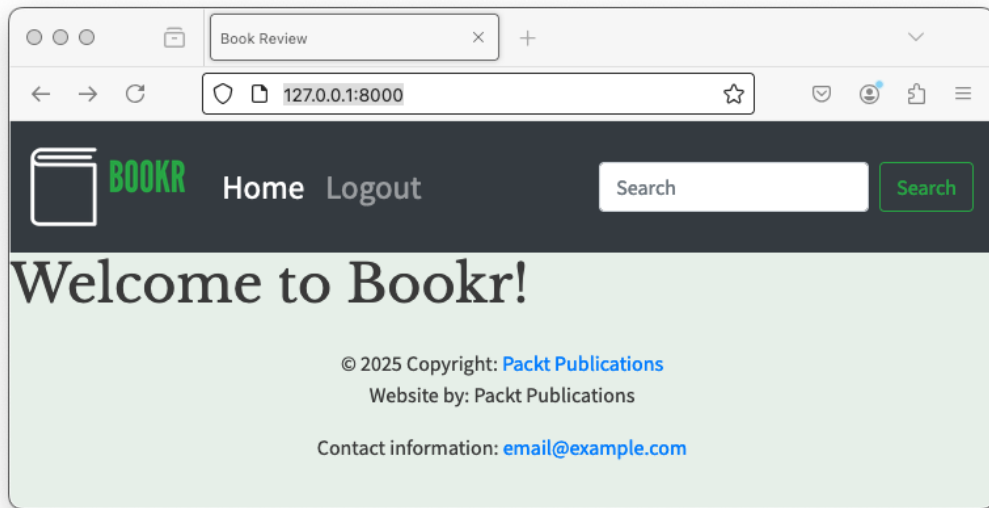


그림 5.24 - 메인 페이지의 Bookr 로고

이전에 Book List 페이지와 같은 페이지를 방문했을 때 Bookr Reviews 로고가 idx_4daf1f9bhad 표시되었다면, 여전히 Bookr Reviews 로고가 표시되어야 합니다:

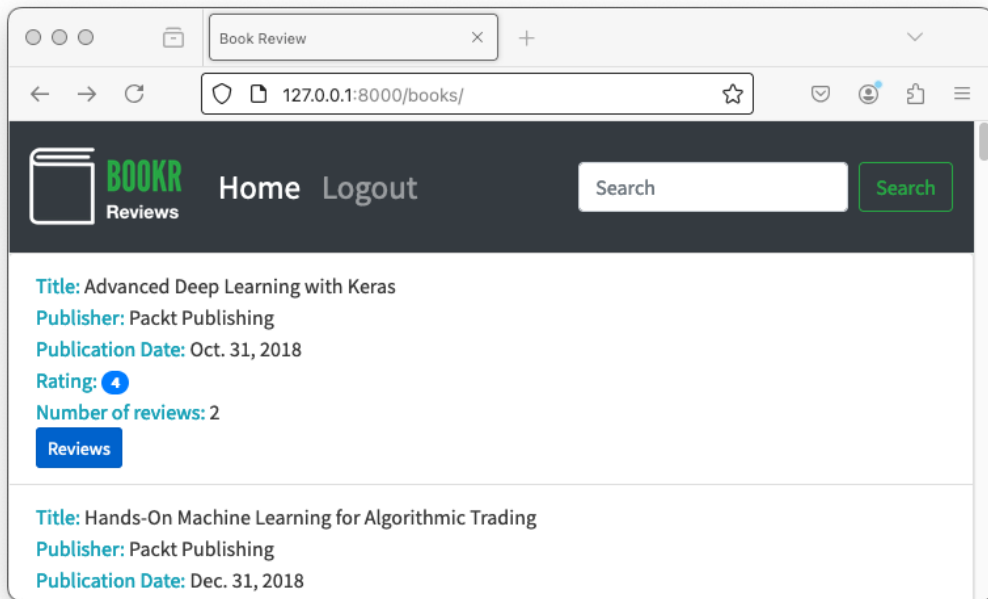


Figure 5.25 – Bookr Reviews logo still appears on the Reviews pages

Summary

In this chapter, we showed you how to use Django's `staticfiles` app to find and serve static files. We used the built-in `static` view to serve these files with the Django dev server in `DEBUG` mode. We showed different places to store static files by using a directory that is global to the project or in a specific directory for the application; global resources should be stored in the former while application-specific resources should be stored in the latter. We showed the importance of namespacing static file directories to prevent conflicts. After serving the assets, we used the `static` tag to include them in our template. We then demoed how the `collectstatic` command copies all the assets into the `STATIC_ROOT` directory, for production deployment. We showed how to use the `findstatic` command to debug the loading of static files. To invalidate caches automatically, we looked at using `ManifestFilesStorage` to add a hash of the file's content to the static file URL. Finally, we briefly talked about using a custom file storage engine.

So far, we have only fetched web pages using content that already exists. In the next chapter, we will start adding forms so that we can interact with web pages by sending data to them over HTTP.

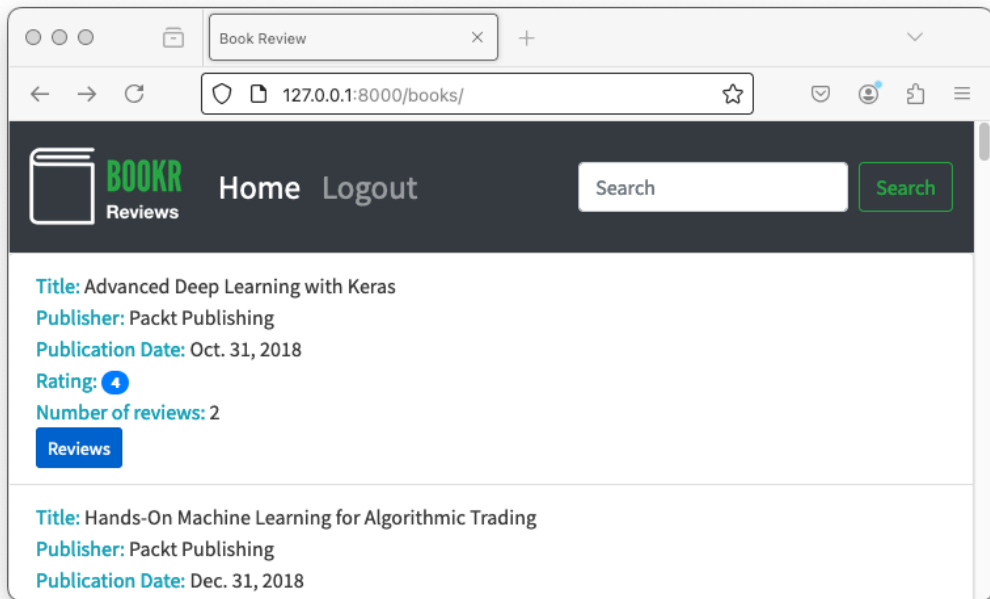


그림 5.25 - Bookr 리뷰 로고가 리뷰 페이지에 여전히 나타납니다

요약

이 장에서는 Django의 staticfiles 앱을 사용하여 정적 파일을 찾고 제공하는 방법을 보여주었습니다. 우리는 내장된 정적 뷰를 사용하여 DEBUG 모드에서 Django 개발 서버로 이러한 파일을 제공했습니다. 우리는 프로젝트에 전역적인 디렉토리 또는 애플리케이션을 위한 특정 디렉토리에 정적 파일을 저장하는 다양한 장소를 보여주었습니다; 전역 리소스는 전자에 저장해야 하며 애플리케이션 특정 리소스는 후자에 저장해야 합니다. 우리는 충돌을 방지하기 위해 정적 파일 디렉토리의 네임스페이싱의 중요성을 보여주었습니다. 자산을 제공한 후, 우리는 정적 태그를 사용하여 이를 템플릿에 포함시켰습니다. 그런 다음 collectstatic 명령이 모든 자산을 STATIC_ROOT 디렉토리로 복사하는 방법을 시연했습니다. 우리는 findstatic 명령을 사용하여 정적 파일 로딩을 디버깅하는 방법을 보여주었습니다. 캐시를 자동으로 무효화하기 위해, 우리는 파일 내용의 해시를 정적 파일 URL에 추가하기 위해 ManifestFilesStorage를 사용하는 방법을 살펴보았습니다. 마지막으로, 우리는 사용자 정의 파일 저장 엔진을 사용하는 것에 대해 간략하게 이야기했습니다.

지금까지 우리는 이미 존재하는 콘텐츠를 사용하여 웹 페이지를 가져오기만 했습니다. 다음 장에서는 HTTP를 통해 데이터를 전송하여 웹 페이지와 상호작용할 수 있도록 양식을 추가하기 시작할 것입니다.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

이 책의 PDF 복사본, 코드 번들 등을 받으세요

QR 코드를 스캔하세요 (또는 packtpub.com/unlock으로 가세요). 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

6

Forms

This chapter introduces **web forms**, a method of sending information from the browser to a web server. We will start by introducing forms in general and discussing how data is encoded to be sent to the server.

So far, the views we have been building for Django have been one-way only. Our browser is retrieving data from the views we have written, but it doesn't send any data back to them. In *Chapter 4, Introduction to Django Admin*, we created model instances by using the Django admin and submitting forms, but those were using views built into Django, not created by us. In this chapter, we will use the Django Forms library to start accepting user-submitted data. The data will be provided through GET requests in the URL parameters and/or POST requests in the body of the request. But before we get into the details, first, let's understand what forms are in Django. You'll learn about the differences between sending form data in a GET HTTP request and sending it in a POST HTTP request, and how to choose which one to use.

In this chapter, we will be covering the following topics:

- Background to forms
- Form security
- The Django Forms library
- Validating forms and retrieving Python values
- Activity 1 – Book search

By the end of this chapter, you will have learned how Django's Forms library is used to build and validate forms automatically and how it cuts down on the amount of manual HTML to write.

6

양식

이 장에서는 웹 양식, 즉 브라우저에서 웹 서버로 정보를 전송하는 방법을 소개합니다. 우리는 일반적인 양식에 대해 소개하고 데이터가 서버로 전송되기 위해 어떻게 인코딩 되는지 논의하는 것으로 시작할 것입니다.

지금까지 우리가 Django를 위해 구축한 뷰는 단방향만 있었습니다. 우리의 브라우저는 우리가 작성한 뷰에서 데이터를 검색하고 있지만, 그들에게 데이터를 다시 보내지 않습니다. 4장, Django Admin 소개에서는 Django 관리자를 사용하고 양식을 제출하여 모델 인스턴스를 생성했지만, 그것들은 우리가 만든 것이 아니라 Django에 내장된 뷰를 사용한 것입니다. 이 장에서는 Django Forms 라이브러리를 사용하여 사용자 제출 데이터를 수락하기 시작할 것입니다. 데이터는 URL 매개변수의 GET 요청과/또는 요청 본문의 POST 요청을 통해 제공됩니다. 그러나 세부 사항에 들어가기 전에, 먼저 Django에서 양식이 무엇인지 이해해 봅시다. GET HTTP 요청에서 양식 데이터를 전송하는 것과 POST HTTP 요청에서 전송하는 것의 차이점과 어떤 것을 사용할지 선택하는 방법에 대해 배우게 될 것입니다.

이 장에서는 다음 주제를 다룰 것입니다:

- 양식의 배경 양식 보안
 - Django Forms 라이브러리
 - 러리
 - 양식 유효성 검사 및 Python 값 검색 • 활동
- 1 – 도서 검색

이 장이 끝날 때쯤이면, Django의 Forms 라이브러리가 양식을 자동으로 생성하고 검증하는 데 어떻게 사용되는지, 그리고 수동으로 작성해야 하는 HTML의 양을 어떻게 줄이는지 배울 것입니다.

Technical requirements

Find the solution in the `Chapter06` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Background to forms

Django forms act as a helper that does three jobs: they build the HTML fields, check that the user's input is valid, and provide clean data to use in a view. They are like a gatekeeper between what a user types and what the application accepts.

When working with an interactive web app, we not only want to provide data to users but also accept data from them to either customize the responses we're generating or to let them submit data to the site. A form is made up of inputs that define key-value pairs of data to submit to the server. For example, when logging in to a website, the data being sent would have the keys `username` and `password`, with values of your username and your password, respectively. Each input in the form has a *name*, and this is how its data is identified on the server side (in a Django view). There can be multiple inputs with the same name, whose data is available in a list containing all the posted values with this name – for example, a list of checkboxes with permissions to apply to users. Each checkbox would have the same name but a different value. The form has attributes that specify which URL the browser should submit the data to and what method it should use to submit the data (browsers only support GET or POST).

The form element

All inputs used during form submission must be contained inside a `<form>` element. There are three HTML attributes that you must use to modify the behavior of the form:

- **Method:** This is the HTTP method for submitting the form; it's either GET or POST. If omitted, this defaults to `get` (because this is the default method when typing in a URL in the browser and hitting *Enter*).
- **Action:** This refers to the URL (or path) to send the form data to. If omitted, the data gets sent back to the current page.
- **Enctype:** This sets the encoding type of the form. You only need to change this if you are using the form to upload files. The most common values are `application/x-www-form-urlencoded` (the default if this value is omitted) or `multipart/form-data` (set this if you're uploading files). Note that you don't have to worry about the encoding type in your view – Django handles the different types automatically.

기술 요구 사항

이 책의 GitHub 저장소의 Chapter06 폴더에서 솔루션을 찾으세요. 저장소 링크에 접근하려면 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르세요.

양식에 대한 배경

Django 양식은 세 가지 작업을 수행하는 도우미 역할을 합니다: HTML 필드를 생성하고, 사용자의 입력이 유효한지 확인하며, 뷰에서 사용할 수 있는 깨끗한 데이터를 제공합니다. 이들은 사용자가 입력하는 것과 애플리케이션이 수용하는 것 사이의 게이트키퍼와 같습니다.

인터랙티브 웹 앱인 idx_49202d5c와 작업할 때, 우리는 사용자에게 데이터를 제공할 뿐만 아니라 그들로부터 데이터를 수집하여 우리가 생성하는 응답을 사용자 맞춤화하거나 그들이 사이트에 데이터를 제출할 수 있도록 허용하고자 합니다. 양식은 서버에 제출할 데이터의 키-값 쌍을 정의하는 입력으로 구성됩니다. 예를 들어, 웹사이트에 로그인할 때 전송되는 데이터는 username과 password라는 키를 가지며, 각각의 값은 귀하의 사용자 이름과 비밀번호입니다. 양식의 각 입력은 이름을 가지며, 이는 서버 측(Django 뷰)에서 데이터가 식별되는 방식입니다. 동일한 이름을 가진 여러 입력이 있을 수 있으며, 이들의 데이터는 이 이름으로 게시된 모든 값을 포함하는 목록에서 사용할 수 있습니다. 예를 들어, 사용자에게 적용할 권한이 있는 체크박스 목록이 있을 수 있습니다. 각 체크박스는 동일한 이름을 가지지만 다른 값을 가집니다. 양식은 브라우저가 데이터를 제출해야 하는 URL과 데이터를 제출하는 데 사용할 방법(브라우저는 GET 또는 POST만 지원)을 지정하는 속성을 가지고 있습니다.

양식 요소

양식 제출에 사용되는 모든 입력은 <form> 요소 안에 포함되어야 합니다. 양식의 동작을 수정하기 위해 사용해야 하는 세 가지 HTML 속성이 있습니다:

- **방법:** 이는 양식을 제출하기 위한 HTTP 방법입니다; GET 또는 POST 중 하나입니다. 생략하면 기본값은 get입니다(브라우저에서 URL을 입력하고 Enter를 누를 때의 기본 방법이기 때문입니다).
- **작업:** 이는 양식 데이터를 전송할 URL(또는 경로)을 나타냅니다. 생략하면 데이터가 현재 페이지로 다시 전송됩니다.
- **인코딩 형식:** 이는 양식의 인코딩 형식을 설정합니다. 파일을 업로드하기 위해 양식을 사용하는 경우에만 이 값을 변경해야 합니다. 가장 일반적인 값은 application/x-www-form-urlencoded(이 값이 생략되면 기본값) 또는 multipart/form-data(파일을 업로드하는 경우 설정)입니다. 뷰에서 인코딩 형식에 대해 걱정할 필요는 없습니다. Django가 다양한 형식을 자동으로 처리합니다.

Here is an example of a form without any of its attributes set:

```
<form>
    <!-- Input elements go here -->
</form>
```

It will submit its data using a GET request to the current URL that the form is being displayed on, using the `application/x-www-form-urlencoded` encoding type.

In the following example, we'll set all three attributes on a form:

```
<form method="post" action="/form-submit" enctype="multipart/form-data">
    <!-- Input elements go here -->
</form>
```

This form will submit its data with a POST request to the `/form-submit` path, encoding the data as `multipart/form-data`.

How do GET and POST requests differ in how the data is sent? Recall in *Chapter 1, Introduction to Django*, that we discussed what the underlying HTTP request and response data that your browser sends look like. In the following two examples, we will submit the same form twice: the first time using GET and the second time using POST. The form will have two inputs: a first name and a last name.

A form submitted using GET sends its data in the URL like this:

```
GET /form-submit?first_name=Joe&last_name=Bloggs HTTP/1.1
Host: www.example.com
```

However, a form submitted using POST sends its data in the body of the request, like this:

```
POST /form-submit HTTP/1.1
Host: www.example.com
Content-Length: 31
Content-Type: application/x-www-form-urlencoded

first_name=Joe&last_name=Bloggs
```

You'll notice that the form data is encoded the same way in both cases; it is just placed differently for the GET and POST requests. In the *Choosing between GET or POST* section, we'll discuss how to choose between these two types of requests.

여기 속성이 설정되지 않은 양식의 예가 있습니다:

```
<form>
    <!-- 입력 요소는 여기로 갑니다 -->
</form>
```

양식이 표시되는 현재 URL에 GET 요청을 사용하여 데이터를 제출합니다. 인코딩 유형은 application/x-www-form-urlencoded입니다.

다음 예에서는 양식에 세 가지 속성을 모두 설정합니다:

```
<form method="post" action="/form-submit" enctype="multipart/form-data">
    <!-- 입력 요소는 여기로 갑니다 -->
</form>
```

이 양식은 데이터를 /form-submit 경로에 POST 요청으로 제출하며, 데이터를 multipart/form-data로 인코딩합니다.

GET과 POST 요청이 데이터 전송 방식에서 어떻게 다른지 설명해 보세요. 1장, Django 소개에서 브라우저가 보내는 기본 HTTP 요청 및 응답 데이터가 어떻게 생겼는지 논의했습니다. 다음 두 예제에서는 동일한 양식을 두 번 제출합니다: 첫 번째는 GET을 사용하고 두 번째는 POST를 사용합니다. 양식에는 이름과 성 두 개의 입력이 있습니다.

GET을 사용하여 제출된 양식은 URL에 다음과 같이 데이터를 보냅니다:

```
GET /form-submit?first_name=Joe&last_name=Bloggs HTTP/1.1
Host: www.example.com
```

그러나 POST를 사용하여 제출된 양식은 요청 본문에 다음과 같이 데이터를 보냅니다:

```
POST /form-submit HTTP/1.1
Host: www.example.com
Content-Length: 31
Content-Type: application/x-www-form-urlencoded

first_name=Joe&last_name=Bloggs
```

양식 데이터가 두 경우 모두 동일한 방식으로 인코딩된다는 것을 알 수 있습니다. GET 및 POST 요청에 대해 다르게 배치되었을 뿐입니다. GET 또는 POST 선택 섹션에서는 이 두 가지 요청 유형 중에서 선택하는 방법에 대해 논의할 것입니다.

Form security

As HTML forms are a source of external data to reach an app, we need to consider the security implications that they pose.

We will briefly discuss the main security considerations that HTML forms introduce to an app and will address them in this and later chapters. This is by no means a complete list of security considerations, as new threats emerge over time.

Note

For further information on security, consider *Attacking and Exploiting Modern Web Applications*, by Simone and Donato Onofri (Packt Publishing, 2023, ISBN: 9781801816298): <https://www.packtpub.com/en-us/product/attacking-and-exploiting-modern-web-applications-9781801816298>.

The **Open Worldwide Application Security Project (OWASP)** provides excellent online resources for application security education, including the OWASP Top Ten vulnerabilities: <https://owasp.org/www-project-top-ten/>.

Input validation

We are accustomed to seeing client-side form validation, such as JavaScript checks, within the HTML page that is executed by the browser, but it is also crucial to carry out these checks server-side. If form validation is only carried out client-side, this can be sidestepped by either switching off JavaScript in the browser or bypassing the browser by submitting the form data using a script or dev tool.

Traditionally, this secure approach has involved duplicated logic by including the frontend validation as well as backend validation. Fortunately, in Django, the `django.forms.Form` class provides a single point of truth for defining validation logic, which we will learn about in the *The Django Forms library* section.

Cross-site request forgery protection

Cross-site request forgery (CSRF) protection prevents a web application from receiving form posts from another site.

Historically, the form action attribute could specify any web resource as a URL, and it was possible to create a remote form that could even masquerade for another purpose, particularly by including malicious content in hidden fields. In fact, the CSRF attack could even be

양식 보안

HTML 양식은 앱에 도달하기 위한 외부 데이터의 원천이므로, 이들이 제기하는 보안 문제를 고려해야 합니다.

HTML 양식이 앱에 도입하는 주요 보안 고려 사항에 대해 간략히 논의하고, 이 장과 이후 장에서 다룰 것입니다. 이는 보안 고려 사항의 완전한 목록이 아니며, 시간이 지남에 따라 새로운 위협이 발생합니다.

참고

보안에 대한 추가 정보는 Simone과 Donato Onofri의 "Attacking and Exploiting Modern Web Applications"(Packt Publishing, 2023, ISBN: 9781801816298): <https://www.packtpub.com/en-us/product/attacking-and-exploiting-modern-web-applications-9781801816298>을 고려하십시오.

OWASP(Open Worldwide Application Security Project)는 OWASP Top Ten을 포함하여 애플리케이션 보안 교육을 위한 훌륭한 온라인 리소스를 제공합니다.

취약점: <https://owasp.org/www-project-top-ten/>.

입력 검증

우리는 브라우저에 의해 실행되는 HTML 페이지 내에서 JavaScript 검증과 같은 클라이언트 측 양식 검증을 보는 데 익숙하지만, 이러한 검증을 서버 측에서도 수행하는 것이 중요합니다. 양식 검증이 클라이언트 측에서만 수행된다면, 이는 브라우저에서 JavaScript를 끄거나 스크립트나 개발 도구를 사용하여 양식 데이터를 제출함으로써 우회될 수 있습니다.

전통적으로, 이 안전한 접근 방식은 프론트엔드 검증과 백엔드 검증을 포함하여 중복된 논리를 포함했습니다. 다행히도, Django에서는 `django.forms.Form` 클래스가 검증 논리를 정의하기 위한 단일 진실의 지점을 제공하며, 이는 Django Forms 라이브러리 섹션에서 배울 것입니다.

교차 사이트 요청 위조 방지

교차 사이트 요청 위조(CSRF) 방지는 웹 애플리케이션이 다른 사이트로부터 양식 제출을 받지 못하도록 합니다.

역사적으로, 양식 액션 속성은 URL로서 어떤 웹 리소스도 지정할 수 있었고, 원격 양식을 생성하여 다른 목적을 가장할 수 있는 것이 가능했습니다. 특히, 숨겨진 필드에 악성 콘텐츠를 포함함으로써 말이죠. 사실, CSRF 공격은 심지어

implemented without a form, being an HTTP POST request implemented in JavaScript on the remote page.

CSRF protection requires a token to be included in the form that is expected on the server side. The CSRF token is stored as a cookie in the browser and is unique on a user-per-session basis.

As we will see in *Exercise 6.01*, Django apps are configured by default with CSRF protection enabled.

Django also gives us a mechanism for including the CSRF token in the form.

The CSRF token must be added to the HTML for every form being sent, and is done with the `{% csrf_token %}` template tag. When the template is rendered, this tag will be interpolated similar to the following:

```
<input type="hidden" name="csrfmiddlewaretoken"
value="tETZjLDUXev1tiYqGCSbMQkhWiesHCnutxpt6mutHI6YH64F0nin5k2JW3B68IeJ">
```

Since this is a hidden field, the form on the page does not look any different from how it did before.

The CSRF token is unique to every visitor on the site. If an attacker were to copy the HTML from our site, they would get their own CSRF token, which would not match that of any other user, so Django would reject the form when it was posted by someone else.

CSRF tokens also change periodically. This limits how long the attacker would have to take advantage of a particular user and token combination. Even if they were able to get the CSRF token of a user that they were trying to exploit, they would have a short window of time to use it.

We will implement a CSRF token in a form in *Exercise 6.02*.

SQL injection prevention

SQL injection attacks were a prevalent security weakness in the early days of the web. A user could inject SQL into a form by carefully crafting inputs that would add additional SQL statements to the database call that is made on the server side during the HTTP response.

This approach relied on the server-side code being poorly written so that user input was sent to the database without any control characters being escaped.

By developing our code with an ORM or other database tool, the coder is taken away from constructing raw SQL from user input, and we can eliminate this risk.

We saw how to construct queries using the Django ORM in *Chapter 3*.

원격 페이지에서 JavaScript로 구현된 HTTP POST 요청으로 양식 없이 구현되었습니다.

CSRF 보호는 서버 측에서 예상되는 양식에 포함될 토큰이 필요합니다. CSRF 토큰은 브라우저의 쿠키로 저장되며 사용자-세션 기준으로 고유합니다.

우리가 연습 6.01에서 볼 수 있듯이, Django 앱은 기본적으로 CSRF 보호가 활성화되어 구성됩니다.

Django는 또한 양식에 CSRF 토큰을 포함하는 메커니즘을 제공합니다.

CSRF 토큰은 전송되는 모든 양식의 HTML에 추가되어야 하며, {%csrf_token%} 템플릿 태그로 수행됩니다. 템플릿이 렌더링될 때, 이 태그는 다음과 유사하게 보간됩니다:

```
<input type="hidden" name="csrfmiddlewaretoken"
value="tETZjLDUXev1tiYqGCSbMQkhWiesHCnutxpt6mutHI6YH64F0nin5k2JW3B68IeJ">
```

이것은 숨겨진 필드이므로, 페이지의 양식은 이전과 다르게 보이지 않습니다.

CSRF 토큰은 사이트의 모든 방문자에게 고유합니다. 공격자가 우리 사이트의 HTML을 복사한다면, 그들은 자신의 CSRF 토큰을 얻게 되며, 이는 다른 사용자의 토큰과 일치하지 않기 때문에 Django는 다른 사람이 게시한 양식을 거부할 것입니다.

CSRF 토큰은 주기적으로 변경됩니다. 이는 공격자가 특정 사용자와 토큰 조합을 이용할 수 있는 시간을 제한합니다. 그들이 악용하려는 사용자의 CSRF 토큰을 얻을 수 있다 하더라도, 사용 가능한 시간은 짧습니다.

우리는 연습 6.02에서 양식에 CSRF 토큰을 구현할 것입니다.

SQL 인젝션 방지

SQL 인젝션 공격은 웹 초창기 시절에 널리 퍼진 보안 취약점이었습니다. 사용자는 HTTP 응답 중 서버 측에서 이루어지는 데이터베이스 호출에 추가 SQL 문을 추가하는 입력을 신중하게 작성하여 양식에 SQL을 주입할 수 있었습니다.

이 접근 방식은 서버 측 코드가 잘못 작성되어 사용자 입력이 제어 문자가 이스케이프되지 않고 데이터베이스로 전송되는 것에 의존했습니다.

ORM 또는 다른 데이터베이스 도구로 코드를 개발함으로써, 코드는 사용자 입력으로부터 원시 SQL을 구성하는 것에서 벗어나게 되며, 이 위험을 제거할 수 있습니다.

우리는 3장에서 Django ORM을 사용하여 쿼리를 구성하는 방법을 보았습니다.

Content Security Policy (CSP) and XSS attacks

Cross-site scripting (XSS) attacks are a broad group of malicious acts that involve including HTML-tagged JavaScript content in input fields that can then be rendered in the display pages of the site for malicious purposes.

Consider a text field in a form on a website that is vulnerable to XSS attacks. A malicious user might enter the following:

```
"><script>malicious_function()</script><p class="
```

If the vulnerable site does not escape the HTML tag syntax, this will be rendered as follows:

```
<input type="text" name="fname" value=""> <script>malicious_function()</script><p  
class="">
```

Fortunately, the Django Forms library has been designed to protect against this kind of attack. This input would be escaped in the following way, and the HTML elements in the input would be escaped so that no script tags are injected into the web page:

```
"><script>malicious_function()</script><p class="
```

However, it would be possible to defeat this security design if a developer were to develop a custom HTML template that specifically allowed unsafe tagged content without escaping it:

```
<input type="text" name="fname" value="UNTRUSTED DATA ">
```

HTTPS and security headers

Hosting an application over HTTP makes it vulnerable to man-in-the-middle attacks, where a third party observes traffic between the client and server and compromises security by capturing that data and replaying it to the server.

By hosting our web app over HTTPS, we are preventing this vulnerability. Our development environment uses HTTP, but we will learn more about deploying our application to HTTPS in *Chapter 18*.

A Django project comes pre-configured with the middleware that supports best practice security.

The main security middleware package is `django.middleware.security.SecurityMiddleware`, which is responsible for configuring

콘텐츠 보안 정책(CSP) 및 XSS 공격

교차 사이트 스크립팅(XSS) 공격은 악의적인 목적을 위해 사이트의 표시 페이지에 렌더링될 수 있는 입력 필드에 HTML 태그가 포함된 JavaScript 콘텐츠를 포함하는 광범위한 악의적 행위 그룹입니다.

XSS 공격에 취약한 웹사이트의 양식에 있는 텍스트 필드를 고려해 보십시오. 악의적인 사용자가 다음을 입력할 수 있습니다:

```
"><script>malicious_function()</script><p class="
```

취약한 사이트가 HTML 태그 구문을 이스케이프하지 않으면, 다음과 같이 렌더링됩니다:

```
<input type="text" name="fname" value=""> <script>malicious_function()</script><p class="">
```

다행히도 Django Forms 라이브러리는 이러한 종류의 공격으로부터 보호하도록 설계되었습니다. 이 입력은 다음과 같이 이스케이프되며, 입력의 HTML 요소는 웹 페이지에 스크립트 태그가 주입되지 않도록 이스케이프됩니다:

```
"><script>malicious_function()</script><p class="
```

그러나 개발자가 안전하지 않은 태그가 있는 콘텐츠를 이스케이프하지 않고 특별히 허용하는 사용자 정의 HTML 템플릿을 개발한다면 이 보안 설계를 무너뜨릴 수 있을 것입니다:

```
<input type="text" name="fname" value="UNTRUSTED DATA ">
```

HTTPS 및 보안 헤더

HTTP를 통해 애플리케이션을 호스팅하면 중간자 공격에 취약해지며, 이 경우 제3자가 클라이언트와 서버 간의 트래픽을 관찰하고 해당 데이터를 캡처하여 서버에 재전송함으로써 보안을 손상시킵니다.

우리의 웹 앱을 HTTPS를 통해 호스팅함으로써 이 취약점을 방지하고 있습니다. 우리의 개발 환경은 HTTP를 사용하지만, 18장에서 애플리케이션을 HTTPS에 배포하는 방법에 대해 더 배울 것입니다.

Django 프로젝트는 모범 사례 보안을 지원하는 미들웨어로 미리 구성되어 있습니다.

주요 보안 미들웨어 패키지는 `django.middleware.security.SecurityMiddleware`로, 이는 구성하는 역할을 합니다.

several important security headers in HTTP responses. The headers are configurable with variables in the project's `settings.py` file.

It implements **HTTP Strict Transport Security (HSTS)** for restricting requests to HTTPS only. When in development mode, `SECURE_HSTS_SECONDS` is set to 0, but upon deployment to production, if HTTPS is configured, then it should be set to a small value, such as 60.

Content type sniffing is a technique to ascertain the content type of a file by examining a part of the data that it contains. It can be a useful technique in some file upload situations, but it is also associated with some security exploits. By default, `SECURE_CONTENT_TYPE_NOSNIFF` is set to `True`. This is the more secure option.

Django also includes `django.middleware.clickjacking.XFrameOptionsMiddleware`. In its default mode, it will add this header to HTTP responses:

```
X-Frame-Options: DENY
```

This setting is necessary for preventing clickjacking attacks on a website.

Clickjacking is an attack that involves setting up an external website that wraps the target website with inobtrusive and often invisible HTML content. The target site is loaded as an `iframe` and the clickjack wrapper site overlays malicious options onto the target site's display, so that the user is unaware that they are clicking on an external overlaid site.

Note

You can read more about security headers in the Django documentation at <https://docs.djangoproject.com/en/5.1/topics/security/>.

With this very brief introduction to form security complete, we will commence an exercise to build a Django project with a form that we can use to demonstrate some of the concepts that we have discussed.

Exercise 6.01 – Building a form in HTML

For the first few exercises of this chapter, we will need an HTML form to test with. In this exercise, we will add an HTML form to a Django project. This will also allow you to experiment with how different fields are validated and submitted. This will be done in a new Django project so that we don't interfere with Bookr. You can refer to *Chapter 1* to refresh your memory on creating a Django project:

1. We'll start by creating the new Django project. You can reuse the Bookr virtual environment that already has Django installed. Open a new terminal and activate the

HTTP 응답에서 여러 중요한 보안 헤더. 헤더는 프로젝트의 `settings.py` 파일에서 변수로 구성할 수 있습니다.

HTTPS로만 요청을 제한하기 위해 HTTP 엄격 전송 보안(HSTS)을 구현합니다. 개발 모드에서는 `SECURE_HSTS_SECONDS`가 0으로 설정되지만, 프로덕션에 배포할 때 HTTPS가 구성되면 작은 값(예: 60)으로 설정해야 합니다.

콘텐츠 유형 스니핑은 파일이 포함하고 있는 데이터의 일부를 검사하여 파일의 콘텐츠 유형을 확인하는 기술입니다. 일부 파일 업로드 상황에서 유용한 기술일 수 있지만, 일부 보안 취약점과도 관련이 있습니다. 기본적으로 `SECURE_CONTENT_TYPE_NOSNIFF`는 `True`로 설정되어 있습니다. 이것이 더 안전한 옵션입니다.

장고는 `django.middleware.clickjacking.XFrameOptionsMiddleware`도 포함합니다. 기본 모드에서는 이 헤더를 HTTP 응답에 추가합니다:

```
X-Frame-Options: DENY
```

이 설정은 웹사이트에서 클릭재킹 공격을 방지하는 데 필요합니다.

클릭재킹은 외부 웹사이트를 설정하여 대상 웹사이트를 방해가 없고 종종 보이지 않는 HTML 콘텐츠로 감싸는 공격입니다. 대상 사이트는 `iframe`으로 로드되고 클릭재킹 래퍼 사이트는 대상 사이트의 표시 위에 악성 옵션을 오버레이하여 사용자가 외부 오버레이 사이트를 클릭하고 있다는 사실을 인식하지 못하게 합니다.

참고

보안 헤더에 대한 자세한 내용은 Django 문서에서 <https://docs.djangoproject.com/en/5.1/topics/security/>를 참조하세요.

이제 양식 보안에 대한 간단한 소개가 끝났으므로, 우리가 논의한 개념 중 일부를 시연할 수 있는 양식을 가진 Django 프로젝트를 구축하는 연습을 시작하겠습니다.

연습 6.01 – HTML로 양식 만들기

이 장의 처음 몇 개의 `idx_16d45e55` 연습 `idx_3d947358`에서는 테스트할 HTML 양식이 필요합니다. 이 연습에서는 Django 프로젝트에 HTML 양식을 추가할 것입니다. 이를 통해 다양한 필드가 어떻게 검증되고 제출되는지 실험할 수 있습니다. 이는 Bookr와 간섭하지 않도록 새로운 Django 프로젝트에서 진행됩니다. Django 프로젝트를 만드는 방법에 대한 기억을 되살리려면 1장을 참조하세요:

1. 새로운 Django 프로젝트를 만드는 것으로 시작하겠습니다. 이미 Django가 설치된 Bookr 가상 환경을 재사용할 수 있습니다. 새 터미널을 열고 활성화하세요.

virtual environment. Then, use `django-admin` to start a Django project named `form_project`. To do this, run the following command:

```
django-admin startproject form_project
```

This will scaffold the Django project in a directory named `form_project`.

2. Create a new Django app in this project by using the `startapp` management command. The app should be called `form_example`. To do this, `cd` into the `form_project` directory, then run the following:

```
python manage.py startapp form_example
```

This will create the `form_example` app directory inside the `form_project` directory.

3. Launch PyCharm, then open the `form_project` directory. If you already have a project open, you can do this by choosing **File | Open**; otherwise, just click **Open** in the **Welcome to PyCharm** window. Navigate to the `form_project` directory, select it, then click **Open**.

If prompted, be sure to choose **Trust Project**.

The `form_project` window should be shown like this:

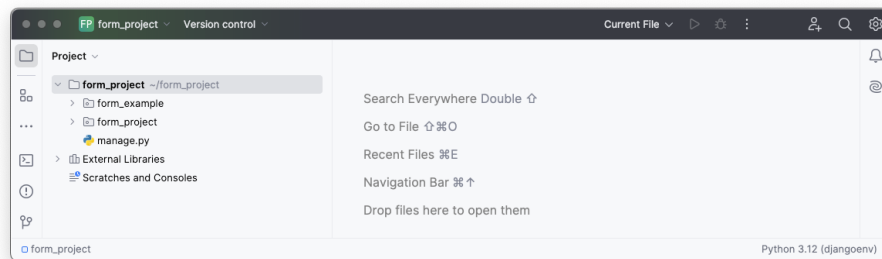


Figure 6.1: The `form_project` project open

4. Create a new run configuration to execute `manage.py runserver` for the project by following the **Edit Configuration** option under the **Current File** menu at the top of the window. You can reuse the virtual environment that you have already created. The **Run/Debug Configurations** window should look similar to the following figure when you're done:

가상 환경. 그런 다음 `django-admin`을 사용하여 `form_project`라는 이름의 Django 프로젝트를 시작합니다. 이렇게 하려면 다음 명령을 실행하십시오:

```
django-admin startproject form_project
```

이렇게 하면 `form_project`라는 이름의 디렉토리에 Django 프로젝트가 생성됩니다.

2. `startapp` 관리 명령을 사용하여 이 프로젝트에 새 Django 앱을 만듭니다. 앱의 이름은 `form_example`이어야 합니다. 이렇게 하려면 `form_project` 디렉토리로 이동한 다음 다음을 실행하십시오:

```
python manage.py startapp form_example
```

이렇게 하면 `form_project` 디렉토리 내에 `form_example` 앱 디렉토리가 생성됩니다.

3. PyCharm을 실행한 후 `form_project` 디렉토리를 엽니다. 이미 프로젝트가 열려 있는 경우 파일 | 열기를 선택하여 이 작업을 수행할 수 있습니다. 그렇지 않으면 PyCharm 환영 창에서 열기를 클릭하십시오. `form_project` 디렉토리로 이동하여 선택한 다음 열기를 클릭합니다.

프롬프트가 표시되면 `idx_e73f4bb0`에서 프로젝트를 신뢰하도록 선택해야 합니다. `form_project` 창은 다음과 같이 표시되어야 합니다:

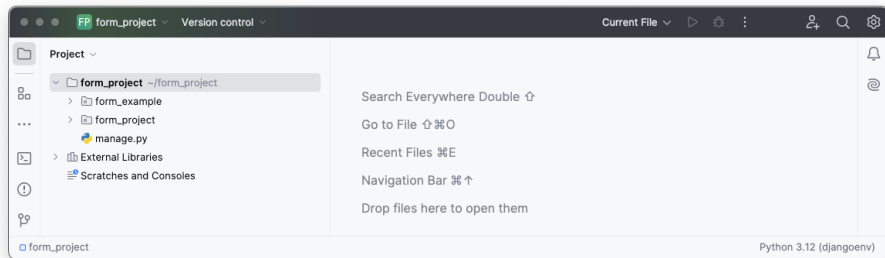


그림 6.1: `form_project` 프로젝트 열기

4. 현재 파일 메뉴의 구성 편집 옵션을 따라 `manage.py runserver`를 실행하기 위해 새 실행 구성을 만듭니다. 이미 생성한 가상 환경을 재사용할 수 있습니다. 실행/디버그 구성 창 `idx_78e99279`는 작업이 완료되면 다음 그림 `idx_90ed30f6`과 유사하게 보여야 합니다:

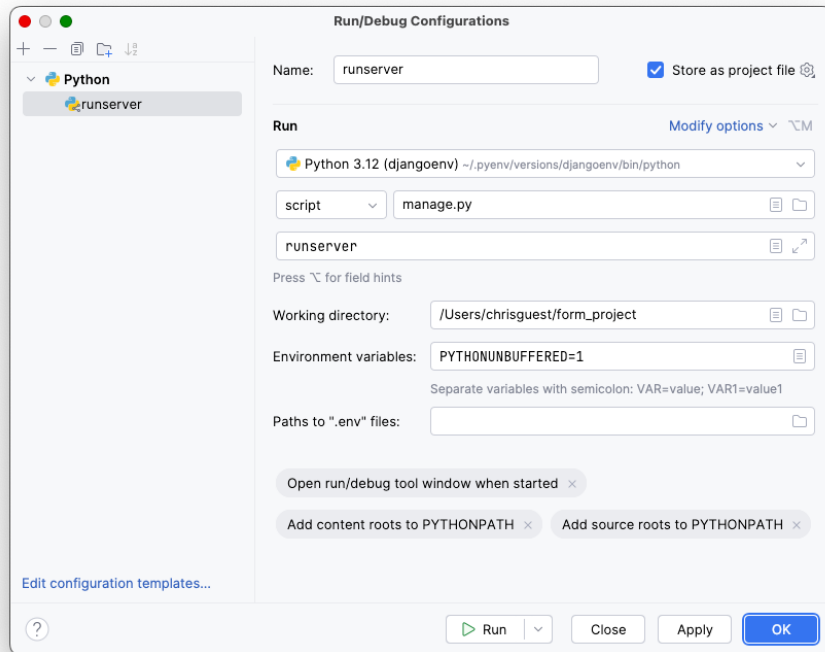


Figure 6.2: Run/Debug Configurations for runserver

You can test that the configuration has been set up correctly by clicking the **Run** button, and then visiting `http://127.0.0.1:8000/` in your browser. You should see the Django welcome screen. If the debug server fails to start or you see the Bookr main page, then you probably still have the Bookr project running. Try stopping the Bookr runserver process and then starting the new one you just set up.

5. Open `settings.py` in the `form_project` directory and add `'form_example'` to the `INSTALLED_APPS` setting.

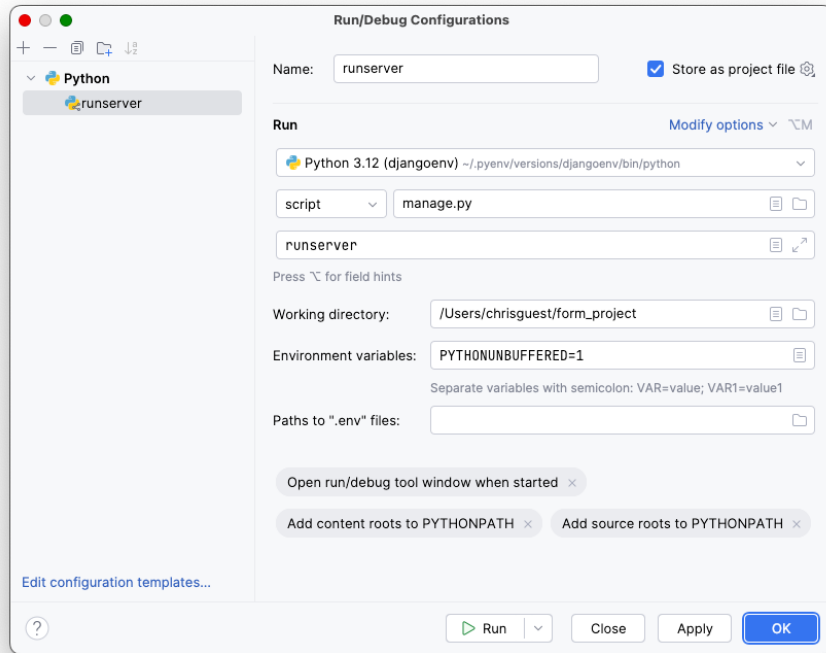


그림 6.2: runserver에 대한 실행/디버그 구성

구성이 올바르게 설정되었는지 확인하려면 실행 버튼을 클릭한 다음 브라우저에서 <http://127.0.0.1:8000/>를 방문하세요. Djangowelcome 화면이 표시되어야 합니다. 디버그 서버가 시작되지 않거나 Bookr 메인 페이지가 표시되면 아마도 여전히 Bookr 프로젝트가 실행 중일 것입니다. Bookr runserver 프로세스를 중지한 다음 방금 설정한 새 프로세스를 시작해 보세요.

5. form_project 디렉토리에서 settings.py를 열고 INSTALLED_APPS 설정에 'form_example'을 추가하세요.

6. The last step in setting up this new project is to create a templates directory for the `form_example` app. Right-click on the `form_example` directory, then select **New | Directory**. Name it `templates`.
7. We need an HTML template to display our form. We can copy the HTML for the form example from the Git repository by visiting this URL and saving the file to the `form_example` directory in the `Chapter06` folder in this book's GitHub repo. .
8. The code for this html page appears in *Figure 6.3*. Its `method` attribute is set to `post`. The `action` attribute is omitted, which means the form will be submitted back to the same URL on which it was loaded:

```
<form method="post">
```

9. Inside the `form` tag are several `<p>` tags, which wrap the form inputs and create some spacing to give the form a visually pleasing layout:

6. 이 새로운 프로젝트를 설정하는 마지막 단계는 form_example 앱을 위한 templates 디렉토리를 만드는 것입니다. form_example 디렉토리를 마우스 오른쪽 버튼으로 클릭한 다음 새로 만들기 | 디렉토리를 선택합니다. 이름을 templates로 지정합니다.
7. 양식을 표시하기 위해 HTML idx_82af8d0btemplate이 필요합니다. 이 URL을 방문하여 Git 리포지토리에서 formexample의 HTML을 복사하고 이 책의 GitHub 리포지토리의 Chapter06 폴더에 있는 form_example 디렉토리에 파일을 저장할 수 있습니다.
8. 이 HTML 페이지의 코드는 그림 6.3에 나타나 있습니다. 그 메서드 속성은 post로 설정되어 있습니다. action 속성은 생략되어 있으며, 이는 양식이 로드된 동일한 URL로 제출된다는 것을 의미합니다:

```
<form method="post">
```

9. 폼 태그 안에는 여러 개의 <p> 태그가 있으며, 이 태그들은 폼 입력을 감싸고 폼에 시각적으로 보기 좋은 레이아웃을 제공하기 위해 약간의 여백을 만듭니다:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
<form method="post">
  <p><label for="id_text_input">Text Input</label><br>
    <input id="id_text_input" type="text" name="text_input"
      value="" placeholder="Enter some text"></p>
  <p><label for="id_password_input">Password Input</label><br>
    <input id="id_password_input" type="password" name="password_input"
      value="" placeholder="Your password"></p>
  <p><input id="id_checkbox_input" type="checkbox" name="checkbox_on"
    value="Checkbox Checked" checked>
    <label for="id_checkbox_input">Checkbox</label></p>
  <p><input id="id_radio_one_input" type="radio" name="radio_input" value="Value One">
    <label for="id_radio_one_input">Value One</label>
    <input id="id_radio_two_input" type="radio" name="radio_input" value="Value Two" checked>
    <label for="id_radio_two_input">Value Two</label>
    <input id="id_radio_three_input" type="radio" name="radio_input" value="Value Three">
    <label for="id_radio_three_input">Value Three</label></p>
  <p><label for="id_favorite_book">Favorite Book</label><br>
    <select id="id_favorite_book" name="favorite_book">
      <optgroup label="Non-Fiction">
        <option value="1">Deep Learning with Keras</option>
        <option value="2">Web Development with Django</option>
      </optgroup>
      <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
      </optgroup>
    </select></p>
  <p><label for="id_books_you_own">Books You Own</label><br>
    <select id="id_books_you_own" name="books_you_own" multiple>
      <optgroup label="Non-Fiction">
        <option value="1">Deep Learning with Keras</option>
        <option value="2">Web Development with Django</option>
      </optgroup>
      <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
      </optgroup>
    </select></p>
  <p><label for="id_text_area">Text Area</label><br>
    <textarea name="text_area" id="id_text_area"
      placeholder="Enter multiple lines of text"></textarea></p>
  <p><label for="id_number_input">Number Input</label><br>
    <input id="id_number_input" type="number" name="number_input"
      value="" step="any" placeholder="A number"></p>
  <p><label for="id_email_input">Email Input</label><br>
    <input id="id_email_input" type="email" name="email_input"
      value="" placeholder="Your email address"></p>
  <p><label for="id_date_input">Date Input</label><br>
    <input id="id_date_input" type="date" name="date_input" value="2019-11-23"></p>
  <p><input type="submit" name="submit_input" value="Submit Input"></p>
  <p>
    <button type="submit" name="button_element" value="Button Element">
      Button With <strong>Styled</strong> Text
    </button></p>
  <input type="hidden" name="hidden_input" value="Hidden Value">
</form>
</body>
</html>

```

Figure 6.3: form-example.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
<form method="post">
  <p><label for="id_text_input">Text Input</label><br>
    <input id="id_text_input" type="text" name="text_input"
      value="" placeholder="Enter some text"></p>
  <p><label for="id_password_input">Password Input</label><br>
    <input id="id_password_input" type="password" name="password_input"
      value="" placeholder="Your password"></p>
  <p><input id="id_checkbox_input" type="checkbox" name="checkbox_on"
    value="Checkbox Checked" checked>
    <label for="id_checkbox_input">Checkbox</label></p>
  <p><input id="id_radio_one_input" type="radio" name="radio_input" value="Value One">
    <label for="id_radio_one_input">Value One</label>
    <input id="id_radio_two_input" type="radio" name="radio_input" value="Value Two" checked>
    <label for="id_radio_two_input">Value Two</label>
    <input id="id_radio_three_input" type="radio" name="radio_input" value="Value Three">
    <label for="id_radio_three_input">Value Three</label></p>
  <p><label for="id_favorite_book">Favorite Book</label><br>
    <select id="id_favorite_book" name="favorite_book">
      <optgroup label="Non-Fiction">
        <option value="1">Deep Learning with Keras</option>
        <option value="2">Web Development with Django</option>
      </optgroup>
      <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
      </optgroup>
    </select></p>
  <p><label for="id_books_you_own">Books You Own</label><br>
    <select id="id_books_you_own" name="books_you_own" multiple>
      <optgroup label="Non-Fiction">
        <option value="1">Deep Learning with Keras</option>
        <option value="2">Web Development with Django</option>
      </optgroup>
      <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
      </optgroup>
    </select></p>
  <p><label for="id_text_area">Text Area</label><br>
    <textarea name="text_area" id="id_text_area"
      placeholder="Enter multiple lines of text"></textarea></p>
  <p><label for="id_number_input">Number Input</label><br>
    <input id="id_number_input" type="number" name="number_input"
      value="" step="any" placeholder="A number"></p>
  <p><label for="id_email_input">Email Input</label><br>
    <input id="id_email_input" type="email" name="email_input"
      value="" placeholder="Your email address"></p>
  <p><label for="id_date_input">Date Input</label><br>
    <input id="id_date_input" type="date" name="date_input" value="2019-11-23"></p>
  <p><input type="submit" name="submit_input" value="Submit Input"></p>
  <p>
    <button type="submit" name="button_element" value="Button Element">
      Button With <strong>Styled</strong> Text
    </button></p>
  <input type="hidden" name="hidden_input" value="Hidden Value">
</form>
</body>
</html>

```

그림 6.3: form-example.html

10. As with any template, we can't see it unless we have a view to render it. Open the `form_example` app's `views.py` file and add a new view called `form_example`. It should render and return the template you just created, like so:

```
def form_example(request):  
    return render(request, "form-example.html")
```

You can now save and close `views.py`.

11. You should be familiar with the next step now, which is to add a URL mapping to the view. Open the `urls.py` file in the `form_project` package directory. Add a mapping for the `form-example` path to your `form_example` view, to the `urlpatterns` variable. It should look like this:

```
path('form-example/', form_example.views.form_example)
```

Make sure you also add an import of `form_example.views`. Save and close `urls.py`.

12. Start the Django dev server (if it is not already running), then load your new view in your web browser; the address is `http://127.0.0.1:8000/form-example/`. Your page should look like this:

10. 어떤 템플릿과 마찬가지로, 렌더링할 뷰가 없으면 볼 수 없습니다. `form_example` 앱의 `views.py` 파일을 열고 `form_example`이라는 새 뷰를 추가하세요. 방금 생성한 템플릿을 렌더링하고 반환해야 합니다. 다음과 같이 작성하세요:

```
def form_example(request):  
    return render(request, "form-example.html")
```

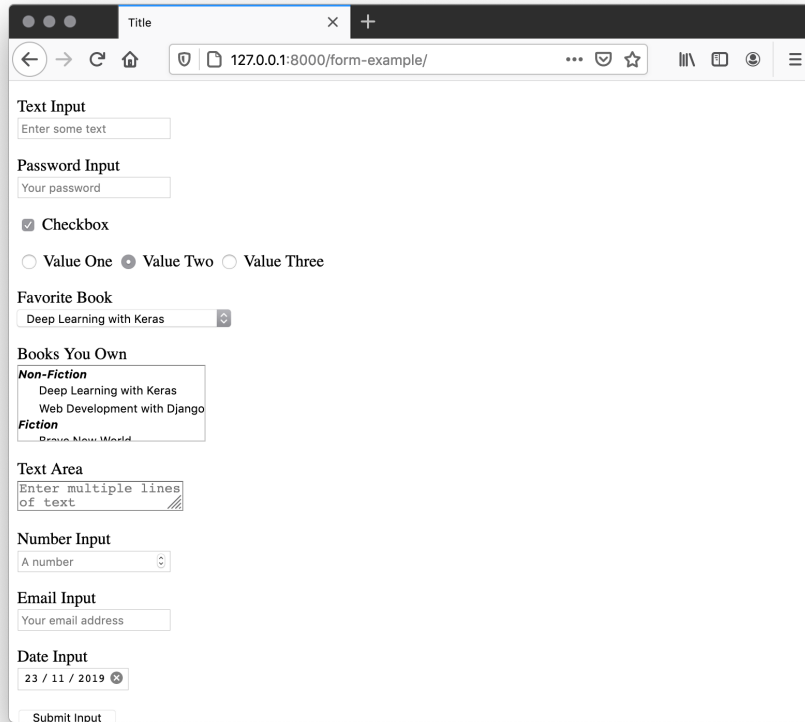
이제 `views.py`를 저장하고 닫을 수 있습니다.

11. 이제 다음 단계에 익숙해져야 합니다. 이는 뷰에 URL 매핑을 추가하는 것입니다. `form_project` 패키지 디렉토리의 `urls.py` 파일을 엽니다. `form-example` 경로에 대한 `form_example` 뷰에 `idx_67229444` 매핑 `idx_7c815636`을 `urlpatterns` 변수에 추가하세요. 다음과 같이 보일 것입니다:

```
path('form-example/', form_example.views.form_example)
```

`form_example.views`를 가져오는 것도 잊지 마세요. `urls.py`를 저장하고 닫으세요.

12. Django 개발 서버를 시작하세요(이미 실행 중이지 않은 경우), 그런 다음 웹 브라우저에서 새 보기를 로드하세요; 주소는 `http://127.0.0.1:8000/form-example/`입니다. 페이지는 다음과 같아야 합니다:



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/form-example/'. The form contains the following elements:

- Text Input:** A text box with the placeholder 'Enter some text'.
- Password Input:** A text box with the placeholder 'Your password'.
- Checkbox:** A checked checkbox labeled 'Checkbox'.
- Radio Buttons:** Three radio buttons labeled 'Value One', 'Value Two' (selected), and 'Value Three'.
- Favorite Book:** A dropdown menu showing 'Deep Learning with Keras'.
- Books You Own:** A section with two categories: 'Non-Fiction' (listing 'Deep Learning with Keras' and 'Web Development with Django') and 'Fiction' (listing 'Django: A Novel').
- Text Area:** A text area with the placeholder 'Enter multiple lines of text'.
- Number Input:** A text box with the placeholder 'A number'.
- Email Input:** A text box with the placeholder 'Your email address'.
- Date Input:** A date picker showing '23 / 11 / 2019'.
- Submit Input:** A button labeled 'Submit Input'.

Figure 6.4: Example inputs page

We have not yet set up everything for form submission, so if you correct all the errors in the form and try to submit it (by clicking either of the **Submit** buttons), you will receive an error stating **CSRF verification failed**, as we can see in the following figure. This demonstrates Django's CSRF protection, which we discussed in the previous section. When you receive this error, just go back in your browser to return to the input example page.

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/form-example/'. The page contains a form with the following elements:

- Text Input:** A text box with the placeholder 'Enter some text'.
- Password Input:** A password box with the placeholder 'Your password'.
- Checkbox:** A checked checkbox labeled 'Checkbox'.
- Radio buttons:** Three radio buttons labeled 'Value One', 'Value Two' (selected), and 'Value Three'.
- Favorite Book:** A dropdown menu showing 'Deep Learning with Keras'.
- Books You Own:** A list of books categorized into 'Non-Fiction' (Deep Learning with Keras, Web Development with Django) and 'Fiction' (Django Atlas Model).
- Text Area:** A text area with the placeholder 'Enter multiple lines of text'.
- Number Input:** A number box with the placeholder 'A number'.
- Email Input:** An email box with the placeholder 'Your email address'.
- Date Input:** A date box showing '23 / 11 / 2019'.
- Submit Input:** A button labeled 'Submit Input'.

그림 6.4: 예제 입력 페이지

아직 양식 제출을 위한 모든 설정을 완료하지 않았으므로, 양식의 모든 오류를 수정하고 제출 버튼 중 하나를 클릭하여 제출을 시도하면 CSRF 검증 실패라는 오류가 발생합니다. 이는 다음 idx_df8fe598 그림에서 볼 수 있습니다. 이는 우리가 이전 섹션에서 논의한 Django의 CSRF 보호 기능을 보여줍니다. 이 오류가 발생하면 브라우저에서 뒤로 가기를 눌러 입력 예제 페이지로 돌아가세요.

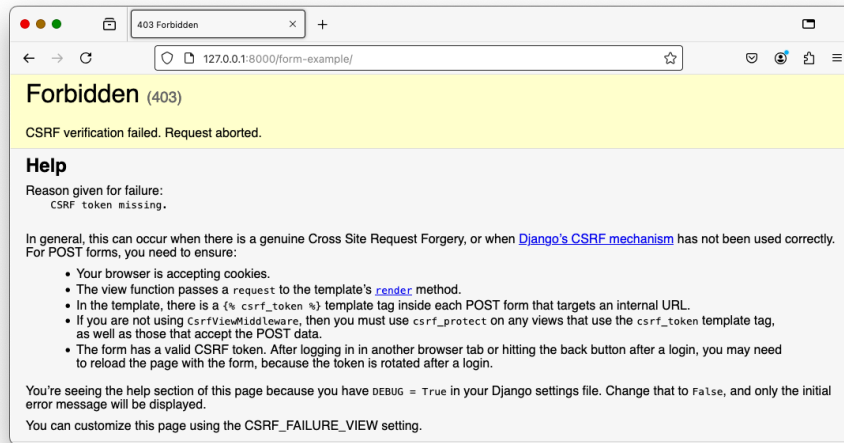


Figure 6.5: CSRF verification error

In this exercise, you created an example page showcasing many HTML inputs, then created a view to render it and a URL to map to it. You loaded the page in your browser and experimented with changing data and trying to submit the form when it contained errors.

We just saw an example of Django's CSRF protection. In the following exercise, we will fix the CSRF token error and learn about how Django handles form submissions. Before we get to that, let's have a quick refresh about accessing data in a request using QueryDict objects.

Accessing data in the view

As we discussed in *Chapter 1*, Django provides two QueryDict objects on the HTTPRequest instances that are passed to the view function. These are `request.GET`, which contains parameters passed in the URL, and `request.POST`, which contains parameters in the HTTP request body. Even though `request.GET` has GET in its name; this variable is populated even for non-GET HTTP requests. This is because the data it contains is parsed from the URL. Since all HTTP requests have a URL, all HTTP requests may contain GET data, even if they are POST or PUT, and so on.

In the next exercise, we will add code to our view to read and display the POST data.

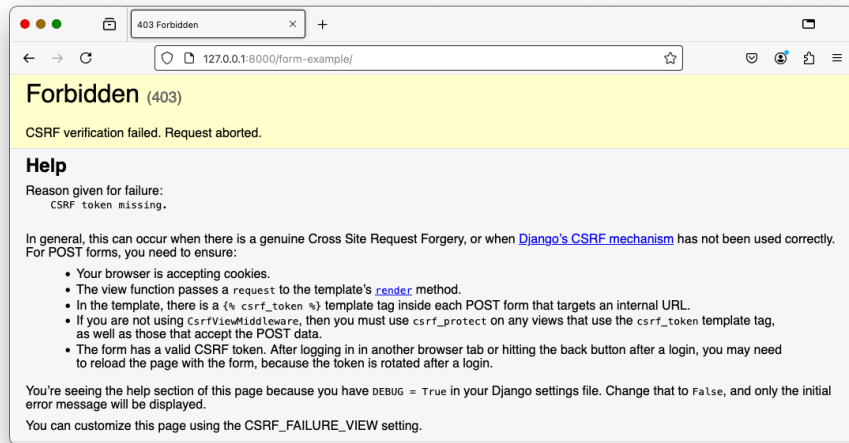


그림 6.5: CSRF 검증 오류

이 연습에서는 많은 HTML 입력을 보여주는 예제 페이지를 만들고, 이를 렌더링할 뷰와 매핑할 URL을 생성했습니다. 브라우저에서 페이지를 로드하고 데이터를 변경하고 오류가 포함된 양식을 제출하려고 실험했습니다.

우리는 방금 Django의 CSRF 보호의 예인 `idx_2875eb38` 예제 `idx_1d28c325`를 보았습니다. 다음 연습에서는 CSRF 토큰 오류를 수정하고 Django가 양식 제출을 처리하는 방법에 대해 배울 것입니다. 그에 앞서, `QueryDict` 객체를 사용하여 요청에서 데이터에 접근하는 방법에 대해 간단히 복습해 보겠습니다.

뷰에서 데이터 접근하기

1장에서 논의한 바와 같이, Django는 뷰 함수에 전달되는 `HttpRequest` 인스턴스에서 두 개의 `QueryDict` 객체를 제공합니다. 이들은 URL에 전달된 매개변수를 포함하는 `request.GET`과 HTTP 요청 본문에 있는 매개변수를 포함하는 `request.POST`입니다. `request.GET`은 이름에 `GET`이 포함되어 있지만, 이 변수는 비-`GET` HTTP 요청에 대해서도 채워집니다. 이는 포함된 데이터가 URL에서 파싱되기 때문입니다. 모든 HTTP 요청은 URL을 가지므로, 모든 HTTP 요청은 `GET` 데이터를 포함할 수 있으며, 비록 그것들이 `POST` 또는 `PUT` 등일지라도 마찬가지입니다.

다음 연습에서는 `POST` 데이터를 읽고 표시하기 위해 뷰에 코드를 추가할 것입니다.

Exercise 6.02 – Working with POST data in a view

We will now add some code to our example view to print out the received POST data to the console. We will also insert the HTTP method that was used to generate the page into the HTML output. This will allow us to be sure of what method was used to generate the page (GET or POST) and see how the form differs for each type:

1. First, in PyCharm, open the `form_example` app's `views.py` file. Alter the `form_example` view so that it prints each value in the POST to the console by adding this code inside the function:

```
for name in request.POST:
    print(f"{name}: {request.POST.getlist(name)}")
```

This code iterates over each key in the POST request data's `QueryDict` object and prints the key and list of values to the console. We already know that each `QueryDict` object can have multiple values for a key, so we use the `getlist` function to get them all.

2. Pass the template's `request.method` into a context variable named `method`. Do this by updating the call to `render` in the view so that it's like this:

```
return render(request, "form-example.html",
              {"method": request.method})
```

3. We will now display the `method` variable in the template. Open the `form-example.html` template and use a `<h4>` tag to show the `method` variable. Put this just after the opening `<body>` tag, like so:

```
<body>
    <h4>Method: {{ method }}</h4>
```

Note that we could access the `method` directly inside the template without passing it in a context dictionary, by using the `request.method` variable. We know from *Chapter 3, URL Mapping, Views, and Templates*, that by using the `render` shortcut function, the `request` is always available in the template. We just demonstrated how to access the `method` in the view here because, later on, we will change the behavior of the page based on the `method`.

연습 6.02 – 뷰에서 POST 데이터 작업하기

이제 예제 뷰에 코드를 추가하여 수신된 POST 데이터를 콘솔에 출력하겠습니다. 또한 페이지를 생성하는 데 사용된 HTTP 메서드를 HTML 출력에 삽입하겠습니다. 이를 통해 페이지를 생성하는 데 사용된 메서드(GET 또는 POST)를 확실히 알 수 있으며 각 유형에 따라 양식이 어떻게 다른지 확인할 수 있습니다:

1. 먼저, PyCharm에서 form_example 앱의 views.py 파일을 엽니다.
form_example 뷰를 수정하여 POST의 각 값을 콘솔에 출력하도록 이 코드를 함수 내부에 추가합니다:

```
for name in request.POST:
    print(f"{name}: {request.POST.getlist(name)}")
```

이 코드는 POST 요청 데이터의 QueryDict 객체에서 각 키를 반복하고 키와 값 목록을 콘솔에 출력합니다. 각 QueryDict 객체는 키에 대해 여러 값을 가질 수 있으므로 getlist 함수를 사용하여 모든 값을 가져옵니다.

2. 템플릿의 request.method를 method라는 이름의 컨텍스트 변수로 전달하세요.
이 작업은 뷰에서 render 호출을 다음과 같이 업데이트하여 수행합니다:

```
return render(request, "form-example.html",
              {"method": request.method})
```

3. 이제 템플릿에 메서드 변수를 표시하겠습니다. form-example.html 파일을 엽니다.
템플릿을 만들고 <h4> 태그를 사용하여 method 변수를 표시하세요. 이것을 여는 태그 바로 다음에 넣으세요.
<body> 태그, 다음과 같이:

```
<body>
...
<h4>방법: {{ method }}</h4>
```

템플릿 내에서 context 딕셔너리를 통해 전달하지 않고도 request.method 변수를 사용하여 메서드에 직접 접근할 수 있다는 점에 유의하세요. 3장 URL 매핑, 뷰, 템플릿에서 render 단축 함수(render shortcut function)를 사용하면 request가 항상 템플릿에서 사용 가능하다는 것을 알 수 있습니다. 여기서는 나중에 메서드에 따라 페이지의 동작을 변경할 것이기 때문에 뷰에서 메서드에 접근하는 방법을 보여주었습니다.

4. We also need to add the CSRF token to the form HTML. We do this by putting the `{% csrf_token %}` template tag after the opening `<form>` tag. The start of the form should look like this:

```
<form method="post">
    {% csrf_token %}
```

Now, save the file.

5. Start the Django dev server if it's not already running. Load the example page (`http://127.0.0.1:8000/form-example/`) in your browser; you should see that it now displays the method at the top of the page (**GET**):

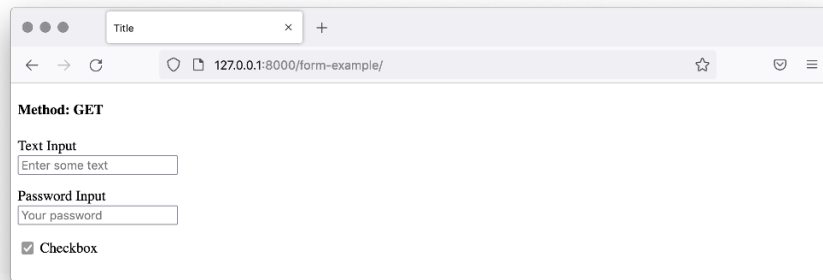


Figure 6.6: Method at the top of the page

6. Enter some text or data in each of the inputs and submit the form by clicking the **Submit Input** button:

4. 우리는 또한 CSRF 토큰을 폼 HTML에 추가해야 합니다. 우리는 {%csrf_token %} 템플릿 태그를 여는 <form> 태그 뒤에 넣음으로써 이를 수행합니다. 폼의 시작은 다음과 같아야 합니다:

```
<form method="post">
    {% csrf_token %}
```

이제 파일을 저장하세요. Django 개발 서버가 이미 실행 중이지 않다면 시작하세요.

5. 브라우저에서 예제 페이지 (<http://127.0.0.1:8000/form-example/>)를 로드하세요; 페이지 상단에 메시드가 표시되는 것을 볼 수 있어야 합니다 (GET):

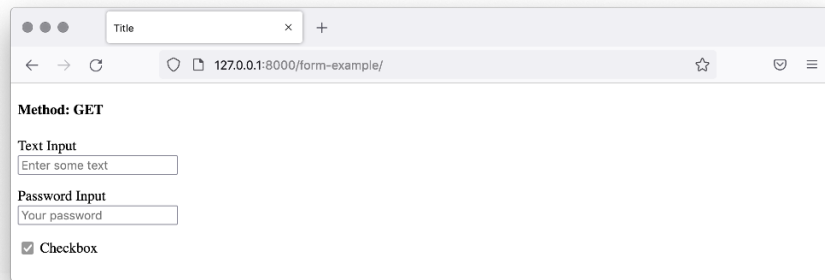


그림 6.6: 페이지 상단의 방법

6. 각 입력란에 텍스트나 데이터를 입력하고 Submit Input 버튼을 클릭하여 양식을 제출하세요:

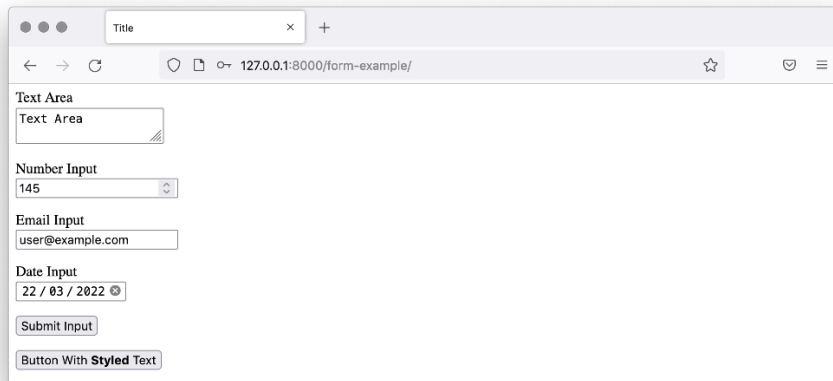


Figure 6.7: Clicking the Submit Input button to submit the form

You should see the page reload and the method display change to **POST**:

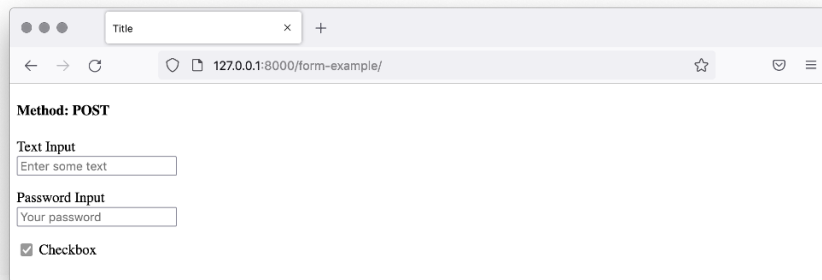


Figure 6.8: Method updated to POST after form is submitted

7. Switch back to PyCharm and look in the **Run** console at the bottom of the window. If it is not visible, click the **Run** button at the bottom of the window to show it:

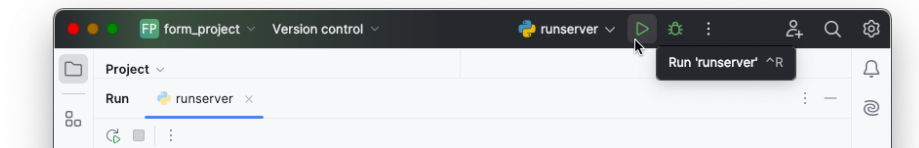


Figure 6.9: Clicking the Run button at the bottom of the window to display the console

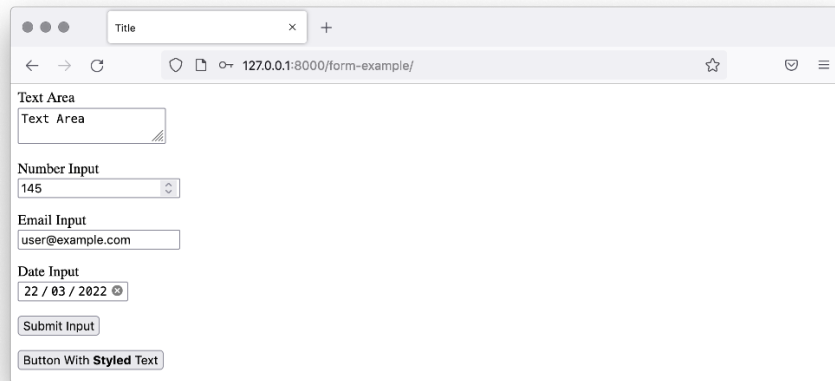


그림 6.7: 제출 입력 버튼을 클릭하여 양식을 제출합니다

페이지가 새로 고쳐지고 메시드가 POST로 변경되는 것을 확인해야 합니다:

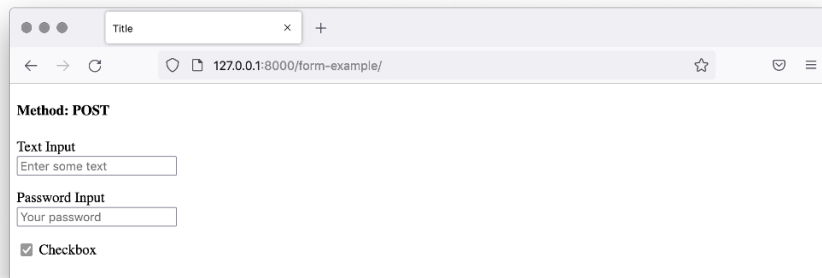


그림 6.8: 양식 제출 후 메시드가 POST로 업데이트됨

7. PyCharm으로 돌아가서 창 하단의 실행 콘솔을 확인하세요. 보이지 않으면 창 하단의 실행 버튼을 클릭하여 표시하세요:

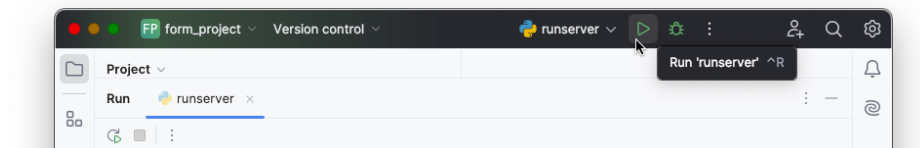
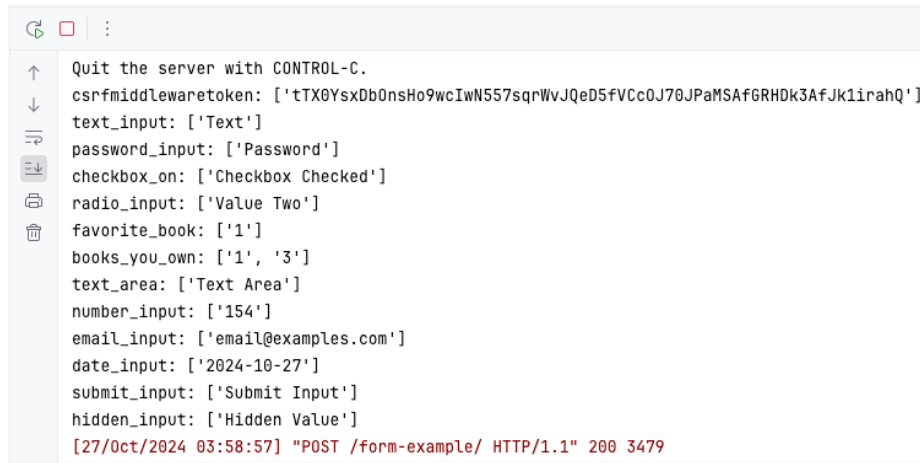


그림 6.9: 창 하단의 실행 버튼을 클릭하여 콘솔을 표시합니다

If you're running the dev server in **Debug** mode, click **Debug** instead.

Inside the **Run** console, the list of values that were posted to the server should be displayed:



```

Quit the server with CONTROL-C.
csrfmiddlewaretoken: ['tTX0YsxD0nsHo9wcIwN557sqrWvJQeD5fVCC0J70JPaMSAf6RHDk3AfJk1irahQ']
text_input: ['Text']
password_input: ['Password']
checkbox_on: ['Checkbox Checked']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
number_input: ['154']
email_input: ['email@example.com']
date_input: ['2024-10-27']
submit_input: ['Submit Input']
hidden_input: ['Hidden Value']
[27/Oct/2024 03:58:57] "POST /form-example/ HTTP/1.1" 200 3479

```

Figure 6.10: Input values shown in the Run console

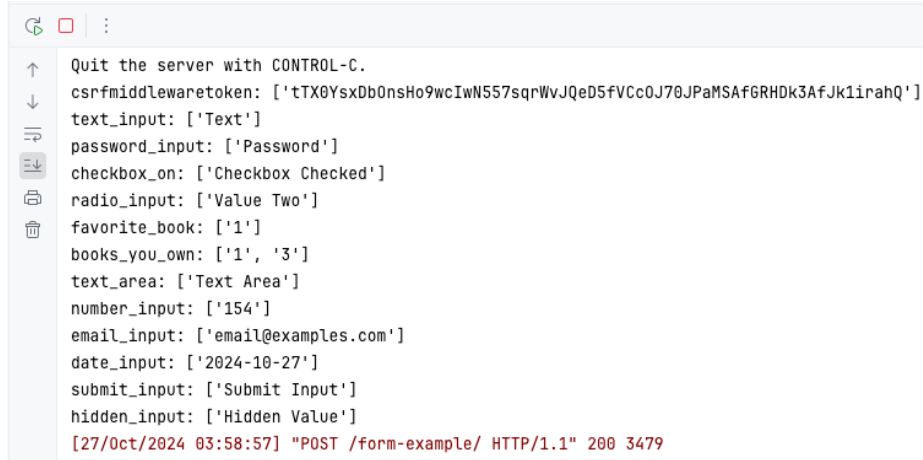
Here are some things you should note:

- All values are sent as text, even number and date inputs.
- For the select inputs, the selected value attributes of the selected options are sent, not the text content of the option tag.
- If you select multiple options for `books_you_own`, then you will see multiple values in the request. This is why we use the `getList` method since multiple values are sent for the same input name.
- If the checkbox was checked, you will have a `checkbox_on` input in the debug output. If it was not checked, then the key will not exist at all (that is, there is no key instead of having the key existing with an empty string or a `None` value).
- We have a value for the `submit_input` name, which is the text **Submit Input**. You submitted the form by clicking the **Submit Input** button, so we received its value. Notice that no value is set for the `button_element` input since that button was not clicked.

8. We will experiment with two other ways of submitting the form – first, by typing Enter when your cursor is in a text-like input (*Text*, *Password*, *Date*, or *Email*, but not *Text Area*, as typing Enter will add a new line).

디버그 모드에서 개발 서버를 실행 중이라면, 대신 디버그를 클릭하세요.

실행 콘솔 안에서 서버에 게시된 값 목록은 다음과 같이 표시되어야 합니다:



```

Quit the server with CONTROL-C.
csrfmiddlewaretoken: ['tTX0YsxD0nsHo9wcIwN557sqrWvJQeD5fVcC0J70JPaMSAf6RHDk3AfJk1irahQ']
text_input: ['Text']
password_input: ['Password']
checkbox_on: ['Checkbox Checked']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
number_input: ['154']
email_input: ['email@examples.com']
date_input: ['2024-10-27']
submit_input: ['Submit Input']
hidden_input: ['Hidden Value']
[27/Oct/2024 03:58:57] "POST /form-example/ HTTP/1.1" 200 3479

```

그림 6.10: 실행 콘솔에 표시된 입력 값들

다음은 주의해야 할 몇 가지 사항입니다:

- 모든 값은 텍스트로 전송되며, 숫자 및 날짜 입력도 포함됩니다.
 - 선택 입력의 경우, 선택된 옵션의 선택된 값 속성이 존재하며, 옵션 태그의 텍스트 내용은 아닙니다.
 - books_you_own에 대해 여러 옵션을 선택하면 요청에서 여러 값이 표시됩니다. 이는 동일한 입력 이름에 대해 여러 값이 전송되기 때문에 getlist 메서드를 사용하는 이유입니다.
 - 체크박스가 선택되었다면, debugoutput에 checkbox_on 입력이 있을 것입니다. 선택되지 않았다면, 키는 전혀 존재하지 않을 것입니다(즉, 빈 문자열이나 None 값이 있는 키가 존재하는 대신 키가 없습니다). submit_input 이름에 대한 값이 있으며, 이는 텍스트 Submit Input입니다.
 - Submit Input 버튼을 클릭하여 양식을 제출했으므로 그 값을 받았습니다. 해당 버튼이 클릭되지 않았기 때문에 button_element 입력에 대한 값은 설정되지 않았습니다.
8. idx_2d3a4943 실험을 위해 양식을 제출하는 두 가지 다른 방법으로 idx_d81e0665를 사용해 보겠습니다 – 첫째, 커서가 텍스트와 유사한 입력(Text, Password, Date 또는 Email)에 있을 때 Enter를 입력하는 것입니다. 단, Text Area에서는 Enter를 입력하면 새 줄이 추가됩니다.

If you submit a form in this way, the form will act as though you had clicked the first **Submit** button on the form, so the `submit_input` input value will be included. The output you see should match that of the previous figure.

The other way to submit the form is by clicking the Button Element submit input, where we will try clicking this button to submit the form. You should see that `submit_button` is no longer in the list of posted values, while `button_element` is now present:

```
csrfmiddlewaretoken: ['tTX0YsxDb0nsHo9wcIwN557sqrWvJQeD5fVCC0J70JPaMSAf6RHDk3AfJk1irahQ']
text_input: ['Text']
password_input: ['']
checkbox_on: ['Checkbox Checked']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
number_input: ['154']
email_input: ['email@example.com']
date_input: ['2024-10-27']
submit_input: ['Submit Input']
hidden_input: ['Hidden Value']
```

Figure 6.11: submit_button is now gone from the inputs, and button_element has been added

You can use this multiple-submit technique to alter how your view behaves, depending on which button was clicked. You can even have multiple submit buttons with the same name attribute to make the logic easier to write.

In this exercise, you added a CSRF token to your form element by using the `{% csrf_token %}` template tag. This means that your form can be submitted to Django successfully without it generating an **HTTP Permission Denied** response. We then added some code to output the values that our form contained when it was submitted. We tried submitting the form with various values to see how they are parsed into Python variables on the `request.POST` part of `QueryDict`. We will now discuss some more of the theory around the difference between GET and POST requests, then move on to the Django Forms library, which makes designing and validating forms easier.

이런 방식으로 양식을 제출하면, 양식은 첫 번째 제출 버튼을 클릭한 것처럼 작동하므로 submit_input 입력 값이 포함됩니다. 당신이 보는 출력은 이전 그림과 일치해야 합니다.

양식을 제출하는 다른 방법은 버튼 요소 제출 입력을 클릭하는 것입니다. 여기서 우리는 이 버튼을 클릭하여 양식을 제출해 보겠습니다. submit_button이 더 이상 게시된 값 목록에 없고, button_element가 이제 존재하는 것을 볼 수 있어야 합니다:

```
csrfmiddlewaretoken: ['tTX0YsxDb0nsHo9wcIwN557sqrWvJQeD5fVCc0J70JPaMSAf6RHDk3AfJk1irahQ']
text_input: ['Text']
password_input: ['']
checkbox_on: ['Checkbox Checked']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
number_input: ['154']
email_input: ['email@examples.com']
date_input: ['2024-10-27']
submit_input: ['Submit Input']
hidden_input: ['Hidden Value']
```

그림 6.11: submit_button이 이제 입력에서 사라지고, button_element가 추가되었습니다

이 다중 제출 기술을 사용하여 클릭된 버튼에 따라 뷰의 동작 방식을 변경할 수 있습니다. 동일한 이름 속성을 가진 여러 제출 버튼을 만들어 논리를 쉽게 작성할 수도 있습니다.

이 연습에서는 {% csrf_token %} 템플릿 태그를 사용하여 양식 요소에 CSRF 토큰을 추가했습니다. 이는 양식이 HTTP 권한 거부 응답을 생성하지 않고 Django에 성공적으로 제출될 수 있음을 의미합니다. 그런 다음 제출할 때 양식이 포함하고 있던 값을 출력하는 코드를 추가했습니다. 우리는 다양한 값으로 양식을 제출하여 그것들이 request.POST의 QueryDict 부분에서 어떻게 파싱되는지 살펴보았습니다. 이제 GET과 POST 요청의 차이에 대한 이론을 좀 더 논의한 후, 양식 설계 및 유효성 검사를 쉽게 해주는 Django Forms 라이브러리로 넘어가겠습니다.

Choosing between GET and POST

Choosing when to use a GET or POST request requires you to consider several factors. The most important thing is deciding whether the request should be idempotent. A request can be said to be idempotent if it can be repeated and produce the same result each time. In other words, if running it multiple times should always leave things the same, use GET. If repeating it could change data or create duplicates, use POST. Let's look at some examples.

If you type in any web address into your browser (such as any of the Bookr pages we have built so far), it will do a GET request to fetch the information. You can refresh the page, and no matter how many times you click **Refresh**, you will get the same data back. The request you are making is not affecting the content on the server. You would say these requests are idempotent.

Now, remember when you added data through the Django admin interface (in *Chapter 4*). You typed the information for the new book into a form, then clicked **Save**. Your browser made a POST request to create a new book on the server. If you repeated that POST request, the server would create *another* book and would do so each time you repeated the request. Since the request is updating information, it is not idempotent. Your browser will warn you about this. If you've ever tried to refresh a page that you were sent to after submitting a form, you may have received a message asking whether you want to "repost form data" (or something more verbose, as shown in the following figure). This is a warning that you're sending the form data again, which might cause the action you just undertook to be repeated:

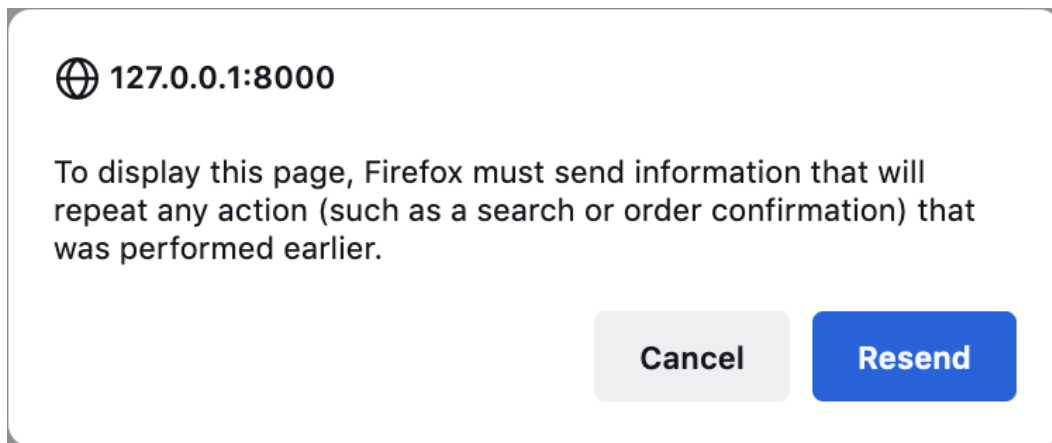


Figure 6.12: Firefox confirming whether information should be resent

This is not to suggest that all GET requests are idempotent and all POST requests are not – your backend application can be designed in any way you want. Although it is not best practice, a

GET과 POST 중 선택하기

GET 또는 POST 요청을 사용할 시기를 선택하는 것은 여러 가지 요소를 고려해야 합니다. 가장 중요한 것은 요청이 멍등성인지 결정하는 것입니다. 요청이 멍등하다고 할 수 있는 것은 반복할 수 있고 매번 동일한 결과를 생성할 수 있을 때입니다. 다시 말해, 여러 번 실행해도 항상 동일한 상태를 유지해야 한다면 GET을 사용하세요. 반복할 경우 데이터가 변경되거나 중복이 생성될 수 있다면 POST를 사용하세요. 몇 가지 예를 살펴보겠습니다.

브라우저에 웹 주소를 입력하면(지금까지 만든 Bookr 페이지 중 하나와 같은) 정보를 가져오기 위해 GET 요청을 수행합니다. 페이지를 새로 고칠 수 있으며, 새로 고침을 몇 번 클릭하더라도 동일한 데이터를 받을 수 있습니다. 당신이 하는 요청은 서버의 콘텐츠에 영향을 미치지 않습니다. 이러한 요청은 멍등하다고 할 수 있습니다.

이제 Django 관리 인터페이스를 통해 데이터를 추가했을 때를 기억하세요(4장). 새로운 책에 대한 정보를 양식에 입력한 후 저장을 클릭했습니다. 브라우저는 서버에 새로운 책을 만들기 위해 POST 요청을 했습니다. 그 POST 요청을 반복하면 서버는 또 다른 책을 생성하고 요청을 반복할 때마다 그렇게 할 것입니다. 요청이 정보를 업데이트하고 있으므로 멍등성이 없습니다. 브라우저는 이에 대해 경고할 것입니다. 양식을 제출한 후에 전송된 페이지를 새로 고치려고 시도한 적이 있다면, "양식 데이터를 다시 제출하시겠습니까?"라는 메시지를 받았을 수도 있습니다(또는 다음 그림에 표시된 것처럼 더 자세한 내용일 수 있습니다). 이는 양식 데이터를 다시 보내고 있다는 경고로, 방금 수행한 작업이 반복될 수 있습니다:

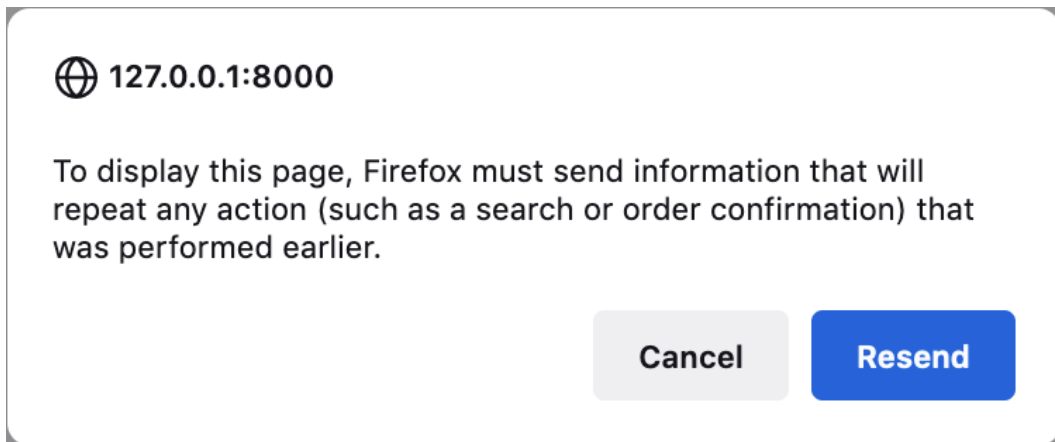


그림 6.12: Firefox가 정보를 다시 전송해야 하는지 확인하는 모습

모든 GET 요청이 멍등하고 모든 POST 요청이 멍등하지 않다는 것을 제안하는 것은 아닙니다. 백엔드 애플리케이션은 원하는 대로 설계할 수 있습니다. 최선의 관행은 아니지만, a

developer might have decided to make data get updated during a GET request in their web application. When you are building your applications, you should try to make sure GET requests are idempotent and leave data-altering to POST requests only. Stick to these principles unless you have a good reason not to.

Another point to consider is that Django only applies CSRF protection to POST requests. Any GET request, including one that alters data, can be accessed without a CSRF token.

Sometimes, it can be hard to decide whether a request is idempotent or not – for example, a login form. Before you submitted your username and password, you were not logged in, and afterward, the server considered you to be logged in, so could we consider that non-idempotent, as it changed your authentication status with the server? On the other hand, once logged in, if you were able to send your credentials again, you would remain logged in. This implies that the request is idempotent and repeatable. So, should the request be GET or POST? This brings us to the second point to consider when choosing what method to use: form data visibility.

If sending form data with a GET request, the form parameters will be visible in the URL. For example, if we made a login form using a GET request, the login URL might be `https://www.example.com/login?username=user&password=password1`. The username, and worse, the password, are visible in the web browser's address bar. It would also be stored in the browser history, so anyone who used the browser after the real user could log in to the site. The URL is often stored in web server log files as well, meaning the credentials would be visible there, too. In short, regardless of the idempotency of a request, don't pass sensitive data through URL parameters.

Sometimes, knowing that the parameter will be visible in the URL might be something you desire. For example, when searching with a search engine, the search parameter will be visible in the URL. To see this in action, try visiting `https://www.google.com` and searching for something. You'll notice that the page with the results has your search term as the `q` parameter. For example, a search for Django will take you to the URL `https://www.google.com/search?q=Django`. This allows you to share search results with someone else by sending them this URL. In *Activity 1*, you will add a search form that passes a parameter similarly.

Another consideration is that the maximum length of a URL allowed by a browser can be short compared to the size of a POST body – sometimes, only around 2,000 characters (or about 2 KB) compared to many megabytes or gigabytes that a POST body can be (assuming your server is set up to allow these size requests).

As we mentioned earlier, URL parameters are available in `request.GET`, regardless of the type of request being made (GET, POST, PUT, and so on). You might find it useful to send some data in

개발자는 웹 애플리케이션에서 GET 요청 중에 데이터가 업데이트되도록 결정했을 수 있습니다. 애플리케이션을 구축할 때 GET 요청이 멍등성을 가지도록 하고 데이터 변경은 POST 요청에만 맡기도록 해야 합니다. 특별한 이유가 없다면 이러한 원칙을 고수하세요.

또 다른 고려 사항은 Django가 POST 요청에만 CSRF 보호를 적용한다는 것입니다. 데이터를 변경하는 GET 요청을 포함하여 모든 GET 요청은 CSRF 토큰 없이 접근할 수 있습니다.

때때로 요청이 멍등인지 아닌지를 결정하는 것이 어려울 수 있습니다. 예를 들어, 로그인 양식입니다. 사용자 이름과 비밀번호를 제출하기 전에는 로그인하지 않았고, 이후 서버는 당신이 로그인한 것으로 간주하므로, 서버와의 인증 상태를 변경했기 때문에 이를 비멍등적이라고 볼 수 있을까요? 반면에, 로그인한 후에 자격 증명을 다시 보낼 수 있다면 여전히 로그인 상태를 유지할 것입니다. 이는 요청이 멍등적이고 반복 가능하다는 것을 의미합니다. 그렇다면 요청은 GET이어야 할까요, POST이어야 할까요? 이는 어떤 방법을 사용할지 선택할 때 고려해야 할 두 번째 사항인 양식 데이터 가시성으로 이어집니다.

GET 요청으로 양식 데이터를 전송하면 양식 매개변수가 URL에 표시됩니다. 예를 들어, GET 요청을 사용하여 로그인 양식을 만들면 로그인 URL은

<https://www.example.com/login?username=user&password=password1>이 될 수 있습니다. 사용자 이름과 더 나쁜 것은 비밀번호가 웹 브라우저의 주소 표시줄에 표시됩니다. 또한 브라우저 기록에 저장되므로 실제 사용자의 다음에 브라우저를 사용한 사람은 사이트에 로그인할 수 있습니다. URL은 웹 서버 로그 파일에도 저장되는 경우가 많아 자격 증명도 거기에 표시됩니다. 요약하자면, 요청의 멍등성과 관계없이 민감한 데이터를 URL 매개변수를 통해 전달하지 마세요.

때때로 매개변수가 URL에 표시될 것이라는 것을 아는 것이 바람직할 수 있습니다. 예를 들어, 검색 엔진으로 검색할 때 검색 매개변수가 URL에 표시됩니다. 이를 확인하려면 <https://www.google.com>을 방문하고 무언가를 검색해 보세요. 결과 페이지에 검색어가 q 매개변수로 표시되는 것을 알 수 있습니다. 예를 들어, Django를 검색하면 URL <https://www.google.com/search?q=Django>로 이동합니다. 이를 통해 이 URL을 보내 다른 사람과 검색 결과를 공유할 수 있습니다. 활동 1에서는 매개변수를 유사하게 전달하는 검색 양식을 추가할 것입니다.

또 다른 고려 사항은 브라우저에서 허용하는 URL의 최대 길이가 POST 본문의 크기에 비해 짧을 수 있다는 것입니다. 때때로, POST 본문은 수 메가바이트 또는 기가바이트가 될 수 있지만, URL의 최대 길이는 약 2,000자(또는 약 2KB) 정도입니다(서버가 이러한 크기 요청을 허용하도록 설정되어 있다고 가정할 때).

앞서 언급했듯이, 요청의 종류(GET, POST, PUT 등)에 관계없이 URL 매개변수는 request.GET에서 사용할 수 있습니다. 일부 데이터를 전송하는 것이 유용할 수 있습니다.

URL parameters and others in the request body (available in `request.POST`). For example, you could specify a `format` argument in the URL that sets what format some output data will be transformed to, but the input data is provided in the `POST` body.

Setting parameters in the `GET` query string might seem redundant when values can be passed as part of the URL path and then parsed by Django to route the URL. Next, we'll look at some reasons why you would prefer using the query string.

Why use GET when we can put parameters in the URL mappings?

Django allows us to easily define URL maps that contain variables. We could, for example, set up a URL mapping for a search view like this:

```
path("/search/<str:search>/", reviews.views.search)
```

This probably looks like a good approach at first, but when we start wanting to customize the results view with arguments, it can get complicated quickly. For example, we might want to be able to move from one results page to the next, so we'll add a `page` argument:

```
path("/search/<str:search>/<int:page>",  
      reviews.views.search)
```

And then, we might also want to order the search results by a specific category, such as author name or date of publishing, so we'll add another argument for that:

```
path("/search/<str:search>/<int:page>/<str:order >",  
      reviews.views.search)
```

You might be able to see the problem with this approach – we can't order the results without providing a page. If we wanted to add a `results_per_page` argument too, we wouldn't be able to use that without setting a page and order key.

Contrast this to using query parameters – all of them are optional, so you could search like this:

```
?search=search+term:
```

Or you could set a page like this:

```
?search=search+term&page=2
```

요청 본문에 있는 URL 매개변수 및 기타 항목(요청.POST에서 사용 가능). 예를 들어, 출력 데이터가 변환될 형식을 설정하는 형식 인수를 URL에 지정할 수 있지만, 입력 데이터는 POST 본문에 제공됩니다.

GET 쿼리 문자열에 매개변수를 설정하는 것은 값이 URL 경로의 일부로 전달되고 Django에 의해 URL이 라우팅되는 경우 중복된 것처럼 보일 수 있습니다. 다음으로, 쿼리 문자열을 사용하는 것을 선호하는 몇 가지 이유를 살펴보겠습니다.

URL 매핑에 매개변수를 넣을 수 있을 때 왜 GET을 사용 할까요?

Django는 우리가 변수를 포함하는 URL 맵을 쉽게 정의할 수 있도록 해줍니다. 예를 들어, 다음과 같이 검색 뷰에 대한 URL 매핑을 설정할 수 있습니다:

```
path("/search/<str:search>/", reviews.views.search)
```

처음에는 좋은 접근 방식처럼 보이지만, 인수로 결과 뷰를 사용자 정의하고 싶어지면 빠르게 복잡해질 수 있습니다. 예를 들어, 한 결과 페이지에서 다음 페이지로 이동할 수 있어야 하므로 페이지 인수를 추가하겠습니다:

```
path("/search/<str:search>/<int:page>",
      reviews.views.search)
```

그리고 나서, 우리는 저자 이름이나 출판 날짜와 같은 특정 카테고리로 검색 결과를 정렬하고 싶을 수도 있으므로, 이를 위한 또 다른 인수를 추가할 것입니다:

```
path("/search/<str:search>/<int:page>/<str:order>",
      reviews.views.search)
```

이 접근 방식의 문제를 알 수 있을 것입니다 – 페이지를 제공하지 않고는 결과를 정렬할 수 없습니다. 만약 `results_per_page` 인수도 추가하고 싶다면, 페이지와 정렬 키를 설정하지 않고는 사용할 수 없습니다.

쿼리 매개변수를 사용하는 것과 대조적으로 – 이 모든 것은 선택 사항이므로, 다음과 같이 검색할 수 있습니다:

```
?search=search+term:
```

또는 다음과 같이 페이지를 설정할 수 있습니다:

```
?search=search+term&page=2
```

Or just set the results ordering like this:

```
?search=search+term&order=author
```

Or you could combine them all:

```
?search=search+term&page=2&order=author
```

Another reason to use URL query parameters is that when submitting a form, the browser always sends the input values in this manner – it cannot be changed so that parameters are submitted as path components in the URL. Therefore, when submitting a form using GET, the URL query parameters must be used as the input data.

Now that we've introduced the fundamentals of forms in HTTP requests and HTML, let's look at the Django Forms library, which makes dealing with forms and requests much easier.

The Django Forms library

We've looked at how to manually write forms in HTML and how to access the data on the request object using `QueryDict`. We saw that the browser provides some validation for us for certain field types (such as email or numbers), but we have not tried validating the data in the Python view. We should validate the form in the Python view for two reasons:

- It is not safe to rely solely on browser-based validation of input data. A browser may not implement certain validation features, meaning the user could post any type of data. For example, older browsers don't validate number fields, so a user can type in a number outside the range we are expecting. Further, a malicious user could try to send harmful data without using a browser at all. The browser validation should be considered a nicety for the user, and that's all.
- The browser does not allow us to do cross-field validation. For example, we can use the `required` attribute for inputs that are mandatory to be filled in. Often, though, we want to set the `required` attribute based on the value of another input. For example, the email address input should only be set as required if the user has checked the **Register My Email** checkbox.

The Django Forms library allows you to quickly define a form using a Python class. This is done by creating a subclass of the base Django Form class. You can then use an instance of this class to render the form in your template and validate the input data. We refer to our classes as forms, similar to how we subclass Django models to create `Model` classes. Forms contain one or more fields of a certain type (such as text fields, number fields, or email fields). You'll notice

결과 정렬을 이렇게 설정하세요:

```
?search=search+term&order=author
```

모두 결합할 수도 있습니다:

```
?search=search+term&page=2&order=author
```

URL 쿼리 매개변수를 사용하는 또 다른 이유는 양식을 제출할 때 브라우저가 항상 입력 값을 이러한 방식으로 전송하기 때문입니다. 매개변수를 URL의 경로 구성 요소로 다시 제출하도록 변경할 수 없습니다. 따라서 GET을 사용하여 양식을 제출할 때 URL 쿼리 매개변수를 입력 데이터로 사용해야 합니다.

이제 HTTP 요청 및 HTML에서 양식의 기본 사항을 소개했으므로, 양식 및 요청을 훨씬 쉽게 처리할 수 있는 Django Forms 라이브러리를 살펴보겠습니다.

Django Forms 라이브러리

HTML에서 양식을 수동으로 작성하는 방법과 QueryDict를 사용하여 요청 객체의 데이터에 접근하는 방법을 살펴보았습니다. 브라우저가 특정 필드 유형(예: 이메일 또는 숫자)에 대해 일부 유효성 검사를 제공하는 것을 보았지만, Python 뷰에서 데이터를 유효성 검사하는 것은 시도하지 않았습니다. 두 가지 이유로 Python 뷰에서 양식을 유효성 검사해야 합니다:

- 입력 데이터의 브라우저 기반 유효성 검사에만 의존하는 것은 안전하지 않습니다. 브라우저가 특정 유효성 검사 기능을 구현하지 않을 수 있으므로 사용자가 어떤 유형의 데이터를 게시할 수 있습니다. 예를 들어, 구형 브라우저는 숫자 필드를 검증하지 않으므로 사용자가 우리가 예상하는 범위를 벗어난 숫자를 입력할 수 있습니다. 또한, 악의적인 사용자가 브라우저를 전혀 사용하지 않고도 해로운 데이터를 전송하려고 할 수 있습니다. 브라우저 유효성 검사는 사용자에게 제공되는 편의 기능으로 간주되어야 하며, 그게 전부입니다.
- 브라우저는 교차 필드 유효성 검사를 허용하지 않습니다. 예를 들어, 필수로 입력해야 하는 입력에 대해 required 속성을 사용할 수 있습니다. 그러나 종종 다른 입력의 값에 따라 required 속성을 설정하고 싶습니다. 예를 들어, 이메일 주소 입력은 사용자가 '내 이메일 등록' 체크박스를 선택한 경우에만 필수로 설정되어야 합니다.

Django Forms 라이브러리를 사용하면 Python 클래스를 사용하여 빠르게 양식을 정의할 수 있습니다. 이는 기본 Django Form 클래스의 하위 클래스를 생성하여 수행됩니다. 그런 다음 이 클래스의 인스턴스를 사용하여 템플릿에서 양식을 렌더링하고 입력 데이터를 검증할 수 있습니다. 우리는 우리의 클래스를 양식이라고 부르며, 이는 Django 모델을 하위 클래스로 만들어 모델 클래스를 생성하는 방식과 유사합니다. 양식에는 특정 유형(예: 텍스트 필드, 숫자 필드 또는 이메일 필드)의 하나 이상의 필드가 포함됩니다. 당신은 주목할 것입니다.

this sounds like Django models, and forms *are* similar to models but use different field classes. You can even automatically create a form from a model – we will cover this in *Chapter 7*.

Defining a form

Creating a Django form is similar to creating a Django model: you define a class that inherits from the `django.forms.Form` class. The class has attributes that are instances of different `django.forms.Field` subclasses. When rendered, the attribute name in the class corresponds to its input name in HTML. To give you a quick idea of what fields there are, some examples are `CharField`, `IntegerField`, `BooleanField`, `ChoiceField`, and `DateField`. Each field generally corresponds to one input when rendered in HTML, but there's not always a one-to-one mapping between a form field class and an input type. Form fields are more coupled to the type of data they collect rather than how they are displayed.

To illustrate this, consider a text input and a password input. They both accept some typed-in text data, but the main difference between them is that the text is displayed in a text input, while in a password input, the text is obscured. In a Django form, both of these fields are represented using `CharField`. Their difference in how they are displayed is set by changing the widget the field is using.

Note

If you're not familiar with the word *widget*, it's a term to describe the actual input that is being interacted with and how it is displayed. Text inputs, password inputs, select menus, checkboxes, and buttons are all examples of different widgets. The inputs we have seen in HTML correspond one-to-one with widgets. In Django, this is not the case, and the same type of `Field` class can be rendered in multiple ways, depending on the widget that is specified.

Django defines several widget classes that define how a field should be rendered as HTML. They inherit from `django.forms.widgets.Widget`. A widget can be passed to the `Field` constructor to change how it is rendered. For example, `CharField` renders as text `<input>` by default. If we use the `PasswordInput` widget, it will instead render as password `<input>`. The other widgets we will use are as follows:

- `RadioSelect`, which renders `ChoiceField` as radio buttons instead of a `<select>` menu
- `Textarea`, which renders `CharField` as `<textarea>`
- `HiddenInput`, which renders a field as a hidden input

이것은 Django 모델처럼 들리며, 폼은 모델과 유사하지만 `idx_16ef2510`의 다른 필드 클래스를 사용합니다. 모델에서 폼을 자동으로 생성할 수도 있습니다 - 이는 7장에서 다룰 것입니다.

폼 정의하기

Django `idx_440f6856` 폼을 만드는 것은 Django `idx_12aa968b` 모델을 만드는 것과 유사합니다: `django.forms.Form` 클래스를 상속하는 클래스를 정의합니다. 이 클래스는 다양한 `django.forms.Field` 하위 클래스의 인스턴스인 속성을 가집니다. 렌더링될 때, 클래스의 속성 이름은 HTML에서의 입력 이름에 해당합니다. 어떤 필드가 있는지 빠르게 이해할 수 있도록 몇 가지 예를 들면 `CharField`, `IntegerField`, `BooleanField`, `ChoiceField`, `DateField`가 있습니다. 각 필드는 일반적으로 HTML에서 렌더링될 때 하나의 입력에 해당하지만, 폼 필드 클래스와 입력 유형 간에는 항상 일대일 매핑이 있는 것은 아닙니다. 폼 필드는 표시되는 방식보다는 수집하는 데이터 유형에 더 밀접하게 연결되어 있습니다.

이를 설명하기 위해 텍스트 입력과 비밀번호 입력을 고려해 보십시오. 두 입력 모두 입력된 텍스트 데이터를 수용하지만, 이들 간의 주요 차이점은 텍스트가 텍스트 입력에서 표시되는 반면, 비밀번호 입력에서는 텍스트가 가려진다는 것입니다. Django 폼에서 이 두 필드는 모두 `CharField`를 사용하여 표현됩니다. 표시되는 방식의 차이는 필드가 사용하는 위젯을 변경하여 설정됩니다.

참고

위젯이라는 단어에 익숙하지 않다면, 이는 실제로 상호작용하는 입력과 그것이 표시되는 방식을 설명하는 용어입니다. 텍스트 입력, 비밀번호 입력, 선택 메뉴, 체크박스 및 버튼은 모두 다양한 위젯의 예입니다. 우리가 HTML에서 본 입력은 위젯과 일대일로 대응합니다. Django에서는 그렇지 않으며, 동일한 유형의 필드 클래스가 지정된 위젯에 따라 여러 방식으로 렌더링될 수 있습니다.

Django는 필드가 HTML로 렌더링되는 방식을 정의하는 여러 위젯 클래스를 정의합니다. 이들은 `django.forms.widgets.Widget`에서 상속됩니다. 위젯은 필드 생성자에 전달되어 렌더링 방식을 변경할 수 있습니다. 예를 들어, `CharField`는 기본적으로 텍스트 `<input>`으로 렌더링됩니다. `PasswordInput` 위젯을 사용하면 대신 비밀번호 `<input>`으로 렌더링됩니다. 우리가 사용할 다른 위젯은 다음과 같습니다:

- 라디오 선택(`RadioSelect`), `<select>` 메뉴 대신 라디오 버튼으로 `ChoiceField`를 렌더링합니다
- 텍스트 영역(`Textarea`), `CharField`를 `<textarea>`로 렌더링합니다
- 숨겨진 입력(`HiddenInput`), 필드를 숨겨진 입력으로 렌더링합니다

We'll look at an example form and add fields and features one by one. First, let's just create a `forms.py` file containing a form with a text input and a password input:

```
from django import forms

class ExampleForm(forms.Form):
    text_input = forms.CharField()
    password_input = forms.CharField(
        widget=forms.PasswordInput
    )
```

The widget argument can be just a widget subclass, which can be fine a lot of the time. If you want to further customize the display of the input and its attributes, you can set the widget argument to an instance of the widget class instead. We will look at customizing the widget displays further soon. In this case, we're using just the `PasswordInput` class, since we are not customizing beyond changing the type of input being displayed.

To provide a form to the user, it must be made available to the template through a view's context. Here is an example:

```
from django.shortcuts import render
from .forms import ExampleForm

def example_view(request):
    form = ExampleForm() # empty form for GET
    return render(request, "example.html", {"form": form})
```

When the form is rendered in a template, it looks like this:

Text input:

Password input:

Figure 6.13: Django form rendered in the browser

Note that the inputs do not contain any content when the page loads; the text has been entered to illustrate the different input types.

예제 양식을 살펴보고 필드와 기능을 하나씩 추가해 보겠습니다. 먼저, 텍스트 입력과 비밀번호 입력이 포함된 aforms.py 파일을 생성해 보겠습니다:

```
from django import forms

class ExampleForm(forms.Form):
    text_input = forms.CharField()
    password_input = forms.CharField(
        widget=forms.PasswordInput
    )
```

widget 인수 `idx_bc1e78ee`can은 단순히 위젯 서브클래스일 수 있으며, 이는 `idx_c1b74adefine`의 경우가 많습니다. 입력의 표시 및 속성을 더 사용자 정의하려면 widget 인수를 위젯 클래스의 인스턴스로 설정할 수 있습니다. 곧 위젯 표시를 사용자 정의하는 방법을 살펴보겠습니다. 이 경우, 표시되는 입력 유형을 변경하는 것 외에는 사용자 정의하지 않기 때문에 PasswordInput 클래스를 사용하고 있습니다.

사용자에게 양식을 제공하려면 뷰의 컨텍스트를 통해 템플릿에 제공되어야 합니다. 다음은 예시입니다:

```
from django.shortcuts import render
from .forms import ExampleForm

def example_view(request):
    form = ExampleForm() # GET을 위한 빈 양식
    return render(request, "example.html", {"form": form})
```

양식이 템플릿에 렌더링될 때, 다음과 같이 보입니다:

Text input:

Password input:

그림 6.13: 브라우저에 렌더링된 Django 양식

페이지가 로드될 때 입력란에 내용이 포함되어 있지 않음을 주의하세요; 텍스트는 다양한 입력 유형을 설명하기 위해 입력되었습니다.

If we examine the page source, we will see the HTML that Django generates. For the first two fields, it looks like this (some spacing has been added for readability):

```
<p>
  <label for="id_text_input">Text input:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</p>
<p>
  <label for="id_password_input">Password input:</label>
  <input type="password" name="password_input" required
    id="id_password_input">
</p>
```

Notice that Django has automatically generated a label with text derived from the field name. The name and id attributes have been set automatically. Django also automatically adds the required attribute to the input. Similar to model fields, form field constructors also accept a required argument – this defaults to True. Setting this to False removes the required attribute from the generated HTML.

Next, we'll look at how a checkbox is added to the form.

A checkbox is represented with BooleanField as it can have only two values: checked or unchecked. It's added to the form in the same way as the other field:

```
class ExampleForm(forms.Form):
    ...
    checkbox_on = forms.BooleanField()
```

The HTML Django generates for this new field is similar to the previous two fields:

```
<label for="id_checkbox_on">Checkbox on:</label>
<input type="checkbox" name="checkbox_on" required
  id="id_checkbox_on">
```

Next are select inputs:

- We need to provide a list of choices to display in the <select> dropdown.
- The field class constructor takes a choices argument. The choices are provided as a tuple of two-element tuples. The first element in each sub-tuple is the value of the

페이지 소스를 살펴보면 Django가 생성하는 HTML을 볼 수 있습니다. 처음 두 개의 필드는 다음과 같이 보입니다 (가독성을 위해 약간의 공백이 추가되었습니다):

```
<p>
  <label for="id_text_input">텍스트 입력:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</p>
<p>
  <label for="id_password_input">비밀번호 입력:</label>
  <input type="password" name="password_input" required
    id="id_password_input">
</p>
```

Django가 필드 이름에서 파생된 텍스트로 자동으로 레이블을 생성했다는 점에 유의하십시오. name 및 id 속성이 자동으로 설정되었습니다. Django는 또한 입력에 required 속성을 자동으로 추가합니다. 모델 필드와 유사하게, 폼 필드 idx_48aff87econstructors idx_e046443d도 required 인수를 허용합니다 - 기본값은 True입니다. 이를 False로 설정하면 생성된 HTML에서 required 속성이 제거됩니다.

다음으로, 체크박스가 폼에 추가되는 방법을 살펴보겠습니다.

체크박스는 두 가지 값만 가질 수 있으므로 BooleanField로 표현됩니다: 체크됨 또는 체크되지 않음. 다른 필드와 동일한 방식으로 폼에 추가됩니다:

```
class ExampleForm(forms.Form):
    ...
    checkbox_on = forms.BooleanField()
```

Django가 이 새로운 필드에 대해 생성하는 HTML은 이전 두 필드와 유사합니다:

```
<label for="id_checkbox_on">체크박스 켜기:</label>
<input type="checkbox" name="checkbox_on" required
  id="id_checkbox_on">
```

다음은 선택 입력입니다:

- <select> 드롭다운에 표시할 선택 목록을 제공해야 합니다. 필드 클래스 생성자는 choices 인수를 사용합니다. 선택지는 두 요소의 튜플로 제공됩니다
- 다. 각 하위 튜플의 첫 번째 요소는 값입니다.

choice, while the second element is the text or description of the choice. For example, choices could be defined like this:

```
BOOK_CHOICES = (  
    ("1", "Deep Learning with Keras"),  
    ("2", "Web Development with Django"),  
    ("3", "Brave New World"),  
    ("4", "The Great Gatsby")  
)
```

Note that you can use lists instead of tuples if you want (or a combination of the two). This can be useful if you want your choices to be mutable:

```
BOOK_CHOICES = (  
    ["1", "Deep Learning with Keras"],  
    ["2", "Web Development with Django"],  
    ["3", "Brave New World"],  
    ["4", "The Great Gatsby"]  
)
```

- To implement `optgroup`, we can nest the choices. To implement the choices as we did in our previous examples, we can use a structure like this:

```
BOOK_CHOICES = (  
    (  
        "Non-Fiction", (  
            ("1", "Deep Learning with Keras"),  
            ("2", "Web Development with Django")  
        )  
    ),  
    (  
        "Fiction", (  
            ("3", "Brave New World"),  
            ("4", "The Great Gatsby")  
        )  
    )  
)
```

선택의 첫 번째 요소는 선택의 값이고 두 번째 요소는 선택의 텍스트 또는 설명입니다. 예를 들어, 선택은 다음과 같이 정의될 수 있습니다:

```
BOOK_CHOICES = (
    ("1", "Keras로 배우는 딥러닝"),
    ("2", "Django로 웹 개발"),
    ("3", "멋진 신세계"),
    ("4", "위대한 개츠비")
)
```

idx_0312ceb8를 주의하세요. idx_59876bd7를 사용하여 튜플 대신 리스트를 사용할 수 있습니다(또는 두 가지의 조합). 선택이 변경 가능하도록 하려면 유용할 수 있습니다:

```
BOOK_CHOICES = (
    ...
    ["1", "케라스와 함께하는 딥러닝"]
    ["2", "장고를 이용한 웹 개발"]
    ["3", "멋진 신세계"],
    ["4", "위대한 개츠비"]
)
...
```

...optgroup을 구현하기 위해 선택 항목을 중첩할 수 있습니다. 우리가 했던 것처럼 선택 항목을 구현하려면 이전 예제들에서, 우리는 다음과 같은 구조를 사용할 수 있습니다:

```
BOOK_CHOICES = (
    ...
    {
        ...
        "논픽션", (
            ("1", "케라스와 함께하는 딥러닝"),
            ("2", "장고를 이용한 웹 개발")
        ),
        ...
    },
    {
        ...
        "소설", (
            ("3", "멋진 신세계"),
            ("4", "위대한 개츠비")
        ),
        ...
    },
    ...
)
```

The select element is added to the form by using `ChoiceField`. The widget defaults to a select input, so that no configuration is necessary apart from setting choices:

```
class ExampleForm(forms.Form):
    ...
    favorite_book =
        forms.ChoiceField(choices=BOOK_CHOICES)
```

This is the HTML that is generated:

```
<label for="id_favorite_book">Favorite book:</label>
<select name="favorite_book" id="id_favorite_book">
  <optgroup label="Non-Fiction">
    <option value="1">Deep Learning with Keras
    </option>
    <option value="2">Web Development with Django
    </option>
  </optgroup>
  <optgroup label="Fiction">
    <option value="3">Brave New World</option>
    <option value="4">The Great Gatsby</option>
  </optgroup>
</select>
```

- Making a multiple select requires the use of `MultipleChoiceField`. It takes a choices argument in the same format as the regular `ChoiceField` for single selects:

```
class ExampleForm(forms.Form):
    ...
    books_you_own =
        forms.MultipleChoiceField(choices=BOOK_CHOICES)
```

And its HTML is similar to that of the single select, except it has the `multiple` attribute added:

```
<label for="id_books_you_own">Books you own:</label>
<select name="books_you_own" required
  id="id_books_you_own" multiple>
  <optgroup label="Non-Fiction">
    <option value="1">Deep Learning with Keras
```

선택 요소는 ChoiceField를 사용하여 양식에 추가됩니다. 위젯은 기본적으로 선택 입력으로 설정되므로 선택 항목을 설정하는 것 외에는 idx_36e3feb3에 대한 구성 필요가 없습니다:

```
class ExampleForm(forms.Form):
    ...
    favorite_book =
        forms.ChoiceField(choices=BOOK_CHOICES)
```

생성된 HTML은 다음과 같습니다:

```
<label for="id_favorite_book">좋아하는 책:</label>
<select name="favorite_book" id="id_favorite_book">
  <optgroup label="논픽션">
    <option value="1">케라스를 이용한 딥러닝
    </option>

    <option value="2">장고를 이용한 웹 개발
    </option>
  </optgroup>
  <optgroup label="소설">
    <option value="3">멋진 신세계</option>
    <option value="4">위대한 개츠비</option>
  </optgroup>
</select>
```

- 다중 선택을 만들려면 MultipleChoiceField를 사용해야 합니다. 선택지를 받습니다. 단일 선택을 위한 일반 ChoiceField와 동일한 형식의 인수:

```
class ExampleForm(forms.Form):
    ...
    books_you_own =
        forms.MultipleChoiceField(choices=BOOK_CHOICES)
```

그리고 그 HTML idx_2ce5a307isidx_87a082f2는 단일 선택과 유사하지만 multiple 속성이 추가되었습니다:

```
<label for="id_books_you_own">소유한 책:</label>
<select name="books_you_own" required
  id="id_books_you_own" multiple>
  <optgroup label="논픽션">
    <option value="1">케라스를 이용한 딥 러닝
```

```
        </option>
        <option value="2">Web Development with Django
        </option>
    </optgroup>
    <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
    </optgroup>
</select>
```

- Choices can also be set after the form has been instantiated. You may want to generate the list/tuple choice inside your view dynamically and then assign it to the field's choices attribute, like so:

```
form = ExampleForm()
form.fields["books_you_own"].choices = [("1", "Deep
Learning with Keras"), ...]
```

Next are radio inputs, which are similar to selects:

- Like selects, radio inputs use ChoiceField, as they provide a single choice between multiple options.
- The options to choose between are passed into the field constructor with the choices argument.
- The choices are provided as a tuple of two-element tuples, also like selects:

```
choices = (
    ("1", "Option One"),
    ("2", "Option Two"),
    ("3", "Option Three")
)
```

- ChoiceField defaults to displaying as a select input, so the widget must be set to RadioSelect to have it rendered as radio buttons. Putting the choice setting together with this, we can add radio buttons to the form like this:

```
RADIO_CHOICES = (
    ("Value One", "Value One"),
    ("Value Two", "Value Two"),
    ("Value Three", "Value Three")
)
```

```

        </option>
        <option value="2">장고로 웹 개발하기
        </option>
    </optgroup>
    <optgroup label="소셜">
        <option value="3">멋진 신세계</option>
        <option value="4">위대한 개츠비</option>
    </optgroup>
</select>

```

- 선택지는 폼이 인스턴스화된 후에도 설정할 수 있습니다. 뷰 내에서 목록/튜플 선택지를 동적으로 생성한 다음 필드의 choices 속성에 할당할 수 있습니다, 다음과 같이:

```

form = ExampleForm()
form.fields["books_you_own"].choices = [("1", "Deep
Learning with Keras"), ...]

```

다음은 선택과 유사한 라디오 입력입니다:

- 선택과 마찬가지로, 라디오 입력은 ChoiceField를 사용합니다. 이는 여러 옵션 중에서 단일 선택을 제공합니다.
- 선택할 옵션은 choices 인수와 함께 idx_f74245c3 필드 생성자에 전달됩니다.

- 선택지는 두 개의 요소로 이루어진 튜플의 튜플로 제공되며, 선택과 유사합니다:

```

choices = (
    ("1", "옵션 하나"),
    ("2", "옵션 두"),
    ("3", "옵션 세")
)

```

- ChoiceField는 기본적으로 선택 입력으로 표시되므로, 위젯을 RadioSelect로 설정해야 라디오 버튼으로 렌더링됩니다. 선택 설정을 이것과 함께 조합하면, 다음과 같이 폼에 라디오 버튼을 추가할 수 있습니다:

```

RADIO_CHOICES = (
    ("값 하나", "값 하나"),
    ("값 둘", "값 둘"),
    ("값 셋", "값 셋")
)

```

```
)

class ExampleForm(forms.Form):
    ...
    radio_input =
        forms.ChoiceField(choices=RADIO_CHOICES,
                           widget=forms.RadioSelect)
```

Here is the HTML that is generated:

```
<label for="id_radio_input_0">Radio input:</label>
<ul id="id_radio_input">
<li>
    <label for="id_radio_input_0">
        <input type="radio" name="radio_input"
            value="Value One" required
            id="id_radio_input_0">
        Value One
    </label>
</li>
<li>
    <label for="id_radio_input_1">
        <input type="radio" name="radio_input"
            value="Value Two" required
            id="id_radio_input_1">
        Value Two
    </label>
</li>
<li>
    <label for="id_radio_input_2">
        <input type="radio" name="radio_input"
            value="Value Three" required
            id="id_radio_input_2">
        Value Three
    </label>
</li>
</ul>
```

Django automatically generates a unique label and ID for each of the three radio buttons.

```
(

class ExampleForm(forms.Form):
    ...
    radio_input =
        forms.ChoiceField(choices=RADIO_CHOICES,
                           widget=forms.RadioSelect)
```

여기 생성된 `idx_7777e3c7HTMLidx_00f63d17`가 있습니다:

```
<label for="id_radio_input_0">라디오 입력:</label>
<ul id="id_radio_input">
<li>
    <label for="id_radio_input_0">
        <input type="radio" name="radio_input"
            value="Value One" required
            id="id_radio_input_0">
            값 하나
    </label>
</li>
<li>
    <label for="id_radio_input_1">
        <input type="radio" name="radio_input"
            value="Value Two" required
            id="id_radio_input_1">
            Value Two
    </label>
</li>
<li>
    <label for="id_radio_input_2">
        <input type="radio" name="radio_input"
            value="Value Three" required
            id="id_radio_input_2">
            값 세 개
    </label>
</li>
</ul>
```

장고는 세 개의 라디오 버튼 각각에 대해 고유한 레이블과 ID를 자동으로 생성합니다.

- To create a text area, use CharField with a Textarea widget:

```
class ExampleForm(forms.Form):  
    ...  
    text_area = forms.CharField(widget=forms.Textarea)
```

You might notice that this text area is much larger than the previous ones we have seen (see the following figure):



Figure 6.14: Normal text area (top) versus Django default text area (bottom)

This is because Django automatically adds `cols` and `rows` attributes. These set the number of columns and rows that the text field displays, respectively:

```
<label for="id_text_area">Text area:</label>  
<textarea name="text_area" cols="40" rows="10"  
required id="id_text_area"></textarea>
```

- Note that the `cols` and `rows` settings do not affect the amount of text that can be entered into a field, only the amount that is displayed at a time. Also, note that the size of `textarea` can be set using CSS (for example, the `height` and `width` properties). This will override the `cols` and `rows` settings.

To create number inputs, you might expect Django to have a `NumberField` type, but it does not.

- 텍스트 영역을 만들려면 CharField와 Textarea 위젯을 사용하세요:

```
class ExampleForm(forms.Form):
    ...
    text_area = forms.CharField(widget=forms.Textarea)
```

이 텍스트 영역이 우리가 이전에 본 것들보다 훨씬 크다는 것을 알 수 있습니다 (다음 그림을 참조하세요):

그림 6.14: 일반 텍스트 영역(상단)과 Django 기본 텍스트 영역(하단)

이는 Django가 자동으로 cols 및 rows 속성을 추가하기 때문입니다. 이 속성들은 각각 텍스트 필드가 표시하는 열과 행의 수를 설정합니다:

```
<label for="id_text_area">텍스트 영역:</label>
<textarea name="text_area" cols="40" rows="10"
required id="id_text_area"></textarea>
```

- cols 및 rows 설정은 필드에 입력할 수 있는 텍스트 양에 영향을 미치지 않으며, 한 번에 표시되는 양에만 영향을 미친다는 점에 유의하세요. 또한, textarea의 크기는 CSS를 사용하여 설정할 수 있습니다(예: height 및 width 속성). 이는 cols 및 rows 설정을 무시합니다.

숫자 입력을 생성하려면 Django에 NumberField 유형이 있을 것으로 예상할 수 있지만, 그렇지 않습니다.

Remember that the Django form fields are data-centric rather than display-centric, so instead, Django provides different `Field` classes, depending on what type of numeric data you want to store:

- For integers, use `IntegerField`.
- For floating point numbers, use `FloatField` or `DecimalField`. The latter two differ in how they convert their data into a Python value. `FloatField` will convert into a float, while `DecimalField` is a decimal.
- Decimal values offer better accuracy in representing numbers than float values, but may not integrate well into your existing Python code.

We'll add all three fields to the form at once:

```
class ExampleForm(forms.Form):  
    ...  
    integer_input = forms.IntegerField()  
    float_input = forms.FloatField()  
    decimal_input = forms.DecimalField()
```

Here's the HTML for all three:

```
<p>  
    <label for="id_integer_input">Integer input:</label>  
    <input type="number" name="integer_input" required  
        id="id_integer_input">  
</p>  
<p>  
    <label for="id_float_input">Float input:</label>  
    <input type="number" name="float_input" step="any"  
        required id="id_float_input">  
</p>  
<p>  
    <label for="id_decimal_input">Decimal input:</label>  
    <input type="number" name="decimal_input" step="any"  
        required id="id_decimal_input">  
</p>
```

The generated HTML of `IntegerField` is missing the `step` attribute that the other two have, which means the widget will only accept integer values.

기억하라 `idx_7e9401ce` Django 폼 필드는 표시 중심이 아니라 데이터 중심이므로, 대신 Django는 저장하려는 숫자 데이터의 유형에 따라 다양한 필드 클래스를 제공합니다:

- 정수의 경우 `IntegerField`를 사용하십시오.
- 부동 소수점 숫자의 경우 `FloatField` 또는 `DecimalField`를 사용하십시오. 후자의 두 가지는 데이터를 Python 값으로 변환하는 방식이 다릅니다. `FloatField`는 `float`로 변환되는 반면, `DecimalField`는 소수입니다.
- 소수 값은 `float` 값보다 숫자를 더 정확하게 표현하지만 기존 Python 코드에 잘 통합되지 않을 수 있습니다

다. 세 가지 필드를 한 번에 폼에 추가하겠습니다:

```
class ExampleForm(forms.Form):
    ...
    integer_input = forms.IntegerField()
    float_input = forms.FloatField()
    decimal_input = forms.DecimalField()
```

여기 `idx_a80e7afball` 세 개의 HTML이 있습니다:

```
<p>
  <label for="id_integer_input">정수 입력:</label>
  <input type="number" name="integer_input" required
    id="id_integer_input">
</p>
<p>
  <label for="id_float_input">부동 소수점 입력:</label>
  <input type="number" name="float_input" step="any"
    required id="id_float_input">
</p>
<p>
  <label for="id_decimal_input">소수점 입력:</label>
  <input type="number" name="decimal_input" step="any"
    required id="id_decimal_input">
</p>
```

The `idx_86206751` generated HTML of `IntegerField` is missing `idx_ef8a0336` the `step` attribute that the other two have, which means the widget will only accept integer values.

The other two fields (`FloatField` and `DecimalField`) generate very similar HTML; their behavior is the same in the browser, and they differ only when their values are used in Django code.

As you might have guessed, an email input can be created with `EmailField`:

```
class ExampleForm(forms.Form):  
    ...  
    email_input = forms.EmailField()
```

Its HTML is similar to the email input we created manually:

```
<label for="id_email_input">Email input:</label>  
<input type="email" name="email_input" required  
      id="id_email_input">
```

Continuing with our manually created form, the next field we will look at is `DateField`:

- By default, Django will render `DateField` as a text input, and the browser will not show a calendar popup when the field is clicked on.

We can add `DateField` to the form with no arguments, like this:

```
class ExampleForm(forms.Form):  
    ...  
    date_input = forms.DateField()
```

When rendered, it just looks like a normal text input:

Date input:

Figure 6.15: Default `DateField` display in the form

Here is the HTML generated by default:

```
<label for="id_date_input">Date input:</label>  
<input type="text" name="date_input" required  
      id="id_date_input">
```

- The reason for using a text input is that it allows the user to enter the date in several different formats. For example, by default, the user can type in the date in *Year-Month-Day* (dash-separated) or *Month/Day/Year* (slash-separated) formats.

다른 두 필드 (FloatField와 DecimalField)는 매우 유사한 HTML을 생성합니다; 브라우저에서의 동작은 동일하며, 값이 Django 코드에서 사용될 때만 다릅니다.

당신이 추측했듯이, 이메일 입력은 EmailField로 생성할 수 있습니다:

```
class ExampleForm(forms.Form):
    ...
    email_input = forms.EmailField()
```

그 HTML은 우리가 수동으로 생성한 이메일 입력과 유사합니다:

```
<label for="id_email_input">이메일 입력:</label>
<input type="email" name="email_input" required
id="id_email_input">
```

수동으로 생성된 양식을 계속 진행하면서, 다음으로 살펴볼 필드는 DateField입니다:

- 기본적으로 Django는 DateField를 텍스트 입력으로 렌더링하며, 필드를 클릭할 때 브라우저는 달력 팝업을 표시하지 않습니다. 이렇게 인수 없이 DateField를 양식에 추가할 수 있습니다:

```
class ExampleForm(forms.Form):
    ...
    date_input = forms.DateField()
```

idx_c6cb20b6이 렌더링되면, 일반 텍스트 입력처럼 보입니다:

Date input:

그림 6.15: 양식에서의 기본 DateField 표시

다음은 기본적으로 생성된 HTML입니다:

```
<label for="id_date_input">날짜 입력:</label>
<input type="text" name="date_input" required
id="id_date_input">
```

- 텍스트 입력을 사용하는 이유는 사용자가 여러 형식으로 날짜를 입력할 수 있도록 허용하기 때문입니다. 예를 들어, 기본적으로 사용자는 날짜를 연도-월-일(대시로 구분) 형식이나 월/일/연도(슬래시로 구분) 형식으로 입력할 수 있습니다.

- The accepted formats can be specified by passing a list of formats to the `DateField` constructor using the `input_formats` argument. For example, we could accept dates in the *Day/Month/Year* or *Day/Month/Year-with-century* format, like this:

```
DateField(input_formats = ["%d/m/%y", "%d/%m/%Y"])
```

- We can override any attributes on a field's widget by passing the `attrs` argument to the widget constructor. This accepts a dictionary of attribute keys/values that will be rendered into the input's HTML.
- We have not used this yet, but we will see it again in the next chapter when we customize the field rendering further. For now, we'll just set one attribute, `type`, that will overwrite the default input type:

```
class ExampleForm(forms.Form):  
    ...  
    date_input = forms.DateField(  
        widget=forms.DateInput(  
            attrs={"type": "date"})
```

When rendered, it now looks like the date field we had before, and clicking on it brings up the calendar date picker:

Date input:

Figure 6.16: `DateField` with date input

Examining the generated HTML now, we can see it uses the date type:

```
<label for="id_date_input">Date input:</label>  
<input type="date" name="date_input" required  
      id="id_date_input">
```

- 허용된 형식은 `input_formats` 인수를 사용하여 `DateField` 생성자에 형식 목록을 전달하여 지정할 수 있습니다. 예를 들어, 우리는 날짜를 일/월/년 또는 일/월/년-세기 형식으로 수락할 수 있습니다. 다음과 같이:

```
DateField(input_formats = ("%d/m/%y", "%d/%m/%Y"))
```

- 우리는 위젯 생성자에 `attrs` 인수를 전달하여 필드의 위젯에서 모든 속성을 재정의할 수 있습니다. 이는 입력의 HTML로 렌더링될 속성 키/값의 사전을 수락합니다.
- 우리는 아직 이것을 사용하지 않았지만, 다음 장에서 `idx_395fe89c` 필드 렌더링을 더 사용자 정의할 때 다시 볼 것입니다. 지금은 기본 입력 유형을 덮어쓸 `type`이라는 하나의 속성만 설정하겠습니다:

```
class ExampleForm(forms.Form):
    ...
    date_input = forms.DateField(
        widget=forms.DateInput(
            attrs={"type": "date"})
```

렌더링되면, 이제 이전에 있던 날짜 필드처럼 보이며, 클릭하면 달력 날짜 선택기가 나타납니다:

Date input:

그림 6.16: 날짜 입력이 있는 `DateField`

생성된 HTML을 살펴보면, 날짜 유형을 사용하고 있음을 알 수 있습니다:

```
<label for="id_date_input">날짜 입력:</label>
<input type="date" name="date_input" required
id="id_date_input">
```

The final input that we are missing is the hidden input:

- Once again, due to the data-centric nature of Django forms, there is no `HiddenField`.
- Instead, we choose the type of field that needs to be hidden and set its widget to `HiddenInput`. We can then set the value of the field using the field constructor's `initial` argument:

```
class ExampleForm(forms.Form):  
    ...  
    hidden_input = forms.CharField(  
        widget=forms.HiddenInput, initial="Hidden Value")
```

Here is the generated HTML:

```
<input type="hidden" name="hidden_input"  
value="Hidden Value" id="id_hidden_input">
```

Note that, as this is a hidden input, Django does not generate a label or surrounding `p` elements.

There are other form fields that Django provides that work in similar ways. These range from `DateTimeField` (for capturing a date and a time) to `GenericIPAddressField` (for either IPv4 or IPv6 addresses) and `URLField` (for URLs). A full list of fields is available at <https://docs.djangoproject.com/en/5.1/ref/forms/fields/>.

Rendering a form in a template

We've now seen how to create a form and add fields, and we've seen what the form looks like and what HTML is generated. But how is the form rendered in the template? We simply instantiate the form class and pass it to the render function in a view, using the context, just like any other variable.

For example, here's how we can pass `ExampleForm` to a template:

```
def view_function(request):  
    form = ExampleForm()  
    return render(request, "template.html", {"form": form})
```

Django does not add the `<form>` element or submit button(s) for you when rendering the template; you should add these around where your form is placed in the template. The form can be rendered like any other variable.

우리가 놓치고 있는 마지막 입력은 숨겨진 입력입니다:

- 다시 말해, Django 양식의 데이터 중심 특성 때문에 HiddenField가 없습니다. 대신, 숨겨야 할 필드의 유형을 선택하고 해당 위젯을 HiddenInput
- 으로 설정합니다. 그런 다음 필드 생성자의 초기 인수를 사용하여 필드의 값을 설정할 수 있습니다:

```
class ExampleForm(forms.Form):
    ...
    hidden_input = forms.CharField(
        widget=forms.HiddenInput, initial="숨겨진 값")
```

여기 생성된 HTML이 있습니다:

```
<input type="hidden" name="hidden_input"
value="Hidden Value" id="id_hidden_input">
```

이것은 숨겨진 입력이므로 Django는 레이블이나 주변 요소를 생성하지 않습니다.

Django가 제공하는 다른 `idx_c4ae50d2` 양식 필드가 있으며, 이들은 `idx_e7b7f6a8` 유사한 방식으로 작동합니다. 이들은 `DateTimeField`(날짜와 시간을 캡처하기 위한)에서 `GenericIPAddressField`(IPv4 또는 IPv6 주소용) 및 `URLField`(URL용)까지 다양합니다. 필드의 전체 목록은 <https://docs.djangoproject.com/en/5.1/ref/forms/fields/>에서 확인할 수 있습니다.

템플릿에서 양식 렌더링하기

이제 우리는 양식을 생성하고 `idx_8f68f0fd` 필드를 추가하는 방법을 보았고, 양식이 어떻게 생겼는지와 생성된 HTML을 보았습니다. 하지만 템플릿에서 양식은 어떻게 렌더링되나요? 우리는 단순히 양식 클래스를 인스턴스화하고 뷰의 `render` 함수에 컨텍스트와 함께 전달합니다. 다른 변수와 마찬가지로요.

예를 들어, 다음과 같이 `ExampleForm`을 템플릿에 전달할 수 있습니다:

```
def view_function(request):
    form = ExampleForm()
    return render(request, "template.html", {"form": form})
```

Django는 템플릿을 렌더링할 때 `<form>` 요소나 제출 버튼을 자동으로 추가하지 않습니다; 템플릿에서 양식이 배치된 부분 주위에 이러한 요소를 추가해야 합니다. 양식은 다른 변수처럼 렌더링할 수 있습니다.

We mentioned briefly earlier that the form was being rendered in the template using the `as_p` method. This layout method was chosen as it most closely matches the example form we built manually. Django offers four layout methods that can be used:

- `as_table`: The form is rendered as table rows, with each input on its own row. Django does not generate the surrounding table element, so you should wrap the form yourself. Here is an example:

```
<form method="post">
  <table>
    {{ form.as_table }}
  </table>
</form>
```

`as_table` is the default rendering method, so `{{ form.as_table }}` and `{{ form }}` are equivalent.

When rendered, the form looks like this:

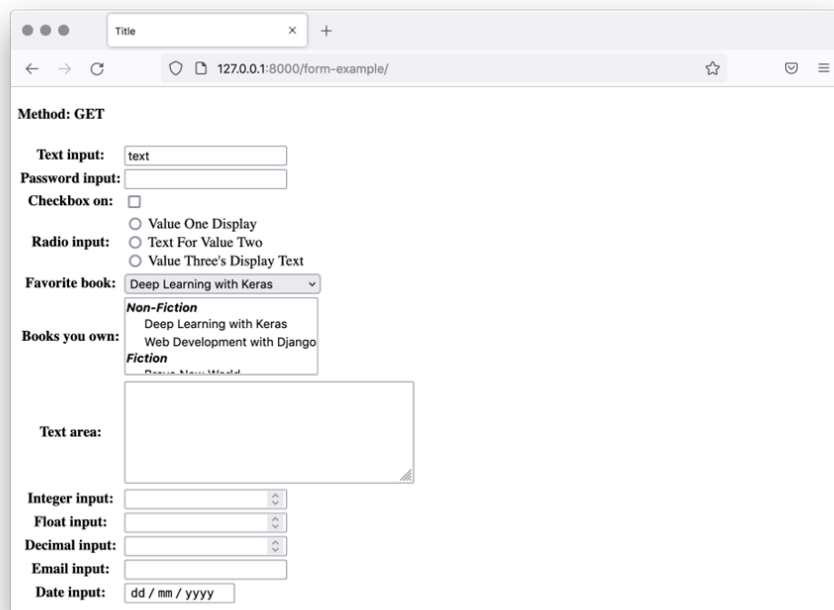


Figure 6.17: Form rendered as a table

우리는 이전에 양식이 as_p 메서드를 사용하여 템플릿에서 렌더링되고 있다고 간략하게 언급했습니다. 이 레이아웃 방법은 우리가 수동으로 만든 예제 양식과 가장 밀접하게 일치하기 때문에 선택되었습니다. Django는 사용할 수 있는 네 가지 레이아웃 방법을 제공합니다:

- **as_table**: 양식이 테이블 행으로 렌더링되며, 각 입력은 자신의 행에 있습니다. Django는 주변 테이블 요소를 생성하지 않으므로 양식을 직접 감싸야 합니다. 다음은 idx_c4f5c95a 예제입니다:

```
<form method="post">
  <table>
    {{ form.as_table }}
  </table>
</form>
```

as_table은 기본 렌더링 방법이므로 {{ form.as_table }}와 {{ form }}은 동일합니다.

렌더링되면, 양식은 다음과 같이 보입니다:

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/form-example/'. The page content is a form with the following elements:

- Method: GET**
- Text input:** A text box containing the word 'text'.
- Password input:** An empty password box.
- Checkbox on:** An unchecked checkbox.
- Radio input:** Three radio buttons with labels 'Value One Display', 'Text For Value Two', and 'Value Three's Display Text'.
- Favorite book:** A dropdown menu with 'Deep Learning with Keras' selected. Below it, a list of books is shown, categorized into 'Non-Fiction' (Deep Learning with Keras, Web Development with Django) and 'Fiction'.
- Books you own:** A section for listing books owned by the user.
- Text area:** A large text input area.
- Integer input:** A numeric input box.
- Float input:** A numeric input box.
- Decimal input:** A numeric input box.
- Email input:** An email address input box.
- Date input:** A date input box with the format 'dd / mm / yyyy'.

그림 6.17: 테이블로 렌더링된 양식

Here is a small sample of HTML that is generated:

```
<tr>
  <th>
    <label for="id_text_input">Text input:</label>
  </th>
  <td>
    <input type="text" name="text_input" required
      id="id_text_input">
    </td>
</tr>
<tr>
  <th>
    <label for="id_password_input">Password input:</label>
  </th>
  <td>
    <input type="password" name="password_input"
      required id="id_password_input">
    </td>
</tr>
```

- `as_ul`: This renders the form fields as list items (`li`) inside either a `ul` or `ol` element. Like with `as_table`, the containing element (`<ul>` or `<ol>`) is not created by Django and must be added by you:

```
<form method="post">
  <ul>
    {{ form.as_ul }}
  </ul>
</form>
```

Here's how the form renders using `as_ul`:

여기 aidx_cfa3794c의 작은 HTML 샘플이 있습니다:

```
<tr>
  <th>
    <label for="id_text_input">텍스트 입력:</label>
  </th>
  <td>
    <input type="text" name="text_input" required
      id="id_text_input">
  </td>
</tr>
<tr>
  <th>
    <label for="id_password_input">비밀번호 입력:</label>
  </th>
  <td>
    <input type="password" name="password_input"
      required id="id_password_input">
  </td>
</tr>
```

- `as_ul`: 이것은 `idx_348b3de3` 양식 필드를 `ul` 또는 `li` 요소 안의 목록 항목 (`li`)으로 렌더링합니다. `as_table`과 마찬가지로, 포함 요소 (`<ul>` 또는 `<li>`)는 Django에 의해 생성되지 않으며 당신이 추가해야 합니다:

```
<form method="post">
  <ul>
    {{ form.as_ul }}
  </ul>
</form>
```

폼이 `as_ul`을 사용하여 렌더링되는 방법은 다음과 같습니다:

Figure 6.18: Form rendered using `as_ul`

And here's a sample of the generated HTML:

```
<li>
  <label for="id_text_input">Text input:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</li>
<li>
  <label for="id_password_input">Password input:</label>
  <input type="password" name="password_input"
    required id="id_password_input">
</li>
```

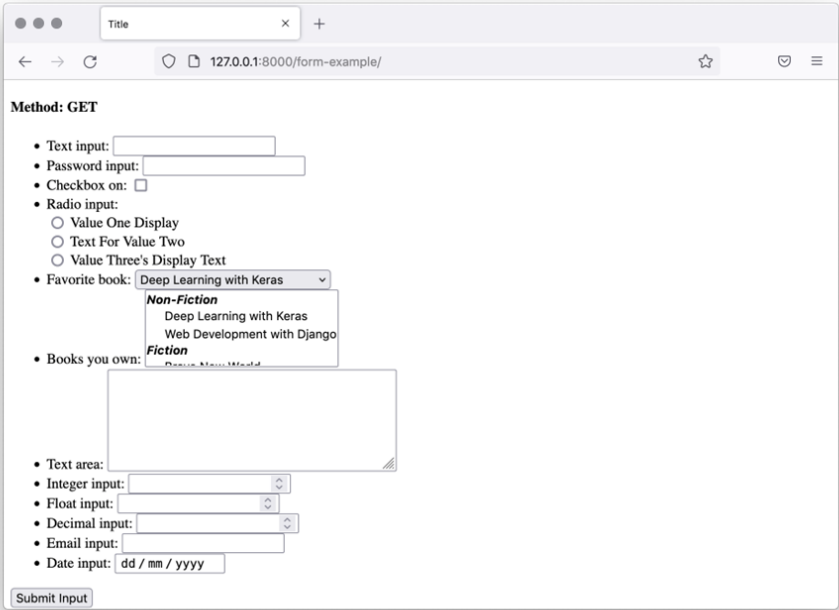


그림 6.18: as_ul을 사용하여 렌더링된 형식

그리고 idx_09114325여기 idx_5cad7632 생성된 HTML의 샘플이 있습니다:

```
<li>
  <label for="id_text_input">텍스트 입력:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</li>
<li>
  <label for="id_password_input">비밀번호 입력:</label>
  <input type="password" name="password_input"
    required id="id_password_input">
</li>
```

- `as_p`: Then there's the `as_p` method, which we used in our previous examples. Each input is wrapped within `p` tags, which means that you don't have to wrap the form manually (in `<table>` or `<ul>`) as you did with the previous methods:

```
<form method="post">
    {{ form.as_p }}
</form>
```

Here's what the rendered form looks like:

Figure 6.19: Form rendered using `as_p`

You've seen this before, but once again, here's a sample of the HTML generated:

```
<p>
  <label for="id_text_input">Text input:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</p>
<p>
  <label for="id_password_input">Password input:</label>
```

as_p: 그런 다음 이전 예제에서 사용한 as_p 메서드가 있습니다. 각 입력은 p 태그로 감싸져 있으므로, 폼을 감쌀 필요가 없습니다. 이전 방법들처럼 수동으로 (<table> 또는 안에) 하세요:

```
<form method="post">
...
    {{ form.as_p }}
</form>
```

렌더링된 양식은 다음과 같습니다:

그림 6.19: as_p를 사용하여 렌더링된 양식

이것을 이전에 본 적이 있지만, 다시 한 번, 생성된 HTML 샘플은 다음과 같습니다:

```
<p>
  <label for="id_text_input">텍스트 입력:</label>
  <input type="text" name="text_input" required
    id="id_text_input">
</p>
<p>
  <label for="id_password_input">비밀번호 입력:</label>
```

```
<input type="password" name="password_input"
      required id="id_password_input">
</p>
```

- `as_field_group`: Django also offers more fine-grained control of layout so that individual form elements can be laid out in a custom design without constraining them to the table, ul, or p structures. `as_field_group` is a method of the field object rather than the form object. It can be called from the individual fields in the form like this:

```
<form method="post">
  {{ form.field.text_input.as_field_group }} <br />
  {{ form.field.password_input.as_field_group }} <br />
</form>
```

This will be rendered as follows:

```
<form method="post">
<label for="id_text_input">Text input:</label>
<input type="text" name="text_input" required id="id_text_input"> <br />
<label for="id_password_input">Password input:</label>
<input type="password" name="password_input" required
id="id_password_input"> <br />
</form>
```

If you wanted to generate a form with fields separated by a div element, you could iterate through the fields of the form like this in the template:

```
<form method="post">
  {% csrf_token %}
  {% for field in form %}
    <div class="field_wrapper"
      id="field_group_{{field.name}}">
      {{ field.as_field_group }}
    </div>
  {% endfor %}
</form>
```

It is up to you to decide which method you want to use to render your form, depending on which suits your application best. In terms of their behavior and use in your view, they are all

```
<input type="password" name="password_input"
      required id="id_password_input">
</p>
```

- **as_field_group**: Django는 개별 양식 요소를 테이블, ul 또는 p 구조에 제약을 두지 않고 사용자 정의 디자인으로 배치할 수 있도록 레이아웃에 대한 보다 세밀한 제어를 제공합니다. **as_field_group**은 양식 객체가 아닌 필드 객체의 메서드입니다. 다음과 같이 양식의 개별 필드에서 호출할 수 있습니다:

```
<form method="post">
  {{ form.field.text_input.as_field_group }} <br />
  {{ form.field.password_input.as_field_group }} <br />
</form>
```

이것은 다음과 같이 렌더링됩니다:

```
<form method="post">
<label for="id_text_input">텍스트 입력:</label>
<input type="text" name="text_input" required id="id_text_input"> <br />
<label for="id_password_input">비밀번호 입력:</label>
<input type="password" name="password_input" required
id="id_password_input"> <br />
</form>
```

양식의 필드를 div 요소로 구분하여 생성하고 싶다면, 템플릿에서 다음과 같이 양식의 필드를 반복할 수 있습니다:

```
<form method="post">{%
  csrf_token %}{%
  for field in form
  %}<div class="
  field_wrapper"
    id="field_group_{{field.name}}">
    {{ field.as_field_group }}
  </div>
  {% endfor %}
</form>
```

어떤 방법을 사용하여 양식을 렌더링할지는 귀하에게 달려 있으며, 귀하의 애플리케이션에 가장 적합한 방법에 따라 결정됩니다. 그들의 동작과 뷰에서의 사용 측면에서, 그들은 모두

identical. In *Chapter 15*, we will also introduce a method of rendering forms that will make use of the Bootstrap CSS classes.

Now that you have been introduced to Django forms, we can update our example form page to use a Django form instead of manually writing all the HTML. We'll do this in the next exercise by replacing hand-written HTML with a Django form.

Exercise 6.03 – Building and rendering a Django form

In this exercise, you will build a Django form using all the fields we have seen. The form and view will behave similarly to the form built manually; however, you will be able to see how much less code is required when writing forms using Django. Your form will also automatically get field validation, and if we make changes to the form, we don't have to then make changes to the HTML, as it will update dynamically based on the form definition:

1. In PyCharm, create a new file called `forms.py` inside the `form_example` app directory.
2. Import the Django forms library at the top of your `forms.py` file:

```
from django import forms
```

3. Define the choices for the radio buttons by creating a `RADIO_CHOICES` variable. Populate it as follows:

```
RADIO_CHOICES = (  
    ("Value One", "Value One Display"),  
    ("Value Two", "Text For Value Two"),  
    ("Value Three", "Value Three's Display Text"),  
)
```

You will use this soon when you create a `ChoiceField` called `radio_input`.

4. Define the nested choices for the book and select inputs by creating a `BOOK_CHOICES` variable. Populate it as follows:

```
BOOK_CHOICES = (  
    (  
        "Non-Fiction", (  
            ("1", "Deep Learning with Keras"),  
            ("2", "Web Development with Django")  
        )  
    ),  
    (  

```

동일합니다. 15장에서는 Bootstrap CSS 클래스를 활용하는 양식을 렌더링하는 방법도 소개할 것입니다.

이제 Django 양식에 대해 소개받았으므로, 수동으로 idx_49f921e6 모든 HTML을 작성하는 대신 idx_2b9f0f0cDjango 양식을 사용하도록 예제 양식 페이지를 업데이트할 수 있습니다. 다음 연습에서는 수동으로 작성한 HTML을 Django 양식으로 교체하여 이를 수행할 것입니다.

연습 6.03 – Django 양식 만들기 및 렌더링

이 연습에서는 우리가 본 모든 필드를 사용하여 Django 양식을 만들 것입니다. 양식과 뷰는 수동으로 만든 양식과 유사하게 작동하지만, Django를 사용하여 양식을 작성할 때 필요한 코드가 얼마나 적은지 볼 수 있습니다. 또한 양식은 자동으로 필드 유효성 검사를 받으며, 양식에 변경을 가하면 HTML을 변경할 필요 없이 양식 정의에 따라 동적으로 업데이트됩니다:

1. PyCharm에서 form_example 앱 디렉토리 내에 forms.py라는 새 파일을 만듭니다
2. 다. forms.py 파일의 맨 위에 Django 양식 라이브러리를 가져옵니다:

```
from django import forms
```

3. 라디오 버튼의 선택지를 정의하려면 RADIO_CHOICES 변수를 생성합니다. 다음과 같이 채웁니다:

```
RADIO_CHOICES = (
    ("Value One", "Value One Display"),
    ("Value Two", "Text For Value Two"),
    ("Value Three", "Value Three's Display Text"),
)
```

라디오 입력이라고 하는 ChoiceField를 생성할 때 곧 이것을 사용할 것입니다.

4. 책과 선택 입력을 위한 중첩 선택지를 정의하려면 BOOK_CHOICES 변수를 생성합니다. 다음과 같이 idx_b41b00a1it을 채웁니다:

```
BOOK_CHOICES = (
    (
        "논픽션", (
            ("1", "케라스를 이용한 딥러닝"),
            ("2", "장고를 이용한 웹 개발")
        )
    ),
    (
```

```
        "Fiction", (  
            ("3", "Brave New World"),  
            ("4", "The Great Gatsby")  
        )  
    )  
)
```

5. Create a class called `ExampleForm`, which inherits from the `forms.Form` class:

```
class ExampleForm(forms.Form):
```

Add the following fields as attributes to the class:

```
    text_input = forms.CharField()  
    password_input = forms.CharField(  
        widget=forms.PasswordInput)  
    checkbox_on = forms.BooleanField()  
    radio_input = forms.ChoiceField(  
        choices=RADIO_CHOICES,  
        widget=forms.RadioSelect)  
    favorite_book = forms.ChoiceField(  
        choices=BOOK_CHOICES)  
    books_you_own = forms.MultipleChoiceField(  
        choices=BOOK_CHOICES)  
    text_area = forms.CharField(widget=forms.Textarea)  
    integer_input = forms.IntegerField()  
    float_input = forms.FloatField()  
    decimal_input = forms.DecimalField()  
    email_input = forms.EmailField()  
    date_input = forms.DateField(  
        widget=forms.DateInput(  
            attrs={"type": "date"})  
        )  
    hidden_input = forms.CharField(  
        widget=forms.HiddenInput,  
        initial="Hidden Value")
```

Save the file.

```

        "소셜", (
            ("3", "멋진 신세계"),
            ("4", "위대한 개츠비")
        )
    )

```

5. forms.Form 클래스를 상속받는 ExampleForm이라는 클래스를 만드세요:

```
class ExampleForm(forms.Form):
```

다음 필드를 클래스의 속성으로 추가하세요:

```

text_input = forms.CharField()
password_input = forms.CharField(
    widget=forms.PasswordInput)
checkbox_on = forms.BooleanField()
radio_input = forms.ChoiceField(
    choices=RADIO_CHOICES,
    widget=forms.RadioSelect)
favorite_book = forms.ChoiceField(
    choices=BOOK_CHOICES)
books_you_own = forms.MultipleChoiceField(
    choices=BOOK_CHOICES)
text_area = forms.CharField(widget=forms.Textarea)
integer_input = forms.IntegerField()
float_input = forms.FloatField()
decimal_input = forms.DecimalField()
email_input = forms.EmailField()
date_input = forms.DateField(
    widget=forms.DateInput(
        attrs={"type": "date"}))
hidden_input = forms.CharField(
    widget=forms.HiddenInput,
    initial="Hidden Value")

```

idx_2306ab06 파일을
저장하세요.

6. Open your `form_example` app's `views.py` file. At the top of the file, add a line to import `ExampleForm` from your `forms.py` file:

```
from .forms import ExampleForm
```

7. Inside the `form_example` view, instantiate the `ExampleForm` class and assign it to the `form` variable:

```
form = ExampleForm()
```

8. Add the `form` variable to the context dictionary using the `form` key. The return line should look like this:

```
return render(request, "form-example.html",
              {"method": request.method, "form": form})
```

Save the file.

Make sure you haven't removed the code that prints out the data the form has sent, as we will use it again later in this exercise.

9. Open the `form-example.html` file inside the `form_example` app's templates directory. You can remove nearly all of the contents of the `form` element, except the `{% csrf_token %}` template tag and the submit buttons. Add a rendering of the `form` variable using the `as_p` method after the `{% csrf_token %}` template tag. The whole `form` element should now look like this:

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <p>
    <input type="submit" name="submit_input"
      value="Submit Input">
  </p>
  <p>
    <button type="submit" name="button_element"
      value="Button Element">
      Button With <strong>Styled</strong> Text
    </button>
```

6. form_example 앱의 views.py 파일을 엽니다. 파일의 맨 위에 forms.py 파일에서 ExampleForm을 가져오는 줄을 추가합니다:

```
from .forms import ExampleForm
```

7. form_example 뷰 안에서 ExampleForm 클래스를 인스턴스화하고 이를 form 변수에 할당합니다:

```
form = ExampleForm()
```

8. form 변수를 form 키를 사용하여 컨텍스트 사전(context dictionary)에 추가합니다. 반환 라인은 다음과 같아야 합니다:

```
return render(request, "form-example.html",
              {"method": request.method, "form": form})
```

파일을 저장하세요.

양식이 보낸 데이터를 출력하는 코드를 제거하지 않았는지 확인하세요. 이 코드는 이 연습에서 다시 사용할 것입니다.

9. form_example 앱의 templates 디렉토리 내의 form-example.html 파일을 엽니다. {% csrf_token %} 템플릿 태그와 제출 버튼을 제외하고는 form 요소의 거의 모든 내용을 제거할 수 있습니다. {% csrf_token %} 템플릿 태그 뒤에 as_p 메서드를 사용하여 form 변수를 렌더링합니다. 전체 form 요소는 이제 다음과 같아야 합니다:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <p>
        <input type="submit" name="submit_input"
            value="입력 제출">
    </p>
    <p>
        <button type="submit" name="button_element"
            value="버튼 요소">
            스타일이 적용된 <strong>텍스트</strong>가 있는 버튼
        </button>
```

```
</p>  
</form>
```

10. Start the Django dev server if it is not already running, then visit the form example page in your browser, at `http://127.0.0.1:8000/form-example/`. It should look as follows:

The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8000/form-example/`. The page content includes a form with the following elements:

- Method: GET**
- Text input:** A single-line text field.
- Password input:** A single-line password field.
- Checkbox on:** A checkbox that is currently unchecked.
- Radio input:** Three radio buttons. The first, labeled "Value One Display", is selected.
- Favorite book:** A dropdown menu with "Deep Learning with Keras" selected.
- Books you own:** A list with two categories: "Non-Fiction" (containing "Deep Learning with Keras" and "Web Development with Django") and "Fiction" (containing "Django REST Framework").
- Text area:** A multi-line text field.
- Integer input:** A single-line field with a spinner.

Figure 6.20: Django example form rendered in the browser

11. Enter some data into the form – since Django marks all fields as required, you will need to enter some text or select values for all fields, including ensuring that the checkbox is checked. Submit the form.
12. Switch back to PyCharm and look in the **Debug** console at the bottom of the window. You should see that all the values being submitted by the form are printed out to the console, similar to in *Exercise 6.02*:

```
</p>
</form>
```

10. Django 개발 서버가 이미 실행 중이 아니라면 시작하고, 브라우저에서 `http://127.0.0.1:8000/form-example/`의 양식 예제 페이지를 방문하세요. 다음과 같이 보여야 합니다:

그림 6.20: 브라우저에 렌더링된 Django 예제 양식

- 양식에 일부 데이터 `idx_930acb85`를 입력하세요. Django는 모든 필드를 필수로 표시하므로 모든 필드에 대해 텍스트를 입력하거나 값을 선택해야 하며, 체크박스가 선택되어 있는지 확인해야 합니다. 양식을 제출하세요.
- PyCharm으로 돌아가서 창 하단의 디버그 콘솔을 확인하세요. 양식에서 제출된 모든 값이 콘솔에 출력되는 것을 볼 수 있어야 합니다. 이는 연습 6.02와 유사합니다:

```
csrfmiddlewaretoken: ['HZ222KpXGzT2PLxMSIfxwHXcgyTtk3qSjl0Eg6Brvu1KUfYvmRqnLFqZzrYg2nt5']
text_input: ['Text']
password_input: ['Password']
checkbox_on: ['on']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
integer_input: ['154']
float_input: ['11.0']
decimal_input: ['1234']
email_input: ['user@example.com']
date_input: ['2024-10-27']
hidden_input: ['Hidden Value']
submit_input: ['Submit Input']
```

Figure 6.21: Values as submitted by the Django form

You can see that the values are still strings, and the names match those of the attributes of the `ExampleForm` class. Notice that the submit button that you clicked is included, as well as the CSRF token. The form you submit can be a mix of Django form fields and arbitrary fields you add; both will be contained in the `request.POST` `QueryDict` object.

In this exercise, you created a Django form with many different types of form fields. You instantiated it into a variable in your view, then passed it to `form-example.html`, where it was rendered as HTML. Finally, you submitted the form and looked at the values it posted. Notice that the amount of code we had to write to generate the same form was greatly reduced. We did not have to manually code any HTML, and we now have one place that defines how the form will display and how it will validate.

In the next section, we will examine how Django forms can automatically validate the submitted data, as well as how the data is converted from strings into Python objects.

Validating forms and retrieving Python values

So far, we have seen how Django forms make it much simpler to define a form using Python code and have it automatically rendered. We will now look at the other part of what makes Django forms useful: their ability to automatically validate the form and then retrieve native Python objects and values from them.

```

csrfmiddlewaretoken: ['HZ222KpX6zT2PLxMSIfxwHXcgyTtk3qSjl0Eg6Brvu1KUfYvmRqnLFqZzrYg2nt5']
text_input: ['Text']
password_input: ['Password']
checkbox_on: ['on']
radio_input: ['Value Two']
favorite_book: ['1']
books_you_own: ['1', '3']
text_area: ['Text Area']
integer_input: ['154']
float_input: ['11.0']
decimal_input: ['1234']
email_input: ['user@example.com']
date_input: ['2024-10-27']
hidden_input: ['Hidden Value']
submit_input: ['Submit Input']

```

그림 6.21: Django 양식에 의해 제출된 값

값이 여전히 문자열이며, 이름이 ExampleForm 클래스의 속성과 일치하는 것을 볼 수 있습니다. 클릭한 제출 버튼이 포함되어 있으며, CSRF 토큰도 포함되어 있습니다. 제출하는 양식은 Django 양식 필드와 추가한 임의의 필드가 혼합될 수 있으며, 둘 다 request.POST QueryDict 객체에 포함됩니다.

이번 연습에서는 다양한 유형의 양식 필드를 가진 Django 양식을 만들었습니다. 이를 뷰의 변수에 인스턴스화한 후 form-example.html로 전달하여 HTML로 렌더링했습니다. 마지막으로 양식을 제출하고 게시된 값을 확인했습니다. 동일한 양식을 생성하기 위해 작성해야 했던 코드의 양이 크게 줄어들었다는 점에 주목하세요. 우리는 HTML을 수동으로 코딩할 필요가 없었고, 이제 양식이 어떻게 표시되고 어떻게 검증될지를 정의하는 한 곳이 있습니다.

다음 섹션에서는 Django 양식이 제출된 데이터를 자동으로 검증하는 방법과 데이터가 문자열에서 Python 객체로 변환되는 방법을 살펴보겠습니다.

양식 검증 및 Python 값 검색

지금까지 Django 양식이 Python 코드를 사용하여 양식을 정의하고 자동으로 렌더링하는 것을 훨씬 간단하게 만드는 방법을 보았습니다. 이제 Django 양식이 유용한 또 다른 부분인 양식을 자동으로 검증하고 그로부터 기본 Python 객체와 값을 검색하는 능력을 살펴보겠습니다.

In Django, a form can either be *unbound* or *bound*. These terms describe whether or not the form has had the submitted POST data sent to it for validation. So far, we have only seen unbound forms – they are instantiated without arguments, like this:

```
form = ExampleForm()
```

A form is bound if it is called with some data to be used for validation, such as the POST data. A bound form can be created like this:

```
form = ExampleForm(request.POST)
```

A bound form allows us to start using built-in validation-related tools on the form instance. First, there's the `is_valid` method, which checks the form's validity, and then the `cleaned_data` attribute, which contains the values converted from strings into Python objects. The `cleaned_data` attribute is only available after the form has been *cleaned*, which is the process of cleaning up the data and converting it from strings into Python objects. The cleaning process runs during the `is_valid` call. An `AttributeError` will be raised if you try to access `cleaned_data` before calling `is_valid`.

A short example of how to access the cleaned data of `ExampleForm` looks like this:

```
form = ExampleForm(request.POST)

if form.is_valid():
    # cleaned_data is only populated if the form is valid
    if form.cleaned_data["integer_input"] > 5:
        do_something()
```

In this example, `form.cleaned_data["integer_input"]` is the integer value, 10, so it can be compared to the number 5. Compare this to the value that was posted, which is the string 10. The cleaning process performs this conversion for us. Other fields, such as dates or Booleans, are converted accordingly.

The cleaning process also sets any errors on the form and fields, which will be displayed when the form is rendered again. Let's see all this in action. Modern browsers provide a large amount of client-side validation, so they prevent forms from being submitted unless their basic validation rules are met. You might have already seen this if you tried to submit the form in the previous exercise with empty fields:

Django에서 폼은 바인드되지 않거나 바인드될 수 있습니다. 이러한 용어는 폼이 제출된 POST 데이터가 유효성 검사를 위해 전송되었는지 여부를 설명합니다. 지금까지 우리는 바인드되지 않은 폼만 보았습니다. 이들은 다음과 같이 인수 없이 인스턴스화됩니다:

```
form = ExampleForm()
```

폼은 유효성 검사를 위해 사용될 데이터를 가지고 호출되면 바인드됩니다. 예를 들어, POST 데이터와 같은 데이터입니다. 바인드된 폼은 다음과 같이 생성할 수 있습니다:

```
form = ExampleForm(request.POST)
```

바인드된 폼은 폼 인스턴스에서 내장된 유효성 검사 관련 도구를 사용하기 시작할 수 있게 해줍니다. 먼저, 폼의 유효성을 검사하는 `is_valid` 메서드가 있고, 그 다음에는 문자열에서 Python 객체로 변환된 값을 포함하는 `cleaned_data` 속성이 있습니다.

`cleaned_data` 속성은 폼이 정리된 후에만 사용할 수 있으며, 이는 데이터를 정리하고 문자열에서 Python 객체로 변환하는 과정입니다. 정리 과정은 `is_valid` 호출 중에 실행됩니다. `is_valid`를 호출하기 전에 `cleaned_data`에 접근하려고 하면 `AttributeError`가 발생합니다.

`ExampleForm`의 정리된 데이터에 접근하는 방법의 짧은 예는 다음과 같습니다:

```
form = ExampleForm(request.POST)

if form.is_valid():
    # cleaned_data는 폼이 유효할 때만 채워집니다.
    if form.cleaned_data["integer_input"] > 5:
        do_something()
```

이 예에서 `form.cleaned_data["integer_input"]`는 정수 값 10이므로 숫자 5와 비교할 수 있습니다. 게시된 값인 문자열 10과 비교해 보세요. 정리 과정이 이 변환을 수행합니다. 날짜나 불리언과 같은 다른 필드도 그에 따라 변환됩니다.

정리 과정은 또한 폼과 필드에 오류를 설정하며, 이는 폼이 다시 렌더링될 때 표시됩니다. 이 모든 것을 실제로 살펴보겠습니다. 현대 브라우저는 많은 양의 클라이언트 측 유효성 검사를 제공하므로 기본 유효성 검사 규칙이 충족되지 않으면 폼이 제출되는 것을 방지합니다. 이전 연습에서 빈 필드로 `idx_0e9d5196` the `idx_1e5789f0` 폼을 제출하려고 시도했다면 이미 이것을 보았을 것입니다:

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/form-example/'. The form contains the following elements:

- Password input:** A text input field.
- Checkbox on:** A checkbox with a tooltip that says 'Please fill out this field.'
- Radio input:** Three radio buttons labeled 'Value One Display', 'Text For Value Two', and 'Value Three's Display Text'.
- Favorite book:** A dropdown menu currently showing 'Deep Learning with Keras'. The dropdown list is open, showing 'Non-Fiction' (with 'Deep Learning with Keras' and 'Web Development with Django' as options) and 'Fiction' (with 'Deep Learning with Keras' as an option).
- Books you own:** A text input field.
- Text area:** A large text input field.
- Integer input:** A text input field with a spin button.
- Float input:** A text input field with a spin button.
- Decimal input:** A text input field with a spin button.
- Email input:** A text input field.

Figure 6.22: Form submission prevented by the browser

Figure 6.22 shows the browser preventing form submission. Since the browser is preventing the submission, Django never gets the opportunity to validate the form itself. To allow the form to be submitted, we need to add some more advanced validation that the browser is unable to validate itself. We will discuss the different types of validations that can be applied to form fields in the next section, but for now, we will just add a `max_digits` setting of 3 to `decimal_input` of `ExampleForm`. This means the user should not enter more than three digits in the form.

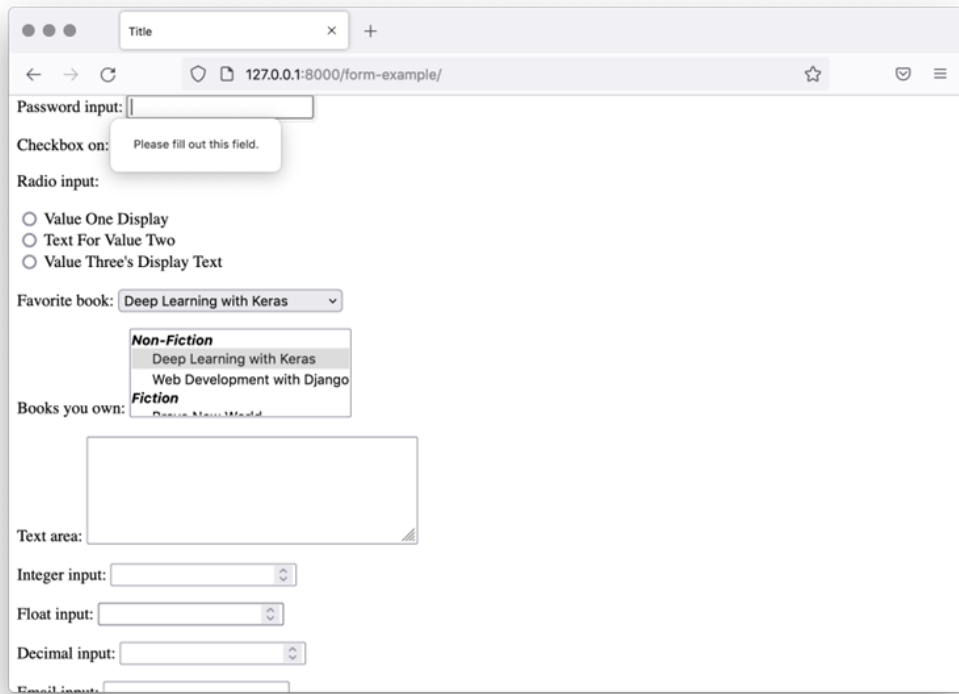


그림 6.22: 브라우저에 의해 양식 제출이 방지됨

그림 6.22는 브라우저가 양식 제출을 방지하는 모습을 보여줍니다. 브라우저가 제출을 방지하고 있기 때문에 Django는 양식 자체를 검증할 기회를 얻지 못합니다. 양식이 제출될 수 있도록 하려면 브라우저가 스스로 검증할 수 없는 좀 더 고급 검증을 추가해야 합니다. 다음 섹션에서는 양식 필드에 적용할 수 있는 다양한 유형의 검증에 대해 논의할 것이지만, 지금은 ExampleForm의 decimal_input에 max_digits 설정을 3으로 추가하겠습니다. 이는 사용자가 양식에 세 자리 이상의 숫자를 입력해서는 안 된다는 것을 의미합니다.

Note

Why should Django validate the form if the browser is already doing this and preventing submission? A server-side application should never trust input from the user: the user might be using an older browser or other HTTP clients to send the request, thus not receiving any errors from their browser. Also, as we have just mentioned, there are types of validation that the browser does not understand, so Django must validate these on its end.

ExampleForm can be updated like this:

```
class ExampleForm(forms.Form):  
    ...  
    decimal_input = forms.DecimalField(max_digits=3)  
    ...
```

Now, the view should be updated to pass `request.POST` to the form class when the method is POST, for example, like this:

```
if request.method == "POST":  
    form = ExampleForm(request.POST)  
else:  
    form = ExampleForm()
```

If you pass `request.POST` into the form constructor when the method is not POST, then the form will always contain errors when first rendered, as `request.POST` will be empty.

Now, the browser will let us submit the form, but we will get an error displayed if `decimal_input` contains more than three digits:

- Ensure that there are no more than 3 digits in total.

Decimal input:

Figure 6.23: An error is displayed when a field is not valid

Django automatically renders the form differently in the template when it has errors. But how can we make the view behave differently depending on the validity of the form? As we

노트

브라우저가 이미 이를 수행하고 제출을 방지하고 있다면 Django가 왜 양식을 검증해야 할까요? 서버 측 애플리케이션은 사용자로부터의 입력을 절대 신뢰해서는 안 됩니다: 사용자가 오래된 브라우저나 다른 HTTP 클라이언트를 사용하여 요청을 보내고 있을 수 있으므로 브라우저에서 오류를 받지 못할 수 있습니다. 또한, 우리가 방금 언급했듯이 브라우저가 이해하지 못하는 검증 유형이 있으므로 Django는 이를 자체적으로 검증해야 합니다.

ExampleForm은 다음과 같이 업데이트할 수 있습니다:

```
class ExampleForm(forms.Form):
    ...
    decimal_input = forms.DecimalField(max_digits=3)
    ...
```

이제, 뷰 `idx_fc409df2`는 POST 메서드일 때 `request.POST`를 양식 클래스에 전달하도록 `idx_9848063fbe` 업데이트되어야 합니다. 예를 들어, 다음과 같이:

```
if request.method == "POST":
    form = ExampleForm(request.POST)
else:
    form = ExampleForm()
```

POST가 아닐 때 `form` 생성자에 `request.POST`를 전달하면, `form`은 처음 렌더링될 때 항상 오류를 포함하게 됩니다. `request.POST`는 비어 있기 때문입니다.

이제 브라우저는 우리가 폼을 제출할 수 있게 해주지만, `decimal_input`이 세 자리를 초과하면 오류가 표시됩니다:

- Ensure that there are no more than 3 digits in total.

Decimal input:

Figure 6.23: 필드가 유효하지 않을 때 오류가 표시됩니다

Django는 오류가 있을 때 템플릿에서 폼을 다르게 렌더링합니다. 하지만 폼의 유효성에 따라 뷰가 다르게 동작하도록 만들려면 어떻게 해야 할까요? 우리가

mentioned earlier, we should use the form's `is_valid` method. A view using this check might contain code like this:

```
form = ExampleForm(request.POST)

if form.is_valid():
    # perform operations from with data from
    # form.cleaned_data
    return redirect("/success-page")
    # redirect to a success page
```

In this example, we are redirecting to a success page if the form is valid. Otherwise, we assume the execution flow continues as before and pass the invalid form back to the render function to be displayed to the user with errors.

Note

Why do we return a redirect on success? For two reasons: first, an early return prevents the execution of the rest of the view (that is, the failure branch); second, it prevents the message about resending the form data if the user then reloads the page.

In the next exercise, we will see the form validation in action and change the view's execution flow based on the validity of the form.

Exercise 6.04 – Validating forms in a view

In this exercise, we will update the example view to instantiate the form differently depending on the HTTP method. We will also change the form so that it prints out the cleaned data instead of the raw POST data, but only if the form is valid:

1. In PyCharm, open the `forms.py` file inside the `form_example` app directory. Add a `max_digits=3` argument to the `decimal_input` of `ExampleForm`:

```
class ExampleForm(forms.Form):
    ...
    decimal_input = forms.DecimalField(max_digits=3)
```

Once this argument has been added, we can submit the form since the browser does not know how to validate this rule, but Django does.

앞서 언급했듯이, 우리는 양식의 `is_valid` 메서드를 사용해야 합니다. 이 검사를 사용하는 뷰는 다음과 같은 코드를 포함할 수 있습니다:

```
form = ExampleForm(request.POST)

if form.is_valid():
    # form.cleaned_data의 데이터로 작업 수행

    return redirect("/success-page")
    # 성공 페이지로 리디렉션
```

이 예제에서는 `form`이 유효한 경우 `idx_2b316d84`로 성공 페이지로 리디렉션하고 있습니다. 그렇지 않으면 실행 흐름이 이전과 같이 계속되고 유효하지 않은 양식을 렌더 함수에 다시 전달하여 오류와 함께 사용자에게 표시됩니다.

참고

성공 시 리디렉션을 반환하는 이유는 무엇인가요? 두 가지 이유가 있습니다: 첫째, 조기 반환은 나머지 뷰의 실행(즉, 실패 분기)을 방지합니다; 둘째, 사용자가 페이지를 새로 고침할 경우 양식 데이터 재전송에 대한 메시지를 방지합니다.

다음 연습에서는 양식 유효성 검사를 실제로 보고 양식의 유효성에 따라 뷰의 실행 흐름을 변경할 것입니다.

연습 6.04 – 뷰에서 양식 유효성 검사

이번 연습에서는 HTTP 메서드에 따라 양식을 다르게 인스턴스화하도록 예제 뷰를 업데이트할 것입니다. 또한 양식이 유효할 경우에만 원시 POST 데이터 대신 정리된 데이터를 출력하도록 양식을 변경할 것입니다:

1. PyCharm에서 `form_example` 앱 디렉토리 내의 `forms.py` 파일을 엽니다. `ExampleForm`의 `decimal_input`에 `max_digits=3` 인수를 추가합니다:

```
class ExampleForm(forms.Form):
    ...
    decimal_input = forms.DecimalField(max_digits=3)
```

이 인수가 추가되면 브라우저가 이 규칙을 검증하는 방법을 모르기 때문에 양식을 제출할 수 있지만 Django는 알고 있습니다.

2. Open the reviews app's `views.py` file. We need to update the `form_example` view so that if the request's method is `POST`, `ExampleForm` is instantiated with the `POST` data; otherwise, it's instantiated without arguments. Replace the current form initialization with this code:

```
def form_example(request):
    if request.method == "POST":
        form = ExampleForm(request.POST)
    else:
        form = ExampleForm()
```

3. Next, also for the `POST` request method, we will check whether the form is valid using the `is_valid` method. If the form is valid, we will print out all the cleaned data. Add a condition after the `ExampleForm` instantiation to check `form.is_valid()`, then move the debug print loop inside this condition. Your `POST` branch should look like this:

```
if request.method == "POST":
    form = ExampleForm(request.POST)
    if form.is_valid():
        for name in request.POST:
            print(
                f"{name}: {request.POST.getlist(name)}")
```

4. Instead of iterating over the raw `request.POST` `QueryDict` (in which all the data is strings), we will iterate over the form's `cleaned_data`. This is a normal dictionary and contains the values converted into Python objects. Replace the `for` line and `print` line with these two:

```
for name, value in form.cleaned_data.items():
    print(f"{name}: ({type(value)}) {value}")
```

We don't need to use `getlist()` anymore since `cleaned_data` has already converted multi-value fields into lists.

5. Start the Django dev server, if it's not already running. Switch to your browser and browse to the example form page at `http://127.0.0.1:8000/form-example/`. The form should look as it did before. Fill in all the fields, but make sure that you enter four or more numbers into the **Decimal input** field to make the form invalid. Submit the

2. 리뷰 앱의 `views.py` 파일을 엽니다. 요청의 메서드가 POST인 경우 `ExampleForm`이 POST 데이터로 인스턴스화되도록 `form_example` 뷰를 업데이트해야 합니다. 그렇지 않으면 인수 없이 인스턴스화됩니다. 현재의 폼 초기화를 다음 코드로 교체하세요:

```
def form_example(request):
    if request.method == "POST":
        form = ExampleForm(request.POST)
    else:
        form = ExampleForm()
```

3. 다음으로, POST 요청 방법에 대해서도, `is_valid` 메서드를 사용하여 양식이 유효한지 확인합니다. 양식이 유효하면 모든 정리된 데이터를 출력합니다. `ExampleForm` 인스턴스화 후에 `form.is_valid()`를 확인하는 조건을 추가한 다음, 이 조건 안으로 디버그 인쇄 루프를 이동합니다. 귀하의 POST 분기는 다음과 같아야 합니다:

```
if request.method == "POST":
    form = ExampleForm(request.POST)
    if form.is_valid():
        for name in request.POST:
            print(
                f"{name}: {request.POST.getlist(name)}")
```

4. 원시 `request.POST QueryDict`를 반복하는 대신(모든 데이터가 문자열인 경우), 우리는 양식의 `cleaned_data`를 반복할 것입니다. 이것은 일반 사전이며 Python 객체로 변환된 값을 포함합니다. `for` 줄과 `print` 줄을 다음 두 줄로 교체하십시오:

```
for name, value in form.cleaned_data.items():
    print(f"{name}: ({type(value)}) {value}")
```

이제 `cleaned_data`가 이미 다중 값 필드를 리스트로 변환했기 때문에 `getlist()`를 사용할 필요가 없습니다.

5. Django 개발 서버를 시작하세요. 이미 실행 중이지 않다면. 브라우저로 전환하고 `http://127.0.0.1:8000/form-example/`의 예제 양식 페이지로 이동하세요. 양식은 이전과 동일하게 보여야 합니다. 모든 필드를 입력하되, 양식이 유효하지 않도록 소수 입력 필드에 네 개 이상의 숫자를 입력해야 합니다. 제출하세요.

form; you should see the error message for the **Decimal input** field show up when the page refreshes:

Figure 6.24: Decimal input error displayed after the form is submitted

6. Fix the form errors by making sure only three digits are in the **Decimal input** field, then submit the form again. Switch back to PyCharm and check the **Debug** console. You should see that all the cleaned data has been printed out:

```
text_input: (<class 'str'>) Text
password_input: (<class 'str'>) Password
checkbox_on: (<class 'bool'>) True
radio_input: (<class 'str'>) Value Two
favorite_book: (<class 'str'>) 1
books_you_own: (<class 'list'>) ['1', '3']
text_area: (<class 'str'>) Text Area
integer_input: (<class 'int'>) 154
float_input: (<class 'float'>) 11.0
decimal_input: (<class 'decimal.Decimal'>) 1234
email_input: (<class 'str'>) user@example.com
date_input: (<class 'datetime.date'>) 2024-10-27
hidden_input: (<class 'str'>) Hidden Value
```

Figure 6.25: The cleaned data from the form has been printed out

양식; 페이지가 새로 고쳐질 때 소수 입력 필드에 대한 오류 메시지가 표시되어야 합니다:

그림 6.24: 양식 제출 후 표시된 소수 입력 오류

6. 소수 입력 필드에 세 자리 숫자만 포함되도록 하여 양식 오류 idx_e5f9ad05를 수정한 후 양식을 다시 제출하십시오. PyCharm으로 돌아가서 디버그 콘솔을 확인하십시오. 모든 정리된 데이터가 출력된 것을 볼 수 있어야 합니다:

```
text_input: (<class 'str'>) Text
password_input: (<class 'str'>) Password
checkbox_on: (<class 'bool'>) True
radio_input: (<class 'str'>) Value Two
favorite_book: (<class 'str'>) 1
books_you_own: (<class 'list'>) ['1', '3']
text_area: (<class 'str'>) Text Area
integer_input: (<class 'int'>) 154
float_input: (<class 'float'>) 11.0
decimal_input: (<class 'decimal.Decimal'>) 1234
email_input: (<class 'str'>) user@example.com
date_input: (<class 'datetime.date'>) 2024-10-27
hidden_input: (<class 'str'>) Hidden Value
```

그림 6.25: 양식에서 정리된 데이터가 출력되었습니다.

Notice the conversions that have taken place. `CharField` has been converted into `str`, `BooleanField` into `bool`, and `IntegerField`, `FloatField`, and `DecimalField` into `int`, `float`, and `Decimal`, respectively. `DateTimeField` becomes a `datetime.date`, and the choice fields will retain the string values of their initial choice values. Notice that `books_you_own` is automatically converted into a list of `str`.

Also, note that, unlike when we iterated over all of the POST data, `cleaned_data` only contains form fields. The other data (such as the CSRF token and the submit button that was clicked) is present in the POST request's `QueryDict` object but is not included, as it is not form fields.

In this exercise, you updated `ExampleForm` so that the browser allowed it to be submitted even though Django would consider it to be invalid. This allowed Django to perform its validation on the form. You then updated the `form_example` view to instantiate the `ExampleForm` class differently, depending on the HTTP method, passing in the request's POST data for a POST request. The view also had its debug output code updated to print out the `cleaned_data` dictionary. Finally, you tested submitting valid and invalid form data to see the different execution paths and the types of data that the form generated. We saw that Django automatically converted the POST data from strings into Python types based on the field class.

Next, we will look at how to add more validation options to fields, which will allow us to more tightly control the values that can be entered.

Built-in field validation

We have not yet discussed the standard validation arguments that can be used on fields. Although we already mentioned the `required` argument (which is `True` by default), many others can be used to control the data being entered into a field more tightly. Here are a few useful ones:

- `max_length`: Sets the maximum number of characters that can be entered into the field, available on `CharField` (and `FileField` – which we will cover in *Chapter 8*).
- `min_length`: Sets the minimum number of characters that must be entered into the field, available on `CharField` (and `FileField` – again, more about it in *Chapter 8*).
- `max_value`: Sets the maximum value that can be entered into a numeric field, available on `IntegerField`, `FloatField`, and `DecimalField`.
- `min_value`: Sets the minimum value that can be entered into a numeric field, available on `IntegerField`, `FloatField`, and `DecimalField`.

변환이 이루어진 것을 주목하세요. CharField는 str로, BooleanField는 bool로, IntegerField, FloatField, DecimalField는 각각 int, float, Decimal로 변환되었습니다. DateField는 datetime.date가 되고, choicefields는 초기 선택 값의 문자열 값을 유지합니다. books_you_own은 자동으로 str의 리스트로 변환됩니다.

또한, idx_bbcfbe06와 달리 모든 POST 데이터를 반복할 때 cleaned_data는 오직 폼 필드만 포함된다는 점에 유의하세요. CSRF 토큰 및 클릭된 제출 버튼과 같은 다른 데이터는 POST 요청의 QueryDict 객체에 존재하지만 폼 필드가 아니므로 포함되지 않습니다.

이 연습에서는 ExampleForm을 업데이트하여 브라우저가 Django가 유효하지 않다고 간주하더라도 제출할 수 있도록 했습니다. 이를 통해 Django는 폼에 대한 유효성 검사를 수행할 수 있었습니다. 그런 다음 HTTP 메서드에 따라 ExampleForm 클래스를 다르게 인스턴스화하도록 form_example 뷰를 업데이트하고, POST 요청에 대한 요청의 POST 데이터를 전달했습니다. 뷰의 디버그 출력 코드도 cleaned_data 사전을 출력하도록 업데이트되었습니다. 마지막으로, 유효한 폼 데이터와 유효하지 않은 폼 데이터를 제출하여 서로 다른 실행 경로와 폼이 생성한 데이터 유형을 확인했습니다. Django가 필드 클래스에 따라 POST 데이터를 문자열에서 Python 유형으로 자동 변환하는 것을 보았습니다.

다음으로, idx_41a146c6 필드에 더 많은 유효성 검사 옵션을 추가하는 방법을 살펴보겠습니다. 이를 통해 입력할 수 있는 값을 더 엄격하게 제어할 수 있습니다.

내장 필드 유효성 검사

우리는 아직 필드에서 사용할 수 있는 표준 유효성 검사 인수에 대해 논의하지 않았습니다. 이미 기본값이 True인 required 인수를 언급했지만, 필드에 입력되는 데이터를 더 엄격하게 제어하기 위해 사용할 수 있는 많은 다른 인수들이 있습니다. 유용한 몇 가지는 다음과 같습니다:

- **max_length**: 필드에 입력할 수 있는 최대 문자 수를 설정합니다. CharField(및 Chapter 8에서 다룰 FileField)에서 사용할 수 있습니다.
- **min_length**: 필드에 입력해야 하는 최소 문자 수를 설정합니다.
- **CharField**(및 다시 Chapter 8에서 더 다룰 FileField)에서 사용할 수 있습니다.
- **max_value**: 숫자 필드에 입력할 수 있는 최대 값을 설정합니다.
- **IntegerField**, **FloatField** 및 **DecimalField**에서 사용할 수 있습니다.
- **min_value**: 숫자 필드에 입력할 수 있는 최소 값을 설정합니다. **IntegerField**, **FloatField** 및 **DecimalField**에서 사용할 수 있습니다.

4. The user may have no books, so the `books_you_own` field should not be required. Add a `required=False` argument to the `MultipleChoiceField` constructor:

```
books_you_own = forms.MultipleChoiceField(
    required=False, choices=BOOK_CHOICES)
```

5. The user should only be able to enter a value between 1 and 10 in `integer_input`. Add `min_value=1` and `max_value=10` arguments to the `IntegerField` constructor:

```
integer_input = forms.IntegerField(
    min_value=1, max_value=10)
```

6. Finally, add `max_digits=5` and `decimal_places=3` to the `DecimalField` constructor:

```
decimal_input = forms.DecimalField(
    max_digits=5, decimal_places=3)
```

Save the file.

7. Start the Django dev server if it's not already running. We do not have to make any changes to any other files to get these new validation rules since Django automatically updates the HTML generation and validation logic. This is a great benefit you get from using Django forms. Just visit or refresh `http://127.0.0.1:8000/form-example/` in your browser, and the new validation will be automatically added. The form should not look any different until you try to submit it with incorrect values, in which case your browser can automatically show errors. Some things to try are as follows:

- Entering more than three characters into the **Text input** field is something you will not be able to do.
- Type less than eight characters into the **Password** field, and then clicking away from it might cause your browser to show an error indicating that this is not valid (only some browsers support this).
- Do not select any values for the **Books you own** field. This will not prevent you from submitting the form anymore.

4. 사용자가 책이 없을 수도 있으므로 `books_you_own` 필드는 필수로 지정하지 않아야 합니다. `MultipleChoiceField` 생성자에 `required=False` 인수를 추가하세요:

```
books_you_own = forms.MultipleChoiceField(
    필수=False, 선택=BOOK_CHOICES)
```

5. 사용자는 `integer_input`에 1에서 10 사이의 값만 입력할 수 있어야 합니다. 추가하세요. `IntegerField` 생성자에 대한 `min_value=1` 및 `max_value=10` 인수:

```
integer_input = forms.IntegerField(
    min_value=1, max_value=10)
```

6. 마지막으로, `max_digits=5`와 `decimal_places=3`을 `DecimalField` 생성자에 추가합니다:

```
decimal_input = forms.DecimalField(
    max_digits=5, decimal_places=3)
```

파일을 저장하세요.

7. Django 개발 서버가 이미 실행 중이지 않다면 시작하세요. 이러한 새로운 유효성 검사 규칙을 적용하기 위해 다른 파일에 변경을 할 필요가 없습니다. Django는 HTML 생성 및 유효성 검사 로직을 자동으로 업데이트합니다. 이것은 Django 폼을 사용하면서 얻는 큰 이점입니다. 브라우저에서 `http://127.0.0.1:8000/form-example/`를 방문하거나 새로 고침하면 새로운 유효성 검사가 자동으로 추가됩니다. 잘못된 값으로 제출을 시도하기 전까지는 폼이 달라 보이지 않아야 하며, 이 경우 브라우저가 자동으로 오류를 표시할 수 있습니다. 시도해 볼 몇 가지 사항은 다음과 같습니다:

- 텍스트 입력 필드에 세 개 이상의 문자를 입력하는 것은 할 수 없습니다.
- 비밀번호 필드에 여덟 개 미만의 문자를 입력한 후 다른 곳을 클릭하면 브라우저에서 유효하지 않다는 오류가 표시될 수 있습니다(일부 브라우저만 지원합니다).
- 소유한 책 필드에 대한 값을 선택하지 마십시오. 더 이상 양식을 제출하는 것을 방해하지 않습니다.

- Use the stepper buttons on **Integer input**. You will only be able to enter a value between 1 and 10. If you type in a value outside this range, your browser should show an error.
- **Decimal input** is the only field that does not validate the Django rules in the browser. You will need to type in an invalid value (such as 123.456) and submit the form before an error (generated by Django) is displayed.

The following figure shows some of the fields that the browser can use to validate itself:

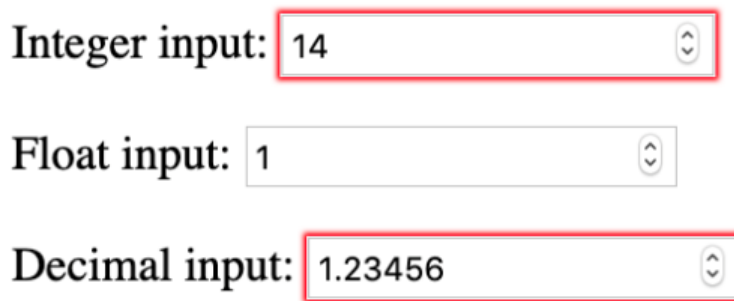


Figure 6.26: Some browsers highlight invalid fields

Figure 6.27 shows an alternate way that validation errors might be presented by the browser – by placing an error message next to a particular field on form submission:

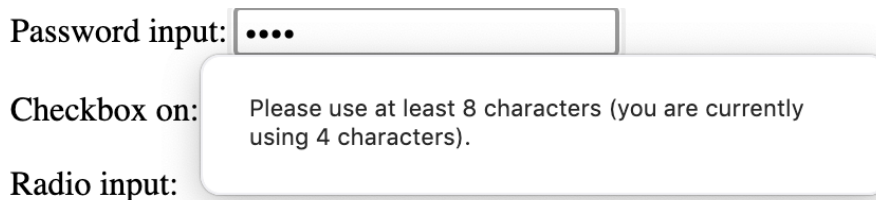


Figure 6.27: Browsers might show messages next to fields on submit

Figure 6.28 shows an error that can only be generated by Django, as the browser does not understand the `DecimalField` validation rules:

- Ensure that there are no more than 3 digits in total.

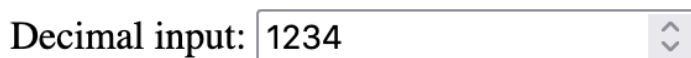


Figure 6.28: The browser considers the form valid, but Django does not

◦ 정수 입력에서 스텝 버튼을 사용하세요. 1과 10 사이의 값만 입력할 수 있습니다. 이 범위를 벗어난 값을 입력하면 브라우저에서 오류가 표시되어야 합니다.

◦ 소수 입력은 브라우저에서 Django 규칙을 검증하지 않는 유일한 필드입니다. 유효하지 않은 값(예: 123.456)을 입력하고 제출해야 합니다.

오류(Django에서 생성됨)가 표시되기 전에 양식을 제출해야 합니다. 다음 그림 idx_d9269a57은 브라우저가 스스로 검증하는 데 사용할 수 있는 일부 필드를 보여줍니다:

The figure shows three input fields with their respective labels and values:

- Integer input:** A text input field containing the value '14'. The field is highlighted with a red border, indicating a validation error.
- Float input:** A text input field containing the value '1'.
- Decimal input:** A text input field containing the value '1.23456'. The field is highlighted with a red border, indicating a validation error.

그림 6.26: 일부 브라우저는 유효하지 않은 필드를 강조 표시합니다

그림 6.27은 브라우저가 제출 시 특정 필드 옆에 오류 메시지를 배치하여 검증 오류를 표시하는 대체 방법을 보여줍니다:

The figure shows a form with three input fields and a validation message:

- Password input:** A text input field containing four dots (••••).
- Checkbox on:** A checkbox that is checked.
- Radio input:** A radio button that is selected.
- Validation message:** A message box that appears when the form is submitted, stating: "Please use at least 8 characters (you are currently using 4 characters)."

그림 6.27: 브라우저는 제출 시 필드 옆에 메시지를 표시할 수 있습니다

그림 6.28은 브라우저가 DecimalField 검증 규칙을 이해하지 못하기 때문에 Django에서만 생성할 수 있는 오류를 보여줍니다:

- Ensure that there are no more than 3 digits in total.

The figure shows a text input field labeled "Decimal input:" containing the value '1234'. The field is highlighted with a red border, indicating a validation error.

그림 6.28: 브라우저는 양식을 유효하다고 간주하지만 Django는 그렇지 않습니다

In this exercise, we implemented some basic validation rules on our form fields. We then loaded the form example page in the browser without having to make any changes to our template or view. We tried to submit the form with different values to check how the browser can validate the form compared to Django.

In the activity for this chapter, we will implement the Book Search view using a Django Form.

Activity 6.01 – Book Search

In this activity, you will finish the Book Search view that was started in *Chapter 1*. You will build a `SearchForm` that submits and accepts a search string from `request.GET`. It will have a select field to choose to search for a title or contributor. It will then search for all books containing the given text in their title or the contributor's first names or last names. You will then render this list of books in the `search-results.html` template. The search term should not be required, but if it exists, it should have a length of more than three characters. Since the view will search even when using the GET method, the form will always have its validation checked. If we made the field required, it would always show an error whenever the page loads.

There will be two ways of performing the search. The first is by submitting the search form that is in the `base.html` template and thus in the top-right corner of every page. This will only search through Book titles. The other method is by submitting a `SearchForm` that is rendered on the `search-results.html` page. This form will display a `ChoiceField` for selecting between a title or contributor search.

These steps will help you complete this activity:

1. Create a `forms.py` file, import the Django forms library, then create a `SearchForm` class.
2. `SearchForm` should have two fields. The first is `CharField` with the name `search`. This field should not be required, but should have a minimum length of 3.
3. The second field on `SearchForm` is `ChoiceField` with the name `search_in`. This will allow you to select between title and contributor (with labels of **Title** and **Contributor**, respectively). It should not be required.
4. Update the `book_search` view so that it instantiates a `SearchForm` using data from `request.GET`.
5. Add code to search for Book models using `title__icontains` (for case-insensitive searching). This should be done if searching by *title*. The search should only be performed if the form is valid and contains some search text. The `search_in` value

이 연습에서는 양식 필드에 대한 기본 유효성 검사 규칙을 구현했습니다. 그런 다음 템플릿이나 `idx_131647bcview`를 변경하지 않고 브라우저에서 양식 예제 페이지를 로드했습니다. 브라우저가 Django와 비교하여 양식을 어떻게 유효성 검사하는지 확인하기 위해 다양한 값으로 양식을 제출해 보았습니다.

이번 장의 활동에서는 Django Form을 사용하여 도서 검색 뷰를 구현할 것입니다.

활동 6.01 – 도서 검색

이 활동에서는 Chapter 1에서 시작된 도서 검색 `idx_c2b869d3view`를 완성할 것입니다. 요청의 GET에서 검색 문자열을 제출하고 수락하는 `SearchForm`을 구축할 것입니다. 제목 또는 기여자를 검색하기 위해 선택할 수 있는 선택 필드가 있을 것입니다. 그런 다음 주어진 텍스트가 제목이나 기여자의 이름 또는 성에 포함된 모든 책을 검색할 것입니다. 이 책 목록은 `search-results.html` 템플릿에서 렌더링할 것입니다. 검색어는 필수는 아니지만, 존재하는 경우 세 글자 이상이어야 합니다. GET 메서드를 사용할 때도 뷰가 검색을 수행하므로 양식은 항상 유효성 검사가 이루어집니다. 필드를 필수로 만들면 페이지가 로드될 때마다 오류가 표시됩니다.

검색을 수행하는 두 가지 방법이 있습니다. 첫 번째는 `base.html` 템플릿에 있는 검색 양식을 제출하는 것이며, 따라서 모든 페이지의 오른쪽 상단 모서리에 있습니다. 이는 오직 도서 제목만 검색합니다. 다른 방법은 `search-results.html` 페이지에 렌더링된 `SearchForm`을 제출하는 것입니다. 이 양식은 제목 또는 기여자 검색 중에서 선택할 수 있는 `ChoiceField`를 표시합니다.

이 단계는 이 활동을 완료하는 데 도움이 될 것입니다:

1. `forms.py` 파일을 생성하고, Django forms 라이브러리를 임포트한 다음, `SearchForm` 클래스를 생성합니다. `SearchForm`은 두 개의 필드를 가져야 합니다. 첫 번째는 이름이 `search`인 `CharField`입니다. 이 필드는 필수가 아니지만 최소 길이는 3이어야 합니다.
3. `SearchForm`의 두 번째 필드 `idx_4747c205`는 이름이 `search_in`인 `ChoiceField`입니다. 이는 제목과 기여자(각각 제목과 기여자의 레이블을 사용) 중에서 선택할 수 있게 해줍니다. 필수가 아닙니다.
4. `book_search` 뷰를 업데이트하여 `request.GET`의 데이터를 사용하여 `SearchForm`을 인스턴스화합니다.
5. 제목으로 검색할 경우 `Book` 모델을 `title__icontains`를 사용하여 검색하는 코드를 추가합니다(대소문자를 구분하지 않는 검색을 위해). 검색은 폼이 유효하고 일부 검색 텍스트가 포함된 경우에만 수행되어야 합니다. `search_in` 값

should be retrieved from `cleaned_data` using the `GET` method since it might not exist, as it's not required. Default it to `title`.

6. When searching for contributors, use `first_names__icontains` or `last_names__icontains`, then iterate the contributors and retrieve the books for each contributor. This should be done if you're searching by *contributor*. The search should only be performed if the form is valid and contains some search text. There are many ways to combine the search results for first or last names. The easiest method using the techniques that you have been introduced to so far is to perform two queries, one for matching first names and then for last names, and iterate them separately.
7. Update the render call so that it includes the `form` variable and books that were retrieved in the context (as well as `search_text`, which was already being passed). The location of the template was changed in *Chapter 3*, so update the second argument to render accordingly.
8. The `search-results.html` template we created in *Chapter 1* is essentially redundant now, so you can clear its content. Update `search-results.html` so that it extends from `base.html` instead of being a standalone template file.
9. Add a title block that will display **Search Results for <search_text>** if the form is valid and `search_text` was set; otherwise, just display **Book Search**. This block will also be added to `base.html` later in this activity.
10. Add a content block, which should show a `<h2>` heading with the text **Search for Books** under the `<h2>` render the form. The `<form>` element can have no attributes, and it will default to making a GET request to the same URL it's on. Add a submit button, as we have used in previous activities, with the `btn btn-primary` class.
11. Under the form, show a message stating **Search results for <search_text>** if the form is valid and search text was entered; otherwise, show no message. This should be displayed in a `<h3>` heading, and the search text should be wrapped in an `<em>`.
12. Iterate over the search results and render each one. Show the book title and contributor(s) first and last names. The book title should link to the `book_detail` page. If the books list is empty, show the text **No results found**. You should wrap the results in a `<ul>` with a class called `list-group`; each result should be a `<li>` with a class of `list-group-item`. This will be like the `book_list` page, but we won't show as much information (just the title and contributors).
13. Update the project-level `base.html` file so that it includes an `action` attribute in the search `<form>` tag. Use the `url` template tag to generate the URL for this attribute.

cleaned_data에서 GET 방법을 사용하여 검색해야 합니다. 존재하지 않을 수 있으므로 필수는 아닙니다. 기본값은 title로 설정합니다.

6. 기여자를 검색할 때는 first_names__icontains 또는 last_names__icontains를 사용하고, 그 후 기여자를 반복하여 각 기여자에 대한 책을 검색합니다. 기여자로 검색하는 경우에만 이 작업을 수행해야 합니다. 검색은 양식이 유효하고 일부 검색 텍스트가 포함된 경우에만 수행되어야 합니다. 이름이나 성에 대한 검색 결과를 결합하는 방법은 여러 가지가 있습니다. 지금까지 소개된 기술을 사용하여 가장 쉬운 방법은 두 개의 쿼리를 수행하는 것입니다. 하나는 이름을 일치시키고, 다른 하나는 성을 일치시키며, 각각을 별도로 반복합니다.
7. 렌더 호출을 업데이트하여 양식 변수와 컨텍스트에서 검색된 책을 포함하도록 합니다(이미 전달되고 있는 search_text도 포함). 템플릿의 위치는 3장에서 변경되었으므로 두 번째 인수를 렌더에 맞게 업데이트합니다.
8. 1장에 만든 search-results.html 템플릿은 본질적으로 중복되므로 내용을 지울 수 있습니다. search-results.html을 독립형 템플릿 파일이 아닌 base.html에서 확장되도록 업데이트하세요.
9. 유효한 경우 <search_text>에 대한 검색 결과를 표시하는 title block idx_0bffa534를 추가하고, 그렇지 않으면 Book Search만 표시하세요. 이 블록은 나중에 이 활동에서 base.html에 추가될 것입니다.
10. <h2> 제목 아래 Books를 검색하라는 텍스트가 있는 content block을 추가하세요. <h2> 아래에 양식을 렌더링합니다. <form> 요소는 속성이 없어야 하며, 현재 URL에 GET 요청을 기본적으로 하게 됩니다. 이전 활동에서 사용한 것처럼 btn btn-primary 클래스를 가진 제출 버튼을 추가하세요.
11. 양식 아래에 유효한 경우 <search_text>에 대한 검색 결과를 나타내는 메시지를 표시하고, 그렇지 않으면 메시지를 표시하지 마세요. 이는 <h3> 제목으로 표시되어야 하며, 검색 텍스트는 으로 감싸야 합니다.
12. 검색 결과를 반복하여 각 결과를 렌더링하세요. 책 제목과 기여자(들)의 이름을 먼저 표시하세요. 책 제목은 book_detail 페이지에 링크되어야 합니다. 책 목록이 비어 있으면 No results found라는 텍스트를 표시하세요. 결과는 list-group이라는 클래스를 가진 로 감싸야 하며, 각 결과는 list-group-item 클래스를 가진 여야 합니다. 이는 book_list 페이지와 비슷하지만, 많은 정보를 표시하지 않을 것입니다(제목과 기여자만).
13. 프로젝트 수준의 base.html 파일을 업데이트하여 검색 <form> 태그에 action 속성을 포함하세요. 이 속성을 위한 URL을 생성하기 위해 url 템플릿 태그를 사용하세요.

14. Set the name attribute of the search field to search, and the value attribute to the entered search_text. Also, ensure that the minimum length of the field is 3.
15. While in base.html, add a title block to the title tag that was overridden by other templates (as in *Step 9*). Add a block template tag inside the <title> HTML element. It should contain the content Bookr.

After completing this activity, you should be able to open the Book Search page at `http://127.0.0.1:8000/book-search/`; it will look like *Figure 6.29*:

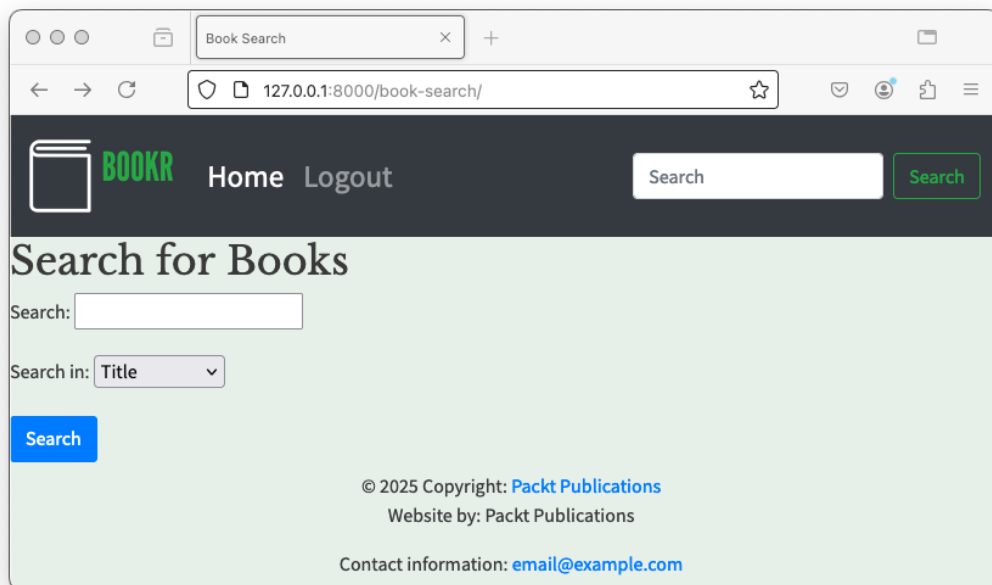


Figure 6.29: The Book Search page without a search

When searching for something using just two characters, your browser should prevent you from submitting either of the search fields.

If you search for something that returns no results, you will see a message that there were no results. Searching by title (this can be done with either field) will show matching results:

14. 검색 필드의 name 속성을 search로 설정하고, value 속성을 입력된 search_text로 설정합니다. 또한, 필드의 최소 길이가 3인지 확인하세요.
15. base.html에서 다른 템플릿에 의해 재정의를 title 태그에 제목 블록을 추가합니다 (9단계와 같이). <title> HTML 요소 안에 블록 템플릿 태그를 추가하세요. 이 태그는 Bookr라는 내용을 포함해야 합니다.

idx_68b21135this idx_fd9b9e11활동을 완료한 후, <http://127.0.0.1:8000/book-search/>에서 도서 검색 페이지를 열 수 있어야 합니다; 이는 그림 6.29와 같이 보일 것입니다:

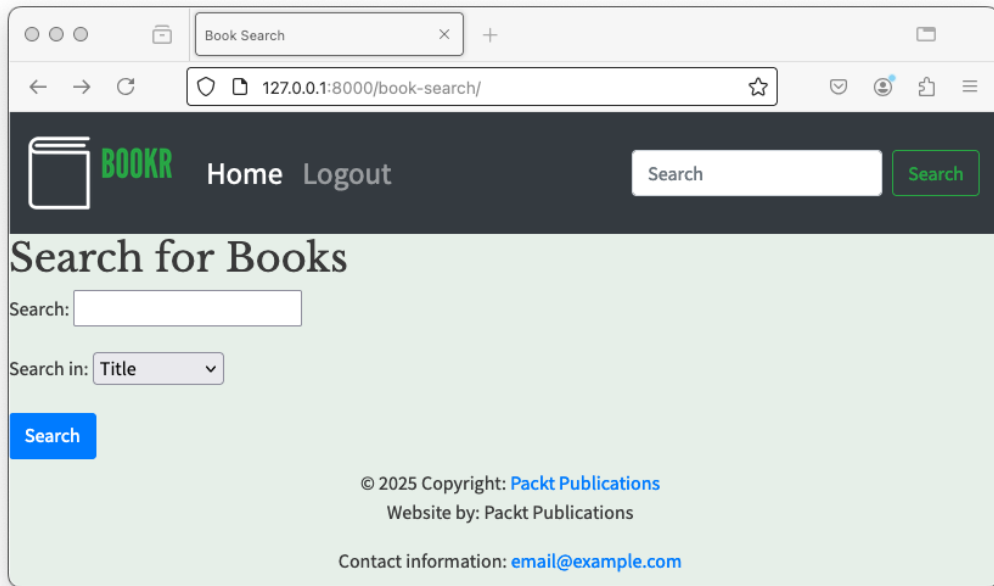


그림 6.29: 검색 없이 도서 검색 페이지

idx_f8fabd22에서 idx_84d5c196무언가를 두 문자만 사용하여 검색할 때, 브라우저는 검색 필드 중 어느 것도 제출하는 것을 방지해야 합니다.

결과가 없는 무언가를 검색하면 결과가 없다는 메시지가 표시됩니다. 제목으로 검색하면(이것은 두 필드 중 어느 것으로도 가능합니다) 일치하는 결과가 표시됩니다:

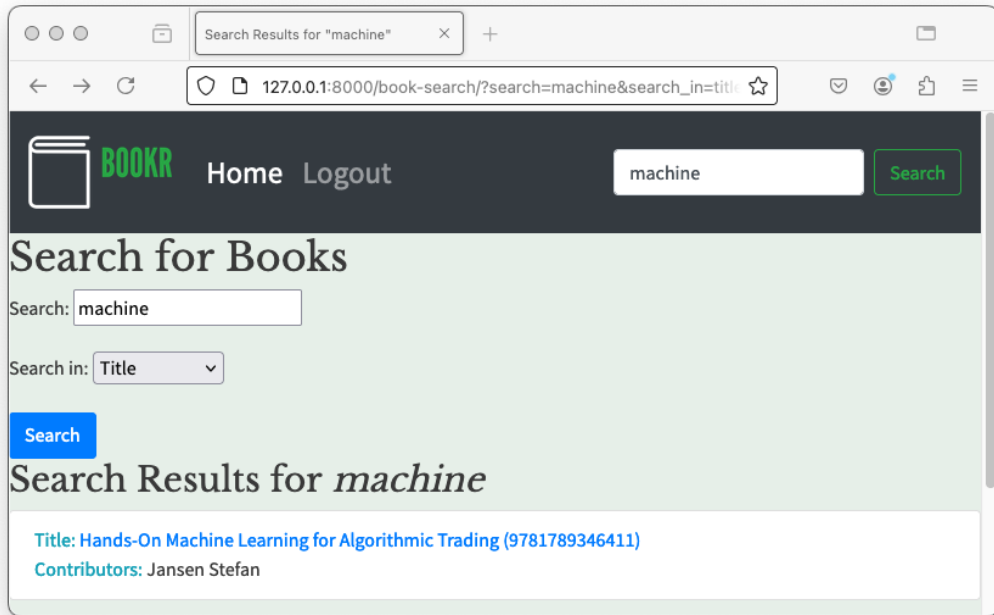


Figure 6.30: Title search

Similarly, when searching by contributor (although this can only be done in the lower form), you'll see matching results:

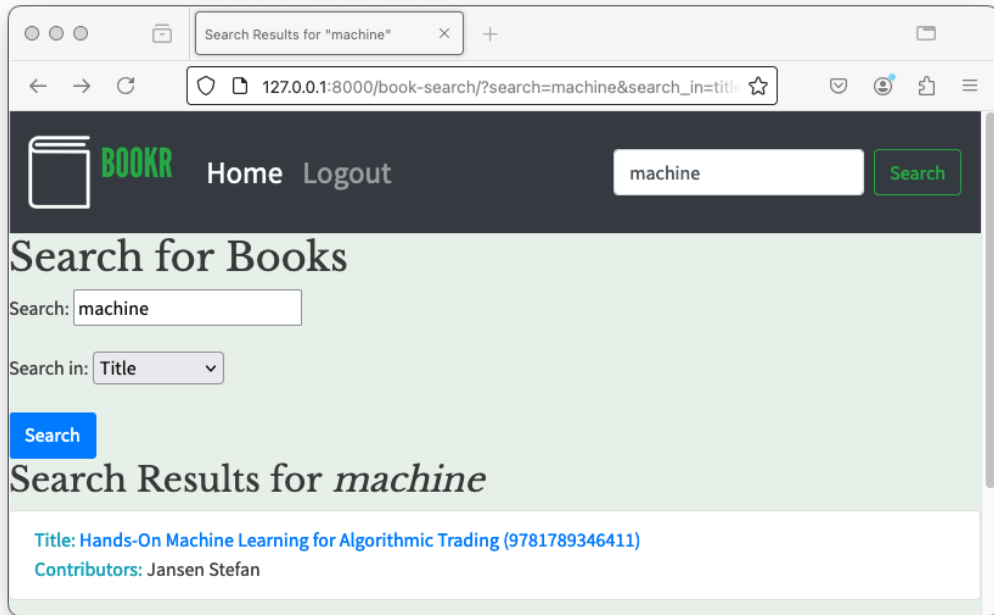


그림 6.30: 제목 검색

유사하게, idx_9b4c3e73가 기여자로 idx_2ddb2547을 검색할 때(비록 이것은 하위 양식에서만 가능하지만), 일치하는 결과를 볼 수 있습니다:

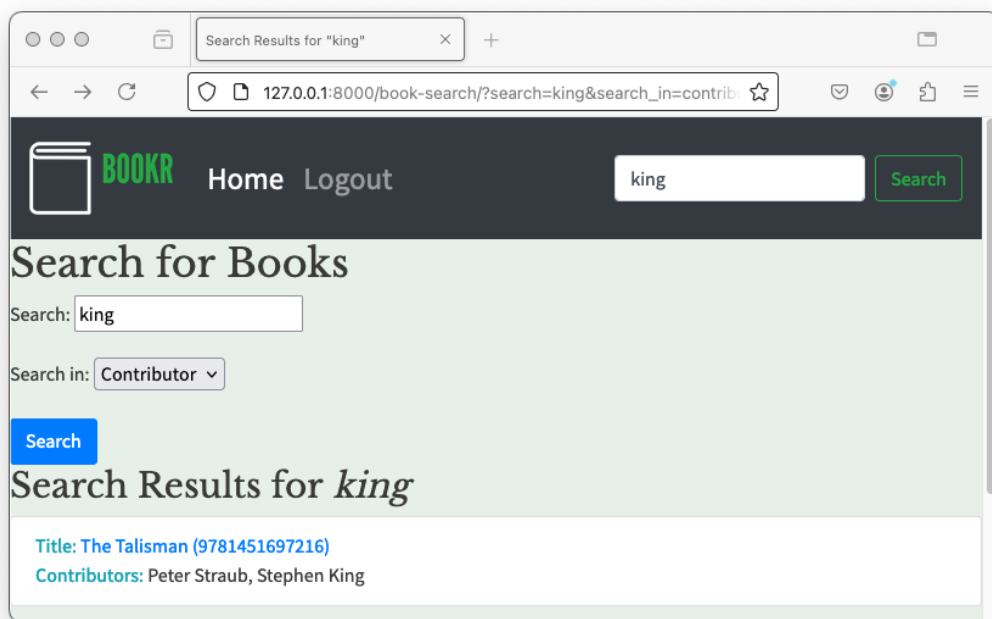


Figure 6.31: A contributor search

Summary

This chapter provided an introduction to forms in Django. We introduced several HTML inputs for entering data onto a web page. We talked about how data is submitted to a web application and when to use GET and POST requests. We then looked at how Django's form classes can make generating the form HTML simpler, as well as allow us to automatically build forms using models. Finally, we enhanced Bookr further by building the Book Search functionality.

In the next chapter, we will dive deeper into forms and learn how to customize the display of form fields, how to add more advanced validation to a form, and how to automatically save model instances by using the `ModelForm` class.

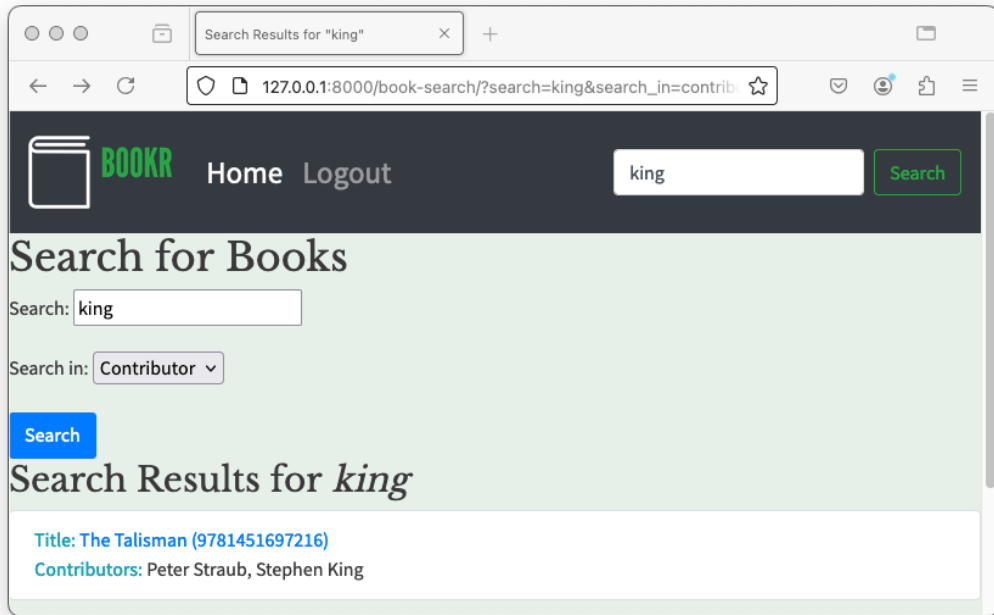


그림 6.31: 기여자 검색

요약

이 장에서는 Django의 폼에 대한 소개를 제공했습니다. 우리는 웹 페이지에 데이터를 입력하기 위한 여러 HTML 입력을 소개했습니다. 우리는 데이터가 웹 애플리케이션에 제출되는 방법과 GET 및 POST 요청을 사용할 때에 대해 이야기했습니다. 그런 다음 Django의 폼 클래스를 사용하여 폼 HTML을 생성하는 과정을 간소화하고 모델을 사용하여 자동으로 폼을 구축하는 방법을 살펴보았습니다. 마지막으로, 우리는 Bookr를 더욱 향상시켜 도서 검색 기능을 구축했습니다.

다음 장에서는 폼에 대해 더 깊이 파고들고 폼 필드의 표시를 사용자 정의하는 방법, 폼에 더 고급 유효성 검사를 추가하는 방법, 그리고 ModelForm 클래스를 사용하여 모델 인스턴스를 자동으로 저장하는 방법을 배울 것입니다.